Table of Contents

Frontmatter . . . 7
Conventions . . . 9
- Code . . . 9
- Formatting . . . 9
- Noisy Configuration Classes . . . 9
- Spring Profiles . . . 10
- Nullability . . . 10
- Spring Initializr Project Configuration . . . 11
- Docker . . . 12
- Databases . . . 12
- A Brief Note on Storing Credentials . . . 13
- Database Migrations . . . 14

Modern Java, Modern Language . . . 17
- The Amazing `AutoCloseable` . . . 17
- Type Inference . . . 21
- Enhanced Switch Expressions . . . 23
- Multiline Strings . . . 25
- Records . . . 26
- Sealed Types . . . 28
- Smart Casts . . . 30
- Functional Interfaces, Lambdas, and Method References . . . 31
- Pattern Matching . . . 33
- The Notation This Book Is Written In . . . 35

Modern Java, Modern Runtime . . . 37
- Operations Improvements . . . 37
- Performance . . . 37
- Java 25, Project Loom, and Virtual Threads . . . 38
- Improving Startup with Project Leyden . . . 40
- Improving Startup and RAM footprint with GraalVM Native Images . . . 40
- Next Steps . . . 41

Dependency Injection and Portable Service Abstractions . . . 43
- The Spring Initializr . . . 43
- The Database . . . 44
- A Repository of Knowledge… About Dogs . . . 45
- Dependency Injection . . . 47
- Portable Service Abstractions . . . 49
- Unified Transaction Handling with `PlatformTransactionManager` . . . 50
- Prefer Composition to Inheritance . . . 54

Simplified Transaction Management with `TransactionTemplate` . . . 55
Aspect-Oriented Programming and the Spring Container . . . 57
Cross-Cutting Concerns . . . 57
Hold My Proxy! . . . 57
What About Concrete Types? . . . 60
Better Configuration with Spring . . . 62
Bean Lifecycles . . . 65
Post-Processors . . . 70
But `@Transactional` Exists, So Let's Use That Instead . . . 75
The Words for What We Just Built . . . 77
Registering Beans and Externalized Configuration . . . 79
Component Scanning . . . 79
Spring Framework 7's `BeanRegistrar`: The Best of Both Worlds . . . 82
The `Environment` . . . 83
The Spring Expression Language . . . 86
Next Steps . . . 90
Spring Boot . . . 91
Spring Data JDBC . . . 92
Conventions over Configuration . . . 92
AutoConfiguration . . . 92
An Amazing Web Server . . . 96
Observable Services . . . 97
GraalVM Native Images . . . 98
Project Leyden . . . 101
The Spring Boot Buildpack integration . . . 102
Testing Spring Boot Applications . . . 105
The Core Spring TestContext Framework . . . 105
Unit Testing . . . 105
Integration Testing . . . 106
Isolated Configuration . . . 108
Testing services with `MockMvc` . . . 113
Endpoint Testing with `RestTestClient` . . . 114
Spring Boot Test Slices . . . 118
Next Steps . . . 119
Spring Security Fundamentals . . . 121
Spring Security, the Project . . . 121
Spring Security for all Contexts . . . 122
Authentication . . . 123
Authentication in Spring Security . . . 123
Filters . . . 127
Spring Session . . . 131

Meet the Persistence Backends. . . . 131
Setting Up Spring Session JDBC 132
Spring Security Integration 135
Keeping Sessions Alive over WebSockets. . . . 139
Conclusion 143
The Trouble with Passwords 145
Authentication in a Web Application 145
In-Memory Users 146
DAO-Backed Authentication 147
Password Encoding 149
But Passwords Are Bad! 152
Bad "Received Wisdom" 153
Beyond Passwords: Magic Links. . . . 155
Authentication Around the World. . . . 155
One-Time Tokens, or "Magic Links". . . . 157
Passkeys and Multi-Factor Authentication. . . . 165
The Powerful Passkeys 165
Concepts We'll Need To Understand for WebAuthn 166
Implementing Passkeys in Spring Security 166
Persistence. . . . 169
Other Interesting Kinds of Authentication 171
Multifactor Authentication 172
Authorization: Deciding Who Can Do What 175
Method-Level Authorization 175
Request-Based Authorization 179
Granular Multi-Factor Authentication 185
Authentication Events in Spring Security 189
Long-Term Credentials vs. Short-Term Credentials 190
The All-or-Nothing Problem 191
Open Authorization for an Open Web 191
OAuth 192
"Continue with Google," Step by Step 192
OpenID Connect (OIDC) 198
What OAuth Gives You 199
Evolution of OAuth 199
OAuth vs. SAML 200
Towards a Passwordless Future. . . . 200
Introducing the Spring Authorization Server 203
Why Spring Authorization Server? 203
The Spring Security OAuth Ship of Theseus 206
One Less `users` Microservice for All 207

Implementing the Spring Authorization Server . . . 209
Users . . . 210
OAuth Clients . . . 211
The OAuth Client . . . 213
The End of the Beginning . . . 216
Persistent State in the Spring Authorization Server with PostgreSQL and JDBC . . . 217
PostgreSQL as the Database . . . 217
Adding Persistence . . . 218
Spring Session . . . 218
Users . . . 219
Persisting Clients . . . 221
Persisting Authorizations . . . 223
Signing Keys in the Spring Authorization Server . . . 229
Generating a Key Pair . . . 229
Rotating Keys . . . 231
A Brief Look at the State of Cryptography with PostgreSQL . . . 231
Generating New Keys Programmatically . . . 232
Key Pair Persistence . . . 233
To Production… and Beyond! . . . 245
Protecting a Simple HTTP API . . . 247
The Flow . . . 247
The OAuth Client . . . 248
The Resource Server . . . 250
Running the Whole Thing . . . 251
Centralizing the `RestClient` Authentication Logic . . . 252
Secure Declarative HTTP Service Clients . . . 253
What Survives, and What Doesn’t . . . 254
A Real OAuth Client with Spring Cloud Gateway . . . 255
PKCE with JavaFX Desktop-based Applications . . . 259
GraphQL . . . 261
gRPC . . . 263
Secure SOAP-based Web Services . . . 265
Protecting Spring AI-based Model Context Protocol (MCP) Services . . . 267
Messaging . . . 269
Spring Integration . . . 269
Defining RabbitMQ Infrastructure . . . 271
A Few Well Known Constants . . . 272
Command Line Interfaces . . . 277
Federated OAuth with the Spring Authorization Server . . . 279
Multitenancy . . . 281
From One to Many Users . . . 281

Earlier Is Easier 281
Arconia's Multitenancy Module 282
What We're Building 282
The Spring Authorization Server 284
The OAuth Client 289
The UI 292
The OAuth Resource Server 293
The Customer's Always Right - If You've Picked the Right Tenant 296
Multitenancy in the Data Layer 297
Tenant-specific DataSource Initialization 298
Tenant-aware DataSources 300
Next Steps 312

Frontmatter

Secure all the things with the Spring Authorization Server by Josh Long

© 2026 Josh Long. All rights reserved.

ISBN: {isbn} Version 1.0.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means - electronic, mechanical, photocopying, recording, or otherwise - without the prior written permission of the publisher, except for brief quotations embodied in reviews and as otherwise permitted under applicable copyright law.

This restriction applies to the text of this work; the code examples are licensed separately, as described below.

Use of Code Examples The code examples in this book are free to use.

You may use, copy, modify, and distribute them for any purpose, including in your own programs, documentation, or written works (such as another book that uses them as samples), without requiring permission and without attribution.

This permission applies only to the code examples and not to the text, figures, or other content of this book, which remain protected as described above.

Disclaimer of Liability While the author and publisher have used good faith efforts to ensure that the information and instructions in this work are accurate, they disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work.

Use of the information and instructions contained in this work is at your own risk.

If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

Use of AI Tools Portions of this work, including code and text, were created or assisted with the use of artificial intelligence tools.

While such content has been reviewed and edited by the author, the author and publisher make no warranties regarding its accuracy, completeness, or fitness for any purpose, and disclaim all liability arising from its use.

Readers applying AI tools to the material in this work do so at their own risk.

Trademarks, service marks, product names, and company names referenced in this work are the property of their respective owners and are used for identification and editorial purposes only. Their use does not imply endorsement or affiliation.

While the author has used reasonable faith efforts to ensure that the information and instructions in this work are accurate, the author disclaims all responsibility for errors or omissions, including

without limitation, responsibility for damages resulting from the use of or reliance on this work.

The use of the information and instructions contained in this work is at your own risk.

If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights. :code: ..

Conventions

In this book I'll do certain things that are for the purposes of this book and not necessarily the best configuration for code. There's a fundamental tension in the way books are written and the way code is written that I wrestle with on every attempt. I've long tried to find ways to write that avoid errors in the reproduced code. I want all the code printed to be compiled and valid. It is for this reason that the vast majority of the code reprinted in this book, and especially of the code reprinted in this book that comes from code I've written for this book, is actually in include of some actual source code. It's *not* code I copied and pasted into some Microsoft Word document or something, and that has long since diverged from the state of the "actual" code.

Code

As I work through these examples, I'll be showing lots of code. You can follow along at home, either by typing things out or simply referencing the code here [https://github.com/secure-all-the-things-book]. It's Apache 2 licensed and yours for the taking.

Formatting

I wrote some tooling to ensure that every example in the book is using the same version of Spring Boot, assumes the same version of Java (or Kotlin), and is using the same version of any projects that aren't managed by Spring Boot's parent `pom.xml`.

I also wrote some tooling to format the `pom.xml` files exactly the same and to format the Java source code exactly the same, too. I'm using the Spring Java Format Maven plugin [https://github.com/spring-io/spring-javaformat]. There's also a variant for Gradle.

The Spring Boot team created this plugin to consistently format the code in Spring Boot itself. This way, it doesn't matter if a contributor to the project is using emacs, vim, IntelliJ IDEA, Visual Studio Code, Eclipse, etc.: their code can be quickly and consistently formatted to match the project's style. Less quibbling about tabs vs. spaces, and more closed issues. This is a *good thing*!

If you want to format the source code:

```
./mvnw spring-javaformat:apply
```

Sometimes the formatter, which internally is based on the Eclipse IDE's formatter machinery, might format things in a way that reads poorly on a printed page. I sometimes force its hand by add empty line comment delimiters (`//`) at the end of a line, which forces the formatter to wrap to the next line.

Noisy Configuration Classes

Often, you'll see me introduce new beans in new configuration classes when they could've just as easily gone in a previous configuration class. This is so that I might avoid reprinting the original beans while keeping code working.

Spring Profiles

Similarly, I might introduce beans that I don't intend for us to land on, but that are interesting to know about. These beans are useful, but not the intended final result. In these cases, I'll use a Spring *profile*, which is a marker annotation that makes the bean inert - Spring doesn't "see" it - until you explicitly *activate* the profile.

```
@Configuration
@Profile("example")
class MyConfig {

 @Bean
 Bar bar(){
   // ...
 }

 @Bean
 @Profile("foo")
 Foo foo(){
   // ...
 }
}
```

Usually, you activate the profile (or profiles) by passing in a comma-delimited command line switch, e.g.:

```
./mvnw -Dspring.profiles.active=example,foo spring-boot:run
```

In this example, both the profile for the configuration class, `example`, and the profile for the nested bean, `foo`, are active and thus both the configuration class and the specific beans inside of it will be active. If the profile `example` is active, but not the `foo` profile, then the result would be that *only* the `bar` bean would be active, since it has no other profile.

As you work through the book, either remove the profile or activate it to ensure the relevant code runs when you run the example.

Generally, however, I try to avoid profiles for "real" code intended for production, as they make it easy to violate the 12-Factor principle of one build for multiple environments. And, there are some sticky issues with the use of profiles when building Spring AOT-powered applications. All in all, it's worth avoiding.

Nullability

Starting from Spring Framework 7 and Spring Boot 4, the entire Spring portfolio adopted the JSpecify.dev annotations [https://jspecify.dev]. These annotations define null restriction on references in the codebase. They give build tools like Apache Maven and Gradle, IDEs like IntelliJ IDEA, and compilers like Java and Kotlin feedback about when a `null` reference is being passed to a place that

it should not be.

Sometimes, in the course of this book, I'll present interfaces or types from Spring that feature these annotations. Spring Security is a very good example of this. It's an old project whose origins assumed a world where all authentication attempts would include a password. Later, new ways to authenticate (and arguably more secure ones) developed and the password field in the original `Authentication` became *nullable*. That is, the password wasn't required because there might be other things in the request - a token, a certificate, etc. - that did the work of identifying the user and so the password wasn't needed.

Adopting these JSpecify.dev annotations has been helpful for even the Spring team because it helped us to see places where our suppositions around null references didn't universally hold true. Considering that Spring Security is more than 20 years old, it's not surprising that these occasional inconsistencies sometimes read their ugly head!

Often, in the Spring codebases, you'll see a `package-info.java` class that contains a single package declaration and `@NullMarked`, like this one from `org.springframework.context`:

```
@NullMarked
package org.springframework.context;

import org.jspecify.annotations.NullMarked;
```

`@NullMarked` says that unless shown otherwise, every reference in this package is meant to be non-`null`. Occasionally, you'll see such exceptions marked with `@Nullable` on specific fields.

Spring Initializr Project Configuration

In this book we'll look at a *lot* of code, and I'll always go to the Spring Initializr and tell you how to consturct a project of similar makeup. I will basically always choose Apache Maven and I will basically always choose the latest and greatest version of Java or the latest and greatest long-term support (LTS) version of Java or Kotlin. At the time of this book's writing, the latest LTS is Java 25.

I may forget to stipulate the artifact ID. Use whatever you want there in this case.

If you're using the Spring Tools [https://spring.io/tools] for Visual Studio Code or Eclipse, or if you're using IntelliJ IDEA, you will have wizards inside of the IDE that'll guide you through the same steps as you would go through on the Spring Initializr. Indeed, those in-IDE wizards are frontends to the same webservice that powers the Spring Initializr, so the results are exactly the same. Feel free to use those, but for consistency, I'll refer to the Spring Initializr experience.

The Spring Initializr gives you a `.zip` file that you have to unzip. Inside you'll find a `pom.xml` (or `build.gradle` or `build.gradle.kts` if you go those routes). I usually then run IntelliJ IDEA: `idea pom.xml` from the command line.

I do this *so often* that I've even got a little Java script you can use.

```
// java ~/josh-env/bin/uao.java $HOME/Downloads/demo.zip
```

```
import java.io.*;
import java.util.function.*;
import java.util.*;

void main(String[] args) throws Exception {
    var zipFile = new File(args[0]);
    var zipFileAbsolutePath = zipFile.getAbsolutePath();
    var folder = new File(zipFileAbsolutePath.substring(0, zipFileAbsolutePath
.lastIndexOf(".")));
    new ProcessBuilder().command("unzip", "-a", zipFileAbsolutePath).inheritIO().
start().waitFor();
    for (var k : Set.of("build.gradle", "build.gradle.kts", "pom.xml")) {
        var buildFile = new File(folder, k);
        if (buildFile.exists()) {
            new ProcessBuilder().command("idea", buildFile.getAbsolutePath())
.inheritIO().start().waitFor();
        }
    }
}
```

I've created an alias to that script in my shell called `uao`.

Docker

I assume the availability of Docker or some sort of container orchestrator on your machines. Where appropriate, I've provided Docker Compose `compose.yml` files in each of the chapters' code.

If you're using the Docker tooling, you can spin up the container images thusly:

```
docker compose up
```

That will run the Docker images and block, keeping the shell on that process in the foreground. If you want to run things and the background them, do:

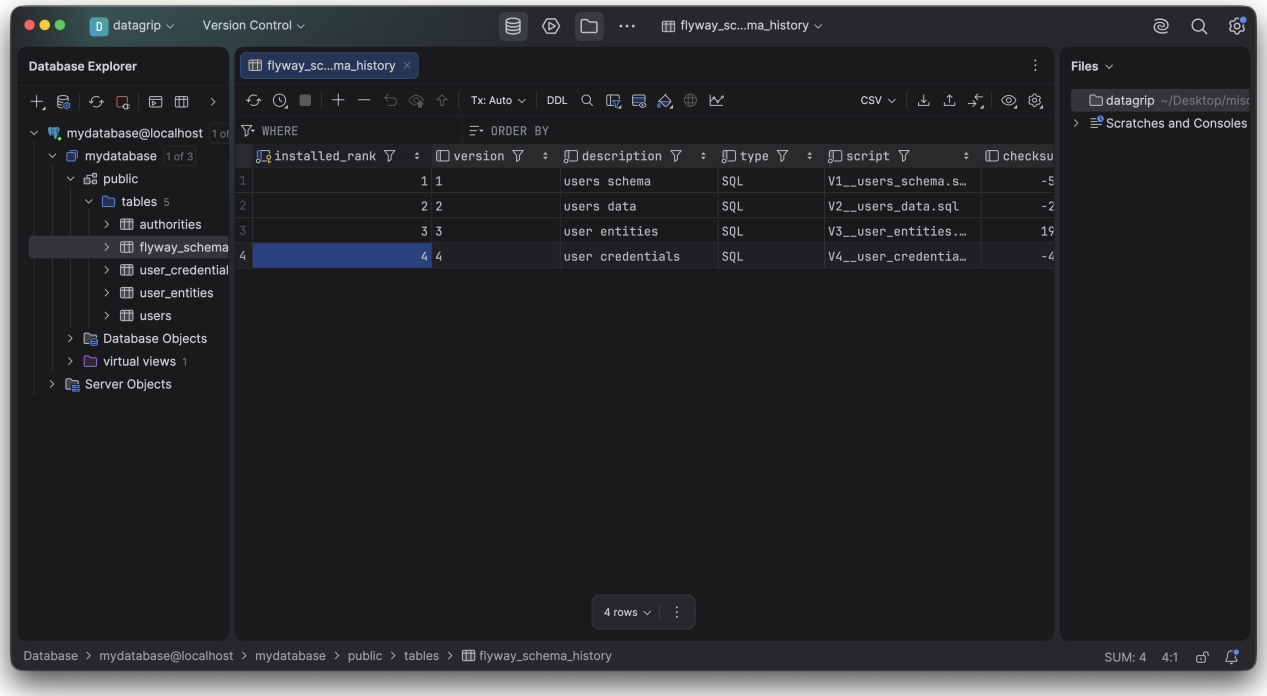
```
docker compose up -d
```

Databases

I'm going to pretty consistently use PostgreSQL in this book. Is it the best choice? No. I mean.. define "best." But it works, and it's widely adopted. I'm a big fan. That said, it's got some peculiarities that make otherwise standard SQL files fail to run on it out of the box, which is a bit of a shame. I'll note those as I go.

I usually use a tool like JetBrains' DataGrip to read data from my various databases, including PostgreSQL.

Here's what I'm (usually) looking at when I talk about consulting the data in a given database table.



1. DataGrip, in all of its glory. It's a very nice database client. That said, I *really* miss Microsoft Access' visual query builder from the distant, distant past, and wish they'd build something like that for this.

I appreciate however that most folks won't do that. They'll probably just the (excellent) PostgreSQL command line interface. If you're on Homebrew, you can simply run the following to install just the command line client.

```
brew install libpq
```

With that, it's trivial to connect to the database. I'm using the default database name, username, and password that you get in the generated Docker Compose files (`compose.yml`) from projects created on the Spring Initializr [https://start.spring.io]. So, here's how you'd connect to such a database using the PostgreSQL CLI:

```
PGPASSWORD=secret psql -U myuser -h localhost mydatabase
```

A Brief Note on Storing Credentials

Every time you see a password, or any kind of credential like a key, in the source code of these programs, remember, this is a *demonstration*. There are a million ways to externalize credentials from the source code, and you should use one of them!

It's essential to keep credentials both secure and accessible for the applications that need them, without risking exposure. Here are some methods and tools to store and make private keys

accessible to a Spring Boot application:

- **Environment Variables**: One of the most common ways is to use environment variables to store sensitive data. These variables can be set on the system where your application runs. This method separates configuration from the application, but it's not the most secure method on its own, especially if multiple applications share the environment.
- **Java's KeyStore (JKS/JCEKS)**: Java provides its built-in mechanism for securely storing cryptographic keys and certificates - Java KeyStore.
- **AWS Secrets Manager or AWS Parameter Store**: If you are deploying on AWS, you can use these services to store and retrieve your application secrets.
- **HashiCorp Vault**: An open-source tool for secret management. It allows you to centrally store, access, and deploy secrets across applications and infrastructure.
- **Azure Key Vault**: If you are on Azure, you can use Azure Key Vault to store, manage, and access secrets.
- **Google Cloud Secret Manager**: For applications deployed on GCP.
- **Encrypted Configuration Files**: Use tools like Jasypt with Spring Boot. Jasypt provides Spring Boot integration and allows you to encrypt property values in your configuration files.
- **Kubernetes Secrets**: If you are deploying your application in Kubernetes, it offers its own secrets management mechanism. Though it is better than plain config maps (`ConfigMap`), Kubernetes Secrets are Base64 encoded by default and not encrypted. For enhanced security, consider integrating with external secrets managers like HashiCorp Vault.
- **Dedicated Hardware Security Modules (HSMs)**: These are physical devices that safeguard and manage digital keys, perform encryption and decryption. Cloud providers like AWS offer their own cloud HSM services.
- **Use Configuration Servers**: For example, Spring Cloud Config Server. Though not a secrets manager *per se*, when combined with encryption, it can serve configurations securely to your applications.

Always encrypt sensitive data in transit and at rest. Rotate secrets regularly. Use IAM roles, policies, and least privilege principles to restrict who can access secrets. Monitor and audit access to secrets. Avoid hardcoding secrets or placing them in a version control system (VCS), even if encrypted. Backup your secrets, but ensure backups are also secured. No matter which approach you choose, it's essential to keep the principle of least privilege in mind. Only the necessary entities should have access to your secrets, and they should only be decrypted at the last possible moment (e.g., by the application when needed).

Database Migrations

I am trying to keep things simple, as I introduce them. Sometimes, I will use Spring's ability to run `data.sql` and a `schema.sql` script at program launch to initialize the database. In Spring Boot, you configure this with the following property.

```
spring.sql.init.mode=always
```

There are other options, `never` and `embedded`. Obviously, if I weren't going to use it, I'd simply not put it on the classpath. So I *never* use `never`. And `embedded` is the default *if* you have an embedded database like H2 or HSQL or the CLASSPATH. Wherever possible, I will use Docker (and Docker Compose, specifically) to standup instances of PostgreSQL. From Spring's perspective, this is a *remote* database (even if Spring Boot can be made to run `docker compose up -d` or to launch the Docker container using Testcontainers), so we must specify `always`.

Another option I may employ as I iteratively evolve the schema, introducing new concepts as we work, is Flyway, which can be used for versioned database migrations. It requires a SQL database table, which it will install for itself, and it'll keep track of which schema files have been applied. It looks by default in `src/main/resources/db/migrations` and runs all the SQL files it finds there in the order in which they're number, `V1_foo.sql`, V2*bar.sql*`, `V3`*baz.sql*, etc. :code: .. :java-lts-version: 25

Modern Java, Modern Language

Are you using the latest version of Java yet?

No? That's a real pity, because it's basically a different language from the one you might've known. As of this writing, the latest Long-Term Support (LTS) release is Java 25. There are new releases of Java coming out every six months. By the time you read this, we could easily be on many versions beyond that. This book was written with Java 25 in mind, but you should be using whatever the latest-and-greatest version of Java available to you happens to be.

That said, I can appreciate that some of you may not have seen all that is new and nifty in the latest and greatest installments of Java, so we'll review some of my favorite language features here. If you're all caught up, then please move on to the next chapters - there's entirely too much to look at, anyway.

Whatever the cause of your hesitation, this chapter contains *spoilers*! If you don't mind spoilers, let's dive right in!

First things first: you need an OpenJDK distribution that works for you. The list of valid and viable distributions scrolls down to the floor and out the door, so I'll refer you to Foojay.io's handy version almanac for Java 17 [https://foojay.io/almanac/java-17/]. One of my favorite tools for managing my Java installations on UNIX-like operating systems is `sdkman` [https://sdkman.io/]. It works on macOS, Windows Subsystem for Linux (WSL), Linux, and, I'm sure, plenty of other places besides. I use the GraalVM [https://www.graalvm.org/] distribution. GraalVM is an OpenJDK distribution with some extra features, including ahead-of-time native image compilation. To get that distribution, you might say:

```
sdk install java 25-graalce ①
sdk default java 25-graalce ②
```

① the first command installs the latest version of the GraalVM distribution of Java 25

② the second makes it the default installed distribution on my box so that all my interactions with Java are going to that distribution

With that done, let's look at some neat features in Java's latest and greatest versions up until Java 25.

The Amazing `AutoCloseable`

Java manages most memory for you, but it can't be responsible for the state outside the humble confines of the JVM. So, for example, you wouldn't like it if Java *garbage-collected* your connection to the database, and you wouldn't like it if Java garbage-collected your open socket connections without warning. Therefore, the interfaces you use to interact with these external resources typically have a `close()` method that you, the client, need to call when your work with the external resource is finished.

You mustn't neglect to call that method! Don't be greedy! You're not greedy, are ya? You want to leave the JVM as clean as you found it. So, you write the boilerplate. We all know the boilerplate:

open the resource; work with it; surround that work with a `try/catch/finally` block; catch any `Throwable` instances if (*when*?) something goes wrong; `close()` the resource if something goes wrong. Add the `close()` call in a `finally` block for extra robustness. All the while, handle the exceptions that might arise when you try to do anything, including calling `close()` on the resource. You'll end up with one or more `try/catch` blocks embedded within the outer `try/catch` blocks.

Let's look at some examples. I'll read a file on the filesystem to demonstrate this new feature. But something needs to ensure that there's a file to read in the first place, so I've extracted all that out into a separate class, `Utils`:

```
package com.example.java.closeable;

import java.io.File;
import java.nio.file.Files;
import java.time.Instant;

public class Utils {

①
    static String CONTENTS = String.format("""
            <html>
            <body><h1> Hello, world, @ %s !</h1> </body>
            </html>
            """, Instant.now().toString()).trim();

②
    static File setup() {
        try {
            var path = Files.createTempFile("bootiful", ".txt");
            var file = path.toFile();
            file.deleteOnExit();
            Files.writeString(path, CONTENTS);
            return file;
        } //
        catch (Throwable t) {
            error(t);
            throw new RuntimeException("could not produce a new file");
        }

    }

③
    static void error(Throwable throwable) {②
        System.err.println("there's been an exception processing the read! " +
throwable.getMessage());
    }

}
```

① the contents of the file, using a handy multiline Java `String`

② this method returns the `java.io.File` for the newly created temporary file

③ a convenience method to log errors. Do not rewrite this code! Otherwise, if you're doing things the old-fashioned way, you'll find yourself rewriting this a *lot*.

Now, let's look at an example of what *not* to do.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.*;

import static com.example.java.closeable.Utils.error;

class TraditionalResourceHandlingTest {

    private final File file = Utils.setup();①

    private static void close(Reader reader) { ③
        if (reader != null) {
            try {
                reader.close();
            } //
            catch (IOException e) {
                error(e);
            }
        }
    }

    @Test
    void read() {
        var bufferedReader = (BufferedReader) null; ②
        try {
            bufferedReader = new BufferedReader(new FileReader(this.file));
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) { ③
            error(e);
        } //
        finally { ④
```

```
            close(bufferedReader);
        }
    }

}
```

① from which file are we reading?

② We need a reference to the `BufferedReader` outside of the scope of the `try/catch` block, but we don't want to initialize that reference until we're inside the `try/catch` block because it might incur an exception.

③ handle any errors. Bear in mind that this solution doesn't even attempt to field the errors and somehow recover. If there is *any* error anywhere, then we abort.

④ also, make sure to close that `Reader`! Err, close that reader *if* it's not `null`! Sorry, close that reader *if* it's not `null`, and don't forget to handle any errors that arise while doing so, either! Be kind, rewind!

Yuck. We can do better with Java 7's `try-with-resources` construct. Let's rework the example accordingly.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

class TryWithResourcesTest {

    private final File file = Utils.setup();

    @Test
    void tryWithResources() {
        try (var fileReader = new FileReader(this.file); //
                var bufferedReader = new BufferedReader(fileReader)) {
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) {
            Utils.error(e);
```

```
        }
    }

}
```

Type Inference

Java 10 introduced type inference for variables given enough context. As a result, Java is a strongly typed language with less typing - pressing of keys - required.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Map;

class TypeInferenceTest {

    @Test
    void infer() throws Exception {
        var map1 = Map.of("key", "value"); ①
        Map<String, String> map2 = Map.of("key", "value");
        Assertions.assertEquals(map2, map1); ①
        var anonymousSubclass = new Object() {
            final String name = "Peanut the Poodle";

            int age = 7;

        };
②
        Assertions.assertEquals(7, anonymousSubclass.age);
        Assertions.assertEquals("Peanut the Poodle", anonymousSubclass.name);
    }

}
```

① both variable definitions contain a type of `Customer`. The compiler knows it, and you know it because the right side of the expression makes it clear. So, all things being equal, why not use the more concise version?

② Indeed, in this case, the compiler knows *more* about the type than it would before, particularly in the case of anonymous subclasses. Suppose you need a throwaway object in which to stash some data temporarily. If you created an anonymous subclass of `Object` and assigned it to a variable of type `Object`, there'd be no way to dereference fields defined on the anonymous subclass. Traditionally, there was no way to reify *anonymous* subclass types because they were *anonymous*. But with `var`, you don't need to account for the type; you can let the compiler infer it.

This last bit - reifying anonymous subclasses - comes in particularly handy when you're working with Java 8 streams abstractions and want to avoid having a host of throwaway classes created as side effects to conduct your data across the transformations.

The new `var` keyword can come in handy in a whole host of other smaller scenarios. You can use the `var` keyword for parameters in a lambda definition. It doesn't buy you much over just omitting `var` (or the type itself), except that you can now decorate the parameter with annotations. Neat!

Lambdas are the one fly in the proverbial ointment, however. Java doesn't have structural lambdas like those in Scala, Kotlin, and other languages. Instead, you must assign the lambda to an instance of a functional interface like `java.util.function.Consumer<T>`. If the literal lambda syntax doesn't clarify what type that is, then the variable type definition itself must. So you can use `var` for every variable definition *except* lambdas. It's so dissatisfying! The agony of having a column of nice, clean `var`s punctuated occasionally by standard variable definitions just because those variables happen to be lambdas! There is a way around it with casts, but I admit it isn't much better. Let's look at some examples.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.List;

class LambdasAndTypeInferenceTest {

    private String delegate(String s, Integer two) {
        return "Hello " + s + ":" + two;
    }

    @Test
    void lambdas() {
        MyHandler defaultHandler = this::delegate; ①
        var withVar = new MyHandler() { ②

            @Override
            public String handle(String one, int two) {
                return delegate(one, two);
            }
        };
        var withCast = (MyHandler) this::delegate; ③
        var string = "hello";
        var integer = 2;
        var set = new HashSet<>( //
                List.of(withCast.handle(string, integer), //
                        withVar.handle(string, integer), //
                        defaultHandler.handle(string, integer)));
        Assertions.assertEquals(1, set.size(), "the 3 entries should all be the same,
and thus deduplicated out");
```

```
    }

    @FunctionalInterface
    interface MyHandler {

        String handle(String one, int two);

    }

}
```

① I can't use `var` here because the compiler doesn't know to which functional interface type the method reference, `delegate(String, Integer)`, should be assigned

② I can use `var` here, but I've lost all the brevity of lambdas!

③ the only way I know around it is to cast the type like this. Ugh.

I tend to use the cast form a lot. Your sensibilities may vary, and given how new this feature is, I wouldn't be surprised if my sensibilities change in the future with respect to this feature, as well.

Enhanced Switch Expressions

Java has a new *enhanced* switch, which forms the basis of the gradual addition of *pattern matching* to the Java language. It simplifies things considerably:

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class TraditionalSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals(respondToEmotionalState(Emotion.HAPPY), "that's
wonderful.");
        Assertions.assertEquals(respondToEmotionalState(Emotion.SAD), "I'm so sorry to
hear that.");
    }

    public String respondToEmotionalState(Emotion emotion) {
        var response = ""; ②
        switch (emotion) {
            case HAPPY:
                response = "that's wonderful.";
                break; ③
            case SAD:
                response = "I'm so sorry to hear that.";
                break;
        }
```

```
        return response;
    }

    enum Emotion {

①
        HAPPY, SAD

    }

}
```

① `Emotion` is an `enum`. There are only two possible values (for this simple example that doesn't at all reflect the rich gradient of human emotions) for a variable of type `Emotion`: `HAPPY` and `SAD`. We have branches for all known states in this `switch` statement. The compiler is satisfied that we have covered every possible value, so it doesn't insist on a `default` branch to handle any unforeseen values. There are no unforeseen values. We say that we've *exhausted* the range of values.

② we define a variable to which we assign the results of our processing.

③ take care to `break` for each branch. Otherwise, the execution flow will drop down to other branches, producing undesirable side effects.

The first example is a classic `switch` statement. There's nothing wrong, *per se*, but it's tedious, and I tend to avoid writing them.

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class EnhancedSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals("that's wonderful.", respondToEmotionalState(Emotion
.HAPPY));
        Assertions.assertEquals("I'm so sorry to hear that.", respondToEmotionalState
(Emotion.SAD));
    }

    private String respondToEmotionalState(Emotion emotion) { ①
        return switch (emotion) {
            case HAPPY -> "that's wonderful.";
            case SAD -> "I'm so sorry to hear that.";
        };
    }

    enum Emotion {
```

```
        HAPPY, SAD;

    }

}
```

① there's no intermediate variable! Each branch produces a value, and that value is the result of the `switch` expression and can be assigned to a variable or, as I do here, returned in one fell swoop from the method as any other expression would.

Multiline Strings

This one is a super-small feature that punches well above its weight: Java supports multiline `String`s. Hooray! There are a million opportunities for multiline `String`s: SQL queries, HTML, Markdown, Asciidoctor, Velocity templating (such as for emails), unit testing, etc.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.time.Instant;

class MultilineStringsTest {

    private final String instant = Instant.now().toString();

①
    private final String multilines = String.format("""
            <html>
            <body>
            <h1> Hello, world, @ %s!</h1> </body>
            </html>
            """, instant).trim();

②
    private final String concatenated = "<html>\n<body>\n" + "<h1> Hello, world, @ " +
instant + "!</h1> </body>\n"
            + "</html>";

    @Test
    void stringTheory() {
        Assertions.assertEquals(this.multilines, this.concatenated);
    }

}
```

① The first example uses a multiline `String` to represent some HTML markup

② The second variable recreates the same HTML markup, down to the padding and the newlines, but uses string concatenation and manually encodes newlines, as we used to have to do.

In the example, the two variables `multilines` and `concatenated` are identical, but the multiline `String` is much easier to wrangle.

Short and simple.

Records

Records are perhaps my second-favorite new feature in Java. If you've ever used case classes in Scala, or data classes in Kotlin, you'll feel *almost* right at home.

Java introduced records, which are a new kind of type. Records, like enums, are a restricted form of a class. As a result, they're ideal for "plain data carriers": classes containing data not meant to be altered, with only the most fundamental methods, such as constructors and accessors. We'll use (and sometimes abuse) them *all* the time in this book. They're wonderful time savers. Need to model a read-only entity in your database that has accessors for all the constituent fields, a constructor, a functional `toString` implementation, a valid `equals` implementation, and a valid `hashCode` implementation? You'd better get codin'! That'll take a while. Or, you could use something like Lombok to code-generate all of that for you based on the presence of a handful of annotations. Or, you could use records.

Here's a trivial example. Suppose you want to return information about `Customer` entities *and* their associated `Order` data. Unfortunately, Java doesn't support multiple return types or provide suitable tuple types, so you need to create something to hold both types. Records to the rescue!

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.List;

class SimpleRecordsTest {

    @Test
    void records() {
        var customer = new Customer(253, "Tammie");
        var order1 = new Order(2232, 74.023);
        var order2 = new Order(9593, 23.44);
        var cos = new CustomerOrders(customer, List.of(order1, order2));
        Assertions.assertEquals(2232, order1.id());
        Assertions.assertEquals(74.023, order1.total());
        Assertions.assertEquals("Tammie", customer.name());
        Assertions.assertEquals(2, cos.orders().size());
        IO.println("order components  " + order1.id() + ':' + order1.total()); ②
    }
```

```
①
    record Customer(Integer id, String name) {
    }

    record Order(Integer id, double total) {
    }

    record CustomerOrders(Customer customer, List<Order> orders) {
    }

}
```

① these three `record` definitions define three new types, each with accessors, constructors, and internal storage. Scarcely more than three lines of code for three brand new types!

② records also automatically expose accessor methods for the fields defined in the constructor. For example, if you want to read the `name` field of the `Customer` type, use `Customer#name()`. If you wish to read the `orders` field of the `CustomerOrders` type, use `orders()`. It took a while to accept that they're in the form `x()`, and not in the form `getX()`, but I've come to love it. Mercifully, all the interesting libraries that need to know about this convention - JSON serializers, for example - already work well with it.

Records make perfect sense for immutable, data-centric types. They alleviate a whole host of boilerplate code. In the first edition of this book, I used the Lombok project [https://projectlombok.org] (which is brilliant) to synthesize the getters, setters, no-args, and all-arg constructors with just a few annotations. It worked, but it was still a handful of lines instead of the one-liners enabled by Java records. I love records!

I still occasionally use Lombok for other things, but it's nice to reduce my reliance on it further.

More controversially, I also sometimes use records to implement services and components quickly. After all, you can have methods on a record. Of course, record implementations can't extend classes, but one doesn't need to do that a lot. There is the undesirable side effect of having accessor methods that expose the state - `dataSource()` or whatever - but, for whatever reason, I don't care. It doesn't cost me anything when I use it. If my code grows large enough to need hierarchies or interface implementations, I'll change it. If the code grows enough that I need to worry about leaking state, I'll change it. But the immediate, short-term effect of having more concise, approachable, readable code seems to make sense to me. Maybe one day that'll change?

Behind the scenes, a record creates a default constructor whose parameters match the types and arity of the record header. Records can have other constructors, but they need to delegate to the default constructor.

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;
import org.springframework.util.StringUtils;
```

```
class RecordConstructorsTest {

    @Test
    void multipleConstructors() {
        var customer1 = new Customer("test@email.com");
        var customer2 = new Customer(2, "test2@gmail.com");
        Assertions.assertEquals(customer1.id(), -1);
        Assertions.assertEquals(customer2.id(), 2);
    }

    record Customer(Integer id, String email) { ①

        Customer { ②
            Assert.notNull(id, () -> "the id must never be null!");
            Assert.isTrue(StringUtils.hasText(email), () -> "the email is invalid");
        }

        Customer(String email) {
            this(-1, email);
        }
    }

}
```

① the default constructor is the one defined in the record header

② If you want to act on the fields passed in the default constructor, create a constructor with no parameters and do the work there. This no-parameter constructor and the record header taken together are the default constructor. If you want to have an alternative constructor, you can do that so long as you forward to the default constructor. I use the default constructor to initialize the `id` to -1. This way, it's impossible to initialize the record with a `null` `id` field.

Sealed Types

Sealed types are a novel feature that hasn't reached its full potential yet. The basic idea is to constrain the number of subtypes for a given type. Why would you do this? Well, it's all a bit *exhausting*! Or should I say, it's all about exhausting the extent to which the runtime needs to support virtual (polymorphic) dispatch?

Sound complicated? It's not really. Ever have a method in a parent type that the child type overrides? The ability to call that method on an instance of that child type, in terms of the parent type's interface, is called polymorphism.

Polymorphic, or *virtual*, dispatch requires a lookup in the virtual function table, which in theory takes time. The runtime wouldn't have to do that if it could say conclusively that a given type can never be subclassed, such as with a final type. The `final` keyword is a bit of a sledgehammer, however. It forecloses entirely on the possibility of any kind of subclassing whatsoever. You might have a hierarchy but wish to keep it shallow and well known. Alternatively, you could make everything package-private (simply omit the `public` modifier on the class), which means that only types in the same package could subclass the type. This would probably work, but of course,

somebody else could come along and create types in the same package in their `.jar`. There haven't traditionally been many good ways to keep a hierarchy shallow - until now. Sealed types can help. They let you constrain the number of subclasses to a known set.

Let's look at an example.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;

class SealedTypesTest {

④
    private String describeShape(Shape shape) {
        Assert.notNull(shape, () -> "the shape should never be null!");
        if (shape instanceof Oval)
            return "round";
        if (shape instanceof Polygon)
            return "straight";
        throw new RuntimeException("we should never get to this point!");
    }

    @Test
    void disjointedUnionTypes() {
        Assertions.assertEquals(describeShape(new Oval()), "round");
        Assertions.assertEquals(describeShape(new Polygon()), "straight");
    }

①
    sealed interface Shape permits Oval, Polygon {

    }

②
    static sealed class Oval implements Shape permits Circle {

    }

③
    // static final class Rhombus implements Shape {}

    static final class Circle extends Oval {

    }

    static final class Polygon implements Shape {

    }
```

```
}
```

① we'll start with a `Shape` and permit two direct subclasses, `Oval` and `Polygon`.

② a subclass of a sealed type must be `final` or `sealed` and explicitly name its subclasses. A sealed type's subclasses may themselves be sealed types, permitting further subtypes.

③ the following declaration does not compile, as it is not one of the explicitly permitted subclasses

④ the `describeShape` method is written so that we can exhaustively handle every subclass.

Sealed types help the compiler, too. The compiler can exhaustively determine every case of every possible subtype, which has implications for the future, as Java looks to better incorporate simple pattern matching into the language. Imagine the possibilities here, and you can kind of see how sealed types might play with the new `switch` expressions, too.

Right now, I don't recommend using `sealed` types. I tend to think types should be open by default. You just don't know what scenario will arise in the future that changes your fundamental assumptions. Furthermore, sealed subtypes are `final`, inhibiting tools like Spring and Hibernate, which must subclass your types to proxy them.

Smart Casts

This feature is one of those things that I don't use all that often in application code, but it's pretty useful when writing infrastructure code that deals with polymorphism. Essentially, the feature spares you from having to create an intermediate variable and casting after you've already done an `instanceof` check.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;

public class SmartCastsTest {

    @Test
    void casts() {
        interface Animal {

            String speak();

        }

        class Cat implements Animal {

            @Override
            public String speak() {
                return "meow!";
```

```
            }

        }

        class Dog implements Animal {

            @Override
            public String speak() {
                return "woof!";
            }

        }

        var newPet = Math.random() < .5 ? new Cat() : new Dog();
        var messages = new HashSet<String>();

①
        if (newPet instanceof Cat) {
            var cat = (Cat) newPet;
            messages.add("the cat says " + cat.speak());
        }
②
        if (newPet instanceof Cat cat) {
            messages.add("the cat says " + cat.speak());
        }

        if (newPet instanceof Dog dog) {
            messages.add("the dog says " + dog.speak());
        }

        Assertions.assertEquals(messages.size(), 1);
        Assertions.assertTrue(messages.contains("the dog says woof!") || messages
.contains("the cat says meow!"));

    }

}
```

① this is a classic example, where we determine the subtype and then create a variable cast to the appropriate type. If ever we change the type definition, we have to replace it both in the cast and in the `instanceof` check.

② the newer, more elegant alternative uses a smart-cast, sparing us the extra variable and cast.

This feature looks almost exactly like a similar feature in Kotlin, and I love it.

Functional Interfaces, Lambdas, and Method References

Java 8 introduced lambdas. They let us treat functions as first-class citizens that can be assigned

and passed around. Well, almost. In Java, lambdas are slightly limited; they're *not* first-class citizens. But, close enough! They *must* have return values and input parameters compatible with the method signature of the sole abstract method of an interface. This interface, one with a single abstract method intended for use as a lambda, is called a *functional interface*. (Interfaces may also have *default*, non-abstract methods where the interface provides an implementation.)

There are some very convenient and oft-used functional interfaces in the JDK itself. Here are some of my favorites.

- `java.util.function.Function<I, O>`: an instance of this interface accepts a type `I` and returns a type `O`. This is useful as a general-purpose function. Every other function could, in theory, be generalized from this. Thankfully, there's no need for that as several other handy functional interfaces exist.
- `java.util.function.Predicate<T>`: an instance of this interface accepts a type `T` and returns a `boolean`. This is an ideal filter when you're trying to sift through a stream of values.
- `java.util.function.Supplier<T>`: A supplier accepts no input and returns an output.
- `java.util.function.Consumer<T>`: A consumer is the mirror image of a `Supplier<T>`; it takes an input and returns no output.

You can also create and use your own custom functional interfaces. Let's look at some examples.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.function.Function;

class LambdasTest {

    @Test
    void lambdas() {
        Function<String, Integer> stringIntegerFunction = str -> 2;①
        interface MyHandler {

            String handle(String one, Integer two);

        }
        MyHandler withExplicit = (one, two) -> one + ':' + two;②
        Assertions.assertEquals(stringIntegerFunction.apply(""), 2);
        Assertions.assertEquals(withExplicit.handle("one", 2), "one:2");
        var withVar = (MyHandler) (one, two) -> one + ':' + two;③
        Assertions.assertEquals(withVar.handle("one", 2), "one:2");
        MyHandler delegate = this::doHandle; ④
        Assertions.assertEquals(delegate.handle("one", 2), "one:2");

    }

    private String doHandle(String one, Integer two) {
```

```
        return one + ':' + two;
    }

}
```

① You can use existing functional interfaces in the JDK…

② you can define your own interfaces and use them as functional interfaces

③ You may use `var` and lambdas together with this (admittedly unsightly) cast.

④ and if you have existing methods whose return types and input parameters line up with the single abstract method of a functional interface, then you can create a method reference and assign it to an instance of that type.

Pattern Matching

We've seen the smart-casting `if` statement, and we've seen the switch expression. At the heart of both of these new things is *pattern matching*, wherein we match a value and then extract the value for use. We've only really seen that it's possible to easily match values thus far, so let's revisit that a bit.

```
package com.example.java.patternmatching;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertTrue;

class PatternMatchingTest {

①
    final String authenticateMessage = """
                please authenticate, so that we can
                get to know you a little better.
            """.stripIndent().stripLeading().stripTrailing();

    final String customerMessage = "the customer's name is %s";

②
    String showMessageWithIf(BrowserClient browserClient) {
        var message = "";
        if (browserClient instanceof Customer(var id, var name)) {
            message = this.customerMessage.formatted(name);
        }
        else if (browserClient instanceof UnknownUser) {
            message = this.authenticateMessage;
        }
        return message;
    }

③
```

```
    String showMessageWithSwitch(BrowserClient browserClient) {
        return switch (browserClient) {
            case Customer(var id, var name) -> this.customerMessage.formatted(name);
            case UnknownUser() -> this.authenticateMessage;
        };
    }

    @Test
    void patternMatching() throws Exception {
        assertTrue(showMessageWithIf(new Customer(1, "Josh")).contains(this
.customerMessage.formatted("Josh")));
        assertTrue(showMessageWithSwitch(new UnknownUser()).contains(this
.authenticateMessage));
    }

④
    sealed interface BrowserClient permits UnknownUser, Customer {

    }

    record UnknownUser() implements BrowserClient {
    }

    record Customer(Integer id, String name) implements BrowserClient {
    }

}
```

① for our examples, we're going to work with a shallow hierarchy rooted in a type called `BrowserClient`. Imagine that when an HTTP request comes in, we adapt the request into one of either a known `Customer` or an `UnknownUser`.

② we'll do some analysis of the incoming user type and display a message as appropriate. Since we're going to prepare these same messages two different ways, I've extracted them out into separate variables.

③ let's use the smart `if` statement to deduce which message to present. Note that we're now switching on an instance of `BrowserClient`. In the first line, we do two things at once: we check that the variable is an instance of `Customer`, and if so, we hoist out - we *extract* - not just the `Customer` variable, but the constituent component fields of the record itself - `id` and `name` - into their own new variables. Super concise. In the second test, we don't need a pointer to the `UnknownUser` or its field, so we do nothing with it save match it.

④ the `if/else` structure worked fine, but it meant that we had to stash a variable and then make sure that only one branch of the test was ever evaluated. The structure as written also gives us nothing in the way of comfort. What happens when we add a new type to the hierarchy? We need to handle that branch, but the compiler will let us ignore it. Let's use our new friend the `switch` expression. The switch expression has access to the same pattern-matching superpowers as the `if`, but it does us two new favors. The compiler will complain if we add a new type to the hierarchy, because it knows that the types are `sealed` and therefore there is an exhaustive set of types, and our `switch` hasn't defined a `default` branch. Thanks, compiler! And, to make things

just that much cleaner, the `switch` is an expression, so we can assign the result of the `switch` to a variable or just return the result of the expression directly.

I bet you never thought you'd see the day that the `switch` was more elegant than an `if`, did you?

Java language architect Brian Goetz talks about the combination of some of the features that we have just covered - pattern matching, sealed types, the smart switch expression - as *data-oriented programming*. Java has reigned supreme in large monolithic codebases, he explains, because it provides good object-oriented support and strong enforcement of privacy, security, and encapsulation. In such a codebase, the boundary on which different parts of the program align is typically an object interface, which the magic of polymorphism incentivizes. But programs aren't usually large, sprawling, deeply rooted graphs of objects these days. More often than not, they're smaller programs dealing with ad hoc messages coming in over the wire and being handled in terms of dumb, but typed, carriers. There's no hierarchy at all! In this context, Java 8 was starting to feel mighty clunky. But no more. With Java 25, it's trivial to define small ad hoc objects and to vary behavior based on them. In a sense, Java 25 introduces a new kind of dispatch: not the dynamic dispatch of yore, but a dispatch for small ad hoc objects.

And just like that, without our ever realizing it, Java 25 manages to introduce support for a completely new style of programming: data-oriented programming. And I bet you were only just getting used to the possibilities of the rudimentary functional programming support in Java 8!

The Notation This Book Is Written In

That's the tour of the language.

Take stock of what we've gathered: `try`-with-resources to tame anything with a lifecycle, `var` to let the compiler do the typing, text blocks for the SQL and HTML and JSON we'll write by the yard, records for the small, immutable data carriers that a security domain is full of, sealed hierarchies and pattern matching and the enhanced `switch` for turning a tangle of `if`/`instanceof` checks into something you'd actually want to read, and lambdas and functional interfaces for the glue in between.

These aren't academic curiosities. They're the notation the rest of this book is written in. Nearly every controller, filter, configuration class, and `RegisteredClient` you'll meet leans on one or more of them, and once your eye is trained you'll stop seeing the syntax and start seeing the intent - which is exactly the point.

But a language is only half the story. Knowing how to *write* modern Java tells you nothing about how it *runs*: how quickly it starts, how much memory it holds, how it behaves boxed inside a container, and - crucially for a system whose entire job is to field authentication requests - how gracefully it scales under load. A Spring Authorization Server that vends tokens for your whole organization lives and dies by exactly those runtime characteristics. So let's go beneath the syntax, to the JVM itself. :code: .. :java-lts-version: 25

Modern Java, Modern Runtime

In the last chapter we looked at modern Java as a *language* - the syntax you'll be reading on every page from here on. This chapter is about modern Java as a *runtime*: the machine beneath the code.

It's tempting to treat the JVM as a black box, and most of the time you can. But a Spring Security deployment - and a Spring Authorization Server most of all - is a long-lived service sitting on the hot path of nearly every request in your system. How quickly it starts, how much memory it holds, how it behaves when it's boxed into a container, and how many concurrent requests it can field before it buckles are not idle curiosities; they're the difference between an auth layer that disappears quietly into the background and one that becomes the bottleneck everyone complains about. The good news is that, much like the garbage collector, most of these gains arrive for free the moment you move to a newer JVM. Let's see what you get.

Operations Improvements

The newer versions of Java now support something called CDS Archives. CDS Archives essentially capture some of the invariant (but freshly computed) state from a given application and cache it for easy reuse on subsequent runs. I consistently shave 0.1 or 0.2 seconds from startup time when using CDS Archives.

The newer versions of Java are also container-aware. So, let's suppose you are running Docker images on your host machine, which has 32GB of RAM. You might configure the JRE with 2GB of RAM and errantly configure the Docker container with only 1GB of RAM. Java would see the 32GB and think it could allocate 2GB, and then fail to start up. In newer versions, Java is aware of the container's limited RAM and won't exceed it.

Java Flight Recorder (JEP 328) monitors and profiles running Java applications. Even better, it does so with a low runtime footprint. Java Mission Control allows ingesting and visualizing Java Flight Recorder data. Java Mission Control takes Java's already stellar support for observability to the next level.

Performance

Java 25 is *fast* and reliable.

Java's garbage collector is the stuff of legend. It's fast, lightweight, and minimally invasive. It's also one of those things where, when it's improved, your application's runtime improves with it, for free. No recompilation is required.

G1 has been the default garbage collector since Java 9, replacing the Parallel garbage collector that had been the default in Java 8. G1 reduces pause times compared to Parallel, though its overall throughput may be lower. Next, Java 11 introduced the ZGC garbage collector to reduce pauses further. Finally, Java 14 introduced the Shenandoah GC, which keeps pause times low and does so in a manner independent of the heap's size.

And it's fast. I can't give you specific numbers or anything because it is so highly workload-sensitive, but large organizations like Netflix have talked about how more recent iterations in Java

have helped them improve their workload performance merely by upgrading. Upgrading the JVM is one of the easiest ways to gain performance and reduce memory utilization, and it's one of the easiest, comparably, too.

Java 25, Project Loom, and Virtual Threads

Project Loom brings transparent fibers to the JVM. As things stand today, in Java 20 or earlier, IO is blocking. Call `int InputStream#read()`, and you might have to wait for the next byte to finally arrive. In `java.io.File`-centric IO, there's very rarely much of a delay. On the network, however, you just can't really ever know. Clients might disconnect. Clients are people, driving through a tunnel and getting spotty signal coverage. During this time, the program flow is said to be *blocked* from proceeding on the thread of execution. In the following snippet, we have no way of knowing when we'll see the word "after" printed. Might be a nanosecond from now. Might be a week from now. It's *blocking*.

```
Executor executor = Executors.newCachedThreadPool();
executor.submit(() -> {
    InputStream in = ...
    IO.println("before");
    int next = in.read();
    IO.println("after");
});
```

This means that we can't address any other requests with this thread until it's finished doing whatever it was doing. We'll need more threads.

This is bad enough, but it's made worse by the architecture of threading in Java prior to Java 25, where each thread maps, more or less, to a native operating system thread. It's expensive to create more threads, too, taking about two megabytes of RAM. We're going to need a *lot* of threads to handle requests if our existing threads are blocked long enough. Many, many more threads. Or, we're going to need to find a way to avoid *blocking* on the precious few threads we do have.

We could use non-blocking IO, as enabled by Java NIO (java.nio.*). In this model, you ask for bytes and register a callback that the runtime executes only when there are actually bytes available. No waiting, no blocking. This approach has the significant benefit of keeping us off threads when there's nothing to be done, allowing others to use those threads in the meantime. It's a bit tedious, however, and low-level. Spring has amazing support for reactive programming, which offers a functional-style programming model on top of non-blocking IO. It works well. But, it requires changing the way you write code. If you're not used to it, it can be a bit daunting, too. Wouldn't it be nice if you could just take that existing code, as demonstrated above, and have it do the right thing, transparently moving the flow of execution off the thread when there's nothing happening, and then resuming the flow of execution when there is? This way, we could keep our simpler code and still get the benefits of non-blocking IO. Absolutely it would. And now you can. Project Loom is straightforward at the high levels: if you run your code in a *virtual thread* (just another type of `java.lang.Thread`), then the runtime will detect *blocking* operations like `InputStream#read()`, `Thread.sleep(long)`, etc., and automatically move the code off the thread it's on if it's in a state where nothing is happening. It'll move the code back onto the thread once there's something to be

done, once the blocking operation has stopped blocking. In the case of reading, that means it'll resume once there are bytes to read. In the case of `Thread.sleep`, it'll resume once the sleep time has elapsed. And you don't have to do anything to get it to work. Let's look at an example that I shamelessly stole from Oracle Java advocate José Paumard.

```
package com.example.java.loom;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.stream.IntStream;

class LoomTest {

    private static Set<String> observe(int index) {
        var before = Thread.currentThread().toString();
        try {
            Thread.sleep(100);
        } //
        catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        var after = Thread.currentThread().toString();
        return index == 0 ? new HashSet<>(Arrays.asList(before, after)) : Set.of();
    }

    @Test
    void threads() throws Exception {
        var switches = 5;
        var observed = new ConcurrentSkipListSet<String>();
        var threads = IntStream.range(0, 1000)①
            .mapToObj(index -> Thread.ofVirtual()②
                .unstarted(() -> {
                    for (var i = 0; i < switches; i++) ③
                        observed.addAll(observe(index));
                }))
            .toList();
        for (var t : threads)
            t.start();
        for (var t : threads)
            t.join();
        Assertions.assertTrue(observed.size() > 1); ④
    }

}
```

① This program launches 1,000 virtual threads. This is an arbitrary number, but it's also enough that it'll create contention - a need to share the precious few resources available to the machine.

② the program sleeps five times in each thread, for about 100 milliseconds each. This is not a long time, but it's enough that it forces the runtime to have to interleave, to *share*, the actual operating system threads.

③ In each thread, we call the `observe` function, which notes the current thread, sleeps, and then notes the current thread again. We don't want all the threads, however, as that would be too many. So we sample: if we're on the first thread launched (whose index would be `0`), note the information; otherwise, do nothing with it and move on.

④ by the end of the run, we've captured the threads on which the first of our thousand virtual threads executed, and stored them in a `Set<String>`, where the values will be deduplicated. Interestingly, there is *probably* more than one value in the set! Remember, we only noted the thread of execution for *one* thread - the *same* thread! - and somehow, without our asking or doing anything, it got moved around. Print the values out, and you'll see the code ran on the same *virtual* thread, but on multiple carrier threads.

The result is that while we were `Thread.sleep`-ing, the carrier thread on which we were executing was made available to whatever else required it in the JVM. This efficient time-sharing means that now we can handle a *lot* more requests with the same codebase. All we had to do was make sure to run the code on a *virtual* thread.

You don't have to use the `Thread.ofVirtual()` construction, either. There's an `Executor` implementation, too: `Executors.newVirtualThreadPerTaskExecutor()`. It's not a pool, however. Threads are no longer expensive, after all.

That's easy enough. In a typical Spring Boot application, however, there are countless places where you'll need to override the default configured `Executor`, but in Spring Boot 3.2 we can do that for you. Specify `spring.threads.virtual.enabled=true`, and you're all set.

Loom is especially important in the context of the Spring Authorization Server, because it uses traditional blocking IO. If you want to easily scale it up, consider using virtual threads and Java 25 when running the Spring Authorization Server. Who knows, you may not need to add new instances. You'll get to have your cake and eat it too: scalable, but simpler, code.

Improving Startup with Project Leyden

Project Leyden is a set of initiatives from the OpenJDK team to improve Java's performance by evaluating more and more about a program's structure earlier, at compilation or as early as possible. This effort is ongoing, but has already borne fruit. You can use Project Leyden with a Spring application to great effect. It provides a dramatic improvement on startup time for *all* Java application workloads.

Improving Startup and RAM footprint with GraalVM Native Images

We're using the GraalVM distribution of OpenJDK. It is not strictly speaking the same as OpenJDK. It

releases more slowly than OpenJDK, for example. GraalVM offers some alluring capabilities, including a polyglot language runtime called Truffle, and the capability to turn Java programs into lightweight, rapid, architecture- and operating system-specific native images. The startup time is akin to or better than Project Leyden, and the RAM reductions could be quite considerable. GraalVM does require some compromises. The resulting binaries are *not* strictly speaking Java programs anymore and must give up compatibility in certain cases.

Next Steps

I didn't intend these two chapters to be a thorough introduction to all things Java - just an amuse-bouche for the cool stuff you might see me do in my code, independent of Spring. Between the language and the runtime, Java 25 is a very compelling way to write software, and it's no coincidence that it's starting to look more and more like languages such as Kotlin, which is also a lovely way to write Spring Boot-based applications. Hopefully, these chapters have been persuasive, and you're ready to turn the page - because from here on, it's Spring all the way down. :code: ..

Dependency Injection and Portable Service Abstractions

I am assuming that by the time you're reading this, you have some understanding of Spring. I assumed the same thing about one of my earlier books, *Reactive Spring*, too, but I included a *very* simple discussion of some of the core principles of Spring in that book and - to my utter shock and chagrin - it was one of the most loved aspects of the book! I couldn't believe it!

So, here we go. We're going to do the same sort of thing, but of course greatly updated to reflect the amazing moment in which we find ourselves.

When Spring first came out, we talked about the "Spring triangle," the three pillars that made the Spring Framework so useful for people:

- portable service abstractions
- dependency injection
- aspect-oriented programming (AOP)

More recently, with the introduction of Spring Boot, we've added a new dimension of utility to the framework:

- autoconfiguration

So I guess it's more like a "Spring square" now, come to think of it.

At the core of Spring as a framework is the fact that it needs to understand how your objects are wired together. But why? Why bother telling it these things? Why not just wire things yourself?

At the micro level, this isn't an unreasonable question. Spring is just Java, and I can write Java without Spring, so why bother with Spring? Let's learn what Spring can help you with by way of an example that we'll start from scratch, iteratively letting Spring do more and more to reduce code and elevate results.

But first, we'll need a project and a database…

The Spring Initializr

We will need to incept a new project. It's just Java and some libraries, so in theory you could create your own with no issues. But I like to use the Spring Initializr (https://start.spring.io) [https://start.spring.io]. Visit the Spring Initializr and make the following dependency selections: `Spring Boot Actuator`, `Thymeleaf`, `Spring Data JDBC`, `PostgreSQL Driver`, `H2`, `Spring Web`, `GraalVM Native Support`, `OpenTelemetry`, `Spring Boot DevTools`, and `Testcontainers`. Select Java 25 (or later), and Apache Maven as the build tool. You could use Gradle if you prefer, though I'll be showing Apache Maven.

Give the application whatever group name and artifact name you like. Click `Generate`. You'll download an application archive (`.zip`). Unzip it and then point your IDE - IntelliJ IDEA, Visual

Studio Code, Eclipse, etc. - at the `pom.xml` in the root of the project.

You should have a project with the usual folder structure: `pom.xml` and `src/{test,main}/{java,resources}`. You'll also have a new class in the `src/main/java` folder with a `public static void main` method.

Rename the file `Main.java`, and gut the code inside. It should look something like this when you're done:

```
package com.example.beans_to_boot;

public class Main {

    public static void main(String[] args) {
①
    }
}
```

1. A *tabula rasa*!

It is on this humble foundation that we build.

The Database

Let's introduce a *trivial* example to reinforce the idea. We're going to read `Dog` data from a SQL database, PostgreSQL, running on our machine. We'll use the following Docker Compose file to stand up the instance of our database.

```
services:
①
  postgres:
    image: 'postgres:18.4'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'

②
  grafana-lgtm:
    image: 'grafana/otel-lgtm:latest'
    ports:
      - '3000:3000'
      - '4317:4317'
      - '4318:4318'
```

1. This will stand up a PostgreSQL database.

2. ...and a Grafana instance. We'll return to this later.

Run this in the usual way: `docker compose up -d`

Put the following SQL files in `src/main/resources/schema.sql` and `src/main/resources/data.sql` relative to your project root.

Here's `schema.sql`:

```
create table if not exists dog
(
    id          serial primary key,
    name        text not null,
    description text not null
);
```

and here's `data.sql`:

```
delete
from dog;
insert into dog (name, description)
values ('Prancer', 'A cute dog');
insert into dog (name, description)
values ('Duke', 'Oh the adventures you will have!');
insert into dog (name, description)
values ('White Dog', 'Oh the adventures you will have!');
```

Run `docker compose up -d`, then make sure to pass that SQL to PostgreSQL:

```
cat schema.sql | PGPASSWORD=secret psql -Umyuser -hlocalhost mydatabase
cat data.sql | PGPASSWORD=secret psql -Umyuser -hlocalhost mydatabase
```

A Repository of Knowledge... About Dogs

We're going to be working with data about dogs, so we'll need a layer to read that data and make it available to us. Here's a straw man first cut at this code, with several less-than-ideal design decisions. Let's look at the moving parts. We'll introduce all of them now and only the things that change as we progress.

First, the holder for data coming from our `dog` table.

```
package com.example.beans_to_boot.raw;

record Dog(int id, String name, String description) {
}
```

1. A Java record is a marvelous thing.

We'll be implementing a repository. Here's the interface for that repository.

```
package com.example.beans_to_boot.raw;

import java.util.Collection;

interface DogRepository {

    Collection<Dog> findAll();

}
```

It seems simple enough...

And here's the default implementation of that interface.

```
package com.example.beans_to_boot.raw;

import org.springframework.jdbc.datasource.DriverManagerDataSource;

import javax.sql.DataSource;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Collection;

class DefaultDogRepository implements DogRepository {

    private final DataSource db;

    DefaultDogRepository() {
①
        this.db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase
", "myuser", "secret");
    }

    @Override
    public Collection<Dog> findAll() {
②
        try (var connection = this.db.getConnection(); //
                var ps = connection.prepareStatement("select * from dog") //
        ) {
            var list = new ArrayList<Dog>();
            try (var rs = ps.executeQuery()) {
                while (rs.next())
                    list.add(new Dog(rs.getInt("id"), rs.getString("name"), rs
.getString("description")));
            }
            return list;
```

```
        } //
        catch (SQLException throwable) {
            throw new RuntimeException(throwable);
        }
    }

}
```

1. Everything about this arrangement makes me itch: `DataSource` objects are expensive and should be pooled, centralized, and reused for many objects. I don't love credentials hardcoded into the source code. And I don't love that we're instantiating the expensive resource in the constructor instead of centralizing its definition and having it passed in via injection.
2. Believe it or not, this demo used to be a *lot* worse when I first started teaching Spring to folks (even if just others on my team) 20+ years ago. The ability to use Java 7's `AutoCloseable` is a game-changer for this sort of code. Just hope you remember to use it! You definitely don't want these things leaking.

And of course here's the entry point into this example. All `Main` classes will have the same basic formula: use an instance of `DogRepository` to read data and iterate over the data.

```
package com.example.beans_to_boot.raw;

public class Main {

    static void main(String[] args) throws Exception {
        var dogRepository = new DefaultDogRepository();
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

1. First we create an instance…
2. …then we pass it to a `test` method, which expects an instance of an interface and doesn't care about how we got it.

Dependency Injection

You probably already saw this coming. We should centralize that `DataSource` definition. Here's the updated `Main` and `DefaultDogRepository`.

```
package com.example.beans_to_boot.di;

import org.springframework.jdbc.datasource.DriverManagerDataSource;
```

```
public class Main {

    static void main(String[] args) throws Exception {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var dogRepository = new DefaultDogRepository(db);
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

And here's the `DogRepository` implementation. There shouldn't be much to say. This is, after all, just using constructors in an object-oriented language. But I'll say a few things anyway, because there was a time when that kind of code used to be fairly common.

When I first started using Java, I'd see a lot of code written like the first example, with the `DataSource` initialized *inside* the thing that used it. This was because a lot of the programmers who first adopted Java when I started using it came from C++ and C backgrounds. In those paradigms, developers were forced to manage memory, and if any of the memory got away from the programmer, you'd get an insidious memory leak! So we had a pattern: RAII. Resource Acquisition Is Initialization. The idea was that we bound things that needed to be managed - memory, sockets, etc. - to the lifecycle of the smallest scope, usually that of the object that used it. Put another way, if a `Foo` needed a `Bar`, it'd create it in the `Foo` constructor and it'd destroy it in the `Foo` *destructor*, a concept we don't really have in Java. Because Java has a garbage collector, it's fine if objects have lifetimes longer than the things that use them.

`DataSource`s tend to be expensive, especially once you have connection pooling. You'll want to keep tabs on where the `DataSource` is built, what its pool size is, and so on. There are several benefits here.

One, it's easier to maintain, as you only need to change things in one place. Two, we're writing our code…

Anyway, here's the updated repository with the constructor. One of the benefits of the constructor is that it's written against the lowest common type, `javax.sql.DataSource`. We're using Spring's built-in (and *not* production-worthy) `DriverManagerDataSource`. It does *not* do connection pooling or anything interesting. It just implements the `DataSource`. In a production environment, you'd likely replace it with a more appropriately configured HikariCP connection pool. In a test environment, you'd replace it with a mock. Either way, you only need to change one thing in one place and pass it into constructors as you go.

```
package com.example.beans_to_boot.di;

import javax.sql.DataSource;
```

```
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Collection;

class DefaultDogRepository implements DogRepository {

    private final DataSource db;

    DefaultDogRepository(DataSource db) {
        this.db = db;
    }

    @Override
    public Collection<Dog> findAll() {
        try (var connection = this.db.getConnection(); //
                var ps = connection.prepareStatement("select * from dog") //
        ) {
            var list = new ArrayList<Dog>();
            try (var rs = ps.executeQuery()) {
                while (rs.next())
                    list.add(new Dog(rs.getInt("id"), rs.getString("name"), rs
.getString("description")));
            }
            return list;
        } //
        catch (SQLException throwable) {
            throw new RuntimeException(throwable);
        }
    }

}
```

Nothing fancy. Just object-oriented programming, constructors, and leveraging the power of polymorphism. Our constructors expect a `DataSource`, and we can give them any implementation we want without violating the contract.

Portable Service Abstractions

Looking at the code for the repository, it is a bit… much. I've written this sort of low-level `DataSource` manipulation code a million times in my career. Trust me when I say that as terrible as it is, it could be *much worse*. Thanks to the elegance of `AutoCloseable`, most of the resource management code and the tedious wrapping of `try` / `catch` go away. But it's still a low-level approach to data access. This isn't even controversial. JDBC is a driver, not an abstraction. It's the lowest level of infrastructure for working with data in a SQL database. I bet if you asked, you'd find that most of the folks working on JDBC drivers never expected people to use them directly. And a good thing, too, because it hurts.

Look at the code again. There are two things that are *essential*: the SQL query and the mapping logic. Everything else - creating the connection, prepared statements, and result sets; iterating over

the result sets; extracting out to lists; and so on - is boilerplate code that is far removed from whatever we're trying to achieve by writing this code in the first place. (Surely there's *some* business justification? Right?)

Let's rework this code to leverage one of Spring's many amazing *portable service abstractions*, the `JdbcClient`.

```
package com.example.beans_to_boot.psb;

import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DefaultDogRepository implements DogRepository {

    private final JdbcClient db;

    DefaultDogRepository(JdbcClient db) {
        this.db = db;
    }

    @Override
    public Collection<Dog> findAll() {
        return db ①
            .sql("select * from dog")//
②
            .query((rs, _) -> new Dog(//
                    rs.getInt("id"), //
                    rs.getString("name"), //
                    rs.getString("description"))//
            ) //
            .list();
    }

}
```

① Look! There's the query we were talking about!

② And there's the mapper logic. Clear as a bell!

I think we can agree, this is *much* better.

Unified Transaction Handling with `PlatformTransactionManager`

This repository has just *one* method, and that method only executes *one* read query. But what happens if that method has two queries? What if they're both writes, modifying database state?

Suppose we're writing to the database and we want to add two records. The first record will go to a new `customers` table that contains a list of all the people who have registered with our pet shop. The

second will go to a table containing the customer's expressed preferences for pets (small enough for condo or apartment living, needs more aggressive routine grooming, etc.). These two records are related to each other. Now imagine that, as soon as we write the first row, the power goes out and we don't manage to write the second row. Now we have a record in an inconsistent state in the database. We can't help the customer until we sort out their preferences. Or imagine, by some contrivance, that the first write succeeds, and then somebody comes along and deletes the customer before the second gets a chance to commit. Now we've got preferences, but no customer!

The data is in an inconsistent state. The normal way to handle this in a database system is to use transactions. Transactions are, broadly speaking, a way of batching and then either committing or rolling back work. It looks conceptually like this:

- start tx
  - write 1
  - write 2
  - write 3
- if everything is good: commit *everything* all at once
- else: rollback

This oversimplifies things a bit, but the main takeaway is that from the perspective of other SQL clients to the database, the system is never in an in-between state. It's all or nothing. They won't ever take action based on inconsistent or invalid database state.

It's a *truly* powerful concept, and it has mirrors in all sorts of technologies. Here's an abbreviated list:

- JMS
- JDBC
- JPA (based on JDBC)
- JDO (based on JDBC)
- JDBI (based on JDBC)
- jOOQ (based on JDBC)
- Kafka
- AMQP (the protocol under RabbitMQ)
- Neo4j
- MongoDB
- Redis (to be fair: here transactions are more about batching than rollbacks)

And as you might imagine, things can become disastrously complicated trying to work with transactions in a consistent way across all these technologies. Some might reach for JTA, but this sort of reminds me of that old XKCD cartoon in which, in introducing the new standard, we've only introduced yet another incompatible variant of the thing being standardized. It doesn't help. Here, Spring has yet another very powerful portable service abstraction called

`PlatformTransactionManager`.



There's also a reactive variant, but let's stick to blocking semantics for this example.

Here's the basic shape of Spring's `PlatformTransactionManager`.

```
package org.springframework.transaction;

import org.jspecify.annotations.Nullable;

public interface PlatformTransactionManager extends TransactionManager {

①
    TransactionStatus getTransaction(@Nullable TransactionDefinition definition)
throws TransactionException;

②
    void commit(TransactionStatus status) throws TransactionException;

③
    void rollback(TransactionStatus status) throws TransactionException;
}
```

1. Create a transaction (and something to which we can ascribe subsequent work).
2. Commit the transaction and the batch of work therein.
3. Or, if something's gone wrong, roll back the work.

We can write code in terms of implementations - the `DataSourceTransactionManager`, for one - of this interface and handle transactions in a consistent way across our code. If we decide to work with Kafka tomorrow, we need only swap out the instance of the `PlatformTransactionManager` in a single place.

So, where to put this code? Remember, we're using Java, a good object-oriented language. We've got to follow the single-responsibility principle; each class should do one thing. Transaction handling feels like a different concern. I suppose we could extract it into another class that implements the same `DogRepository` interface, sets up and tears down the transactions, and then calls our actual target `DogRepository`.

Here's our new `TransactionalDogRepository`.

```
package com.example.beans_to_boot.tx1;

import org.springframework.transaction.PlatformTransactionManager;

import java.util.Collection;

class TransactionalDogRepository implements DogRepository {
```

```
    private final DogRepository target;

    private final PlatformTransactionManager txm;

    TransactionalDogRepository(PlatformTransactionManager txm, DogRepository target) {
        this.target = target;
        this.txm = txm;
    }

    @Override
    public Collection<Dog> findAll() {
        IO.println("starting transaction");
        var status = this.txm.getTransaction(null);
        try {
            var results = this.target.findAll();
            this.txm.commit(status);
            IO.println("committing transaction");
            return results;
        } //
        catch (Throwable throwable) {
            IO.println("rolling transaction back");
            this.txm.rollback(status);
            throw new RuntimeException(throwable);
        }
    }

}
```

It's wrapped the core functionality, finding records, in a transaction.

In order for this arrangement to work, we have to spin up a `PlatformTransactionManager` and rework our `Main` class.

```
package com.example.beans_to_boot.tx1;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        var transactionalDogRepository = new TransactionalDogRepository
(dbPlatformTransactionManager, dogRepository);
```

```
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

① The instance given to the `test` method is still a `DogRepository`, so it is none the wiser.

Prefer Composition to Inheritance

This new wrapper class is a *composite*. In object-oriented design there are two ways to introduce new features to a type system: specialization (subclassing) and composition.

Prefer composition to inheritance. It buys you lots of *optionality*. Optionality is wiggle room, the freedom to make different decisions later on with little cost or disadvantage. Optionality is the bedrock of architecture: given the constraints, what choices should I make that most advantage me now and tomorrow?

- Flexibility at runtime. Composition lets an object hold references to other objects and swap them out, so behavior can change dynamically. Inheritance fixes the relationship at compile time.
- Avoids the fragile base class problem. With inheritance, changes to a parent class can silently break subclasses that depended on its internals. Composition isolates components behind their interfaces, so changes propagate less unpredictably.
- Escapes deep hierarchy rigidity. Inheritance tends to grow tall, tangled class trees. When you need behavior from multiple sources, single-inheritance languages force awkward workarounds; composition lets you assemble capabilities from independent pieces.
- Better encapsulation. Subclasses often need to know how the parent works internally (protected members, method call order). Composition only exposes the public interface of the parts you use, keeping coupling loose.
- Models "has-a" more honestly than forced "is-a". Inheritance implies a true subtype relationship. Developers frequently abuse it just to reuse code, creating relationships that aren't logically sound (a `Stack` is not really a `List`, even if it reuses one). Composition expresses "uses" or "has" without that false claim.
- Easier testing. Composed dependencies can be mocked or injected; inherited behavior is baked in and harder to isolate.

The tradeoff is a bit more boilerplate - you delegate calls to the contained objects rather than getting them "for free." Inheritance still fits genuine is-a relationships and cases in which you want polymorphism through a shared base type. The guidance is a default, not an absolute.

If you run `Main`, you'll see that we've got printouts and that we've started and stopped a transaction for every call.

Simplified Transaction Management with `TransactionTemplate`

I'm happy with what we've done, but the result is certainly more verbose. Imagine having to write all that boilerplate transaction code in a new composite method for every method we introduce in the original implementation. It wouldn't be so bad if we could do away with the fragile resource management and error-handling logic.

The core arrangement is similar from one use to another. Start transaction, do work. Commit. Or catch an exception and roll back. The only thing we really care about is the stuff in the middle - the work itself. Another very interesting design pattern we care about is the *template* pattern. Here, we extract that which changes from that which doesn't, and we hide that which doesn't change in a *template method*. It's a template, you see, because you only need to fill out the parts that are unique to your situation.

Spring's `TransactionTemplate` and `JdbcTemplate` - the precursor to Spring's `JdbcClient` - were gateway drugs for me. I deleted thousands of lines of code when I first found these gems.

Let's refactor the `TransactionalDogRepository` to build on the `TransactionTemplate` instead.

```
package com.example.beans_to_boot.tx2;

import org.springframework.transaction.support.TransactionTemplate;

import java.util.Collection;

class TransactionalDogRepository implements DogRepository {

    private final DogRepository target;

    private final TransactionTemplate txm;

    TransactionalDogRepository(TransactionTemplate txm, DogRepository target) {
        this.target = target;
        this.txm = txm;
    }

    @Override
    public Collection<Dog> findAll() {
        return this.txm.execute(_ -> target.findAll());
    }

}
```

So much better!

Here's the refactored `Main` class, too.

```
package com.example.beans_to_boot.tx2;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.support.TransactionTemplate;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
①
        var transactionTemplate = new TransactionTemplate(
dbPlatformTransactionManager);
        var transactionalDogRepository = new TransactionalDogRepository
(transactionTemplate, dogRepository);
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

① All we've done is introduce the `TransactionTemplate`, which of course takes a pointer to the `PlatformTransactionManager`.

This is a real improvement, but it only handles *one* concern: transactions. In the next chapter we'll see what happens when the concerns start to multiply, and how Spring's proxies and its container let us stop writing this boilerplate by hand altogether. :code: ..

Aspect-Oriented Programming and the Spring Container

In the last chapter we leaned on composition and Spring's `TransactionTemplate` to wrap our repository in a transaction. It worked, but you could already feel the strain: every new concern - transactions today, logging or security or metrics tomorrow - meant another hand-written composite, and, in each one, another method to maintain for every method on the underlying type. These are *cross-cutting concerns*, and object-oriented composition alone doesn't scale to meet them. In this chapter we'll solve that properly - first by generating proxies at runtime, and then by handing the whole job to Spring's container, which turns out to be exactly the sort of thing a good framework should do for you.

Cross-Cutting Concerns

Transactions are only the first of these. Suppose we wanted to add logging. Or metrics. Or security checks. Each one is another composite, and every new method on `DogRepository` means a corresponding method in every composite we've written. Three concerns and ten methods is thirty delegating methods, and not one of them does anything interesting.

Then it gets worse, because the composites have to be ordered. Which one wraps which? Suppose we wanted auditing, so that for every write operation we also write a row to an audit table. That has to happen *inside* the transactional composite, so that the underlying write and the audit row commit or roll back together. Get the nesting backwards and your audit trail will confidently lie to you.

So the composites aren't independent, they aren't reusable across repositories, and there are more of them every time we touch the code. Composition is the right tool for modeling what an object *is*. It's the wrong tool for describing something that must happen on the way into *every* method of *every* repository.

Hold My Proxy!

The plan of attack for an object-oriented language is to use composition or inheritance to introduce new concerns. But we can't scale that. We'd get too many permutations of too many types in no time at all. So what if we could do this automatically? What if we could, at runtime, *programmatically* create a class that implements the interface?

Java has had a way to do exactly this since version 1.3. It lives in `java.lang.reflect`, it's called `Proxy`, and given a list of interfaces it will manufacture a class implementing all of them, then hand you an instance. Every call to every method on that instance is routed to a single callback that you supply.

Let's put it to work in a new class called `Transactions`, which is where all of the metaprogramming will live from here on out.

Fair warning: this level of high-wire metaprogramming can be a little intense for folks not prepared for it. You don't need to retain all of it. Let's just understand conceptually what's happening. We're going to ask Java to do two things: create a class that implements the interface we care about

(`DogRepository`), and then create a single instance of that new class and return it to us.

We provide a callback - shown here as a lambda, though it's an implementation of the JDK's `InvocationHandler` type - that will be invoked whenever anybody calls a method on the newly generated instance.

```
package com.example.beans_to_boot.jdkproxy;

import org.springframework.transaction.support.TransactionTemplate;

import java.lang.reflect.Method;
import java.lang.reflect.Proxy;

class Transactions {

    static Object createProxy(TransactionTemplate tt, Object target) {
        var classLoader = target.getClass().getClassLoader();
        var interfaces = target.getClass().getInterfaces();
①
        return Proxy.newProxyInstance(classLoader, interfaces, (_, method, args) ->
doInTx(tt, target, method, args));
    }

②
    private static Object doInTx(TransactionTemplate tt,
③
            Object target,
④
            Method method,
⑤
            Object[] args) {
        return tt.execute(_ -> {
            try {
                try {
                    IO.println("before tx");
⑥
                    return method.invoke(target, args);
                } //
                finally {
                    IO.println("after tx");
                }
            } //
            catch (Exception e) {
                throw new RuntimeException(e);
            }
        });
    }

}
```

① Call `java.lang.reflect.Proxy.newProxyInstance(ClassLoader, Class[] interfaces, InvocationHandler)`. The `InvocationHandler` is the lambda at the end. Its first parameter is the proxy instance that `newProxyInstance` hands back to us - `this`, from the proxy's point of view. We don't need it here, so we discard it using Java's unnamed variable, `_`.

② The handler forwards everything on to `doInTx`, starting with the `TransactionTemplate` that will run the call inside a transaction.

③ We're given a pointer to the `target`: the real, unproxied object that the proxy stands in front of.

④ We're given a pointer to the `java.lang.reflect.Method` that is being invoked. Did the caller invoke `toString` on the proxy instance? Then this will be the `Method` for the `toString` method.

⑤ We're given a pointer to the `Object[]` arguments that were passed to the method when it was called. In our case, we're going to be calling `findAll`, which takes no arguments, so this `Object[] args` will be empty.

⑥ Reflectively invoke the original method on the target, inside the transaction and sandwiched between our two log statements.

Weird, isn't it? But it's a force multiplier that we need. It'll allow us to achieve the same sort of runtime metaprogramming shenanigans you'd expect from a duck-typed language like JavaScript or Ruby, without giving up the type system for most of our code.

Note what we *didn't* have to write: a method for `findAll`, a method for anything else on the interface, or indeed any method at all. One handler covers every method the interface declares, and every method it might declare tomorrow. That's the escape from the combinatorial explosion we walked into a page ago.

Let's rework our `Main` class, jettisoning the old, artisanal, hand-crafted `TransactionalDogRepository` and instead using this new shortcut to simplify things.

```
package com.example.beans_to_boot.jdkproxy;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.support.TransactionTemplate;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        var transactionTemplate = new TransactionTemplate(
dbPlatformTransactionManager);
        var transactionalDogRepository = (DogRepository) Transactions.createProxy
(transactionTemplate, dogRepository);
        test(transactionalDogRepository);
```

```
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

Again, from the perspective of our "test," nothing has changed. It expected a `DogRepository`, and a `DogRepository` it got.

What About Concrete Types?

OK, we've come this far. Let's talk about the two elephants in the room.

First: we now have a single concrete implementation of the interface and a programmatically generated one. But, really, since the programmatically generated one is not *our* work product, it feels like we've got a single interface for a single concrete type. Opinions vary here, but I'd wager the consensus is that if you have just one concrete implementation, you don't need the interface.

Second: the interface isn't optional. The JDK's `Proxy` class only works on *interfaces* - it has no way to automatically create new implementations of concrete classes. So the interface we'd most like to delete is the very thing holding the proxy up.

Conceptually, the fix is simple. Subclass a concrete type and then use that subclass as a sort of composite, forwarding requests to the actual target. In this way the subclass has the same shape as the concrete type and can wrap the target just as before. But how? The easiest way to do this these days is bytecode weaving. We can programmatically write - with bytecode - a new class file, add it to the classloader, and then reflectively construct a new instance of it. There are many libraries that can help with this sort of thing. ASM. Byte Buddy. The JDK itself, in more recent versions, can do it too. And, a perennial favorite, CGLIB.

Spring vendors CGLIB. That is to say, it actually has a copy of CGLIB in the framework itself. We use it to generate classes for all sorts of reasons. And there's even a super-convenient type, the `ProxyFactoryBean`, which we can use to achieve the same results as with the JDK's `Proxy` class.

Here's the rewritten `Transactions` class. No other changes are required. You can see we've even kept the same `doInTx` method. The concepts are the same; both the JDK implementation and the CGLIB implementation need a pointer to the target object, the method being invoked, and the arguments passed to it.

```
package com.example.beans_to_boot.cglibproxy;

import org.aopalliance.intercept.MethodInterceptor;
import org.springframework.aop.framework.ProxyFactoryBean;
import org.springframework.transaction.support.TransactionTemplate;

import java.lang.reflect.Method;
```

```
class Transactions {

    static Object createProxy(TransactionTemplate tt, Object target) {
        var pfb = new ProxyFactoryBean();
        pfb.setProxyTargetClass(true);
        pfb.setTarget(target);
        for (var i : target.getClass().getInterfaces())
            pfb.addInterface(i);
        pfb.addAdvice((MethodInterceptor) invocation -> doInTx(tt, target, invocation
.getMethod(),
                invocation.getArguments()));
        return pfb.getObject();
    }

    private static Object doInTx(TransactionTemplate tt, Object target, Method method,
Object[] args) {
        return tt.execute(_ -> {
            try {
                try {
                    IO.println("before tx");
                    return method.invoke(target, args);
                } //
                finally {
                    IO.println("after tx");
                }
            } //
            catch (Exception e) {
                throw new RuntimeException(e);
            }
        });
    }

}
```

In this iteration, we've deleted the interface and renamed our `DefaultDogRepository` to just `DogRepository`. Here's `Main` with the interface gone.

```
package com.example.beans_to_boot.cglibproxy;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.support.TransactionTemplate;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
```

```
        var dogRepository = new DogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        var transactionTemplate = new TransactionTemplate(
dbPlatformTransactionManager);
        var transactionalDogRepository = (DogRepository) Transactions.createProxy
(transactionTemplate, dogRepository);
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

Look at the cast: `Transactions.createProxy` still returns an `Object`, but what comes back now is a *subclass* of `DogRepository` rather than an implementation of some interface. The `test` method never changed, and it still can't tell the difference. Less is more.

Subclass proxies buy us that deleted interface, but they aren't free, and the constraints all follow from the one mechanism. If the proxy is a subclass, then:

- The class can't be `final`, and neither can any method you want proxied. A subclass has to be able to override what it's wrapping. `private` and `static` methods are out for the same reason.
- Self-invocation doesn't work. When one method on the target calls another method on itself, that second call goes directly to `this` - the real, unproxied object - and never passes through the subclass at all.

Hold on to that second one. It is, by a comfortable margin, the most common source of "why isn't my transactional method actually transactional?" in the wild, and we're only a few pages from meeting the annotation that provokes the question. The answer is always the same: something called the method from *inside* the object, so the proxy never saw it.

Better Configuration with Spring

The code is still approachable, but the wiring is starting to get out of hand. Every object in `Main` is constructed by hand, in the right order, with its dependencies passed in by hand. Want a second entry point - a scheduled job, a test harness, a second `main`? Duplicate the whole assembly. Change one constructor signature and go fix every copy. And this is just a single database query in a single repository; we haven't begun to think about web layers, messaging, or observability.

Let's see if Spring can improve things. The idea is to turn our objects into *beans* (basically, objects that Spring creates and manages) and let Spring's `ApplicationContext` wire them together. Conceptually, the flow is simple: Spring ingests your configuration, produces `BeanDefinition`s, and from those definitions instantiates the actual beans.

Here's our rewritten `Main` class, this time using Spring Framework to wire things together.

```
package com.example.beans_to_boot.springframework;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    static void main(String[] args) {
①
        var applicationContext = new AnnotationConfigApplicationContext
(DogConfiguration.class);
        var dogRepository = applicationContext.getBean(DogRepository.class);
②
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

① We create the `ApplicationContext` and tell it to source the wiring of these objects from a configuration class. Spring needs to know how your objects are wired together, and there are several ways to tell it. You can use all of them, or just one, to get the job done. We're starting with one way: Java configuration - in this case, a class called `DogConfiguration`.

② The `test` method is unchanged. It takes a `DogRepository`, and it neither knows nor cares that Spring built the one it got.

Here's that configuration class.

```
package com.example.beans_to_boot.springframework;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
class DogConfiguration {

①
    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
```

```
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(TransactionTemplate tt, JdbcClient jdbcClient) {
        var target = new DogRepository(jdbcClient);
        return (DogRepository) Transactions.createProxy(tt, target);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

① The wiring should look familiar. Each method constructs and returns one fully formed object. If the object requires any dependencies, it declares them as parameters on the method. Spring will attempt to resolve those dependencies for you.

We've simply retooled our `main` method into an admittedly longer Java configuration class. The order is gone, which is the point: we no longer say *when* to build anything, only *what* each thing needs, and Spring works out the sequence.

Notice, though, that the `dogRepository` method still calls `Transactions.createProxy` by hand. Spring is wiring our objects now, but the proxying is still our problem. Hold that thought - we'll take it away from ourselves shortly.

One habit to pick up while we're here. In Spring, you want the return type of each `@Bean` method to be as specific as possible, because Spring resolves dependencies by *type*, and that declared return type is all it has to go on before the method has ever been called. Declare `driverManagerDataSource()` as returning `DataSource` and nothing can ever ask for a `DriverManagerDataSource`. Declare it as returning `DriverManagerDataSource`, as we do here, and both requests are satisfiable. The parameters run the other way: ask for the most general type that does the job - `jdbcClient(DataSource)`, not `jdbcClient(DriverManagerDataSource)` - and you stay free to swap the implementation later without touching the consumer. This maxim appears all over the computer science ecosystem and is called *Postel's Law*. It says: *Be conservative in what you send, and liberal in what you accept*. The maxim is more about network services and API design, but it helps here when thinking about object wiring, too.

Bean Lifecycles

Earlier we spoke about C++ and RAII. That particular pattern isn't great, but it does hint at something important: objects often have interesting and delicate state. They're created. They're put into service. They're destroyed. Java handles memory management, but more things than just RAM can leak. Network sockets, for one. So it's important to have a concept of lifecycle. Spring provides *many* different mechanisms that let interested beans participate in lifecycle events. These are essentially callbacks triggered when objects are created or destroyed.

In a typical Spring application, objects are created when the `ApplicationContext` starts up and destroyed when the `ApplicationContext` shuts down. There's a one-to-one mapping between the life of the context and the life of the beans inside it. But objects can have *scope*. They could be created anew for each and every HTTP request, for example. Or for each new unique HTTP session. Or for each new run of a *step* in a Spring Batch *job*. By default, Spring beans are *singleton*-scoped: they exist once and only once per application context.

There are lots of ways to specify dependencies - *dependency injection* - for a bean. Thus far we've been relying on constructors. But some objects are designed in such a way that they have a constructor and *setter* methods that might be called with *optional* dependencies after the object's been created. In this case, the constructor wouldn't be the best place for initialization code, because the initialization would necessarily need to happen *after* the object's dependencies (constructor and setter alike) have been specified. Spring provides two ways of participating in this lifecycle phase: `InitializingBean` and `@PostConstruct`. Similarly, when the application shuts down, there are two equivalent approaches for cleaning up state: `DisposableBean` and `@PreDestroy`.

Here's our reworked `DogRepository` to demonstrate this.

```
package com.example.beans_to_boot.lifecycle;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DogRepository implements InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"), rs
.getString("description")))
```

```
                .list();
    }

    @PostConstruct
    void postInit() {
        IO.println("postInit");
    }

    @PreDestroy
    void preDestroy() {
        IO.println("preDestroy");
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("afterPropertiesSet");
    }

}
```

Most of these callbacks are, in this particular example, pointless - but hopefully illustrative.

Run it and you'll see `postInit` printed before `afterPropertiesSet`, and `preDestroy` before `destroy`. That ordering is not arbitrary, and it's worth having the whole sequence in one place, because Spring hands you six or seven ways to get a word in edgewise and says surprisingly little about which one happens when. Here's the whole thing, in the order it runs.

What runs	Notes
The constructor	Constructor injection happens here. The object exists, but Spring isn't finished with it, so this is the wrong place for anything that needs the object to be fully configured.
Field and setter injection	Anything declared with `@Autowired` on a field or a setter gets populated now. This is the whole reason initialization logic doesn't belong in the constructor.
The `*Aware` callbacks	`BeanNameAware`, `BeanClassLoaderAware`, and `BeanFactoryAware` fire here; `ApplicationContextAware` and its siblings arrive a moment later, in the row below, because they're delivered by a post-processor. Either way, this is where a bean is handed pointers back to the container.
`BeanPostProcessor.postProcessBeforeInitialization`	Every registered `BeanPostProcessor` gets a look at the bean. We'll write one of these ourselves later in the chapter.

What runs	Notes
`@PostConstruct`	First of the three initialization hooks. It lands here rather than later because it's dispatched *by* one of the post-processors in the row above - Spring's own `CommonAnnotationBeanPostProcessor`.
`InitializingBean.afterPropertiesSet()`	Second. The interface-based equivalent of `@PostConstruct`, and it runs *after* it, not before. That surprises people who assume the interface wins.
`@Bean(initMethod = "…")`	Third and last. A custom method named in the bean definition rather than marked on the class - handy for third-party types you can't annotate.
`BeanPostProcessor.postProcessAfterInitialization`	The final hook before the bean is published to the rest of the application. This is where proxies get swapped in - ours, later in this chapter, and `@Transactional's after that.
The bean is in service	Everything above ran before your application did a lick of work. Everything below runs after it has stopped.
`@PreDestroy`	First of three again - shutdown runs the same mechanisms in the same relative order. Also dispatched by `CommonAnnotationBeanPostProcessor`.
`DisposableBean.destroy()`	Second, mirroring `afterPropertiesSet`.
`@Bean(destroyMethod = "…")`	Third, mirroring `initMethod`. Spring will also infer a `close` or `shutdown` method if the type has one, which is why an unclosed connection pool is rarer than it used to be.

Two takeaways worth keeping. The annotation always beats the interface, in both directions - `@PostConstruct` before `afterPropertiesSet`, `@PreDestroy` before `destroy`. And a bean is only handed to the rest of the application after that last `BeanPostProcessor` has had its say, which is the seam the whole back half of this chapter lives in.

Now notice what these callbacks have in common: each one is the container telling a bean that something happened to it. That turns out to be a good idea in general, and Spring doesn't restrict it to creation and destruction. The same notion generalizes into a full publish-and-subscribe event bus, and once you have one of those, any component can announce something without knowing who, if anyone, is listening.

There are tons of interesting things that happen in a typical application. Somebody has authenticated in a Spring Security application. A Spring Batch `Job` has finished executing. A new HTTP request has been made. A customer has added an item to a cart. *Whatever*. Any Spring bean can inject a reference to an `ApplicationEventPublisher` and publish an event - which can be any object, though it's commonly a subclass of Spring's `ApplicationEvent`. And any interested bean can listen for that event.

In the old days, beans did this by implementing the `ApplicationListener` interface; they'd get *every* event and would have to sift through the types to determine whether they were interested. Then we

got the generics-capable `ApplicationListener<T>`, where listeners would be delivered only events of the type they were interested in. Then, finally, we got `@EventListener`, which is my go-to approach these days. Here's an example showing both ends of that history: an `ApplicationListener` that sees every event, including Spring's `ContextRefreshedEvent` - which is published right after Spring has finished building the bean graph from the input configuration - and an `@EventListener` method that receives only the `ContextClosedEvent`.

```
package com.example.beans_to_boot.lifecycle;

import org.springframework.context.ApplicationEvent;
import org.springframework.context.ApplicationListener;
import org.springframework.context.event.ContextClosedEvent;
import org.springframework.context.event.ContextRefreshedEvent;
import org.springframework.context.event.EventListener;

class Listener implements ApplicationListener<ApplicationEvent> {

    @Override
    public void onApplicationEvent(ApplicationEvent event) {
        IO.println("got [" + event.getClass().getName() + "]");
        if (event instanceof ContextRefreshedEvent contextRefreshedEvent) {
            IO.println("got contextRefreshedEvent [" + contextRefreshedEvent + "]");
        }
    }

    @EventListener
    void onClose(ContextClosedEvent contextClosedEvent) {
        IO.println("context closed event [" + contextClosedEvent.getTimestamp() + "]"
);
    }

}
```

And, of course, we've wired that up in the `DogConfiguration` class.

```
package com.example.beans_to_boot.lifecycle;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
class DogConfiguration {
```

```
①
    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(TransactionTemplate tt, JdbcClient jdbcClient) {
        var target = new DogRepository(jdbcClient);
        return (DogRepository) Transactions.createProxy(tt, target);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

① The newly registered `Listener` class.

Run `Main` and watch the console. Because that first listener implements `ApplicationListener<ApplicationEvent>`, it receives *everything*, and you'll see the container narrate its own startup - a handful of events, `ContextRefreshedEvent` among them - before a single dog is printed. A real application publishes dozens. Which is the argument for `@EventListener` in one screenful: the `onClose` method declares what it wants, and Spring never bothers it with anything else.

Everything we've done so far, though, we've done from *inside* a bean - implement an interface, annotate a method, wait to be called. To centralize a concern the way we set out to, we need to get outside the beans entirely and change them on their way in.

Post-Processors

This is a good moment to make a distinction I've been glossing over. Inside every `ApplicationContext` is a `BeanFactory`. The `BeanFactory` is the part that does the real work: it holds the bean definitions and performs dependency injection. The `ApplicationContext` wraps it and glues it into the surrounding application environment - events, resource loading, externalized configuration, internationalization. It's the context, after all. The distinction matters right now because the two hooks we're about to meet operate at those two different levels.

As you might've surmised, every Spring application goes through a well-known set of phases as it starts up. There's a lot more to it than this, but conceptually I find it helps to talk about three distinct phases.

- *ingest* - Spring reads the definitions of how you want your objects wired. So far we've seen Java configuration. But there are others, including component scanning, XML definitions, and `BeanRegistrar`s.
- *define* - after reading all the configuration, Spring turns it into `BeanDefinition` objects - metamodel objects that describe how an object should be created, its lifecycle, its dependencies, and so on - and stores them in the `BeanFactory`. At this stage nothing has been created yet, but we know enough to validate how objects relate to each other and to spot missing dependencies. Think of `BeanDefinition`s as the primordial soup for your application. No strong types at this point. Just designs and aspirations.
- *instantiate* - at this point we have live-fire objects. Spring has called constructors, passed in dependencies, and turned the `BeanDefinition`s into a valid graph of objects that, taken together, are your application.

Spring has a callback interface for each of the two seams between those phases.

The earlier one is `BeanFactoryPostProcessor`, which runs after *ingest* and before *instantiate*. Implement it and Spring will let you visit all the `BeanDefinition` instances in the `BeanFactory`, potentially deleting them, replacing them, or adding new ones.

We can see that middle phase with our own eyes. Here's a `BeanFactoryPostProcessor` that does nothing but walk the `BeanFactory` and print what it finds.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;

class MyBeanFactoryPostProcessor implements BeanFactoryPostProcessor {

    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory beanFactory)
throws BeansException {

        for (var beanName : beanFactory.getBeanDefinitionNames()) {
```

```
            var beanDefinition = beanFactory.getBeanDefinition(beanName);
            var clzz = beanFactory.getType(beanName);
            IO.println("got the bean definition for [" + clzz.getName() + "] with bean
name [" + beanName + "]: "
                    + beanDefinition.getDescription());
        }

    }

}
```

The bean it inspects is deliberately boring: a one-line `InitializingBean` lambda whose only distinguishing feature is a `@Description` annotation. The bean isn't the point. The point is that we can read its type, its name, and its description at a moment when the object itself does not yet exist. That's the primordial soup, and a `BeanFactoryPostProcessor` is a spoon.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.InitializingBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Description;

@Configuration
class BppConfiguration {

    @Bean
    static MyBeanFactoryPostProcessor myBeanFactoryPostProcessor() {
        return new MyBeanFactoryPostProcessor();
    }

    @Bean
    @Description("A simple demo that has a description")
    InitializingBean simpleDemoThatHasADescription() {
        return () -> IO.println("a demo with a description ");
    }

}
```

Note the `static` on that bean method. It matters more than it looks. A non-`static` `@Bean` method can't be called until Spring has instantiated the `@Configuration` class that declares it - so if Spring must build a configuration class in order to find your `BeanFactoryPostProcessor`, then that configuration class, and anything it eagerly drags along with it, gets created *before* post-processing runs and escapes it entirely. A `static` method needs no instance, so Spring can call it first. Any time you register a post-processor, make the method `static`.

Now for the later seam. Once the `BeanFactory` is full of viable `BeanDefinition` objects, Spring creates actual *beans* out of them by calling the constructors and setting the dependencies. Here we have one interface, `BeanPostProcessor`, that lets us get involved both *before* and *after* each bean's

initialization callbacks run - which is to say, on either side of the `@PostConstruct` and `afterPropertiesSet` methods we wrote a few pages ago. A `BeanPostProcessor` is one of the last lines to cross before the program is up and running. It may replace a bean of a given type with another bean of the same polymorphic type, but not much more. It is, however, an ideal place in which to introduce generic, cross-cutting concerns like transaction demarcation.

Which is exactly what we're going to do, because we still owe ourselves that. So far we've been calling the `Transactions` class by hand, and we'd have to remember to do it for every object that needs it. But suppose we had some generic mechanism for signaling that a given object wants to participate in transactions - a marker interface, say. Let's call it `Tx`.

```
package com.example.beans_to_boot.pp;

interface Tx {

}
```

Add that to the implementation of `DogRepository`.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DogRepository implements Tx, InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"), rs
.getString("description")))
            .list();
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
```

```
        IO.println("afterPropertiesSet");
    }

}
```

What we need now is some code to sift through all the objects right as they're about to be pressed into service and, if they implement this marker interface, wrap them in a transactional proxy.

```
package com.example.beans_to_boot.pp;

import org.jspecify.annotations.Nullable;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.transaction.support.TransactionTemplate;

class TxBeanPostProcessor implements BeanPostProcessor {

    private final BeanFactory beanFactory;

①
    TxBeanPostProcessor(BeanFactory beanFactory) {
        this.beanFactory = beanFactory;
    }

    @Override
    public @Nullable Object postProcessAfterInitialization(Object bean, String
beanName) throws BeansException {
        if (bean instanceof Tx tx) {
            IO.println("making a transactional proxy for [" + tx.getClass().getName()
+ "] with bean name [" + beanName
                    + "]");
            var transactionTemplate = this.beanFactory.getBean(TransactionTemplate
.class);
            return Transactions.createProxy(transactionTemplate, tx);
        }
        return bean;
    }

}
```

① Inject the `BeanFactory` so that we can ask it for whatever user-configured instance of `TransactionTemplate` is available. Note that we look up the bean only at the very last moment, at the time of the transactional method's invocation. We don't want to force eager initialization of the `TransactionTemplate` or any of its dependencies by looking up the object in the constructor.

With this done, we no longer need to manually call `Transactions` for any particular bean - and the `dogRepository` method in our configuration class goes back to simply constructing a `DogRepository`, with nothing about transactions in sight. The framework will do it automatically for any bean that

implements `Tx`. Register the new `TxBeanPostProcessor` with a `static` `@Bean` method, for the reason we just covered.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@Import(BppConfiguration.class)
class DogConfiguration {

    @Bean
    static TxBeanPostProcessor txBeanFactoryPostProcessor(BeanFactory beanFactory) {
        return new TxBeanPostProcessor(beanFactory);
    }

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(JdbcClient jdbcClient) {
```

```
        return new DogRepository(jdbcClient);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

This is where a good framework like Spring really comes into its own. Spring has a bird's-eye view of everything you're doing and can dramatically simplify things accordingly. Count what we just bought with one class: transactions apply to every current repository, every future repository, and every method on all of them, and not one of those repositories mentions a transaction anywhere in its source. That is a cross-cutting concern, centralized. It's also the essence of aspect-oriented programming (AOP).

But `@Transactional` Exists, So Let's Use That Instead

If you have used Spring at all before, then you probably know where this is going: Spring has already done this exact declarative transaction-demarcation magic trick. It's called `@Transactional`, and it's been in the framework for more than 20 years. Let's refactor to use it. Spring won't apply it for us unless we ask it to, and we ask with the `@EnableTransactionManagement` annotation on a `@Configuration` class.

```
package com.example.beans_to_boot.transactional;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@EnableTransactionManagement ①
class DogConfiguration {

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
```

```
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(JdbcClient jdbcClient) {
        return new DogRepository(jdbcClient);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

① Here's the most important bit. We can replace everything we wrote with this one annotation, plus the `@Transactional` annotation on the repository itself.

And now we can delete `Tx` and remove its use from `DogRepository`. We can also delete the `Transactions` class and the `TxBeanPostProcessor`.

Ahh. Much better.

The `@Transactional` annotation is more granular and capable than our homebrewed `Tx` marker interface. Where `Tx` could only say "all of this object, always," `@Transactional` can be placed on a class, in which case every method in the class will be wrapped in a transaction, or on individual methods. You can also stipulate things like propagation, isolation, and timeouts. Or you could place it on the class for some sensible defaults and then override those at the method level. *Much* better.

Two defaults are worth knowing before you lean on it. The first is the one we set up earlier: `@Transactional` works by proxy, so only calls arriving from *outside* the object are intercepted. A method that calls another `@Transactional` method on `this` gets no transaction, no warning, and no error. The second is that `@Transactional` rolls back on unchecked exceptions - `RuntimeException` and `Error` - and *commits* on checked ones. That surprises people, and it tends to surprise them in production. If you throw a checked exception to signal failure, say so: `@Transactional(rollbackFor = Exception.class)`.

This exercise - implementing a `BeanPostProcessor` and using proxies to apply a change centrally - is

the core of how Spring works. Spring does this sort of indirection *everywhere*. You put annotations on a class; Spring discovers them and replaces your beans with proxies that do the thing you asked for. And, thanks to the magic of polymorphism, consumers of these beans are none the wiser. They interact with your code through its plain interface, exactly as before. Everybody is better for it.

The Words for What We Just Built

We arrived at aspect-oriented programming by building it, which is my favorite way to arrive anywhere, but it does leave us without the vocabulary. Spring's documentation assumes you have it, and so does every stack trace you'll ever read from inside a proxy, so here are the words, mapped onto the code we just wrote.

- An *aspect* is a cross-cutting concern, modularized - packaged up as a thing you can name, apply, and reuse. Transaction demarcation is an aspect. So are security, caching, retries, and metrics. Our `Transactions` class, together with the `TxBeanPostProcessor` that applied it, was an aspect assembled by hand.
- A *join point* is a place in a running program where an aspect could be applied. In Spring, a join point is always a method invocation on a Spring bean. Other AOP systems allow more - field access, constructor calls, static initializers - but Spring's proxies can only intercept method calls, so that is the entire universe here. This is the same fact as the self-invocation problem, seen from a different angle.
- A *pointcut* is the expression that selects which join points you actually mean. Our pointcut was `instanceof Tx`. The pointcut for `@Transactional` is "any method carrying the annotation, or belonging to a class that carries it."
- *Advice* is the code that runs at a matched join point. The body of our `doInTx` method was advice. Because it wraps the target call on both sides, it's specifically *around* advice - the most powerful kind, and the only kind that can decline to call the target at all. Spring also offers *before*, *after returning*, *after throwing*, and *after (finally)* advice, which are all just around advice with the tedious parts filled in for you.
- *Weaving* is the act of connecting advice to join points. Some AOP systems weave at compile time, or at class-load time, by rewriting the bytecode of the target class itself. Spring weaves at runtime, by proxy, which is why every page of this chapter has been about proxies.

Spring exposes all of this directly if you want it: put `@EnableAspectJAutoProxy` on a `@Configuration` class, write a class annotated with `@Aspect`, and declare your advice with `@Around`, `@Before`, and friends, using AspectJ's pointcut expression language to describe what to match. It's the same machinery `@Transactional` rides on. In practice I reach for it rarely, because the concerns most people need have already been written and shipped as annotations. But when you need it, it's there, and now you know what it's doing.

We've gone from hand-written composites, to hand-written proxies, to a container that writes the proxies for us. What we haven't questioned is how that container learns about our objects in the first place. We've used exactly one mechanism - Java configuration - and we've hardcoded a database URL, a username, and a password directly into it. Both of those are the next chapter's problem. :code: ..

Registering Beans and Externalized Configuration

By now Spring's container is wiring our objects together and applying cross-cutting concerns for us. But we've only seen *one* way to tell Spring how our beans fit together - Java configuration - and we've been hardcoding configuration values like database credentials right into our source code. This chapter rounds out the picture. First we'll look at the other ways Spring can ingest bean definitions: component scanning and, new in Spring Framework 7, the `BeanRegistrar`. Then we'll pull those hardcoded values out of the code entirely using the Spring `Environment` and the Spring Expression Language.

Component Scanning

We saw in the previous chapter that Spring can ingest your beans in a number of different ways, and that Java configuration was just one. Another one is *component scanning*, where Spring can scan my packages and identify beans to manage by the presence of marker annotations, which we call *stereotype* annotations, from the Unified Modeling Language (UML).

The root stereotype annotation is `@Component`. Place it on a class and instruct Spring to discover classes in the package(s) in which that class lives and Spring will apply some useful heuristics to create the bean for you. It'll attempt to satisfy any of the constructor's dependencies by type. It'll look for interfaces like `InitializingBean` and `DisposableBean`, and annotations like `@PostConstruct`, and honor them just as before. It'll create a singleton, just as before. Assuming Spring can construct the object without any intervention from you, this is a great way to make short work of registering beans for Spring to manage. Indeed, it's easier to think about the places where Spring *couldn't* do the work for you.

- If you have custom factory methods (e.g., `JdbcClient.create(DataSource)`).
- If you need to pass in parameters that Spring can't satisfy by looking at other beans in the `BeanFactory`.
- If you don't have access to the source code. Obviously. From this arises an OK rule of thumb: use component scanning for your own code, use Java configuration for other people's code.

Spring also supports *meta-annotations*. You can put `@Component` on other annotations and then put those meta-annotations on your code and Spring will pick it up as well.

Spring does this quite famously with a handful of other annotations. These annotations are documentary *and* technical in nature. They tell the framework that the thing should be treated a certain way *and* they tell the programmer the intended role of the component in the codebase.

- `@Component` - a generic, managed bean
- `@Repository` - indicates that the component handles data-access logic. Reading, writing, etc.
- `@Service` - indicates that the component handles coarse-grained business logic.
- `@Controller` - indicates that the component is a *controller*, adapting requests of a particular protocol to server-side types, routing them to handler methods, and forwarding the output

model onto a handler to produce a response.

- `@RestController` - indicates that the component is a *controller*, specifically adapted to HTTP and the tenets of the REST constraint on HTTP.

Interestingly, `@RestController` is *not* annotated with `@Component`. It's annotated with `@Controller`, so there you have *two* levels of meta-annotations!

Best part? You can create your own meta-annotations. Just add `@Component` to it and Spring will discover it just like it does any of the other meta-annotations.

Here's our reworked `DogConfiguration` supporting component scanning, and with no explicit Java bean configuration method for the `DogRepository`.

```
package com.example.beans_to_boot.componentscan;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@ComponentScan ①
@EnableTransactionManagement
class DogConfiguration {

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
```

```
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

1. We have enabled component-scanning with `@ComponentScan`. By default this will look in the current package (`com.example.beans_to_boot`) and below (e.g., `com.example.beans_to_boot.a.b.c`) and find any class marked with `@Component`.

The `DogRepository` no longer appears in the `DogConfiguration` class. We've simply annotated it with a stereotype annotation, `@Repository`.

```
package com.example.beans_to_boot.componentscan;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.util.Collection;

@Repository ①
@Transactional
class DogRepository implements InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"), rs
.getString("description")))
            .list();
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }
```

```
    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("afterPropertiesSet");
    }

}
```

1. It's the little things that make everything so much nicer.

Component scanning supports a kind of *implicit* configuration. Spring discovers the class and applies heuristics to configure and construct a valid instance of your type. Java configuration supports a kind of *explicit* configuration. You must tell Spring almost everything, but you also have full control. It's a bit more verbose, but sometimes it's the best option.

Spring Framework 7's `BeanRegistrar`: The Best of Both Worlds

I love Java configuration and component scanning. (I certainly don't miss XML!) Component scanning was introduced as a first-class feature in Spring Framework 2.5 in 2007. Java configuration was introduced as a first-class citizen in Spring Framework 3.0 in 2009.

But they both have some limitations, too. Java configuration is very verbose. A whole method plus an annotation for one bean registration. They're also inflexible. How do I register beans dynamically, subject to some ambient thing like environment variables? What if I wanted to register five beans based on some property value? One does not simply programmatically add methods to a class in a for-loop. The same limitation exists with stereotype annotations.

Spring Framework 7 (in 2025) introduced a new mechanism that nicely addresses some limitations of the other two models. If you need the dynamism it affords, it's the best way to get it.



I will confess, there *are* some existing ways to access that runtime dynamism that long predate `BeanRegistrar`, but they come with their own set of limitations and 99% of people won't use or need them, especially with `BeanRegistrar` now available.

Let's look at an example where we rewire *almost* the entire application using `BeanRegistrar`. We'll retain one bean with component scanning (the `DogRepository`) and we'll retain one bean (the `DriverManagerDataSource`) using Java configuration.

```
package com.example.beans_to_boot.beanregistrar;

import org.springframework.beans.factory.BeanRegistrar;
import org.springframework.beans.factory.BeanRegistry;
import org.springframework.core.env.Environment;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;
```

```
import javax.sql.DataSource;

class DogBeanRegistrar implements BeanRegistrar {

①
    @Override
    public void register(BeanRegistry registry, Environment env) {

②
        registry.registerBean(Listener.class);
        registry.registerBean(MyBeanFactoryPostProcessor.class);
③
        registry.registerBean(TransactionTemplate.class,
                a -> a.supplier(ctx -> new TransactionTemplate(ctx.bean
(DataSourceTransactionManager.class))));
        registry.registerBean(JdbcClient.class, a -> a.supplier(ctx -> JdbcClient
.create(ctx.bean(DataSource.class))));
        registry.registerBean(DataSourceTransactionManager.class, a -> a
            .supplier(supplierContext -> new DataSourceTransactionManager
(supplierContext.bean(DataSource.class))));
    }

}
```

1. You do your wiring in a method. You can do iteration, load things, do `if`/`else`, etc., all from within a method. What a time to be alive!
2. If you want Spring to apply the same heuristics and automatically configure a bean, as it would when doing component scanning, then you can simply call `registerBean(Class)` and Spring will take it from there.
3. If you want to exert more control over how things are wired up, or look up other beans for reference, pass in the second parameter and specify a `Supplier<T>` on the `spec` parameter.

All in all, this is a huge improvement in total lines of code. I don't know how I feel about readability, though. I think it'd be pretty natural to factor those lambda suppliers out to separate methods. But then, I might as well just use Java bean configuration. That said, in Kotlin, this DSL is even more succinct and elegant and inarguably readable while taking less code.

This style grows on me more and more. You can use it. Or don't. You've got options depending on your style and use case.

The `Environment`

Let's revisit the one bean we left as Java configuration: the `DriverManagerDataSource`. This entire time we've left that bean configured with values hardcoded in the Java source code. This is terribad for a number of reasons.

First, in a book about security, it is *woefully* insecure! Obviously you should store your passwords in a more secure medium than in plaintext buried in the source code for a program developers

interact with every day. Especially if it's the production database password. Heaven forbid!

Second, this is super fragile. It doesn't conform to the principles behind *The 12 Factor Manifesto* [https://12factor.net], which offers a ton of good advice for how to structure and deploy cloud-native applications. Factor III (oh, excuse me, I meant "three") is *Config*. This factor says that "an app's config is everything that is likely to vary between deploys (staging, production, developer environments, etc)." This includes things like database connection credentials, backing services like Amazon S3, a cache, etc. It poses the litmus test for whether your application is well enough configured as follows: if the codebase were open-sourced right now, would any credentials be compromised? If so, you've not done a good enough job with your configuration.

Let's use the Spring Framework `Environment` abstraction. It's a layer of indirection between the application and the outside world. Think: property files, YAML files, environment variables, JNDI keys, HashiCorp Vault, etc. The `Environment` is an object that any Spring bean might inject and use to load configuration values. It in turn delegates to a chain of `PropertySource` instances that know how to load values from various sources. Spring comes loaded with some useful out-of-the-box defaults. As you move up the abstraction stack, you get even more sophisticated alternatives for things like HashiCorp Vault, the Spring Cloud Config Server, etc.

Let's load our configuration values from a local configuration file called `application.properties`.

```
package com.example.beans_to_boot.environment;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.core.env.Environment;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import java.net.URI;
import java.util.Objects;

@Configuration
@ComponentScan
①
@PropertySource("classpath:application.properties")
@EnableTransactionManagement
@Import(DogBeanRegistrar.class)
class DogConfiguration {

    @Bean
    DriverManagerDataSource driverManagerDataSource(Environment environment,
②
            @Value("${spring.datasource.username}") String usernameValue) {
        IO.println("configuring the DataSource username: [" + usernameValue + "]");
③
        var username = environment.getProperty("spring.datasource.username");
④
        var url = environment.getProperty("spring.datasource.url", URI.class);
        var pw = environment.getProperty("spring.datasource.password");
```

```
        return new DriverManagerDataSource(Objects.requireNonNull(url).toASCIIString(
), username, pw);
    }

}
```

1. The `@PropertySource` annotation is a convenient way to add the contents of a `.properties` file to the `Environment`.
2. You can inject a reference to the `Environment` directly *or* you can have Spring provide the value for you in Java configuration methods by annotating parameters with `@Value` and then using a `\${…}` syntax.
3. Here we're just loading a value and getting it back as a `String`, the default.
4. Spring's built-in `ConversionService` can translate all manner of different String values into useful domain types. You only need ask, in this case by passing in the preferred target conversion type. I'm asking that the URI be converted to a `java.net.URI`. Why not? If it's not a valid URI, the conversion will fail and the program will stop just that much earlier.

We've added a new `.properties` file to the example.

```
spring.application.name=beans-to-boot
#
①
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.password=secret
spring.datasource.username=myuser
#
②
spring.threads.virtual.enabled=true
#
③
spring.sql.init.mode=always
④
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=*
management.tracing.sampling.probability=1.0
```

1. I've chosen to use the keys `spring.datasource.url`, `spring.datasource.password`, and `spring.datasource.username` as the keys for my DataSource's URL, password, and username for, um, no reason at all…
2. We'll return to this soon.
3. We'll return to this soon.

I've got the password for the PostgreSQL server as it will be configured when we use the Docker Compose `compose.yml` file that ships with this codebase. This isn't the end of the world for local-only development. But obviously in production environments, you'd never have the production database username and password and URL in plaintext. You'd pass them in through environment

variables or use something like HashiCorp Vault.

This is why Spring's `Environment` has the notion of precedence. External contributions - those further away from the codebase - override contributions closer to the codebase, conceptually. So, if you specify an environment variable, it'll override the values provided in `application.properties`. There's even some goodness in there to normalize environment variables. So, `SPRING_DATASOURCE_URL` will be normalized to `spring.datasource.url`. In this way, developers can keep the hardcoded development-only environment variable, but at runtime in production the real connection credentials can be specified with environment variables.

The Spring Expression Language

We used the `Environment` to look up provisioned keys and values from the external environment. Sometimes, the values we need are more dynamic than that. Sometimes we need to do a little massaging of the data before using it. For this, we have the oh-so-handy Spring Expression Language (SpEL).

There are many amazing uses where, once you've tried it, you won't be able to live without something as convenient as the SpEL. Suppose you want to define a simple predicate with which to evaluate headers in a Spring Integration messaging flow. You could use SpEL. What if you wanted to define authorization rules for Spring Security? You could use SpEL. And suppose you simply wanted to know the meaning of life, the universe, and everything? *You could use SpEL*.

```
package com.example.beans_to_boot.spel;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

@Component
class Inquisitor {

    Inquisitor(@Value("#{ theAnswerToLifeTheUniverseAndEverything.answer() } ") int
answer) {
①
        Assert.state(answer == 42, "the values should match");
    }

}

@Component
class TheAnswerToLifeTheUniverseAndEverything {

    public int answer() {
        return 42;
    }

}
```

In this example, we're using the familiar `@Value` annotation, but this time instead of using it to inject a property placeholder (e.g.: `${foo.bar}`), we're using it to evaluate an expression and coerce the returned value from that expression into the value being injected. In this case, an `int`. We arrive at that `int` by calling another method on a bean in the Spring context: `TheAnswerToLifeTheUniverseAndEverything`. This is a bit of a contrived example. However, the `@Value` annotation is also the simplest and most accessible interaction most folks will have with the SpEL. You can use the `@Value` annotation in Java configuration classes and `@Bean` methods, or in regular old components anywhere you could otherwise use `@Value`.

The Spring Expression Language is a full-blown expression language that offers most of (if not more than) what the competitors - like OGNL, the Java Server Faces Expression Language, the Java Server Pages Expression Language, etc. - offer. One of my favorite features? You can turn the expressions into *compiled* code!

I'll never forget the SpEL's introduction into the Spring Framework. I was trawling the updates to Spring Framework 3.0 back in 2008 or 2009, watching as Andy Clement added this amazing new mechanism to the codebase. The first incarnation of the Spring Expression Language was written using Antlr, which is the Java analog to Yacc and Bison. These are tools with which you can write grammar, then a generator will turn the grammar into a recursive descent parser. Basically, it's a way to build interpreters or the foundations of a compiler. It makes perfect sense for even a gifted craftsman like Andy Clement to use these venerable tools to handle such a dedicated task. After all, Antlr's basically a standard! There is no better toolchain for building lexers and parsers on the JVM, full stop. Plenty of other projects do: Hibernate, Apache Spark, Apache Camel, Apache KIE Drools, GraphQL Java, Apache Groovy, Neo4j, Elasticsearch, PrestoDB, Apache Hive, Apache Cassandra, etc. But not Andy! He got the code working but wasn't satisfied. So a few weeks later, I did `svn up` and saw that Andy had thrown the Antlr version out. He'd just rolled up his sleeves and written a recursive descent parser - with a compiler, mind you! - by hand! *Legendary*.

Most of the time, you'll use it via expressions specified in annotations, since it lets you embed dynamic programming in otherwise static contexts. But you can also use it directly in your own application. Here's a simple example where we do some basic math.

```
package com.example.beans_to_boot.spel;

import org.springframework.context.annotation.Configuration;
import org.springframework.expression.ExpressionParser;
import org.springframework.expression.spel.standard.SpelExpressionParser;
import org.springframework.util.Assert;

@Configuration
class SimpleSpel {

    SimpleSpel() {
①
        var spelExpressionParser = new SpelExpressionParser();

②
        simple(spelExpressionParser);
    }
```

```
    private void simple(ExpressionParser expressionParser) {
        var expression = expressionParser.parseExpression("1 + 2");
        var evaluatedValue = expression.getValue(Integer.class).intValue();
        Assert.state(3 == evaluatedValue, "the values should match");
    }

}
```

We can also evaluate expressions against variables assigned to a *context*.

For this demo, we've got an `Order` class.

```
package com.example.beans_to_boot.spel;

record Order(String sku) {
}
```

In the next few iterations of our examples, we'll refer to a `List<Order>`.

```
package com.example.beans_to_boot.spel;

import org.springframework.context.annotation.Configuration;
import org.springframework.expression.ExpressionParser;
import org.springframework.expression.spel.standard.SpelExpressionParser;
import org.springframework.util.Assert;

import java.util.List;

@Configuration
class ValueSpel {

    private final List<Order> orders = List.of(new Order("sku1"), new Order("sku2"));

    ValueSpel() {
        var spelExpressionParser = new SpelExpressionParser();
        value(spelExpressionParser);
    }

    private void value(ExpressionParser expressionParser) {
①
        Assert.state(expressionParser.parseExpression("size()").getValue(orders,
Integer.class).intValue() == 2,
                "the values should match");
    }

}
```

1. we can evaluate methods against the object given as the context. In this case the `size()` method on `List<Order>`.

Indeed, there's even a `EvaluationContext`, which can act as a, well, a *context* for all sorts of references that can be made available to the expression during evaluation.

```
package com.example.beans_to_boot.spel;

import org.springframework.context.annotation.Configuration;
import org.springframework.expression.ExpressionParser;
import org.springframework.expression.spel.standard.SpelExpressionParser;
import org.springframework.expression.spel.support.StandardEvaluationContext;
import org.springframework.util.Assert;

import java.util.List;

@Configuration
class ContextSpel {

    private final List<Order> orders = List.of(new Order("sku1"), new Order("sku2"));

    ContextSpel() {
        var spelExpressionParser = new SpelExpressionParser();
        context(spelExpressionParser);
    }

    private void context(ExpressionParser expressionParser) {
        var context = new StandardEvaluationContext();
①
        context.setVariable("orders", orders);
②
        Assert.state(
                expressionParser.parseExpression("#orders.size()").getValue(context,
Integer.class).intValue() == 2,
                "the values should match");
    }

}
```

1. we can add as many variables to the `StandardEvaluationContext` as we like.
2. we can then reference those variables with `#variableName` from within the expression.

If you're intending to re-evaluate the same expression over-and-over again, then consider *compiling* the expression. That's right, you can turn your `Expression` into byte code that's loaded in the `ClassLoader` and evaluated much more efficiently on later evaluations.

```
package com.example.beans_to_boot.spel;

import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.expression.ExpressionParser;
import org.springframework.expression.spel.SpelCompilerMode;
import org.springframework.expression.spel.SpelParserConfiguration;
import org.springframework.expression.spel.standard.SpelExpressionParser;
import org.springframework.util.Assert;

import java.util.List;

@Configuration
class CompiledSpel {

    private final List<Order> orders = List.of(new Order("sku1"), new Order("sku2"));

    CompiledSpel() {
①
        var configuration = new SpelParserConfiguration(SpelCompilerMode.IMMEDIATE,
                Thread.currentThread().getContextClassLoader());
        var spelExpressionParser = new SpelExpressionParser(configuration);
        compiled(spelExpressionParser);
    }

    private void compiled(ExpressionParser expressionParser) {
        Assert.state(expressionParser.parseExpression("size()").getValue(orders,
Integer.class).intValue() == 2,
                "the values should match");
    }

}
```

1. Here we only need to change how we construct the `SpelExpressionParser` to switch its `SpelCompilerMode` to `IMMEDIATE`.

The Spring Expression Language is one of those amazing and pervasive features that's found all over the framework.

Next Steps

We've built up quite the application, and in the next chapter we'll see how much we can delete with the introduction of Spring Boot! :code: ..

Spring Boot

We've come a long way, but done very, very little. For all our troubles, we've only got one repository with one method. Now imagine we were trying to build a real application. We'd need security; we'd need to stand up a web server. We'd need observability. We'd need to package the application up for production.

Let's leverage Spring Boot. It's already on the CLASSPATH. Spring Boot builds on Spring Framework (and the rest of the Spring ecosystem). It's just more Spring, taken a step further thanks to the power of something called *autoconfiguration*. But first, let's refactor.

Here's the updated `Main` class.

```
package com.example.beans_to_boot.boot;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Import;

①
@SpringBootApplication
②
@Import(DogBeanRegistrar.class)
public class Main {

    static void main(String[] args) {
③
        SpringApplication.run(Main.class, args);
    }

④
    @Bean
    ApplicationRunner testRunner(DogRepository repository) {
        return _ -> repository.findAll().forEach(IO::println);
    }

}
```

1. Spring Boot's `@SpringBootApplication` annotation is itself a *meta-annotation*. It is meta-annotated with `@Configuration`, which means this `Main` class is also a `@Configuration` class. It's meta-annotated with `@ComponentScan`, so we don't need to worry about that either.
2. because it's a `@Configuration` class, we can use `@Import` on it, just like before.
3. through most of the code we've looked at, we've created a Spring `ApplicationContext`. Here, we're doing the same thing. The return value of `SpringApplication.run` is an `ApplicationContext`. But we don't need it, because we don't need to explicitly look up the `DogRepository` to pass to our

`test` method. Instead…

4. …we can define a bean of type `ApplicationRunner`. `ApplicationRunner` beans are given a chance to run as close as possible to the moment when the application is fully initialized and right before it's put into service. So, here, rather than squatting on the `main` method as we did in every earlier example, we simply register this bean. We can register as many as we want, for whatever purpose, and know that this logic will run as the service initializes.

Spring Data JDBC

We're using Spring Boot now. We've got an object mapper (not quite a *full* ORM, but close enough) framework on the CLASSPATH. Spring Data JDBC is capable, and it's already been configured. Let's rework our repository to leverage it, rather than hand-rolling SQL ourselves.

```
package com.example.beans_to_boot.boot;

import org.springframework.data.repository.ListCrudRepository;

import java.util.Collection;

interface DogRepository extends ListCrudRepository<Dog, Integer> {

    Collection<Dog> findByName(String name);

}
```

I know, you're thinking: "Didn't we get rid of the interface ages ago?" But fear not! The interface is back, but this is *way* better: we got rid of the implementation! This interface, which extends Spring Data's `ListCrudRepository`, already has a method called `findAll`. It also has methods to `save`, `delete`, `findById`, etc. Spring will create a proxy implementation of this interface at startup time and provide implementations for all those methods by default. You can even define custom finder methods. In this example, I've got a finder method called `findByName` that, when invoked, will execute a SQL query of the shape `select * from dog where name = ?`. It does this by convention. You can, of course, annotate the method in the interface with `@Query`, and Spring will use your query instead. *Much better!*

Conventions over Configuration

Spring Boot has a ton of convenient conventions. For example, it'll create a *real* `DataSource` backed by the HikariCP connection pool, with properties loaded by convention from the Spring `Environment`. Spring Boot will, by convention, load any `application.properties` or `application.yml` it finds in `src/main/resources`. So we no longer need our `@PropertySource` annotation.

AutoConfiguration

We imported the `DogBeanRegistrar`, so we might as well look at it.

```
package com.example.beans_to_boot.boot;

import org.springframework.beans.factory.BeanRegistrar;
import org.springframework.beans.factory.BeanRegistry;
import org.springframework.core.env.Environment;

class DogBeanRegistrar implements BeanRegistrar {

    @Override
    public void register(BeanRegistry registry, Environment env) {
        registry.registerBean(MyBeanFactoryPostProcessor.class);
        registry.registerBean(Listener.class);
    }

}
```

What happened? Where are all our beans?! They're gone. (Almost) all gone. Spring Boot provides those beans for us, thanks to its marvelous *autoconfiguration*. So all that remains are our beans.

When Spring Boot starts up, it scans the CLASSPATH for any `.jar` containing a file called `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. That file, in turn, contains a newline-delimited list of Java configuration classes' fully qualified names. Functionally, these classes are the same as the other Java configuration classes we've seen so far, with some small refinements. First, keep in mind that Spring Boot will *try* to load every single one of these configuration classes on startup. Sometimes the beans in these configuration classes are not important or useful in a particular situation. It would be grossly inefficient if Spring Boot registered beans that we didn't need. So, autoconfiguration classes and the `@Bean` methods within are guarded with `@Conditional` annotations. These annotations point to implementations of the `Condition` type. These `Condition` instances test something about the application or its environment, and if the test comes back positive, the beans are contributed to the `BeanFactory`. If not, they are not contributed.

There are many such conditions. Conditions to test for the presence or specific value of a property in the `Environment`. Conditions to see whether you and I, the users, have or have not already defined beans of a particular type. Conditions to see whether certain classes are present in the CLASSPATH. These conditions let Spring Boot avoid defining beans that aren't needed, and degrade gracefully. If we'd kept our `DriverManagerDataSource` from earlier, Spring Boot would've backed off on contributing its `DataSource`, because it would infer that we want to exert control here. In this way, we get the best of both worlds: Spring Boot has sensible defaults that drop away whenever we want to override something.

We can (and should) create our own autoconfiguration if we want to leverage the same convention-over-configuration experience that Spring Boot promotes.

Create an autoconfiguration class of your own. Be sure to put the code in a package that is *not* scanned by Spring's component scanning mechanism. In my application, my root package is `com.example.beans_to_boot`, so Spring will discover all classes in that package or deeper. So, I'll put the autoconfiguration in a separate, adjacent, package called `com.example.autoconfigure`, not because it's required but because it confirms that we've wired up the autoconfiguration correctly

and that this configuration class will be loaded and evaluated regardless of the fact that it's not discovered by the component scanning machinery.

```
package com.example.beans_to_boot.autoconfigure;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;

@AutoConfiguration ①
@ConditionalOnRandomness ②
class SillyAutoConfigurationExample {

    @Bean
③
    @ConditionalOnProperty(value = "my.config", matchIfMissing = true, havingValue =
"bar")
    ApplicationRunner myConditionalBean() {
        return _ -> IO.println("randomly initialized bean");
    }

}
```

1. `@AutoConfiguration` is yet another meta-annotation. You could use `@Configuration` here if you wanted.
2. `@ConditionalOnRandomness` is a conditional annotation that we'll write ourselves in just a moment.
3. `@ConditionalOnProperty` is a Spring Boot conditional annotation that will evaluate to true if the property called `my.config` exists in the `Environment`. We further stipulate that the value will be matched if it's not specified. If it is specified, and only if the value is `bar`, then this Java configuration bean will be registered. If this test evaluates to true, you'll see the output of our `ApplicationRunner` when the application starts up. If not, nothing will happen.

Let's look at the implementation of our (admittedly very contrived) condition.

```
package com.example.beans_to_boot.autoconfigure;

import org.springframework.context.annotation.Condition;
import org.springframework.context.annotation.ConditionContext;
import org.springframework.context.annotation.Conditional;
import org.springframework.core.type.AnnotatedTypeMetadata;

import java.lang.annotation.*;

①
@Target({ ElementType.TYPE, ElementType.METHOD })
@Retention(RetentionPolicy.RUNTIME)
```

```
@Documented
@Conditional(RandomnessCondition.class)
@interface ConditionalOnRandomness {

}

②
class RandomnessCondition implements Condition {

    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
        return Math.random() > .5;
    }

}
```

1. this annotation is a meta-annotation which we've annotated with Spring Framework's `@Conditional` annotation. We pointed it to an implementation of `Condition` that we've written called...
2. `RandomnessCondition`, which evaluates to true whenever the result of `Math.random()` is greater than `0.5`.

Now, create a text file in `src/main/resources/META-INF/spring/` called `org.springframework.boot.autoconfigure.AutoConfiguration.imports` in your project.

```
com.example.beans_to_boot.autoconfigure.SillyAutoConfigurationExample
```

Here, we've got just the one class. This is a bit of a toy, but if you restart the program a few times, you'll see your `ApplicationRunner` run (or not run) in seemingly random alternation.

We could have put this autoconfiguration class and the text file in a separate `.jar`, added that to the CLASSPATH of any Spring Boot application, and Spring Boot would give your code a chance to run *without* requiring input from the user at all. This is how you provide seamless, out-of-the-box, sensible defaults and configurations for your users.

But how do you distribute them? In a `.jar`, of course. But here again, Spring Boot's design philosophies pay dividends. Spring Boot uses something called *starter* dependencies. These are dependencies that serve only to aggregate other Maven dependencies and to manage their versions. When you work with a Spring Boot project, as generated from the Spring Initializr, you should only ever have to change one version - at most, four. All the other Spring dependencies for the rest of the codebase come from the Spring Boot parent artifact, which ensures that versions line up. If one dependency uses one version of Log4J, for example, and another uses a different one, Spring Boot will pick one and ensure it works for all the dependencies it supports.

Recall that Spring Boot has conditional annotations, one of which is `@ConditionalOnClass`. A bean so annotated will only be registered *if* the class is on the CLASSPATH. Spring Boot *starters activate* different conditionals because they bring in different cohorts of dependencies.

An Amazing Web Server

Spring Boot autoconfiguration does a whole heckuva lot more than register a `DataSource`. You might notice that when you start the program, it doesn't just exit as soon as the `Dog` entities have been enumerated. Spring Boot started a web server. Why? Because we asked it to by adding `Spring Web` to the project *way* back at the very beginning of this chapter. So we now have a full web framework and web server in play.

Here, I even wrote a trivial Spring MVC controller that loads all the results from the repository.

```
package com.example.beans_to_boot.boot;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Collection;

@Controller
@ResponseBody
class DogController {

    private final DogRepository repository;

    DogController(DogRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/dogs")
    Collection<Dog> dogs() {
        return this.repository.findAll();
    }

}
```

Run the program and then hit `http://localhost:8080/dogs` in your browser to see that it's working.

By the way, when you ran the program, did you notice the pretty colored log output? And the *amazing* ASCII art banner?



If you put a custom banner in `src/main/resources/banner.txt`, Spring Boot will use it instead of the default built-in ASCII art.

Remember when we first introduced `application.properties`? I didn't speak to some of the configuration values in the property file. Only those concerned with the `DataSource`. Let's revisit some of those.

```
spring.application.name=beans-to-boot
```

```
#
①
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.password=secret
spring.datasource.username=myuser
#
②
spring.threads.virtual.enabled=true
#
③
spring.sql.init.mode=always
④
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=*
management.tracing.sampling.probability=1.0
```

1. the original database credentials… moving on.
2. this property enables virtual threads, a game-changing feature in Java 21 or later. This will dramatically improve the scalability of blocking I/O.
3. this property tells Spring Boot to run `src/main/resources/schema.sql` and `src/main/resources/data.sql` on startup. Super convenient to ensure the system is correctly configured.

Observable Services

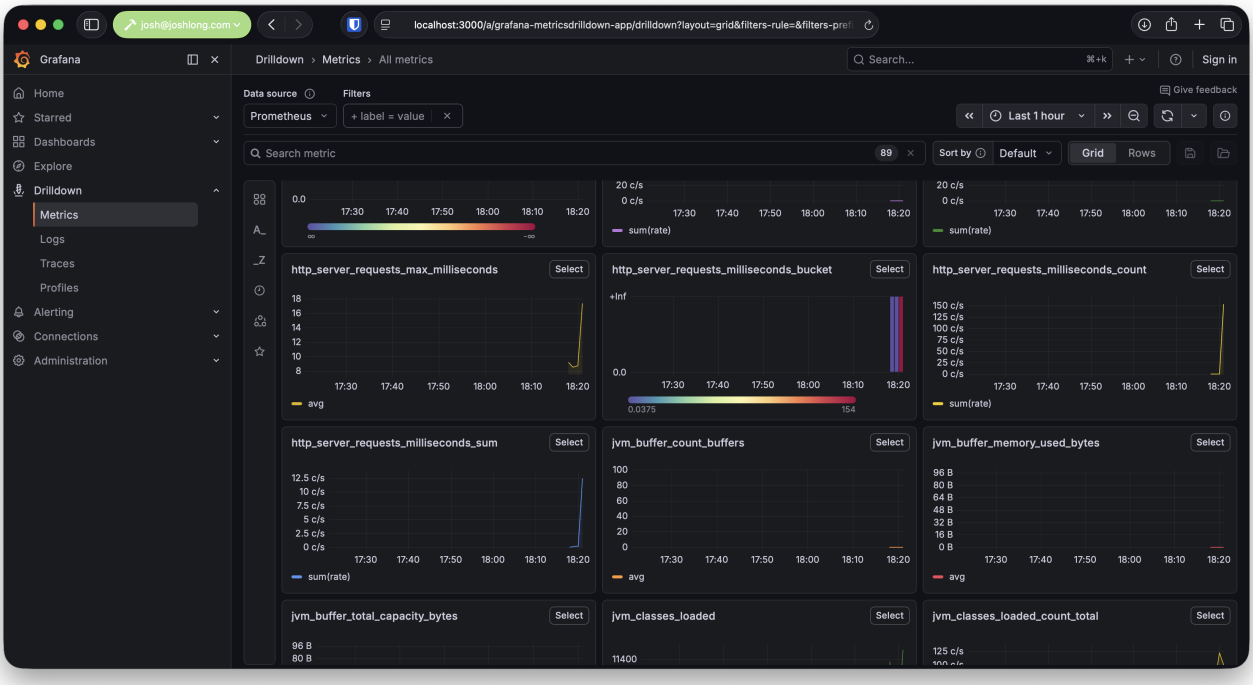
Spring Boot is production-ready and worthy out of the box. When you configured the application back on the Spring Initializr, you added the `Actuator` and `OpenTelemetry` dependencies. The Actuator dependency brings in a slew of endpoints, usually mounted under `/actuator/*`. There are many endpoints, but most of them are not visible unless you *expose* them with another property added to `application.properties`.

If you want to see all the Actuator endpoints and the full health endpoint, you might use the following configuration properties.

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Now you can visit `/actuator/metrics`, `/actuator/health`, `/actuator/beans`, `/actuator/configprops`, etc. Visit just `/actuator` to see all the sub-endpoints.

We've also got Grafana running in the background. As you work through the code in Spring Boot, you might notice that, first of all, you're seeing a good deal more application events. Second, if you visit http://localhost:3000 and then navigate to `Drilldown → Metrics`, you can see the same metrics you saw under `/actuator/metrics`, but now graphed over time. Neat!



GraalVM Native Images

This application runs really quickly and scalably, but could it be faster and use less RAM? Sure! One way you might help with that is to embrace GraalVM native images. GraalVM [https://graalvm.org] is an OpenJDK distribution that carries with it some amazing extras. You'll need to have it installed on your machine.

I'm interested in the `native-image` compiler tool that comes with GraalVM. It takes a Java program's `.class` files and turns them into native, operating-system- and architecture-specific machine code. The kind you'd get from using C, Go, or Rust. The result is a *markedly* smaller memory footprint and a *markedly* faster startup time, at the dubious expense of portability. The resulting binaries only run on the platform for which they were compiled. If you compile the binary in a Linux CI environment, it'll be fine in a Linux production environment, so it's not as big a deal-breaker as it might sound.



Keep in mind that GraalVM native images are *not* standard Java and in fact run counter in some ways to Java's write-once, run-anywhere mantra and promise of portability.

GraalVM was the first scenario where Spring Boot could leverage ahead-of-time code generation to transform the program into a form that would execute more efficiently at runtime. One of the AOT transformations that's just been made possible in Spring Boot 4 and Spring Framework 7 is compile-time, ahead-of-time code generation for Spring Data repositories like our Spring Data JDBC repository. For it to work, however, the AOT engine will need to know which SQL query dialect is being used *at compilation time*.

Add the following bean to help it along.

```
package com.example.beans_to_boot.boot;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.jdbc.core.dialect.JdbcPostgresDialect;

@Configuration
class DataAotConfiguration {

    @Bean
    JdbcPostgresDialect jdbcPostgresDialect() {
        return JdbcPostgresDialect.INSTANCE;
    }

}
```

You can take advantage of native-image technology thusly:

```
./mvnw -DskipTests -Pnative native:compile
```

Then, stand back. It might take a while. Maybe even minutes. On my machine, for this application, it takes about 30 seconds to do the full compile. Your mileage may vary.

Try it. Go to the `target` folder and run `target/beans-to-boot` or whatever your Maven artifact name was.

GraalVM can have *amazing* results, but it does require some compromises about which you need to be aware. The way GraalVM works is by starting from your `main` method entry point into the application, scanning every class that you use, and every class that those classes use, and every class that those classes use, and so forth, and onward until it's built up a giant graph of which classes your program will use. It retains those and throws every other class away before it compiles your program. It also throws away reflective metadata (you know, the types and instances that you get back when you try to do something like `Customer.class.getMethods()`). It also throws away the metadata for JNI references. And `Serialization`. Also, all the stuff that aren't `.class` files, but that ship in the `.jar` files for your application - things like `.properties` files, images, etc.? It throws all that away, too. Basically, all the stuff that the JVM normally would create and/or load on every startup, but that your program may never need: it throws it all away. The result is a *lightning* fast native image which uses a tiny fraction of the usual RAM.

But it's not perfect. The analysis that results in this much smaller set of classes happens during compilation. Java's a Turing-complete language. It's impossible to completely understand at compile time what will happen at runtime. So the GraalVM compiler might throw things out that you might actually end up needing during runtime. So you'll need to tell it what to keep. What *not* to throw out. Normally, in a GraalVM application, you do this with `.json` configuration files that live in `.jar` files under `/META-INF/native-image/${groupId}/${artifactId}/*.json`.

The trouble is that these configuration files are fiddly. If you refactor a type in your Java code, your IDE might forget to update the entry in the `.json` configuration file. So, you'll need to update it yourself. It gets tedious.

Spring Framework 6 and Spring Boot 3 introduced an Ahead-of-Time (AOT) component model that gets evaluated *during compilation*. Remember when we talked about the Spring `BeanFactory` and said that it handles *ingest* (the loading of bean configuration from various sources like component scanning, Java configuration, etc.), then turns everything into metamodel objects of the class `BeanDefinition`? Those `BeanDefinition` objects give any would-be examiner enough information about the objects in the application, including their classes.

Spring's AOT component model builds your application, to that point, giving you access to the `BeanDefinition` objects, and allowing you to programmatically register *hints*, which get translated into the aforementioned `.json` files.

We won't belabor the point, but there are different ways to register these AOT hints, depending on the level of power you need.

For most applications, users should probably get comfortable with `RuntimeHintsRegistrar`.

Let's suppose we wanted to read in a text file (`message`) and write out its contents to the command line when the program starts up. You could easily construct a `Resource` object and then read its contents in the JVM. But that text file won't exist in a GraalVM native image, because it'll have been thrown out. We need to tell GraalVM not to.

Here's the text file.

```
hello, AOT!
```

Now we need to register some code to contribute the AOT hints.

```
package com.example.beans_to_boot.boot;

import org.jspecify.annotations.Nullable;
import org.springframework.aot.hint.RuntimeHints;
import org.springframework.aot.hint.RuntimeHintsRegistrar;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ImportRuntimeHints;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import java.nio.charset.Charset;

@Configuration
①
@ImportRuntimeHints(AotConfiguration.Hints.class)
class AotConfiguration {

②
    private static final Resource MESSAGE = new ClassPathResource("/message");
```

```
③
    @Bean
    ApplicationRunner messageLoader() {
        return _ -> IO.println(MESSAGE.getContentAsString(Charset.defaultCharset()));
    }

    static class Hints implements RuntimeHintsRegistrar {

        @Override
        public void registerHints(RuntimeHints hints, @Nullable ClassLoader
classLoader) {
④
            hints.resources().registerResource(MESSAGE);
        }

    }

}
```

1. The `AotConfiguration` class is a stock-standard Java configuration class. The `@ImportRuntimeHints` annotation loads the AOT `RuntimeHintsRegistrar` class. This class will be invoked during compilation.
2. Spring's `Resource` abstraction is a nice way to represent anything that could be read (that is, it produces `byte`) or written to. There are many implementations. Here we're using the one to represent resources on the CLASSPATH.
3. the `RuntimeHints` object offers convenient DSL methods to register resources for loading, types for reflection, or serialization, etc.
4. By the time this `ApplicationRunner` executes at runtime, the `message` file *will* be in the memory of the native image and everything will continue to work, all because of the work we did at compile time to make it so.

Conceptually, that's all there is to it! There are other callback classes besides `RuntimeHintsRegistrar`, but that's a topic for discussion some other day. What's important to know is that you'll almost never need to worry about registering these AOT hints. Spring Boot will do it. Or there'll be configuration in the libraries that you consume. But not *always*. In which case, you'll have to do the work yourself. But at least you now know how.

Project Leyden

GraalVM native images are my favorite approach for optimization, but they do come with their own caveats. They don't always work without sometimes considerable finessing, and I won't get into all that here, but there is a whole Spring Ahead-of-Time (AOT) component model that allows you to add supplementary configuration to your Spring Boot applications so that they behave well in a GraalVM native-image context.

If you want a standard way to improve the startup time (and not much else) of your application, you can also use Project Leyden, part of the core Java SDK. It works reliably on *every* program, with

no compromises, but the results aren't quite as impressive, either.

The thrust of it is that you run the program in a special mode: the JVM dumps out information about which things are used into a file, and that file can then be fed into the `java` CLI on a subsequent run so it'll start up a lot faster. On my machine, this application is almost 50% faster on startup as a result of using Project Leyden. Here's an example of doing this training run and then getting great results.

```
#!/usr/bin/env bash
①
rm -rf extracted
rm -rf aot

②
mvn -DskipTests -Pnative  clean package
③
cp target/*jar application.jar
④
java -Djarmode=tools -jar application.jar extract --destination aot

cd aot

⑤
java -XX:AOTCacheOutput=app.aot   -Dspring.aot.enabled=true
-Dspring.context.exit=onRefresh -jar application.jar

⑥
rm -rf ../application.jar

⑦
java -XX:AOTCache=app.aot   -Dspring.aot.enabled=true -jar application.jar
```

1. clean up and staging
2. use Spring's AOT mode and do a package
3. move the resulting packaged `.jar` to a well-known name, `application.jar`
4. run the program using Spring Boot's `.jar` tools to extract the normally nested Spring Boot `.jar` into a folder called `autoconfigure`.
5. run the Java program up until the application context's been refreshed, and dump the analysis out to `app.aot`.
6. cleanup
7. finally, run the program but use the `app.aot` index to dramatically speed up the application.

The Spring Boot Buildpack integration

For most scheduling solutions, you'll need a container containing your application. This doesn't mean that you'll need a `Dockerfile`, however. Quite the contrary. You're better off automatically

building the container rather than manually defining it. In the world of mystery-meat cloud containers, *less is more*.

Spring Boot's Maven and Gradle plugins can do the work for you by deferring to the Buildpacks project [https://buildpacks.io]. Buildpacks provide recipes for taking application artifacts and packaging them up as containers. These recipes represent the collective wisdom of the Spring and Java communities that maintain them. There are also buildpacks for scores of other languages and ecosystems, including COBOL!

Here's how you can use it for your own application. If you want to build a container for a standard JVM-based application, do the following.

```
./mvnw -DskipTests spring-boot:build-image
```

If you want to build a container for a GraalVM native image, do the following:

```
./mvnw -DskipTests -Pnative spring-boot:build-image
```

Let this run for a bit, and it'll give you the name of the container that's just been created. You can run it or `docker push` it to whatever container registry you want. From there, you're off to production!

Spring Boot makes it easy to do the right thing. But how do you know you've *actually* done the right thing? That's what the next chapter, on testing, is all about.

Testing Spring Boot Applications

Spring makes it easy to both do the right thing and to focus on doing the right thing. But how do you know if you've actually done the right thing? How do you know if your changes resulted in outcomes you desired when you started writing the code in the first place? Testing, of course.

In this book, I'll sometimes use tests as ways to demonstrate features. Other times, I'll just give you code you can run and watch work. So, let's quickly take a look at the testing story for Spring and Spring Boot.

In a plain JUnit test, you could, of course, quite trivially stand-up Spring Boot and your whole application, obtain references to the beans, etc. This works, but it's sort of all-or-nothing. The real art comes in skillfully carving out that which is under test from that which isn't. That which is variable from the invariant. And Spring has amazing support here.

Spring has great integration with JUnit, which is the industry standard. Spring also supports Spock for the Groovy community and TestNG. But JUnit's the main thing. Use that, and things will be alright. One of the core Spring Framework engineers, Sam Brannen, is also a major contributor to the JUnit project. It's no coincidence then that Spring and JUnit have enjoyed a nice symbiotic relationship for decades.

The Core Spring TestContext Framework

The Spring TestContext Framework in Spring is a test-framework agnostic framework that provides lifecycle management, dependency injection, support classes, annotations, and more for working with Spring components in a test class. It works with JUnit Jupiter (which shipped starting in JUnit 5), JUnit 4, and TestNG. By default, when you configure a Spring Boot project, it configures a JUnit 5-based testing stack. You can switch to the others, but.. why would you?

In this book, we'll use JUnit 5 when we work with tests. The core Spring Framework support is massive. Broadly speaking, there are at least two interesting layers of support: unit-testing, and pseudo integration testing.

Unit Testing

I assume you understand the gist of unit testing. Spring even provides some *mock* implementations of key Spring Framework types like `Environment` (`org.springframework.mock.env.MockEnvironment`, and the related `org.springframework.mock.env.MockPropertySource`), a bevy of mock-ready Servlet types (see `org.springframework.mock.web`). These mock objects are for use with Spring's Web MVC framework and are generally more convenient to use than dynamic mock objects with mocking frameworks like EasyMock. There is even a slew of *utility* classes designed to construct interesting objects or answer interesting questions in the context of a test: `AopTestUtils`, `TestSocketUtils`, `ReflectionTestUtils`, `JdbcTestUtils`, etc. I won't dive too much into this stuff because it's of little import: for our application, we're probably going to do very little unit testing. Instead, we're going to be working with (secure) applications which will need to have several layers of concerns setup and working to do anything interesting. We're therefore far more interested in integration-testing.

Integration Testing

The support for integration testing in Spring is simple: I want to stand up a Spring `ApplicationContext` and inject the beans I've configured to ensure that they, and they alone, work as expected in the ensemble in which they're configured. This isn't plain unit-testing, nor is it full-bore integration testing. It's something in between.

Here's the simplest possible test using only the facilities in Spring Framework.

```
package com.example.beans_to_boot.springframework;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.env.Environment;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit.jupiter.SpringExtension;

①
@ExtendWith(SpringExtension.class)
②
@ContextConfiguration(classes = SpringTest.SpringTestConfiguration.class)
class SpringTest {

    final static String NAME = "Spring fans";

    @Test
    void test(@Autowired Announcer announcer) {
        Assertions.assertEquals(NAME, announcer.name());
    }

③
    @Configuration
    static class SpringTestConfiguration {

        @Bean
        Announcer announcer(Environment env) {
            return new Announcer(env, NAME);
        }

    }

}
```

1. this annotation comes from JUnit and allows customization of the default JUnit lifecycle and instances

2. this annotation tells the JUnit Spring support to load a Java configuration class and make available the beans therein.
3. and here, inside our test class (though this isn't required, it is convenient and portable), is a Java configuration class with but one bean `Announcer`.

The code for `Announcer` looks like this:

```
package com.example.beans_to_boot.boot;

import org.springframework.beans.factory.InitializingBean;
import org.springframework.core.env.Environment;
import org.springframework.util.Assert;

class Announcer implements InitializingBean {

    private final String name;

    private final Environment environment;

    Announcer(Environment environment, String name) {
        this.name = name;
        this.environment = environment;
        Assert.notNull(name, "Name must not be null");
        Assert.notNull(environment, "Environment must not be null");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("Hello, " + name + "!");
    }

    String name() {
        return name;
    }

}
```

It implements `InitializingBean`, which is a Spring lifecycle callback. In a unit-test, we'd have to call that method ourselves to ensure proper setup. Similarly, we can trust that the dependency on `Environment` has been satisfied by the time we start poking and prodding at the `Announcer` bean.

At this point there's a good deal more we could do. We could *delve* (yes, I know that's a telltale identifier of AI-written prose, but I didn't use AI to write it!) more deeply into the web framework support. Or the database support and the transaction support. Tons.

In the example above, we injected a *real* `Environment`.

Isolated Configuration

There's a tension we face when doing integration tests. We want things to execute in an environment as close to production as possible - otherwise why bother with integration testing at all? - but we also need to avoid complicating test setup with unnecessary dependencies. These extra dependencies add no clarity to our determination that some bit of code works or not while simultaneously creating undue burden on the people writing and maintaining the test. Mercifully, we're using a dependency injection framework. A framework follows the open-closed principle: it's open for extension, closed for modification. Put another way: you can change the way Spring behaves without recompiling or changing Spring's code. This works because you can swap out, conceptually, one bean for another at any time.

Spring already knows how to wire up your entire production application. Because of this, its testing support isn't about adding more configuration - it's about removing or carving that configuration up. You don't need to waste time teaching a testing framework how to mimic your production environment; Spring already has the blueprint. Instead, your job is to tell Spring how to run less of the application, swapping out or isolating the specific pieces you want to test from the ones you don't.

In this section, we'll look at ways to make this work trivial.

We'll demonstrate these concepts using a simple class, called `StoreService`, which is meant to be a service for, you know, a merchant. A merchant's store. Its storefront.

Here's the `StoreService`. It depends on a bean of type `java.time.Clock` to do its work.

```
package com.example.beans_to_boot.springframework;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Clock;
import java.time.LocalTime;

@Service
@Transactional
class StoreService {

    private final Clock clock;

    StoreService(Clock clock) {
        this.clock = clock;
    }

    boolean isOpen() {
        var hour = LocalTime.now(this.clock).getHour();
        return hour >= 9 && hour < 17;
    }
```

```
}
```

The `Clock` is configured in `StoreConfiguration`.

```
package com.example.beans_to_boot.springframework;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Clock;

@Configuration
class StoreConfiguration {

    @Bean
    StoreService storeService(Clock clock) {
        return new StoreService(clock);
    }

    @Bean
    Clock clock() {
        return Clock.systemDefaultZone();
    }

}
```

Now suppose I want to test that my logic works correctly. I can't run this using the production implementation of the `Clock` because, of course, that'll change as time goes by. I need to pin it down so that it's invariant relative to my test code.

TestBean

The first approach is a Spring Framework homegrown approach that might be enough in plenty of cases, and is my first go-to solution when I want to swap out a part of the bean graph. It requires no external dependencies besides Spring.

When you annotate a field with `@TestBean`, you're telling Spring that you'd like a chance to provide a *bean override* for beans of that particular type. In this instance, we provide the bean that overrides the existing one using a `static` factory method.

```
package com.example.beans_to_boot.springframework;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.bean.override.convention.TestBean;
```

```
import org.springframework.test.context.junit.jupiter.SpringExtension;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = StoreConfiguration.class)
class TestBeanTest {

①
    @TestBean(enforceOverride = true)
    Clock clock;

②
    static Clock clock() throws Exception {
        var tenAm = Instant.parse("2026-07-17T10:00:00Z");
        return Clock.fixed(tenAm, ZoneOffset.UTC);
    }

    @Test
    void businessIsOpenAt10Am(@Autowired StoreService service) {
        Assertions.assertTrue(service.isOpen());
    }

}
```

1. here we're telling Spring that we will provide an *override* for the missing bean and, further, that if there is no other bean to be overridden, then this should be treated as an error.
2. here's our `static` factory method which resolves the fixed `Clock` instance. Spring will invoke this.
3. finally: the test.

Mockito Beans

In the last example we provided our own instance of `Clock` to override the configured one. The instance we configured returned pre-programmed results on certain calls. For a more complex class with more methods, this would involve surgical overrides and finagling state in the parent class.

It could get ugly really quickly. An alternative is to use Mockito, a mocking library that allows us to define objects that *look* like our object, but whose responses (and behavior) we can provide to suit the purposes of our tests.

With a *mock*, it's up to us to provide implementations of all the methods we intend to invoke in the course of our test. No methods of the original class will be called. If you remember our section on creating proxies, then this sort of thing should be familiar.

We'll use `@MockitoBean` to instruct Spring to create a Mockito proxy for us for a given bean of a given

type. Then, before our tests run, we'll need to program the mock.

```
package com.example.beans_to_boot.springframework;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.context.junit.jupiter.SpringExtension;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneId;

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = StoreConfiguration.class)
class MockitoBeanTest {

①
    @MockitoBean(enforceOverride = true)
    Clock clock;

②
    @BeforeEach
    void before() throws Exception {
        var tenAm = Instant.parse("2026-07-17T10:00:00Z");
        Mockito.when(this.clock.instant()).thenReturn(tenAm);
        Mockito.when(this.clock.getZone()).thenReturn(ZoneId.of("Z"));
    }

③
    @Test
    void businessIsOpenAt10Am(@Autowired StoreService service) {
        Assertions.assertTrue(service.isOpen());
    }

}
```

1. instead of us providing a factory method, we'll tell Spring to wire up a `Mock` and replace the bean in the context with that Mock.
2. then we'll use Mockito's API to instruct the `Clock` on what to do.
3. and finally, we'll kick the whole chain of operations off. If we've done our work correctly, `isOpen` will result in a call to the methods whose responses we just provided. And the results will work no matter the date.

Mockito Spy Beans

A *mock* is an object whose behavior you can provide. A *spy* is a wrapper around a real object. Methods call the real implementation unless you stub them.

```
package com.example.beans_to_boot.springframework;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;
import org.springframework.test.context.junit.jupiter.SpringExtension;

import static org.assertj.core.api.AssertionsForClassTypes.assertThat;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.verify;

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = StoreConfiguration.class)
class MockitoSpyBeanTest {

①
    @MockitoSpyBean
    StoreService storeService;

    @Test
    void callsTheRealMethodByDefault() {

②
        var open = storeService.isOpen();

③
        verify(storeService).isOpen();
        assertThat(open).isIn(true, false);
    }

    @Test
    void canStubASingleMethodWhenYouWantTo() {
④
        given(storeService.isOpen()).willReturn(true);

⑤
        assertThat(storeService.isOpen()).isTrue();
    }

}
```

1. the real `isOpen()` actually executes against the real `Clock`
2. ...but it's a spy, so the invocation is still recorded

3. we'll override just this call
4. and then run the test as usual

A good test tests only that which is under test and isolates all the other invariants. These three annotations - `@TestBean`, `@MockitoBean`, and `@MockitoSpyBean` - make this trivial for common dependencies. Sometimes, however, you will need to isolate entire subsystems, like the web tier itself.

Testing services with `MockMvc`

`MockMvc` is Spring's tool for testing your web layer (controllers) without actually starting a real server or opening a real network socket. Here's the core idea. Normally when you test a Spring MVC controller end-to-end, you'd have to boot up Tomcat, bind to a port, and fire real HTTP requests at it (that's what `TestRestTemplate` or `WebTestClient` against a live port do). That works, but it's slow and it's more of an integration test. MockMvc instead spins up the Spring MVC machinery - the `DispatcherServlet`, your handler mappings, argument resolvers, message converters, validation, exception handlers, filters - in memory, and lets you send mock requests straight into that dispatcher. No socket, no port, no real HTTP. So what it actually does is give you a fluent way to say "pretend an HTTP request came in, and let me assert on what the controller layer did with it." Here's an example.

First, we'll introduce a trivial Spring MVC controller. We won't get too much into this right now, but you've seen a similar controller already.

```
package com.example.beans_to_boot.springframework;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Map;

@ResponseBody
@Controller
class SimpleController {

    @GetMapping("/greetings")
    Map<String, String> greetings(@RequestParam String name) {
        return Map.of("greeting", "Hello, " + name + "!");
    }

}
```

Now let's write a simple test that exercises this controller.

```
package com.example.beans_to_boot.springframework;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.request.MockMvcRequestBuilders;
import org.springframework.test.web.servlet.result.MockMvcResultMatchers;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

class SimpleControllerTest {

①
    private final MockMvc mockMvc = MockMvcBuilders.standaloneSetup(new
SimpleController()).build();

    @Test
    void greetings() throws Exception {
        this.mockMvc
            .perform(MockMvcRequestBuilders.get("/greetings").param("name", "Josh")
.accept(MediaType.APPLICATION_JSON))
            .andExpect(MockMvcResultMatchers.status().isOk())
            .andExpect(MockMvcResultMatchers.jsonPath("$.greeting").value("Hello,
Josh!"));
    }

}
```

1. first we tell Spring's `MockMvc` machinery to set up the web tier around this particular controller
2. we use the configured `MockMvc` and use it to make requests against the HTTP API and make assertions about the request and response.

Endpoint Testing with `RestTestClient`

`MockMvc` gives you a servlet-centric API for driving your controllers. `RestTestClient` targets those same server-side endpoints, but through a fluent, `RestClient`-style API - the same ergonomics as `WebTestClient`, which developers on the reactive stack will recognize. You can point it at a running server, at the whole `ApplicationContext`, or - as we do here - at a single controller, in which case there's no socket, port, or real HTTP involved at all. Let's try it out.

```
package com.example.beans_to_boot.springframework;

import org.junit.jupiter.api.Test;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.client.RestTestClient;

class SimpleClientTest {

    private final RestTestClient client = RestTestClient.bindToController(new
SimpleController()).build();
```

```
    @Test
    void greetings() {
        this.client.get()
            .uri("/greetings?name=Josh")
            .accept(MediaType.APPLICATION_JSON)
            .exchange()
            .expectStatus()
            .isOk()
            .expectBody()
            .jsonPath("$.greeting")
            .isEqualTo("Hello, Josh!");
    }

}
```

Here we bind the client directly to the `SimpleController` and use its fluent API to issue a request against the endpoint, then make assertions about the response. == Testing in Spring Boot

So far we've looked at isolated pieces in Spring Framework and seen how they layer one on top of the other. But a typical, secure application has a good many more moving parts, parts that are usually supplied by Spring Boot autoconfiguration. Spring Framework doesn't know about Spring Boot autoconfiguration. And we'd have to recreate all of it to sufficiently test things as to reproduce them in production. Instead, we'll use Spring Boot's testing support, which builds on the support introduced in the Spring Framework, to do this.



Spring Boot automatically pulls in a whole spate of interesting testing libraries, like Awaitility (for testing asynchronous systems), JSONPath (XPath for JSON), Mockito (a mocking library), AssertJ (for various assertions), Hamcrest (for still more assertions and matchers), and JUnit (the de facto standard testing framework).

Everything you've learned so far still applies, there's just *more* to the story. Let's start with a jack-of-all trades annotation, `@SpringBootTest`, which works every time, but might be more than you need in some cases. Add `@SpringBootTest` to a test class and it'll launch the production configuration and environment and then your tests will run with access to all the requisite beans. You can continue using `@TestBean` and `@MockitoBean` and `@MockitoSpyBean`, of course. Let's look at an example.

```
package com.example.beans_to_boot.boot;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BootifulTest {

①
```

```
    @Test
    void dogs(@Autowired DogRepository repository) {
        Assertions.assertEquals(3, repository.findAll().size());
    }

}
```

1. we have access to everything Spring Boot will have autoconfigured, including the `DogRepository`.

Good news! we can inject the *live* `DogRepository` and poke at it, just as if you were in your production code. Bad news: it's *live*! Don't forget to start your Docker image before running this test!

There are, of course, ways around this. If you visit the Spring Initializr and choose `Docker Compose` support and then choose, say PostgreSQL, you'll get a Docker Compose `compose.yml` file that looks like this.

```
services:
  postgres:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432'
```

this is fantastic during development because you can live that `git clone`-and-run lifestyle. Spring Boot will detect it and automatically run it *and* it'll automatically connect to it. No need to specify the connection details, e.g.: `spring.datasource.url`, `spring.datasource.username`, etc. Indeed, the `compose.yml` we've used for this chapter and the last one? I sourced it from the Spring Initializr and took the liberty of exposing the ports. The default generated on the Spring Initializr does not do this, because it doesn't need to.

But what about during development? Here you might want to use Testcontainers. Testcontainers are an amazing Java library for running Docker images from Java code. There's a core `Container` class which you can instantiate to launch any arbitrary Docker image. Some of these Docker images have supported configuration properties, environment variables, required mounts, etc. So the Testcontainers project has put together a library of specializations that take strongly typed parameters to help you construct a Docker image correctly. If you choose PostgreSQL and Testcontainers on the Spring Initializr, you'll get two new classes in your `src/test/java` folder:

- a `Test*Application.java` - you can run this class as you develop, usually in tandem with the Spring Boot Devtools, and you'll have your full production object graph *and* bean(s) for the applicable Testcontainer(s). You don't need to even define the requisite properties to connect to this new instance. Spring Boot will derive them for you and connect automatically. It'll do this by importing the next class…
- …which is `TestcontainersConfiguration.java`. This class defines the actual Testcontainers

objects.

It is this latter class that we'll use to automatically stand up a PostgreSQL database process. Here's our improved test.

```
package com.example.beans_to_boot.boot;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;

①
@Import(TestcontainersConfiguration.class)
@SpringBootTest
class BootifulTestcontainersTest {

    @Test
    void dogs(@Autowired DogRepository repository) {
        Assertions.assertEquals(3, repository.findAll().size());
    }

}
```

1. this pulls in our Testcontainers configuration

And here's the actual Testcontainers configuration class, defining a bean that we've identified as a `@ServiceConnection`.

```
package com.example.beans_to_boot.boot;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.postgresql.PostgreSQLContainer;
import org.testcontainers.utility.DockerImageName;

①
@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {

②
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer(DockerImageName.parse("postgres:latest"));
    }
```

```
}
```

1. this is a meta-annotation for `@Configuration`.
2. a `@ServiceConnection` is a bean that Spring Boot will identify and use to hydrate connection details automatically, obviating the need for us to specify, e.g., `spring.datasource.url`, `spring.datasource.username`, etc.

Shut down all your Docker images and then run this test, and it'll still work, because the service connection will supply the PostgreSQL instance and our SQL initialization logic (`schema.sql` and `data.sql`, defined earlier) will initialize the schema, all before we ever run a single test.

The test is green. Love to see it.

Spring Boot Test Slices

Spring Boot's got the ability to stand up your entire application and let you swap out individual components in the object graph. This works well, but it means that you're paying the cost of constructing the entire object graph each and every time the program starts. Spring Boot has the notion of *test slices*, which can activate only the objects you need to test a component in its particular subsystem - data, JSON, the web, security, etc.

In most cases, the user would simply add the relevant annotation from one of the dozens of supported test modules (`@JsonTest`, `@DataJdbcTest`, `@WebMvcTest`, etc.) to get the requisite autoconfiguration for just that subsystem. Then proceed as normal. All of these special annotations in turn are meta-annotations and use `@AutoConfigure*` variants and `@ImportAutoConfiguration`. You can usually test with just one main annotation, e.g, `@DataJdbcTest`.



You can't have more than one main annotation, but you can have one main annotation *and* one or more of the `@AutoConfigure*` variants.

Let's look at an example.

```
package com.example.beans_to_boot.boot;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.data.jdbc.test.autoconfigure.DataJdbcTest;
import org.springframework.context.annotation.Import;

@DataJdbcTest
class DataJdbcTestTest {

    @Test
    void dogs(@Autowired DogRepository repository) {
        Assertions.assertEquals(3, repository.findAll().size());
    }
```

```
}
```

the `@DataJdbcTest` starts an embedded SQL database for us. Recall that when we started this project we added `H2` to the specification on the Spring Initializr. Likewise, it runs all tests in a transaction that is automatically rolled back once the test is done, ensuring that the database state is consistent between tests. You get all this *while* writing less code and while executing a bit faster than when running the full Spring `ApplicationContext`. These gains pay off in a larger suite of tests.

All in all, Spring Boot's testing support is a crucial way to think about how to build software.

Next Steps

In this section we explored the awesome force-multiplying potential of Spring Boot and saw how it frees you to focus on that which matters: building the best, most valuable application possible for your customers.

At this point, you're grounded enough in the concepts that you at least know where to start looking when working with new features in a Spring Boot world. This will serve you well as we move up the abstraction stack in subsequent chapters. :code: ..

Spring Security Fundamentals

In this section we'll look at the fundamentals of Spring Security. After all, we won't get far without those core concepts in place.

Spring Security, the Project

Spring Security began in late 2003 when Ben Alex created what was originally called Acegi Security (or the Acegi Security System for Spring) after discussions on the early Spring developer mailing lists about the need for a comprehensive security framework. At the time, the Spring Framework lacked a robust security architecture, leaving developers to build custom, application-specific authentication and authorization solutions. Acegi was designed from the ground up to leverage Spring's core principles of dependency injection and aspect-oriented programming, providing a configurable, pluggable security architecture built around a flexible servlet `FilterChain` that allowed developers to bypass rigid container-managed security entirely. It quickly gained traction by handling both robust authentication and complex authorization directly within the Spring application context.

Ben released Acegi as an open source project under the Apache License in March 2004. The project rapidly attracted a small but enthusiastic community and gained traction within the Java ecosystem. By 2006, Acegi had matured into a production-ready framework, and in 2007 it was officially adopted into the Spring portfolio as an official Spring subproject.

This era, for me, was where I started to really get excited about Spring. It's where I decided I wanted to be a part of the story. I'd used Spring Framework and all of its wonderful little `Template` classes and its web framework and dependency injection framework and all that in my various applications, but 2006 is where I started to appreciate the big-picture potential: in knowing the Spring component model, I had purchase in all the other domains that Spring would take me to. Spring Security meant that I was already halfway towards having secure applications and services. Spring Batch, also released around this time, let me learn the domain of batch processing in terms of the familiar idioms in Spring. I was already halfway there! Spring Integration, also released around this time, let me learn the domain of enterprise application integration, messaging, and event-driven architecture in terms of the familiar idioms of Spring. I was already halfway there. To be clear, there were alternatives for these particular use cases, but none of them integrated so deeply with Spring and its guiding philosophies. I'd have to learn a whole different paradigm, ecosystem, and maybe even language and runtime to get any footing in those alternatives. Spring opened doors and let me cut to the heart of the matter.

Later that year, Acegi was rebranded as *Spring Security*, with Ben Alex handing the leadership reins over to Luke Taylor to guide the framework into its next major chapter: the release of Spring Security 2.0 in April 2008.

The transition into the Spring portfolio brought even tighter integration with the Spring ecosystem and continued refinement of the framework's architecture. Under this new era of stewardship, Spring Security 2.0 introduced namespace-based XML configuration, *dramatically* reducing the boilerplate that had made earlier Acegi configurations notoriously verbose. I can't stress this enough. Spring Framework's main configuration mechanism *used* to be XML, and in 1.0, the only way to wire up objects was to define instances of every bean required for a given application, no

matter how many were required. There was no mechanism for composition. No way to say, *I want all **these** beans in **that** namespace, now*. We'd have to type them out, one by one, and configure them all, one by one. As you can imagine, in the domain of security, there are *many* moving parts, and every wayward developer had to understand all of them to get even the simplest application up and running. You won't have to deal with that in this book, of course. We've had nearly two decades of Java configuration, and more than two decades since XML namespaces and schemas were introduced. I just wanted to register that it used to be unwieldy. I don't know if writing a book is cheaper than therapy, but it's definitely more validating.

The framework evolved well beyond basic authentication and authorization, cementing a flexible architecture centered around servlet filters, authentication providers, method security, and deep integration with the Spring Framework.

As stewardship expanded to a group of maintainers - including Luke Taylor, Rob Winch, and a growing community of contributors - Spring Security continued to modernize alongside both the Java ecosystem and the changing web. At this point, no single engineer has done more to improve Spring Security than its current lead Rob Winch, about whom you'll no doubt hear more from me in the course of this book.

The 3.x generation introduced features such as *Spring Expression Language* (SpEL)-based access control, improved LDAP integration, OpenID support, and built-in protection against common web vulnerabilities like Cross-Site Request Forgery (CSRF). Spring Security 3.2 and 4.x shifted the preferred programming model away from verbose XML toward Java-based, type-safe configuration and fluent APIs.

The framework's evolution accelerated as application architectures shifted toward cloud-native systems and distributed services. Spring Security 5 introduced first-class support for OAuth 2.0, OpenID Connect, reactive applications built with Spring WebFlux, and a modernized password encoding strategy. Spring Security 6, aligned with Spring Framework 6 and Java 17+, further refined its Lambda-based DSL, strengthened secure-by-default behavior, retired legacy APIs, introduced native support for modern WebAuthn passkeys, and fully embraced modern Java and Spring practices. Spring Security 7 introduced multifactor authentication, pulled in the Spring Authorization Server and Spring Security Kerberos projects, unifying them with the rest of the Spring Security efforts.

Today, Spring Security is a highly modular, extensible framework that secures everything from traditional servlet-based applications to reactive services and OAuth 2.0 Authorization Servers. It provides first-class support for JWTs, decentralized identity, modern authorization models, and deep integration with Spring Boot and the broader Spring ecosystem. Despite more than two decades of evolution, it remains faithful to Ben Alex's original vision: providing a powerful, reusable, and extensible security framework that integrates naturally with Spring applications.

With that high-level overview out of the way, let's talk specifics.

Spring Security for all Contexts

Spring Security has a good story for securing all manner of different workloads. Most of this book assumes that you're building HTTP-based clients and services, but part of the reason I wanted to write this book is precisely because it can do *so much more*! You can use it to secure HTTP clients

and services, MCP clients and services, command line interface (CLI) clients, back-office messaging code, and so much more.

Authentication

Authentication answers the question: *who is this request from*? What's their name? Do we know about them? And with that information, you can answer other more interesting questions like *do they work for our organization?* or *are they a customer with a paid subscription in good standing*? Identity is everything. There's no continuity of service if there are no users for whom to ensure continuity. There are no users if there is no identity.

Authentication works by *challenging* the user for proof of some sort of *factor*, the demonstration of which confirms the identity of the user. A password is a widespread example of this. Anybody can claim to be Rob Winch, lead of Spring Security, but if they don't have his supersecret, well-guarded password, they're not logging into his email or gaining access to his systems. A password demonstrates that the user has knowledge that presumably only they should have.

There are many other factors. One-time tokens, passkeys, and certificates are all examples of authentication factors.

A well-secured system will use multiple factors for authentication, sometimes called multifactor auth or *MFA*. When should you choose which factor? Authentication factors usually fall into three buckets:

- **knowledge** - something you know. Examples are passwords, PINs, passphrases, security questions, etc. Some weaknesses of this factor are that it can be guessed, phished, reused, leaked, or written down.
- **possession** - something you have. Examples of this are a phone, a hardware security key, a smart card, an authenticator app, a passkey stored on your device, etc. Some weaknesses of this factor are that the device or token can be lost, stolen, cloned in a weak system, or intercepted if it relies on text messages (SMS).
- **inherence** - something you are. Examples of this are fingerprints, face scans, iris scans, voiceprints, etc. Some weaknesses of this factor are that biometrics can be spoofed, mishandled, or impossible to *rotate* if compromised.

Sometimes the demonstrated ability of the user to authenticate in more than one way is, itself, a factor. A user might have access to most of a given system if they present *just* a single factor, like a password, but might have access to more sensitive facets of the system if they present multiple factors.

Authentication in Spring Security

In Spring Security, the core concept of authentication is represented by an object named, unsurprisingly, `Authentication`. Spring Security features an `AuthenticationManager` which in turn delegates to a chain of `AuthenticationProvider` types. There are *dozens* of these included in Spring Security, many of them having specific knowledge of how to authenticate a given user in terms of a given specification or protocol or scenario.

Here's the definition of `AuthenticationManager` in Spring Security.

```
package org.springframework.security.authentication;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;

@FunctionalInterface
public interface AuthenticationManager {

    Authentication authenticate(Authentication authentication) throws
AuthenticationException;
}
```

The most common implementation of `AuthenticationManager` is `ProviderManager`, which as I say manages a chain of `AuthenticationProvider` instances.

Here's the definition of `AuthenticationProvider` in Spring Security.

```
package org.springframework.security.authentication;

import org.jspecify.annotations.Nullable;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;

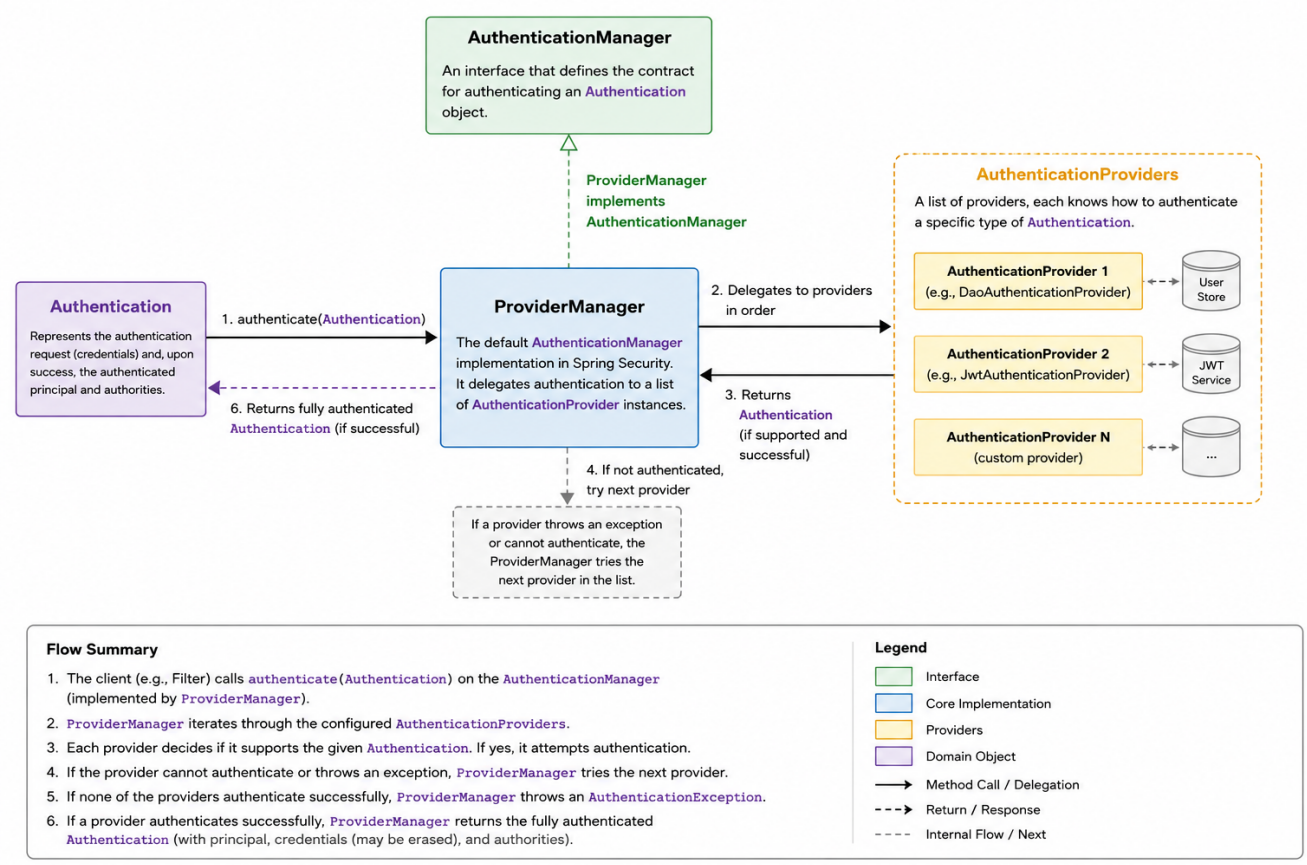
public interface AuthenticationProvider {

    @Nullable Authentication authenticate(Authentication authentication) throws
AuthenticationException;

    boolean supports(Class<?> authentication);
}
```

One of the most commonly used `AuthenticationProvider` types is the `DaoAuthenticationProvider`, which ties authentication to users stored in some implementation of `UserDetailsService`. `UserDetailsService` in turn assumes you can load the entire user, including its (encoded and hopefully not plaintext) password from some backing store, like a SQL database, or in-memory.

It sounds like a lot, but keep in mind that you don't have to interact with most of this stuff almost ever. Here's a high-level diagram to make clear the relationship between these types.



1. here's a high-level overview of the flow of authentication requests

And now let's actually pull these pieces together in a simple test. We'll construct these objects ourselves, using Spring Security directly.

```
package com.example.security;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.ProviderManager;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

import java.util.List;

@SpringBootTest
class SecurityApplicationTests {

    @Test
    void authentication() throws Exception {
```

```
①
        var pw = PasswordEncoderFactories.createDelegatingPasswordEncoder();

②
        var josh = User.withUsername("josh").password(pw.encode("pw")).roles("USER")
.build();
        var rob = User.withUsername("rob").password(pw.encode("pw")).roles("USER",
"ADMIN").build();
        var users = List.of(josh, rob);
        var userDetailsService = new InMemoryUserDetailsManager(users);

③
        var daoAuthenticationProvider = new DaoAuthenticationProvider
(userDetailsService);
        daoAuthenticationProvider.afterPropertiesSet();

④
        var authenticationProviders = List.of((AuthenticationProvider)
daoAuthenticationProvider);
        var authenticationManager = new ProviderManager(authenticationProviders);
        authenticationManager.afterPropertiesSet();

⑤
        var authentication = new UsernamePasswordAuthenticationToken("josh", "pw");
        Assertions.assertFalse(authentication.isAuthenticated(), "the user is not
authenticated");
        var authenticated = authenticationManager.authenticate(authentication);
        Assertions.assertTrue(authenticated.isAuthenticated(), "the user is
authenticated");
    }

}
```

1. you don't store passwords in plaintext, do you? Of course not! You need to encode it. And Spring Security has a nifty composite implementation of the `PasswordEncoder` hierarchy that I try to use for basically everything.
2. we're going to store users *in-memory*. Obviously, you could source these users from a SQL database. Any implementation of `UserDetailsService` will work. We're just going to hardcode our users for this example. (Which does make the act of encoding the passwords seem a bit silly, doesn't it?)
3. We'll wire up a lonesome `AuthenticationProvider` of type `DaoAuthenticationProvider` which expects a reference to the aforementioned `UserDetailsService`.
4. We'll wire up an implementation of `AuthenticationManager` called `ProviderManager` to handle any and all `AuthenticationProvider` instances. `ProviderManager` will give incoming `Authentication` requests to all configured instances of `AuthenticationProvider`. If any of them can handle it, then processing stops. If not, they either throw an exception or return an `Authentication` with `authenticated == false`.
5. finally, the rubber meets the road. Imagine we're an HTTP client using an HTTP form or an

HTTP basic request. Spring Security's filter would intercept those requests, wrap them into a `UsernamePasswordAuthenticationToken` instance, and then ask the `AuthenticationManager` to try to authenticate it. Here, we see that it does so successfully.

Filters

Now that you know, conceptually, about how to *do authentication*, you'll hopefully appreciate that the hardest part about security is getting your code in the right place to do authentication in the first place. In a web application, Spring Security installs a filter that intercepts requests bound for your application, and it is there that code of a kind with the code in that test actually runs.

Filters are a natural way to implement this sort of gatekeeper logic on your application. Spring Security uses filters all over the place to ensure that it is able to put up certain pages for certain use cases (like login pages). These filters aren't routes in your application, they are seen *before* your application gets a chance to respond. Filters have ordering relative to each other. Some filters run earlier, some run later. If you want to override the behavior of one of Spring Security's filters, you could in the worst case put another filter earlier in the chain and duplicate or vary the code in the configured instance. You can leverage this mechanism too. Spring has two modalities: reactive and non-reactive. In this book I'm assuming we're using the non-reactive stack. When you bring in Spring Boot's `spring-boot-starter-webmvc` starter, it configures Apache Tomcat by default. You could optionally also use Jetty. Spring Boot provides a natural way to contribute filters, side-stepping the tedious configuration required when interacting with the lower-level Servlet engine.

Here's an example.

```
package com.example.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpFilter;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;

import java.io.IOException;

@Configuration
class CustomFilterConfiguration {

①
    @Bean
    @Order(Ordered.LOWEST_PRECEDENCE)
    FilterRegistrationBean<MyFilter> myFilterFilterRegistrationBean() {
```

```
        var myFilter = new MyFilter();
        var frb = new FilterRegistrationBean<>(myFilter);
        frb.addUrlPatterns("/test", "/test/*");
        frb.setAsyncSupported(true);
        return frb;
    }

②
    static class MyFilter extends HttpFilter {

        @Override
        protected void doFilter(HttpServletRequest request, HttpServletResponse
response, FilterChain chain)
                throws IOException, ServletException {

            IO.println("before " + request.getRequestURL());
③
            if (request.getUserPrincipal() == null) {
④
                chain.doFilter(request, response);
            } //
            else {
                response.setContentType(MediaType.TEXT_HTML_VALUE);
                response.setStatus(HttpStatus.OK.value());
                var userPrincipal = request.getUserPrincipal();
                response.getWriter().write("""
                        <html>
                         <body>
                            <h1>
                             Hello, %s!
                            </h1>
                         </body>
                        </html>
                        """.formatted(userPrincipal.getName()));
            }
            IO.println("after " + request.getRequestURL());
        }

    }

}
```

1. You can register beans of type `FilterRegistrationBean` and they'll let you wire up all the same things you might've configured when using `web.xml` or programmatic `Servlet` registration. We can specify in which order the filter is to be processed. I want this to be processed dead-last, *after* all the other filters have run. Here, we're specifying the URL patterns to which to map our `Filter` and whether the `Filter` supports the *async* processing introduced in Servlet 3.0, in 2009.
2. I use the word *filter* to refer to a conceptual thing, but in the Servlet API there are at least two interesting classes to be aware of: `Filter` and `HttpFilter`. We're interested in `HttpFilter` here

because we want to intercept HTTP requests, not just generic unspecified requests bound for the server.

3. Spring Security installs a filter *before* our filter, and one of the things it does is *wrap* the raw native `HttpServletRequest` that comes from the native Servlet container in a new class that provides answers to `getUserPrincipal()` by sourcing the information from the Spring Security authenticated principal.
4. We *should* be last, and the request *should* be secured, but if for some reason there's no authenticated principal in the request, then let's just continue the filter chain, letting somebody else have a chance at resolving this. There's nothing we can do.

Where does that user come from? Who put it there? Behind the scenes, when Spring Security authenticates a request, it creates an object of type `Authentication` and stores it in a place that the current ongoing request can get to it. Often, but not always, this is backed by a `ThreadLocal<?>`. You can access the current authenticated user with `SecurityContextHolder`:

```
var authentication = SecurityContextHolder
                        .getContextHolderStrategy()
                        .getContext()
                        .getAuthentication();
var authenticationName = authentication.getName();
```

The name returned here and the name returned from `getUserPrincipal` in the example above should be the same, pointing to the same reference.

Spring Session

Spring Session provides an API and a generic framework for managing a user's session state. It introduces an abstraction layer between the web application and the backing session storage. By default, traditional web applications rely on the Servlet container (such as Apache Tomcat or Jetty) to hold user sessions in memory, binding each session to the individual node that created it.

What happens if that node is destroyed? What happens to the state? Many (but not nearly all!) web servers support session replication, in which state is copied from one node to another so that, no matter which node answers a request, the client experiences continuity of state. But replication is off by default, and in multi-node, auto-scaling, or microservices deployments, in-memory session management causes requests to fail when they're routed to an instance without the local session data — unless you resort to complex sticky-session load balancing.

Spring Session solves this with a shared session storage model. Every node in the cluster connects to a single external backing service (such as MongoDB, Redis, a relational database, InfiniSpan, Caffeine, etc.), so any node can read, update, or invalidate any active session, regardless of where the request is routed. It works by installing a `Filter` that replaces the native container `HttpSession` with a Spring Session wrapper, then forwarding writes to that external store. Your code keeps using the standard `HttpServletRequest` and `HttpSession` interfaces, so the migration is completely transparent.

This is a big improvement over the status quo. Just because your Servlet container *can* replicate sessions doesn't mean it will do a better job than something like Redis or GemFire, both of which were purpose-built for exactly this sort of replication. Statistically, it almost never will. Remember: the people who wrote those blazing-fast HTTP servers aren't the same people who wrote the blazing-fast distributed databases. Session replication is even a premium, paid feature in some containers, so being able to plug in a better solution at no cost is a real win.

Spring Session also lets you customize how session identifiers are read from and written to HTTP requests and responses. It ships with resolvers like `HeaderHttpSessionIdResolver`, which tracks sessions using a custom HTTP header (such as `X-Auth-Token`) instead of a cookie. That makes Spring Session well-suited to stateless RESTful microservices and single-page applications that don't rely on browser cookies.

Meet the Persistence Backends

Redis is one of the primary and most commonly used backends supported by Spring Session. It provides fast, in-memory key-value persistence with scalable pub/sub capabilities. Spring Session handles the complex Redis mechanics for you, including serializing session attributes (JSON or Java serialization), managing session expiration through Redis TTL keys, and listening to Redis keyspace notifications to propagate session expiration and deletion events across nodes.

There are also community modules that integrate alternative caching and database technologies such as Infinispan [https://infinispan.org/docs/stable/titles/spring/spring.html], GemFire [https://techdocs.broadcom.com/us/en/vmware-tanzu/data-solutions/spring-session-for-tanzu-gemfire/2-0/gf-ss/index.html], and Caffeine [https://github.com/gotson/spring-session-caffeine], letting teams choose the technology that aligns best with their existing infrastructure.

Setting Up Spring Session JDBC

To keep things simple while still getting most of the benefits, we'll use Spring Session JDBC, which is ideal for applications that prefer relational storage without standing up additional infrastructure like Redis. It includes mechanisms to query active sessions, clean up expired ones, and index sessions by attribute keys (such as username). You can customize key properties such as the maximum inactive interval (session timeout), the session table and key-namespace prefixes, the cleanup cron schedule, and the cookie or header strategy.



> Spring Session works with both Servlet-based and reactive, non-blocking web servers. In this book, we'll focus exclusively on the blocking, synchronous Servlet-based approach. Ensure that you're using virtual threads by setting `spring.threads.virtual.enabled=true` in your `application.properties` or `application.yml` file!

Let's build a new stateful web application. Go to the Spring Initializr, choose `Spring Web`, `Spring Session`, `JDBC API`, `Spring Security`, `PostgreSQL Driver`, `Thymeleaf`, `WebSocket`, and `Flyway Migration`. Click `Generate` and open the resulting `.zip` file in your favorite IDE.

First things first, we need to configure our SQL datasource connection. Open up `application.properties` and make it look like this.

```
spring.application.name=spring-session
spring.datasource.password=secret
spring.datasource.username=myuser
spring.datasource.url=jdbc:postgresql://localhost:5432/mydatabase
spring.threads.virtual.enabled=true
server.servlet.session.timeout=3m
```

Next, we'll need to initialize the requisite SQL schema for Spring Session JDBC to do its work. Create `src/main/resources/db/migration/V01__session.sql`:

```
-- sessions
create table if not exists spring_session
(
    primary_id            character(36) primary key not null,
    session_id            character(36)             not null,
    creation_time         bigint                    not null,
    last_access_time      bigint                    not null,
    max_inactive_interval integer                   not null,
    expiry_time           bigint                    not null,
    principal_name        character varying(100)
);
create unique index if not exists spring_session_ix1 on spring_session using btree
(session_id);
create index if not exists spring_session_ix2 on spring_session using btree
(expiry_time);
create index if not exists spring_session_ix3 on spring_session using btree
```

```
(principal_name);

-- session attributes
create table if not exists spring_session_attributes
(
    session_primary_id character(36)          not null,
    attribute_name     character varying(200) not null,
    attribute_bytes    bytea                  not null,
    primary key (session_primary_id, attribute_name),
    foreign key (session_primary_id) references spring_session (primary_id)
        match simple on update no action on delete cascade
);
```

We're going to see Spring Security in a bit. But for now, we need to disable it since the default autoconfiguration will lock the application down.

```
package com.example.spring_session;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
class SecurityConfiguration {

①
    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

②
    @Bean
    InMemoryUserDetailsManager userDetailsManager(PasswordEncoder pw) {
        var josh = User //
            .withUsername("josh@joshlong.com") //
            .password(pw.encode("pw")) //
            .roles("USER") //
            .build();
        var rob = User //
            .withUsername("rob@springsecurity.site") //
            .password(pw.encode("pw"))//
            .roles("USER", "ADMIN")//
            .build();
```

```
        return new InMemoryUserDetailsManager(rob, josh);
    }

③
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity security) {
        return security.formLogin(Customizer.withDefaults())//
            .authorizeHttpRequests(a -> a //
                .requestMatchers("/sessions", "/secure-increment")
                .authenticated() //
                .anyRequest()
                .permitAll()//
            )
            .build();
    }

}
```

1. define a common-sense `PasswordEncoder`
2. install two users, `josh@joshlong.com` and `rob@springsecurity.site`, both of whom have the password `pw`.
3. allow all requests *except* those to `/sessions` and `/secure-increment`, which require authentication. We will look at the `/secure-increment` and `/sessions` endpoints later.

Now, finally, we'll need to demonstrate that Spring Session is doing its job. We're going to create a controller that increments a number stored in the HTTP session. If the value doesn't exist, it initializes it to `0`, then increments it.

```
package com.example.spring_session;

import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Map;

@Controller
@ResponseBody
class IncrementingController {

    @GetMapping("/increment")
    Map<String, Integer> increment(HttpSession session) {
①
        var existingValue = (Integer) session.getAttribute("count");
        if (existingValue == null) {
            session.setAttribute("count", 0);
        }
        var count = (Integer) session.getAttribute("count");
```

```
②
        session.setAttribute("count", count + 1);
        return Map.of("count", count);
    }

}
```

1. did you know you can inject the current `HttpServletRequest` or `HttpSession` associated with the current `HttpServletRequest` into Spring MVC controller handler methods?
2. finally, we write a value to the `HttpSession`.

Run the program and visit `localhost:8080/increment` in a browser a few times. You should see the number incrementing as you go. Inspect your SQL database. You should see the Flyway table and two tables for Spring Session. Restart the JVM. Visit the same page in the same browser tab, and the value should keep incrementing.

The HTTP session survived the restart, and the session would just as easily survive load balancing and rebalancing.

Spring Security Integration

Spring Session integrates tightly with Spring Security to unify authentication state and session management across distributed systems. Its features include finding a user's active sessions by principal/username across the cluster, enforcing a maximum number of concurrent sessions per user across multiple nodes, and ensuring sessions are invalidated on logout or other security events.

Recall that we've already declared some authorization rules for certain paths: `/secure-increment` and `/sessions`.

Let's define the controller for `/secure-increment`.

```
package com.example.spring_session;

import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class SecureIncrementingController {

    @GetMapping("/secure-increment")
    Map<String, Object> increment(HttpSession session, Principal principal) {
```

```
①
        var existingValue = (Integer) session.getAttribute("secureCount");
        if (existingValue == null) {
            session.setAttribute("secureCount", 0);
        }
        var count = (Integer) session.getAttribute("secureCount");
②
        session.setAttribute("secureCount", count + 1);
        return Map.of("count", count, "name", principal.getName());
    }

}
```

This controller is basically the same as before, except now we're also injecting the authenticated principal and incorporating it into the response. By the time this method is invoked, we're already authenticated; if we weren't, Spring Security would force us to a login page. Inspect the SQL tables and you'll see that Spring Session has captured the `Principal#getName()` in the `principal_name` column of the `spring_session` table.

Now, an amazing possibility arises: we can list all the given HTTP sessions tied to a particular user! Imagine a user logs in using Chrome, then later logs in from another Firefox. In both cases, it's the same user, but they've got unique HTTP sessions on the server.

We can list all of those sessions now. You could just read the `spring_session` table using JDBC. But Spring Session provides an even more convenient way to do it: the `SpringSessionBackedSessionRegistry`. Define an instance.

```
package com.example.spring_session;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.session.FindByIndexNameSessionRepository;
import org.springframework.session.Session;
import org.springframework.session.security.SpringSessionBackedSessionRegistry;

@Configuration
class SpringSecuritySessionConfiguration {

①
    @Bean
    <S extends Session> SpringSessionBackedSessionRegistry<S> sessionRegistry(
            FindByIndexNameSessionRepository<S> sessionRepository) {
        return new SpringSessionBackedSessionRegistry<>(sessionRepository);
    }

}
```

1. define a Spring Security `SessionRegistry` backed by Spring Session.

Now, let's define a controller in which we list all the HTTP sessions tied to a given user.

```
package com.example.spring_session;

import org.springframework.security.core.session.SessionRegistry;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class SessionsController {

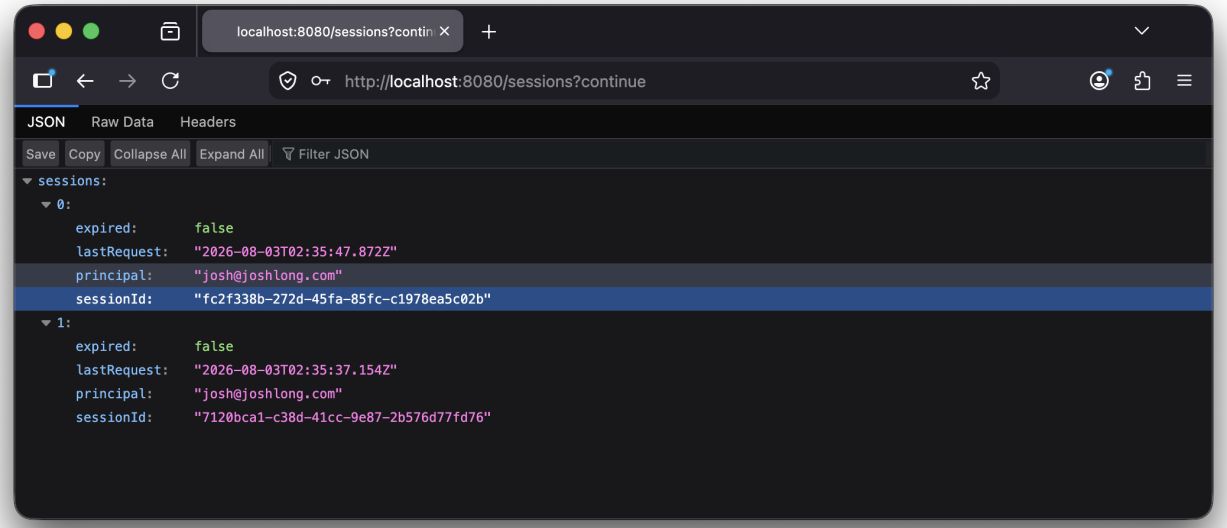
    private final SessionRegistry registry;

    SessionsController(SessionRegistry registry) {
        this.registry = registry;
    }

    @GetMapping("/sessions")
    Map<String, Object> sessions(Principal principal) {
        return Map.of("sessions", this.registry.getAllSessions(principal, false));
    }

}
```

Visit `localhost:8080/sessions`. You'll be prompted to authenticate and then shown all the sessions for the user `josh@joshlong.com`. Visit the same page in another browser, or in an incognito window in your current browser, or in a different profile in your current browser. And you should now see more than one session.



Spring Session knows about your logged-in sessions. Which means you could, in theory, use it to control how many sessions you're allowed.

Spring Security has just such a feature! Let's try it out.

First, we need to customize Spring Security to enable the session management features.

```
package com.example.spring_session;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.session.SessionRegistry;

@Configuration
class SpringSecuritySessionsConfiguration {

    @Bean
    Customizer<HttpSecurity> securitySessionsCustomizer(SessionRegistry
sessionRegistry) {
        return http -> http //
            .sessionManagement(session -> session//
                .maximumSessions(2) ①
                .sessionRegistry(sessionRegistry) ②
            ); //
    }

}
```

1. ensure that there are no more than two sessions for a given user
2. point Spring Security to the `SessionRegistry` instance we defined earlier that's backed by Spring

Session.

Now, if you attempt the same experiment we performed earlier, you'll see that we cannot create more than two sessions. You might be able to authenticate more than two times, but go back through the other instances and refresh, and you'll find you've been logged out. You're only allowed two instances.

Keeping Sessions Alive over WebSockets

If a user interacts only through a WebSocket, their HTTP session might expire due to inactivity. Spring Session provides built-in WebSocket integration that intercepts WebSocket messages to keep the underlying `HttpSession` active, keeping the persistent store and the active real-time connection in sync.

Let's look at a simple example.

```
package com.example.spring_session;

import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.handler.annotation.SendTo;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import java.security.Principal;

@Controller
class ChatController {

①
    @GetMapping("/chat")
    String chatPage(Principal principal, Model model) {
        model.addAttribute("username", principal.getName());
        return "chat";
    }

②
    @MessageMapping("/send")
    @SendTo("/topic/public")
    ChatMessage send(@Payload ChatMessage message, Principal principal) {
        return new ChatMessage(principal.getName(), message.content());
    }

}
```

1. this controller handler method is for Spring MVC. It sets up a template, adds data to the Spring MVC `Model` (which is a `Map`), and then returns `chat`, which Spring MVC will use to resolve the view to render. In this case: `src/main/resources/templates/chat.html`.

2. this controller handler method is for Spring MVC's WebSocket controller handler machinery. In this case, it listens for WebSocket requests to `/send`, taking in the payload and the current authenticated user as parameters. Whatever response is produced is sent to the public WebSocket topic called `/topic/public`.

This requires a DTO, called `ChatMessage`.

```
package com.example.spring_session;

record ChatMessage(String sender, String content) {
}
```

Here's the `.html` page, rendered using Thymeleaf templates.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="en">
<head>
    <meta charset="UTF-8">
    <title>Authenticated Chat</title>

    <style>
        #chat {
            display: grid;
            gap: .5em;
            justify-items: start;
        }

        #chat > #prompt,
        #chat > #chat-box,
        #chat > #messageInput {
            width: 100%;
            box-sizing: border-box;
        }

        #chat > button {
            justify-self: start;
        }

        #chat-box {
            padding: .5em;
            border: 1px solid black;
            height: 10em;
            overflow-y: scroll;
        }
    </style>
</head>
<body>
<div id="chat">
    <div id="prompt">
```

```
        <h2>WebSocket Chat</h2>
        <p>Logged in as: <strong th:text="${username}" id="username">User</strong></p>
    </div>
    <div id="chat-box"></div>
    <input type="text" id="messageInput" placeholder="Type a message..."/>
    <button onclick="send()">Send</button>
</div>
<script>
    function load(src) {
        return new Promise(ok => {
            const s = document.createElement('script');
            s.src = src;
            s.onload = ok;
            document.head.appendChild(s);
        })
    }

    function startChat() {
        const box = document.getElementById('chat-box'),
            input = document.getElementById('messageInput');
        const stomp = Stomp.over(new SockJS('/messages'));
        stomp.connect({}, () => stomp.subscribe('/topic/public', m => {
            const {sender, content} = JSON.parse(m.body);
            box.insertAdjacentHTML('beforeend',
                `<div><span class="sender">${sender}:</span> ${content}</div>`);
            box.scrollTop = box.scrollHeight;
        }));
        window.send = () => {
            const content = input.value.trim();
            if (content) stomp.send('/app/send', {}, JSON.stringify({content}));
            input.value = '';
        };
    }
    const cdn = 'https://cdnjs.cloudflare.com/ajax/libs/';
    const promises = ['sockjs-client/1.6.1/sockjs.min.js',
'stomp.js/2.3.3/stomp.min.js']
        .map(src => load(cdn + src));
    Promise.all(promises).then(startChat);
</script>
</body>
</html>
```

1. Connect to the SockJS endpoint defined in your WebSocket configuration.
2. Subscribe to `/topic/public` broadcast messages.

All of this requires that we establish some WebSocket configuration.

```
package com.example.spring_session;
```

```
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.session.Session;
import
org.springframework.session.web.socket.config.annotation.AbstractSessionWebSocketMessa
geBrokerConfigurer;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;

@Configuration
@EnableWebSocketMessageBroker
class WebSocketConfiguration ①
        extends AbstractSessionWebSocketMessageBrokerConfigurer<Session> {

②
    @Override
    public void configureStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/messages").withSockJS();
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/queue/", "/topic/");
        registry.setApplicationDestinationPrefixes("/app");
    }

}
```

1. in a non-Spring Session-backed application, you'd normally override `WebSocketMessageBrokerConfigurer`
2. normally, in a raw `WebSocketMessageBrokerConfigurer`, you'd override `registerStompEndpoints`, not `configureStompEndpoints`.

And finally, we need to lock down the newly minted `/chat` endpoint.

```
package com.example.spring_session;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
class ChatSecurityConfiguration {

    @Bean
    Customizer<HttpSecurity> chatHttpSecurityCustomizer() {
        return http -> http.authorizeHttpRequests(a -> a.requestMatchers("/chat")
.authenticated());
```

```
    }

}
```

With all that in place, open up one browser or one incognito tab and visit `http://localhost:8080/chat`. Log in as `josh@joshlong.com`. In another incognito tab, log in as `rob@springsecurity.site`. In one tab, enter a message and see that the message is visible to the other. Keep typing for a few minutes. Don't reload the pages. Send a message once a minute. Do so for three minutes. If you did this in a normal WebSocket-based application, the server would time out the HTTP session even though the WebSocket communication was ongoing. In this case, however, as long as you're communicating via WebSockets, the HTTP session will be perpetuated.

Conclusion

All applications, even the "stateless" single-page applications, have state. We encounter it first when we think about identity. And since identity is key to everything else we do when building systems, it is *not* something you can afford to think about later on. It's front and center in everything. Spring Session gives us a convenient way to scale and optimize the bookkeeping associated with that state. The choice between speed and security is no choice at all! And with Spring Session, it doesn't have to be a choice: choose both. :code: ..

The Trouble with Passwords

In the last chapter we assembled the moving parts of authentication by hand. Now let's put them to work in a real, Spring Boot-powered web application - and confront the single most common (and most troublesome) authentication factor of all: the password. We'll wire up users, first in memory and then from a SQL database, learn how Spring Security stores and *migrates* passwords so they're never left in plaintext, and then take an honest look at why, despite all this machinery, passwords remain a weak foundation to build on.

Authentication in a Web Application

We'll see how to do so in various other contexts at other places in this book. But for now, let's start with a web application. Let's take these simple concepts and look at them in practice, in a more fleshed-out example implemented using Spring Boot and Spring Security.

Visit the Spring Initializr, add `PostgreSQL Driver`, `Flyway`, `JDBC API`, `GraalVM Native Support`, `Spring Web`, `Testcontainers`, `Spring Security`, `DevTools`, and `WebAuthn for Spring Security` to the build. I always choose Apache Maven and, of course, the latest version of Java or the latest long-term support (LTS) version of Java. Click `Generate` and open the resulting zip file in your IDE.

We're going to connect this application to a SQL database. As always, we'll use Docker Compose to make short work of this. I've created a `compose.yml` file here.

```
services:
  postgres:
    image: 'postgres:18.4'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'
```

Run that in the usual way:

```
docker compose up -d
```

Here's our initial `application.properties` configuration.

```
spring.application.name=security
①
spring.datasource.username=myuser
spring.datasource.password=secret
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
②
spring.flyway.baseline-on-migrate=true
```

```
#
spring.threads.virtual.enabled=true
```

1. define SQL database connection information
2. tell Spring Boot to evaluate `src/main/resources/{schema,data}.sql`

Let's set up our Spring Security application. Let's install a single controller that we will protect and that will require an authenticated user to access.

```
package com.example.security;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class MeController {

    @GetMapping("/")
    Map<String, String> me(Principal principal) {
        return Map.of("name", principal.getName());
    }

}
```

This endpoint is protected. You'll need to authenticate to visit this endpoint: `http://localhost:8080/`.

By default, a Spring Boot application with `org.springframework.boot : spring-boot-starter-security` is completely locked down, and all endpoints will require authentication.

Spring Boot registers a *lot* of common-sense and useful defaults, locking down the application, applying useful authentication methods like HTTP BASIC and form-based login, applying cross-site request forgery (CSRF) protection, and more.

To access any endpoint, a user would need to authenticate. But how? We could specify a single username and password, for demonstration, using properties, but it's almost too contrived. Even for me. So let's instead register a bean of type `UserDetailsService`.

In-Memory Users

We'll start by registering an `InMemoryUserDetailsManager` to see everything work.

```
package com.example.security;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
@Profile("memory")
class InMemoryConfiguration {

    @Bean
    InMemoryUserDetailsManager memoryUserDetailsManager(PasswordEncoder pw) {
        var user1 = User.withUsername("user@anotherone.site") ①
            .roles("ADMIN") //
            .password(pw.encode("p@ssw0rd")) //
            .build();
        var user2 = User.withUsername("josh@joshlong.com") //
            .roles("USER", "ADMIN") ②
            .password(pw.encode("pw")) //
            .build();
        return new InMemoryUserDetailsManager(user1, user2);
    }

}
```

1. we can specify whatever we want for the `username`. In this case I'm using something that looks like an email, but there's no schema or requirements here.
2. users have *authorities*, which help us determine what privileges a given user has. We'll look at these in more depth when we look at authorization.

Start the application and visit the endpoint, and you'll be prompted to authenticate. Use `josh@joshlong.com` and `pw` as the username and password, respectively.

You should see the username returned to you in a JSON response. It worked!

DAO-Backed Authentication

Let's change the `UserDetailsService` implementation to read from our SQL database instead of from in-memory. We'll need some schema to define the tables in which our users and the enumeration of their *authorities* will reside. Since we're using Flyway, we'll put the SQL schema migration files in `src/main/resources/db/migration`. I've placed the schema for the `users` and `authorities` table in the following file in `src/main/resources/db/migration/V1__users_schema.sql`.

```
create table if not exists users
(
    username varchar(200) not null primary key,
    password varchar(500) not null,
```

```
    enabled  boolean      not null
);

create table if not exists authorities
(
    username  varchar(200) not null,
    authority varchar(50)  not null,
    constraint fk_authorities_users foreign key (username) references users (username
),
    constraint username_authority UNIQUE (username, authority)
);
```

I've placed the following file in `src/main/resources/db/migration/V2__users_data.sql`.

```
delete
from authorities;
delete
from users;

insert into users (username, password, enabled)
values ('josh@joshlong.com',
'{bcrypt}$2a$10$i4a5jw4y9LnKS//8DNDVYe2VfC6Jr2m0Ds1j5un5JNhnkJxg6jAT.', true);

insert into users (username, password, enabled)
values ('user@anotherdomain.site',

'{sha256}0a598ce65440a7c4ad18b48fdc2cf28c4e58aec803780f59548535696a23b16624e47ff4f59e6
f39', true);

--
insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_USER');

insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_ADMIN');

insert into authorities (username, authority)
values ('user@anotherdomain.site', 'ROLE_USER');

insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_EDITOR');
```

I've taken this schema directly from the Spring Security documentation and the source code for Spring Security. We're using PostgreSQL, but there are equivalent schemas that you could use for various other databases, too.

Delete the conflicting `InMemoryUserDetailsManager` bean from earlier. Let's replace it with one that delegates to these newly minted SQL tables and their data.

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.provisioning.JdbcUserDetailsManager;

import javax.sql.DataSource;

@Configuration
class SecurityConfiguration {

    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
①
        var userDetailsManager = new JdbcUserDetailsManager(dataSource);
②
        userDetailsManager.setEnableUpdatePassword(true);
        return userDetailsManager;
    }

}
```

1. this implementation of `UserDetailsService` reads user data from the aforementioned SQL tables
2. this property is a relatively new feature.

It allows Spring Security to update the password in the SQL database if the encoding used isn't the default configured encoding in the system.

Restart the program. You should see data in your PostgreSQL `users` and `authorities` tables.

Open up the browser and hit the same endpoint and authenticate with username `josh@joshlong.com` and password `pw`. You should authenticate just fine. But *how*? We'll need to dive into password encoding before we have a good answer.

Password Encoding

Looking at the output, you'll notice that the passwords written to the database were basically garbled (*encoded*) text preceded by a prefix in curly brackets: {, the algorithm name, and }.

This was intentional: I wanted these passwords to work with the `PasswordEncoder` configured here:

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
@Configuration
class PasswordConfiguration {

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

}
```

A `PasswordEncoder` is an object that takes plaintext Strings and *encodes* them with an algorithm. Spring Security ships with dozens of implementations for algorithms like SHA (*do not* use this one!), SHA-256 (maybe don't use this one either), PBKDF2, SCrypt, BCrypt, Argon2, etc. Many of the implementations are duplicated, in fact, because there is dual support in Spring Security for various algorithms from the third-party BouncyCastle library *and* from the third-party Password4J library. Indeed, most of the good implementations come from either the BouncyCastle or Password4J libraries and require their presence on the CLASSPATH.

You could use these algorithms directly by constructing the relevant instances of the `PasswordEncoder`, like so:

```
var bcrypt = new BCryptPasswordEncoder();
```

I always prefer instead to use the composite that comes from `PasswordEncoderFactories`, which is itself a composite `PasswordEncoder`. It maintains a dictionary of different algorithm names mapped to an instance of their implementation. Security is a fast-moving space. Algorithms that are deemed secure and uncrackable today might be deemed cracked and insecure tomorrow. Spring Security reserves the right to bump the default algorithm as it evolves, should the need arise. Right now, the best algorithm provided out-of-the-box with no need for any other dependency is `BCryptPasswordEncoder`.

When you use the `DelegatingPasswordEncoder` from `PasswordEncoderFactories` today, at the time of this writing in mid-2026, the default algorithm used is `bcrypt` (which maps to an instance of `BCryptPasswordEncoder`, as shown above). That might change tomorrow as the algorithms are discovered to be less secure. If you have BouncyCastle or Password4J on the CLASSPATH, I'd prefer Argon2. Use Argon2 when available because it is memory-hard and tunable for memory, CPU time, and parallelism, making large-scale GPU/ASIC cracking harder. OWASP currently recommends Argon2 as the first choice.

The `DelegatingPasswordEncoder` prefixes all passwords with the name of the algorithm used to encode it.

Let's walk through the (rough) authentication flow, with the `PasswordEncoder` in mind.

- a user enters a username and a password in a form on your website and then submits the request
- Spring Security intercepts the request and constructs a `UsernamePasswordAuthenticationToken` with the username and password as parameters.

- the configured `AuthenticationManager` is asked to authenticate the request.
- the `AuthenticationManager`, in this case, is probably an instance of `ProviderManager`, so it in turn delegates to a chain of `AuthenticationProvider` instances, which get a chance to authenticate the `Authentication` request.
- finally, the `DaoAuthenticationProvider` instance loads the user from the `UserDetailsService` using the presented username.
- It needs to check that the passwords match, so it obtains an instance of the `DelegatingPasswordEncoder` and checks that the presented password (in plaintext) matches the one stored in the database (encoded, and prefixed with an algorithm name) by calling `PasswordEncoder#matches(CharSequence plain, String encoded)`. Internally, this uses the prefix to identify the ultimate implementation of `PasswordEncoder` to use, then encodes the presented password with that implementation.

You can of course configure your own `DelegatingPasswordEncoder` if you like to use Argon2 (or anything else). Add Password4J to the CLASSPATH: `com.password4j : password4j : 1.8.4`. I'm using that version in the middle of 2026. (You should use whichever version is the latest-and-greatest as you're reading this.)

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.DelegatingPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.crypto.password.StandardPasswordEncoder;
import org.springframework.security.crypto.password4j.Argon2Password4jPasswordEncoder;

import java.util.Map;

①
@Configuration
@Profile("password4j")
class CustomPasswordEncoderConfiguration {

    @Bean
    DelegatingPasswordEncoder delegatingPasswordEncoder() {
        var argon = new Argon2Password4jPasswordEncoder();
        var defaultPasswordEncoder = "argon@Password4j";
        var mapOfEncoders = Map.<String, PasswordEncoder>of( //
                defaultPasswordEncoder, argon, //
                "bcrypt", new BCryptPasswordEncoder(), //
                "sha256", new StandardPasswordEncoder()②
        );
        return new DelegatingPasswordEncoder(defaultPasswordEncoder, mapOfEncoders);
    }
```

```
}
```

1. this implementation configures the Password4J-powered Argon2 implementation as the *default* encoder, with optional support for BCrypt.
2. we're registering an encoder for `sha256`, but keep in mind that this implementation is deprecated! In a happier world, we wouldn't use it.

Make sure you replace the other `PasswordEncoder` implementation from the `PasswordConfiguration` class with this one if you'd like to use it.

We've configured custom password encoding so that we can handle the data in the SQL database that uses `sha256` encoding. It's a deprecated algorithm, and we'd certainly prefer it if people weren't using it at this point. The trouble is, there's always *legacy*. Maybe people used that algorithm before, and now they need to migrate the passwords to the new algorithm. Encoded passwords are, by definition, *challenging* to unencode, and so we can't convert them to a new password unless we somehow have the plaintext. The only time the system has the plaintext version of the password is when the user logs in and presents a password that, when encoded with the same algorithm, matches the one in the database. Earlier, when we configured the `JdbcUserDetailsManager`, we set `setEnableUpdatePassword` to `true`. This allows Spring Security to re-encode a password in the database that's encoded using something besides the current *default* password encoder when the user authenticates successfully.

Consult the `data.sql`, or your database, and you'll see that the user `user@anotherdomain.site` has a password encoded with `sha256`. So, restart the application and visit http://localhost:8080/ and then authenticate as `user@anotherdomain.site` and `pw`. Check the database again, and you'll see it's been migrated to use the *default* password encoder. If you're using the `PasswordEncoder` from `PasswordEncoderFactories`, you'll see it's been encoded using `bcrypt`. If you're using the custom `PasswordEncoder` we set up above, you'll see it's been encoded using `argon@Password4j`. Either way, *much* better than `sha256`! Take a victory lap, you've earned it. You can say that you're storing passwords as safely as possible.

But Passwords Are Bad!

Yah, I said it! Passwords are a terrible *factor* for authentication. Which is a shame because, for the vast majority of systems in production today, they're all we've got. But they're also harder to maintain, most people don't have the tools to deploy them correctly, and - to make matters worse - so much of the received wisdom around them has worked against us.

Humans aren't built to remember dozens of high-quality, unique, and complex passwords. It would be great if everybody used a good password manager like Bitwarden, 1Password, LastPass, etc. But they don't. Or, worse, they might've accidentally started using password managers in one context (e.g., Google Chrome) but gotten locked out because they switched away from whatever that context was (e.g., Windows to macOS, Android to iPhone, Google Chrome to Safari, etc.). The result? People don't develop good password hygiene. They reuse passwords. They scrawl them on stickies affixed above their monitor. Or, worse, in a text file on the public cloud.

Insufficient engineering practices are often to blame. In the example above, we showed how to store passwords in an encoded fashion *and* how to migrate them to better encodings whenever

possible. Sometimes folks don't do the right thing. Meta (formerly known as Facebook) was caught storing hundreds of millions of user passwords in plain text [https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/] for years. In 2019, it also came out that Google had stored some passwords in plaintext [https://www.wired.com/story/google-stored-gsuite-passwords-plaintext/] since 2005. The 2012 Yahoo! Voices attack [https://en.wikipedia.org/wiki/2012_Yahoo_Voices_hack] revealed that Yahoo! had stored around 450,000 passwords in plaintext. If it can happen to these organizations, it can happen to anyone. (Why can't everyone just use Spring Security?)

Now you might say that those issues are a sign of poor password management, and I'd say you can't have poor password management if you don't have passwords. We can both be right.

Bad "Received Wisdom"

Sometimes, it boils down to bad "best practices." Should you rotate your passwords every month? No. Password rotation used to be recommended because the assumption was that passwords could be stolen without anyone noticing. Changing them periodically would reduce the window during which an attacker could use them. This made sense before the widespread adoption of multifactor authentication, breach detection, password managers, and monitoring for compromised credentials. But periodic rotation makes security *worse*. Research shows that people tend to increment the number (`Passw0rd1` → `Passw0rd2`) or make tiny, predictable modifications, or write passwords down, or choose weaker passwords because they're tired of changing them. All of which makes passwords *easier* to guess or derive. It's much better to securely store a long, secure password in a password manager rather than changing it without proper handling every month (or whenever).

Beyond Passwords: Magic Links

We ended the last chapter on a sour note: passwords are hard to store, hard to remember, and hard to defend, and even decades of received wisdom about them has often made things worse. We're not alone in reaching that conclusion. Governments and regulators the world over are actively pushing users and organizations away from password-only authentication - so let's start there, and then look at our first genuinely passwordless option: the one-time token, better known as the "magic link." But before we can wire up anything new, we need to learn how to customize a Spring Security application beyond simply registering beans and hoping for the best.

Authentication Around the World

Indeed, in many government contexts, passwords are discouraged or disallowed.

In the **United States**, the Office of Management and Budget issued the federal Zero Trust strategy requiring agencies to adopt phishing-resistant multifactor authentication (MFA) for users whenever possible.

The **European Union** is one of the strongest proponents of MFA: the NIS2 directive explicitly includes the use of multifactor authentication or continuous authentication solutions, where appropriate, as one of the required cybersecurity risk-management measures for covered entities (energy, healthcare, banking, transportation, cloud providers, public administration, digital infrastructure, etc.). The European Union's Digital Operational Resilience Act (DORA) for financial institutions requires strong identity and access management. While it does not literally say "passwords are banned," password-only authentication generally won't satisfy modern expectations for privileged or high-risk access. And the Revised Payment Services Directive introduced *Strong Customer Authentication* (SCA), which requires authentication using at least two independent factors from categories such as knowledge, possession, and inherence. A password alone is not enough for many electronic payment transactions.

The **United Kingdom**'s *National Cyber Security Centre* strongly recommends MFA for important accounts and increasingly recommends phishing-resistant authentication for government systems.

The Monetary Authority of **Singapore** requires strong authentication for many banking and financial services. Banks generally must use multifactor authentication for customer access and sensitive transactions.

Japan does not broadly prohibit passwords. However, government cybersecurity guidance strongly encourages MFA, and major financial institutions require additional authentication. Passkeys are increasingly being adopted across both government and private sectors.

South Korea has historically had some of the world's strongest online authentication requirements. Although earlier systems relied heavily on certificates, modern regulations increasingly permit FIDO authentication, biometrics, and MFA instead of password-only systems.

We won't be completely escaping passwords for a while, but every journey of a thousand miles starts with but a single step. It starts with us. Let's explore some alternatives to passwords. But before we can do that, we need to think about how to customize a Spring Security application. So

far we've just registered beans of the right type in the right place, and things have sort of worked out.

Remember when I told you that there's a giant filter sitting in between your requests and the back-end application logic? Well, we can configure how that works by defining the `SecurityFilterChain` ourselves, like this.

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@Profile("sfc")
class SecurityFilterChainConfiguration {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity security) throws Exception {
        return security //
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults())
            .csrf(Customizer.withDefaults()) //
            .cors(AbstractHttpConfigurer::disable) //
            .authorizeHttpRequests(a -> a //
                .requestMatchers("/admin")
                .hasRole("ADMIN")//
                .requestMatchers("/**")
                .authenticated() //
            )//
            .build();
    }

}
```

We won't get into what's happening here just yet. Suffice it to say that in defining this bean, we are overriding the defaults that Spring Boot provides for Spring Security and that you get for free when you add `spring-boot-starter-security` to the CLASSPATH. By default, Spring Boot declares a bean of type `SecurityFilterChain` that:

- enables HTTP Basic authentication
- enables form login
- requires that all access to any endpoint be authenticated

Historically, if you wanted to change this, you'd have to define a `SecurityFilterChain` bean of your own and you'd instantly lose the default protection Spring Boot gave you. So, your first order of business was to immediately redefine the same bean, which looks more or less like this:

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Profile("defaultsfc")
@Configuration
class BootsDefaultSecurityFilterChainConfiguration {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity security) throws Exception {
        return security //
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults())
            .authorizeHttpRequests(a -> a.anyRequest().authenticated())
            .build();
    }

}
```

Only then could you start customizing the security for the application. But for the brief moment between overriding the old bean and fleshing out the new one, your application was *less* secure than if you'd done nothing at all!

Not a good result. In Spring Security 7, we introduced a way to iteratively *add* to the default security, instead of replacing it wholesale; we only need to define a bean of type `Customizer<HttpSecurity>`. For most of this book, I will do exactly that *almost* everywhere. I show you the `SecurityFilterChain` approach because on the occasions when you want to make authorization more granular, or when reading older code and older text, you'll need to appreciate what's happening.

One-Time Tokens, or "Magic Links"

You ever use Slack?



If so, then I am so very sorry. I am not a huge fan of the huge fans required to cool the computer running Slack. Or indeed almost any Chromium or Electron-based software.

When you authenticate, it prompts you to enter your email. You oblige it, and it sends you an email.

You click on the link in your email, and now you're authenticated. You have access to your email, so therefore you are who you say you are. Here, it's an email. But it could be any out-of-band messaging medium. Email, WhatsApp/SMS (though these two, and anything tied to a spoofable phone number, are increasingly discouraged these days), Discord, etc.

Do these services require a password? Almost certainly. And probably another factor. And you can trust that Google (behind Gmail), Apple (and iCloud), Microsoft (and Active Directory and Outlook), etc. have all done a ton to ensure the integrity of passwords for their services. They have whole teams working on the security of passwords for their products. So in a way this just kicks the can down the road, but it does remove a significant burden from your organization, and it simplifies the user experience while also making their lives (hopefully) more secure.

I don't want to set up Twilio, JavaMail, or SendGrid here, so let's simplify things and just imagine that the out-of-band messaging medium by which people are communicating is our application's logs. Is this *real*? No, of course not. But it'll help to see things work end-to-end.

Let's try it out. First, we'll need to enable one-time tokens. Do so within a `Customizer<HttpSecurity>`.

```
package com.example.security.ott;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
class OttConfiguration {

    static final String SENT_URL = "/login/ott/sent";

    @Bean
    Customizer<HttpSecurity> ottCustomizer(ConsoleOneTimeTokenGenerationSuccessHandler
handler) {
        return security -> security ①
            .oneTimeTokenLogin(config -> config //
                .tokenGenerationSuccessHandler(handler))//
            .authorizeHttpRequests(a -> a
②
                .requestMatchers(SENT_URL)
                .permitAll()); //
    }

}
```

1. first, tell Spring Security to allow one-time tokens as an authentication mechanism, pointing it to an implementation of `OneTimeTokenGenerationSuccessHandler`, which we'll look at momentarily.
2. then we need to make publicly accessible an HTML page (visible under the path `/login/ott/sent`) that we intend for the users to see when authenticating.

In the configuration class, I've extracted out the HTML page's path into a `static final` constant so that I can reference it in a few other places for consistency. To be 100% honest with you, I'd normally write basically all of the code to do with wiring this up in the same outer `OttConfiguration` class, using a lambda and private static inner classes to do the work. There just isn't that much complexity here. But this is a book, and I want the examples to be clear and to introduce the concepts one at a time, so I've extracted them into separate classes. And since I had more than one class about the same *thing*, I've put them all in the same nested package, `ott`.

What is this `OneTimeTokenGenerationSuccessHandler`, I hear you say? It's a callback interface that gets involved once the user has entered their username on the login page. You can use this as a chance to set up any state and then look the user up and email them, call or text their phone number or WhatsApp, etc.

But again, I want to keep things simple, so I'll simply print on the console the URL to which the authenticating user should go.

```
package com.example.security.ott;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.ott.OneTimeToken;
import
org.springframework.security.web.authentication.ott.OneTimeTokenGenerationSuccessHandl
er;
import
org.springframework.security.web.authentication.ott.RedirectOneTimeTokenGenerationSucc
essHandler;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.io.IOException;

import static com.example.security.ott.OttConfiguration.SENT_URL;

@Component
class ConsoleOneTimeTokenGenerationSuccessHandler implements
OneTimeTokenGenerationSuccessHandler {

①
    private final OneTimeTokenGenerationSuccessHandler successHandler = new
RedirectOneTimeTokenGenerationSuccessHandler(
            SENT_URL);

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
OneTimeToken oneTimeToken)
            throws IOException, ServletException {

②
```

```
        var url = ServletUriComponentsBuilder.fromContextPath(request) //
            .path("/login/ott") //
            .queryParam("token", oneTimeToken.getTokenValue()) //
            .build();

③
        IO.println("sending email " + oneTimeToken.getUsername() + " who should go to
" + url);

④
        this.successHandler.handle(request, response, oneTimeToken);
    }

}
```

1. this is a built-in `OneTimeTokenGenerationSuccessHandler` that simply redirects to a particular view. We will want to do that, eventually.
2. but first, we need to know where to send the user. Spring MVC provides this handy convenience class called `ServletUriComponentsBuilder` that makes building up a relative URL pretty darned easy.
3. finally, we print out where we expect the user to go to access the system. Here, we're conveying this link on the system console, but in a real application we'd send an email or SMS or something.
4. finally, with all the setup done, we need to respond to the HTTP request. We do so by redirecting to a URL on the server that'll render an HTML page.

Let's quickly touch on that `ServletUriComponentsBuilder` use. Spring uses the current `HttpServletRequest` to build up a fully qualified URL. But what happens if your service is behind a gateway or a router? You don't want your users trying to click on a URL bound for `http://192.168.2.5:20482`, you want them going to `http://example.com/`. If your gateway forwards proxy headers like `Forwarded` or the `X-Forwarded-*` headers, then Spring can be made to honor those with the `ForwardedHeaderFilter`. You can customize how this is set up with `server.forward-headers-strategy` in a Spring Boot application. I usually use `server.forward-headers-strategy=framework`. If you use this, then be sure to *strip* those headers at the trust boundary (your gateway), and then add them again before sending it to your downstream service. Spring Boot has no way of knowing whether these headers are trustworthy!

When we set up the application, we configured Thymeleaf so that we could produce HTML views like the one we're presenting to the user after sending the authentication link. Let's do that here.

```
package com.example.security.ott;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

import static com.example.security.ott.OttConfiguration.SENT_URL;
```

```
@Controller
class OttController {

    @GetMapping(SENT_URL)
    String sent() {
①
        return "ott-sent";
    }

}
```

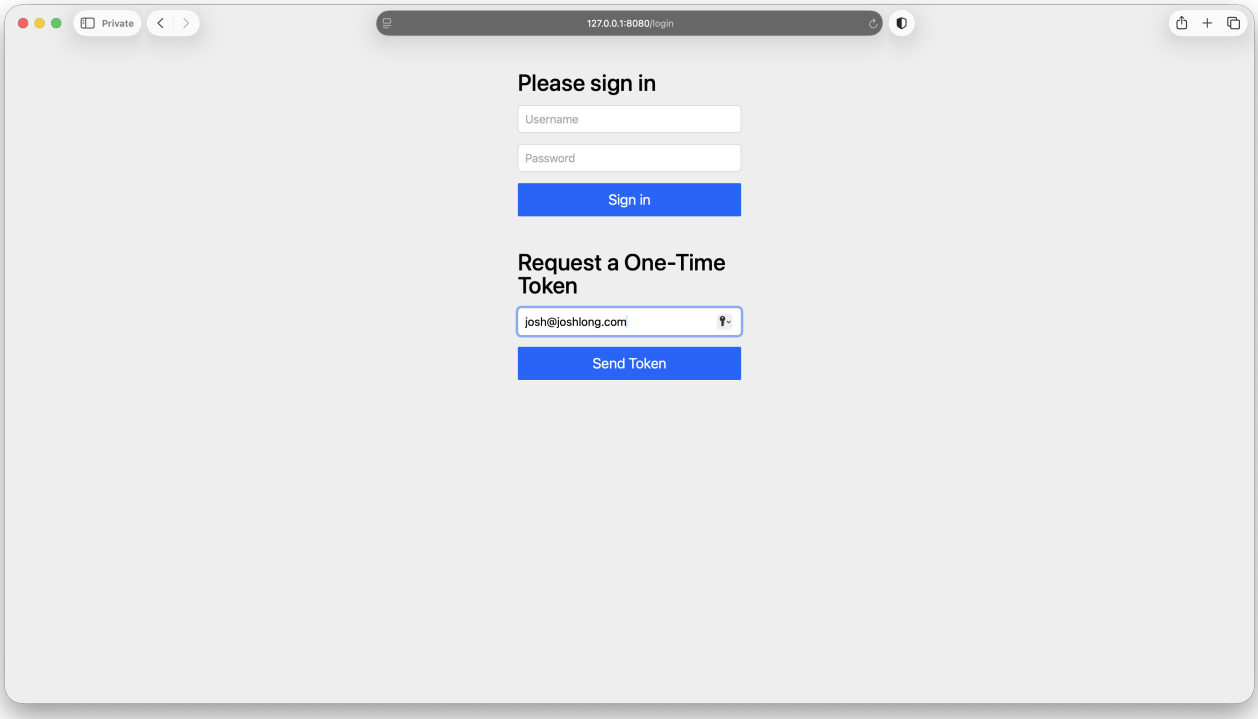
1. this is a regular `@Controller` with no `@ResponseBody` to be found. Spring interprets the return value as a template to be resolved via the autoconfigured `ViewResolver`. In this case, it'll resolve to `src/main/resources/templates/ott-sent.html`.

Here's the Thymeleaf page:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sent</title>
</head>
<body>
<h1>
    You've got console mail!
</h1>
</body>
</html>
```

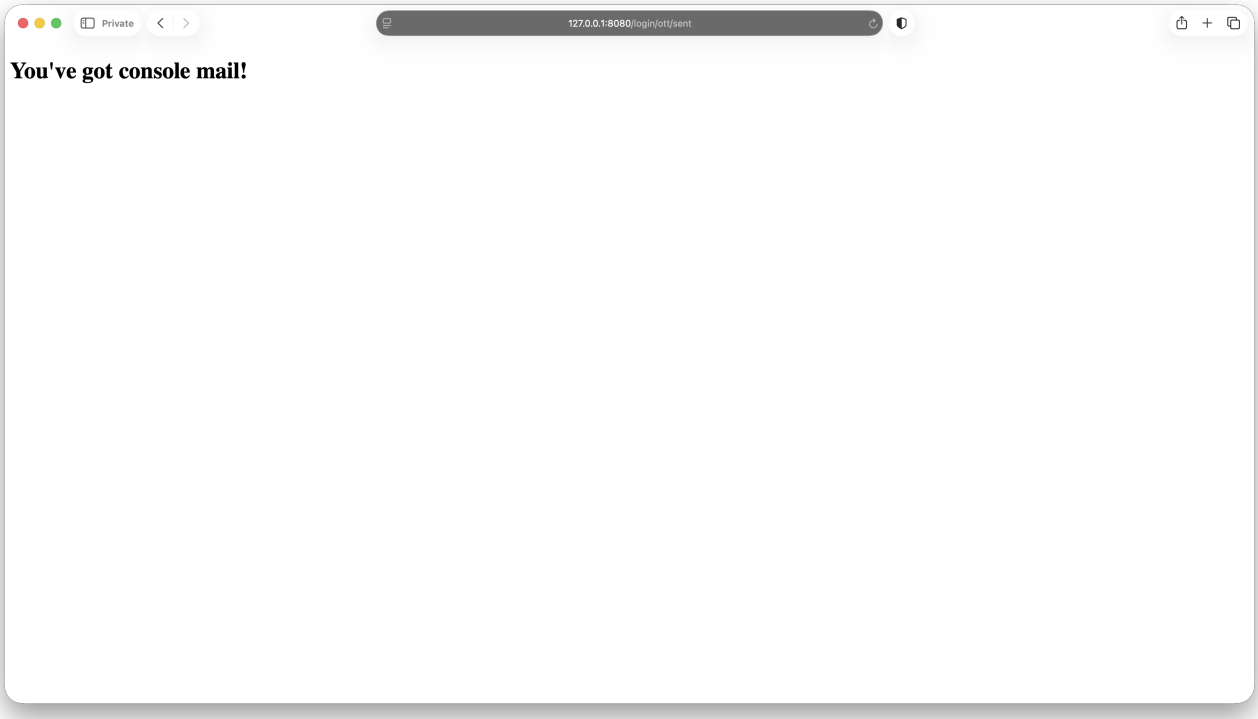
Nothing fancy.

With all this in place, start the application and hit http://localhost:8080. Enter the username (`josh@joshlong.com`).



1. the signin form now has a new capability: *Request a One-Time Token*.

then you'll get that, um, *minimal* Thymeleaf page we put together.



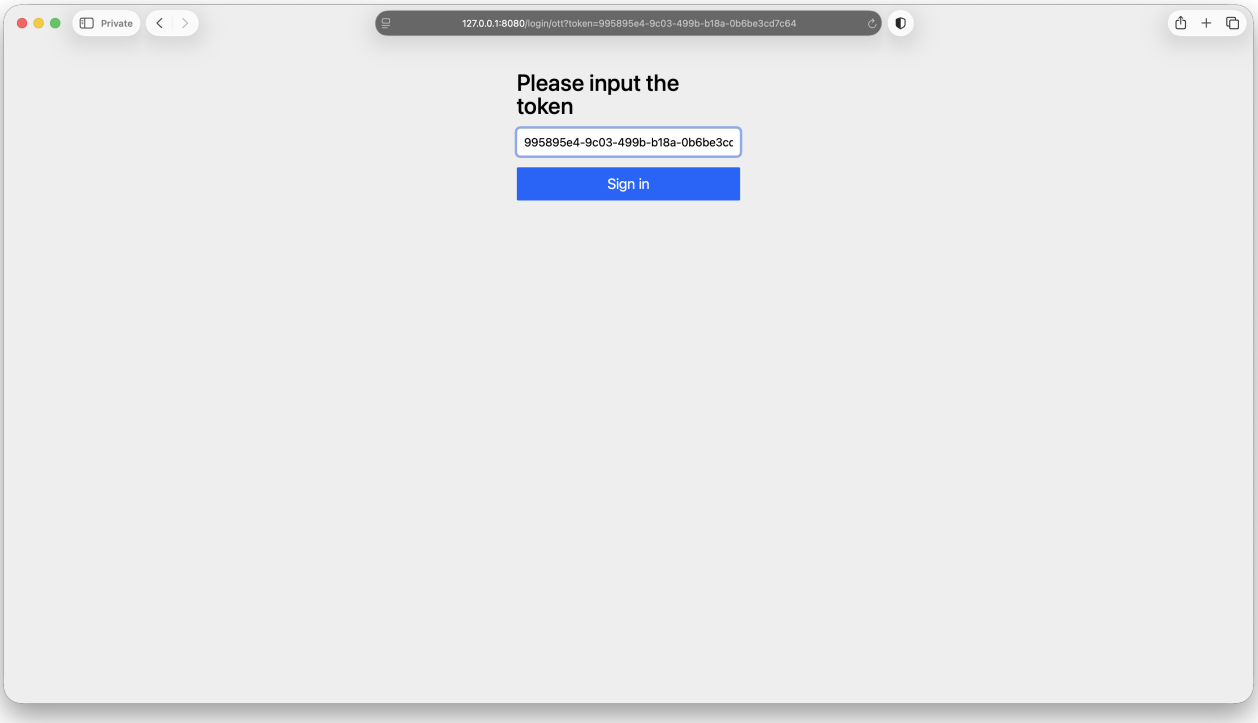
1. I know, I know. Stick to my day job. Web design's not my thing.

Meanwhile, an out-of-band message has been sent, in this case, to my IDE's console logs.

```
2026-07-06T13:43:51.672+09:00  INFO 79054 --- [security] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Complet
sending email josh@joshlong.com who should go to http://127.0.0.1:8080/login/ott?token=995895e4-9c03-499b-b18a-0b6be3cd7c64
```

1. I’ve got console mail!

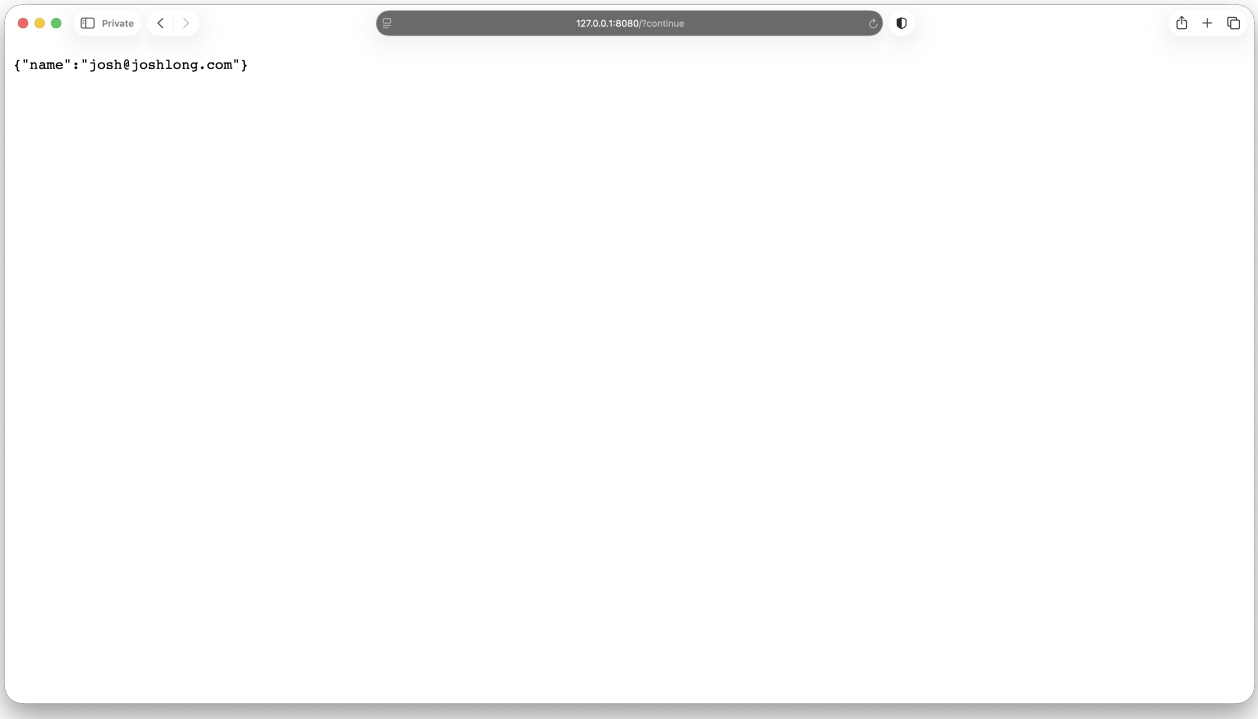
Click on the link, and it’ll take you to an input where you can plug in the one-time token.





If you are trying to follow the authentication flow in an incognito window in, say, Apple’s Safari browser, make sure to copy and paste the link from the console into the browser. Clicking on the link will start a *brand-new* browser whose session is divorced from the first one in the incognito window.

And now you should see your username in the HTTP response.



Easy!

My friend James Ward [https://linkedin.com/in/jamesward] did something interesting with this mechanism for another book we're jointly writing (and we hope to do it for this book, too!), where users who bought the book on a website would have had to enter their emails. Those emails are turned into users in a database. Somebody comes along, signs in, and that grants an OAuth token which is then used to provision and provide access to an MCP service that in turn has access to the chapters of the book. Users can point their MCP client of choice - Claude Code, Goose, JetBrains Air, whatever - at the MCP service and consume the content and ask questions about it as MCP resources. Super innovative!

One-time tokens are great for passwordless authentication. It's important to remember their limitations, too. They are *not* signup links to confirm emails. Nor are they password-reset links. And of course, if somebody loses access to the account where the out-of-band communication is sent (email, SMS, whatever), then they're now effectively locked out of *two* systems!

One-time tokens are an easy way to build upon the considerable technical *and* legal infrastructure that companies like Google, Microsoft, Apple, etc., have built up to manage passwords and fight breaches and the like. They allow you to secure a system while avoiding passwords. That said, there is a password in the mix. It's just not *your* password to manage.

Passkeys and Multi-Factor Authentication

One-time tokens got us most of the way out of the password business, but they still lean on an out-of-band channel - and, ultimately, on somebody else's password. Passkeys go further. Built on the FIDO2 and WebAuthn standards, they replace the shared secret with public-key cryptography that can't be phished, reused, or leaked from a breached database. In this chapter we'll learn what makes passkeys tick, implement and persist them in Spring Security, survey a few other authentication mechanisms worth knowing about, and finally combine factors into true multifactor authentication.

The Powerful Passkeys

Passwords are the weakest link. By the early 2000s, companies like PayPal began exploring alternative authentication methods, recognizing that the password model was fundamentally broken. The industry-wide acknowledgment that passwords needed to be eliminated entirely came in 2013 with the founding of the FIDO Alliance (Fast Identity Online). Major technology companies like Microsoft, Google, Apple, Meta, and others created the FIDO Alliance with a mission to eliminate password-based authentication through open standards. You can see the full roster of contributing members [https://fidoalliance.org/members/] here.

The FIDO Alliance's work produced two major specifications. FIDO U2F (Universal Second Factor) enabled secure two-factor authentication using hardware security keys, while FIDO2 - developed in close collaboration with the World Wide Web Consortium (W3C) - introduced a more comprehensive approach. FIDO2 consists of two complementary standards: the Client to Authenticator Protocol (CTAP), which defines how external authenticators communicate with client devices, and Web Authentication (WebAuthn), which provides the standardized browser API that allows websites to register and authenticate users using cryptographic credentials. WebAuthn was officially released as a W3C Recommendation in 2019 and is now widely supported across modern browsers. This is the bit that makes this interesting for me: what we're about to look at isn't some hypothetical, would-be-nice thing that might ship in the not-too-distant future; it's already here!

Rather than creating a shared secret like a password, a user's device generates a unique public/private key pair for each website or application. The private key never leaves the user's device. It's kept secure in hardware such as a phone's Secure Enclave, while only the corresponding public key is registered with the service. When signing in, the device proves possession of the private key by cryptographically signing a challenge from the server, typically after the user authenticates locally using biometrics, a PIN, or another device-unlock mechanism. Because there is no reusable password to steal, passkeys dramatically reduce the risk of phishing (a public key stored by the server is useless to attackers), credential stuffing, and password database breaches (because servers do not store reusable secrets).

However, early FIDO2 deployments had significant limitations. They often required dedicated hardware security keys such as YubiKeys or were tied to a single device, meaning users could lose access if that device was lost. The major breakthrough came in 2022-2023 when Apple, Google, and Microsoft committed to expanding the FIDO standard to support what became known as "passkeys" - FIDO credentials that could be securely synchronized across a user's trusted devices through cloud-based credential managers such as iCloud Keychain, Google Password Manager, 1Password,

and Bitwarden. (Platform authenticators like Windows Hello initially stored credentials bound to a single device, with cross-device passkey sync arriving later.) Now a passkey could be created once and used seamlessly across phones, tablets, and computers while remaining protected by each device's secure hardware and local authentication. This eliminated much of the backup-and-recovery problem while maintaining strong security.

The term "passkey" itself represents an evolution in how these technologies are presented to end users. By synchronizing FIDO credentials across devices and making the underlying cryptography largely invisible, the industry adopted "passkey" to describe these user-friendly, discoverable credentials. When you use a passkey today, you're benefiting from over a decade of standards work and billions of dollars in platform investment. FIDO Alliance standards provided the vision and interoperability framework; the W3C standardized the browser-facing WebAuthn API; browser vendors and operating system providers implemented the technology; and online services integrated support into their applications.

So, all together now: Passkeys rely on public-key cryptography. Each passkey consists of a unique public/private key pair, with the private key securely stored on one of your trusted devices. When you authenticate, the private key is unlocked using something that's easier for you to remember or present - such as your face, your fingerprint, or your device PIN - and then used to prove your identity cryptographically.

Concepts We'll Need To Understand for WebAuthn

Remember, WebAuthn works by creating a key tied to a particular domain (to prevent spoofing). So when you work with WebAuthn you need to specify at minimum three things:

- **the relying party ID** - The RP ID is the domain that a passkey is bound to. It's the security anchor for WebAuthn. When a passkey is created, it is cryptographically tied to this RP ID. The browser will only allow that passkey to be used on that domain (or a matching subdomain, depending on the RP ID).
- **the relying party name** - a human-readable display name.
- **allowed origins** - The allowed origins are the exact web origins that are permitted to call the WebAuthn APIs. You need to specify *everything*: the scheme (`http` or `https`), the exact domain, and the port (if applicable).

Implementing Passkeys in Spring Security

Now for the easy part. Whereas the explanation took a fair bit of time, the implementation is trivial. Let's get a minimum viable product working, first.

```
package com.example.security.webauthn;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

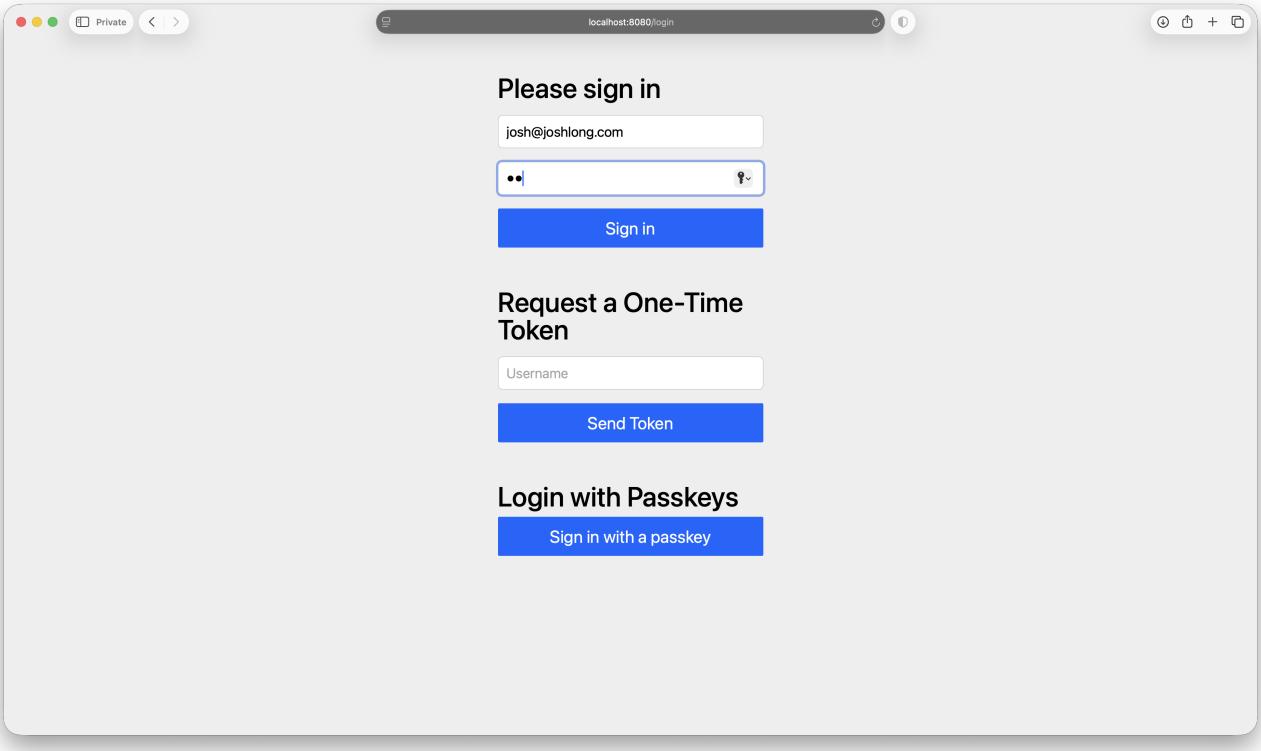
```
@Configuration
class WebAuthnConfiguration {

    @Bean
    Customizer<HttpSecurity> webAuthnCustomizer() {
        return security -> security //
            .webAuthn(c -> c //
                .allowedOrigins("http://localhost:8080")//
                .rpId("localhost")//
                .rpName("bootiful")//
            );
    }

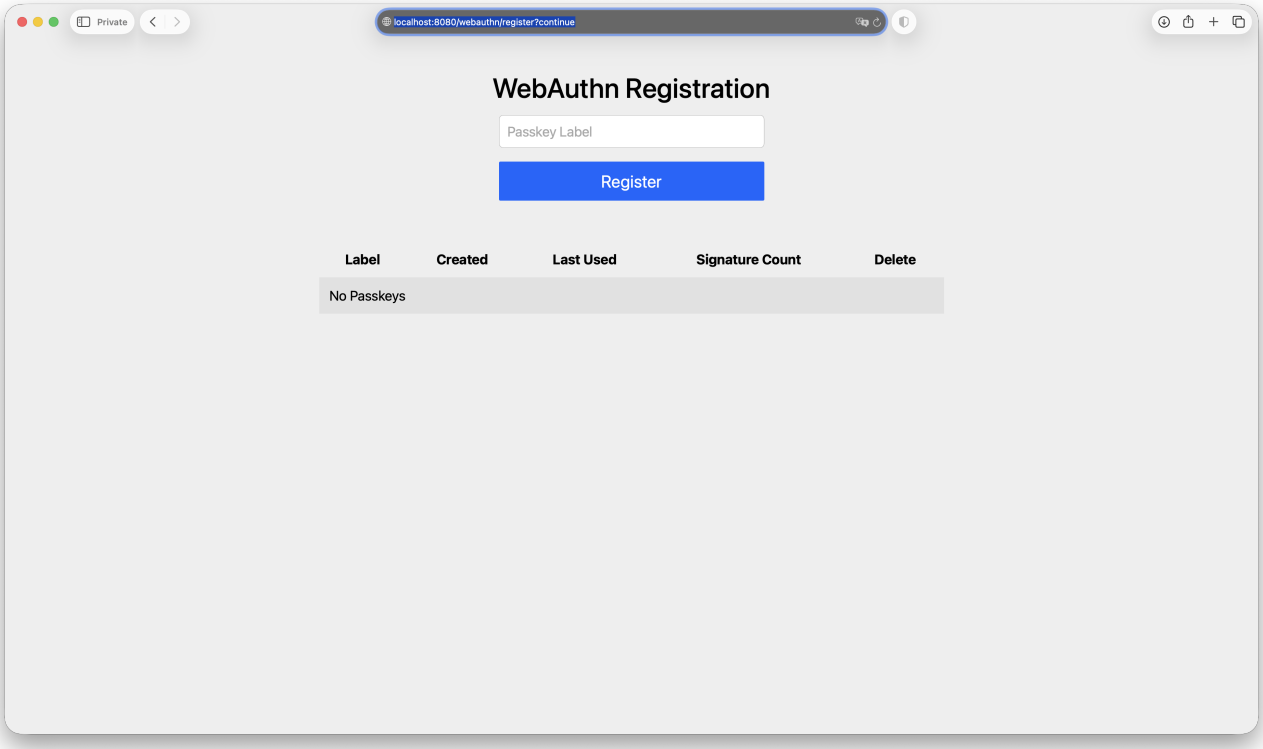
}
```

It's not even worth annotating this code. Now that you know the concepts, the code is as plain as can be.

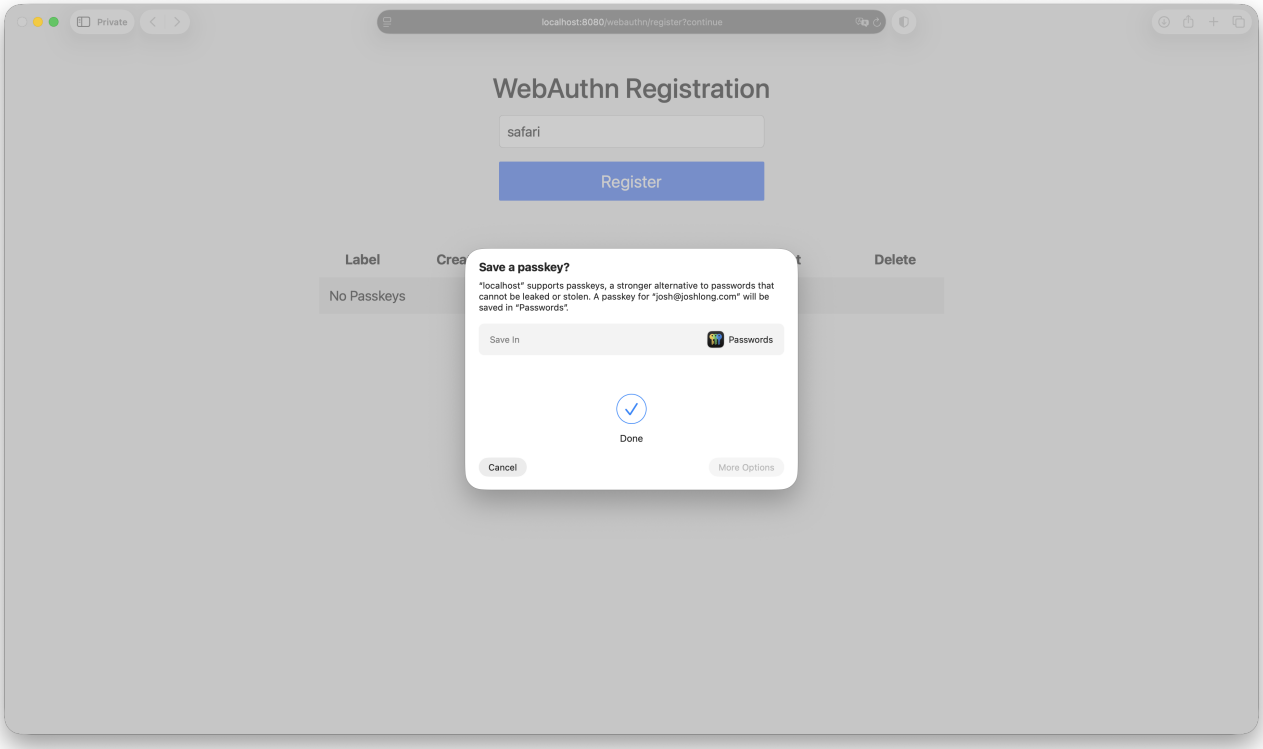
Let's try it out. Open your browser to http://localhost:8080, *not* http://127.0.0.1:8080, as that's a different origin URL! First order of business: we need to *register* a passkey. You can't authenticate with a passkey if the server doesn't have your public key. So, unfortunately, we need to authenticate using one of the other pre-configured mechanisms (like one-time tokens or usernames and passwords), *then* register the passkey.



1. sign in in the usual way



1. once you're authenticated, visit `/webauthn/register`



1. Register the passkey

I'm on an Apple laptop, and I am using an incognito browser instance. As a result the only option I have is to use Apple's Passwords.app, whose secrets are stored securely in iCloud, and federated across various Apple devices like my iPhone, Apple Watch, iPad, and MacBook Pro. If I were using a non-incognito browser window, I'd have access to my plugins and there I'd have access to my

Bitwarden secrets manager. The secret I'm saving is tied to my Apple TouchID. My *finger*! But the key is used to authenticate access to the private key, and I can authenticate that in many ways. If I'm on my iPhone, I can authenticate access to my private key by using Apple FaceID (my face!) too. So, even though I'm registering the key on my MacBook Pro with my fingerprint, I can still access the same website on my iPhone using my FaceID. No need to register a second key.

What if I don't trust this device, for some reason? Perhaps I'm signed in on a corporate laptop, and they have full access to all my sensors, input readers, etc. If I click *More Options*, it'll show me the option to create a QR Code, which I can then scan with my private and trusted computing device (like my iPhone). I can finish registering the Passkey on the iPhone, and it'll be set on this browser instance.

If you're using the Google ecosystem, you'll have (I'd imagine) roughly the same streamlined experience. It works on all the major browsers on Linux, too, but there you'd probably store the Passkey in your password manager of choice. This all even works on *Windows*!

Now logout by visiting `/logout` or simply starting a new incognito window. You'll see the same sign in options as before. This time, choose *Sign in with a Passkey*. On an Apple device, I just present my fingerprint and (in my best "hacker voice") *I'm in*!

Persistence

If you restart the Spring Boot application, you'll have to re-register your Passkey, because everything was stored in memory. Mercifully, Spring Security ships with JDBC-backed implementations of the persistence for the parts that need it. Let's look at that.

```
package com.example.security.webauthn;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcOperations;
import
org.springframework.security.web.webauthn.management.JdbcPublicKeyCredentialUserEntity
Repository;
import
org.springframework.security.web.webauthn.management.JdbcUserCredentialRepository;

@Configuration
class WebAuthnPersistenceConfiguration {

①
    @Bean
    JdbcPublicKeyCredentialUserEntityRepository jdbcPublicKeyCredentialRepository
(JdbcOperations jdbc) {
        return new JdbcPublicKeyCredentialUserEntityRepository(jdbc);
    }

②
    @Bean
```

```
    JdbcUserCredentialRepository jdbcUserCredentialRepository(JdbcOperations jdbc) {
        return new JdbcUserCredentialRepository(jdbc);
    }

}
```

Do I know exactly what these things do? No. Don't need to. I just know that they make important parts of the WebAuthn support durable, and that's good enough for me.

I also know that they need us to define some database tables for them to work. So, I've added two more Flyway migrations.

The first (`src/main/resources/db/migration/V3__user_entities.sql`) creates the `user_entities` table.

```
create table user_entities
(
    id           varchar(1000) not null,
    name         varchar(100)  not null,
    display_name varchar(200),
    primary key (id)
);
```

The second (`src/main/resources/db/migration/V4__user_credentials.sql`) creates the `user_credentials` table:

```
create table user_credentials
(
    credential_id               varchar(1000) not null,
    user_entity_user_id         varchar(1000) not null,
    public_key                  bytea         not null,
    signature_count             bigint,
    uv_initialized              boolean,
    backup_eligible             boolean       not null,
    authenticator_transports    varchar(1000),
    public_key_credential_type  varchar(100),
    backup_state                boolean       not null,
    attestation_object          bytea,
    attestation_client_data_json bytea,
    created                     timestamp,
    last_used                   timestamp,
    label                       varchar(1000) not null,
    primary key (credential_id)
);
```

These `.sql` files are in the Spring Security `.jar` files, but I copied and pasted them here for our use. Note that I've had to replace the uses of the SQL `blob` type in the `user_credentials` table with `bytea`, which works well in PostgreSQL.

If you're using macOS, make sure to open up Passwords.app, then go to the Passkeys section, and then delete any passkeys you might've created when using this Spring Security application, on `localhost`.

Restart the Spring Security application. You'll see the tables created in your database. Redo the whole process:

- authenticate
- register a Passkey with your Spring Security application
- logout
- log back in, this time using Passkeys

You should see data in the newly minted database tables. You can restart the Spring Security application and attempt to log in again with a Passkey and this time, it'll work just fine.

Other Interesting Kinds of Authentication

We've looked at usernames and passwords, one-time tokens, Passkeys, and we've only just begun to scratch the surface! There are so many other ways to authenticate a user, and Spring Security supports these and many more. I'll mention a few of them. If you know you have these particular use cases, feel free to consult the relevant documentation for the details.

One interesting approach to authentication is **HTTP BASIC**, where you send username and password (Base64-encoded) in an `Authorization` header.

Another option is to look a user up in an **LDAP** or **Active Directory** instance, instead of a username and password database.

Pre-Authentication

Another interesting approach to authentication is **pre-authentication**. In this scenario, Spring Security still needs to set up some sort of authentication and assign it to the current request, but the actual work of authenticating the user is done elsewhere. Spring Security is instead given something it can use to understand that authentication and present it to application code as a valid `Authentication`.

If you're using an authentication and authorization platform like SiteMinder, it'll communicate a header conveying information about the authenticated user. Spring Security can work with that through the `RequestHeaderAuthenticationFilter`.

Another approach is to use mutual **X.509** authentication. In this scheme, clients present a certificate issued by an authority that our server knows about and trusts.

Spring Security supports **Java EE Security Services**. In the distant, distant past, it was common to run something called an *Application Server*. These application servers had their own very primitive facilities for authentication such that the user might've already authenticated before a request ever reached the Spring Security filter or Spring Framework code. In such a scenario, the `J2eePreAuthenticatedProcessingFilter` class extracts the username from the `userPrincipal` property

of the `HttpServletRequest`. Use of this filter would usually be combined with the use of Java EE roles. But… don't use any of it, if you can avoid it. It adds nothing but complexity. Moving on…

Multifactor Authentication

There are tons of options here, but which is best? Sometimes, things are so sensitive that, to be extra certain that you're talking to the right user, you want more than one factor.

Multifactor authentication is the idea that, while any single factor has weaknesses that could conceivably be exploited, it's much less likely that a hacker could exploit multiple factors, especially if they're of different categories. We spoke earlier about the categories of authentication factors, including *knowledge*, *possession*, and *inherence*.

Spring Security 7 introduces multifactor authentication, where the system can store information about which authentication factors have been provided for a given authentication (a user) and then reconcile that against a list of outstanding factors required for that same authentication.

The simplest possible configuration is global, and annotation-powered.

```
package com.example.security.mfa;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import
org.springframework.security.config.annotation.authorization.EnableMultiFactorAuthenti
cation;
import
org.springframework.security.config.annotation.authorization.MultiFactorCondition;
import org.springframework.security.core.authority.FactorGrantedAuthority;

@Profile("globalmfa")
@Configuration
①
@EnableMultiFactorAuthentication(
        authorities = { FactorGrantedAuthority.WEBAUTHN_AUTHORITY,
FactorGrantedAuthority.PASSWORD_AUTHORITY },
        when = { MultiFactorCondition.WEBAUTHN_REGISTERED })
class GlobalConditionalMfaConfiguration {

}
```

1. this configuration requires multifactor authentication for every endpoint in the system, but it's *conditional* on the user having registered WebAuthn. After all, they can't log in and register the Passkey if they can't log in in the first place. If they haven't, then there's no sense in asking for it.

In the above example, the factors are combined together. It's as though you're asking for WebAuthn *AND* Passwords. But that's silly. If the user can present a passkey, then they don't really need to present a password. Passkeys are much safer. So, let's customize the conditions so that the user has

to present one of the following:

- BOTH a password and a one-time token
- a Passkey

The specifics of multifactor auth that we've seen so far were specified as parts of the annotation, but behind the scenes those stipulations are translated onto the configuration of a global bean of type `AuthorizationManagerFactory`. You can provide your own, if you like. You still need to define the annotation, but you can override the specifics in the bean itself.

```
package com.example.security.mfa;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.authorization.AllRequiredFactorsAuthorizationManager;
import org.springframework.security.authorization.AuthorizationManagerFactory;
import org.springframework.security.authorization.DefaultAuthorizationManagerFactory;
import org.springframework.security.authorization.RequiredFactor;
import
org.springframework.security.config.annotation.authorization.EnableMultiFactorAuthenti
cation;

@Configuration
@EnableMultiFactorAuthentication(authorities = {})
class GlobalEitherOrMfaConfiguration {

    @Bean
    AuthorizationManagerFactory<Object> authorizationManagerFactory() {
①
        var webauthn = AllRequiredFactorsAuthorizationManager.builder()
            .requireFactor(RequiredFactor.Builder::webauthnAuthority)
            .build();

②
        var passwordAndOtt = AllRequiredFactorsAuthorizationManager.builder()
            .requireFactor(RequiredFactor.Builder::passwordAuthority)
            .requireFactor(RequiredFactor.Builder::ottAuthority)
            .build();

        var mfa = new DefaultAuthorizationManagerFactory<>();
        mfa.setAdditionalAuthorization(AllRequiredFactorsAuthorizationManager.anyOf
(webauthn, passwordAndOtt));
        return mfa;
    }

}
```

1. We're going to register two required additional authorizations. One required authorization

specifies that the user must authenticate with *both* one-time tokens and passwords. The other just says the user must authenticate with Passkeys.

Restart the application and enter the Passkey, and you'll be staring at an authenticated page. Logout and then enter a username or a one-time token, and you'll be asked to do another authentication.

Authorization: Deciding Who Can Do What

Everything so far has answered one question: *who is this request from?* That's authentication. The multifactor configuration we built in the previous chapter was simple and *global* - one rule applied to every request and every endpoint. Now we turn to the second half of security: given that we know who you are, *what are you allowed to do?* This is *authorization*, and Spring Security supports it as *defense-in-depth* - multiple independent layers, at the method level and at the request level, so that a breach of one layer doesn't hand an attacker the keys to the others.

Request-based authorization is the kind we'll mostly be dealing with here, but we'll start with method-level authorization, which has fewer moving parts.

Method-Level Authorization

Method-level authorization in Spring Security uses annotations to restrict access to individual Java methods according to the user's roles or permissions. By applying annotations such as `@PreAuthorize` or `@Secured` directly to bean methods, Spring enforces fine-grained access control before or after method execution. Because the check lives on a proxy rather than in the method body, it applies to every call that arrives through the bean's public API - no matter which controller, listener, or scheduled job made it. The corollary is worth internalizing: a call a bean makes to *itself* doesn't go through the proxy, so `@PreAuthorize` on a method invoked via `this` is silently skipped. The same goes for `private` methods.



`@Secured` requires that you use `@EnableMethodSecurity(securedEnabled = true)`.

Method-level authorization can be quite useful, particularly when securing things where the granularity of the service might be smaller than the ongoing HTTP request, as with Spring GraphQL.

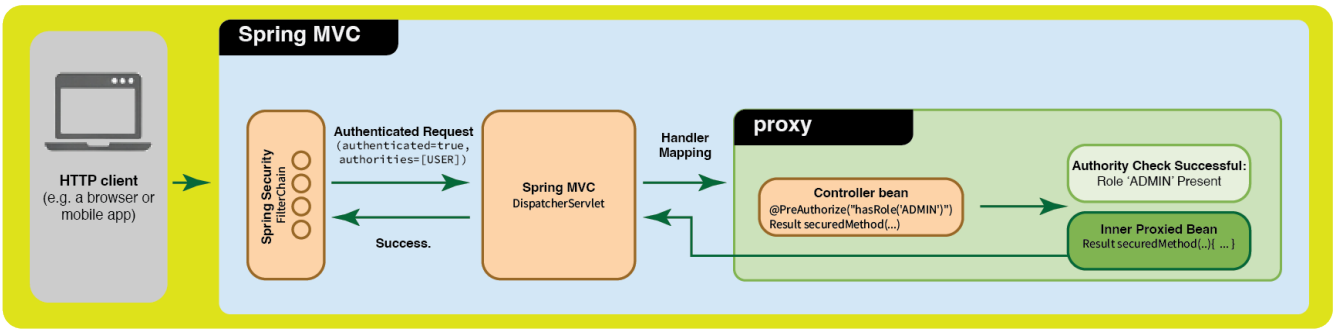
In method-level security, all interactions with your application's beans are gated by a sort of filter (in this case, a proxy on your bean) that verifies there's a user of some sort with a particular set of permissions. These gates happen right as you make a call to a bean.

There are two kinds of checks: pre-authorization and post-authorization. Pre-authorization checks are a good place to ensure that a given user has the necessary rights (permissions, roles, authorities, whatever) to make this call. A post-authorization check is a good place to ensure that the returned value is one to which the authenticated principal actually has rights. Both checks throw exceptions of type `AccessDeniedException`, which is a `RuntimeException`. You should handle this exception in your client code if you want to do something on the spot. Otherwise, if you let it bubble up in a web container, Spring Security's default filter machinery will intercept the exception and translate it into the usual HTTP error codes.

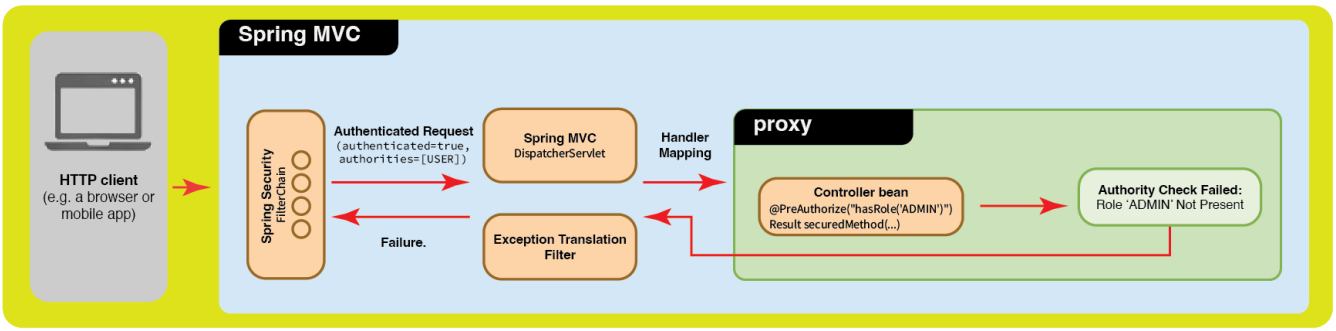


Mind the difference in *when* these two checks run. A pre-authorization check happens before the method body executes, so a rejected call changes nothing. A post-authorization check happens *after*: whatever the method did on its way to producing that return value - rows written, records deleted, messages published, third-party APIs called - has already happened. Denying access at that point

> withholds the result; it does not undo the work. Reserve `@PostAuthorize` for methods that only read.



1. a successful request makes its way past the proxy boundary to the underlying bean



1. a failed request will be rejected and sent back through the exception translation layer before ever reaching the underlying bean

Let's look at some examples. First things first: we need to *enable* method security.



> You might see `@EnableGlobalMethodSecurity` in older code. This is deprecated, and you're encouraged to move to the newer approach shown here.

You can put `@EnableMethodSecurity` on any `@Configuration` class or on your `@SpringBootApplication`, which is, of course, a `@Configuration` class. Of note here: Spring Boot does *not* enable method security for you by default.

```
package com.example.security.authorization.methods;

import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecuri
ty;

@Configuration
@EnableMethodSecurity
class CustomerServiceConfiguration {

}
```

Let's look at a trivial service that handles potentially sensitive data. We want to be sure both that

the person making the request is an `ADMIN` *and* that the data returned belongs to them.

```
package com.example.security.authorization.methods;

import org.springframework.security.access.prepost.PostAuthorize;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Service;

import java.util.Map;

@Service
class CustomerService {

    private final Map<Long, Customer> customers = Map.of(//
            1L, new Customer("josh@joshlong.com"), //
            2L, new Customer("user@anotherdomain.site")//
    );

    @PreAuthorize("hasRole('ADMIN')") ①
    @PostAuthorize(" returnObject  != null  && returnObject.username ==
authentication.name ")
②
    Customer readPrivilegedCustomerData(long id) {
        return this.customers.getOrDefault(id, null);
    }

}

record Customer(String username) {
}
```

1. the `@PreAuthorize` annotation's Spring Expression Language expression has access to all the methods on Spring Security's `SecurityExpressionRoot` class. This class has convenience methods to check for all authorities or any authority, to wave the attempt blindly on through, to reject it outright, and so on.
2. we've already looked up the domain data, but let's inspect the method's return value to make one last check that it belongs to the requester. Imagine the kinds of tenancy checks you could build with this!

Now let's exercise this code.

First, we'll need a `TestcontainersConfiguration` class. Let's look at that.

```
package com.example.security.authorization.methods;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.postgresql.PostgreSQLContainer;
```

```
import org.testcontainers.utility.DockerImageName;

@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer(DockerImageName.parse("postgres:latest"));
    }

}
```

Standard stuff. You can have this generated for you by the Spring Initializr.

Now let's look at a test that depends on this test database and that furnishes mock users with Spring Security's `@WithMockUser`.

```
package com.example.security.authorization.methods;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.test.context.support.WithMockUser;

①
@Import(TestcontainersConfiguration.class)
@SpringBootTest
class CustomerServiceTest {

    private final CustomerService customerService;

    @Autowired
    CustomerServiceTest(CustomerService customerService) {
        this.customerService = customerService;
    }

②
    @Test
    @WithMockUser(username = "josh@joshlong.com", roles = { "ADMIN", "USER" })
    void good() {
        try {
            var customer = this.customerService.readPrivilegedCustomerData(1);
            Assertions.assertEquals("josh@joshlong.com", customer.username(), "the id
should be 1");
        } //
        catch (Exception e) {
```

```
            throw new RuntimeException(e);
        }
    }

③
    @Test
    @WithMockUser(username = "bob")
    void badPre() {
        Assertions.assertThrows(AccessDeniedException.class, () -> this
.customerService.readPrivilegedCustomerData(1));
    }

    @Test
    @WithMockUser(username = "bob", roles = { "USER" })
    void badPost() {
        Assertions.assertThrows(AccessDeniedException.class, () -> this
.customerService.readPrivilegedCustomerData(2));
    }
④

}
```

1. we're using Spring Boot's very convenient `@SpringBootTest` to autoconfigure the application so that we can interact with the fully configured runtime.
2. Spring Security's `@WithMockUser` annotation lets us describe a contrived, *mock* user to plug into the context. In this case, the user has a username of `josh@joshlong.com` and two roles.
3. this call should be rejected because our mock user doesn't have the roles the pre-authorization check requires.
4. this call should be rejected because the returned `Customer` belongs to another user.



Spring Security provides other annotations that give you progressively more control over the mock object installed in the context. `@WithMockUser` lets you describe the attributes of a user, which it will then place in the `SecurityContext`. `@WithUserDetails` lets you specify a username along with the name of the `UserDetailsService` bean from which to source that `User`.

Neat, eh? You can use this in addition to request-based authorization, which we'll look at next.

Request-Based Authorization

We looked at method-level authorization first because, conceptually, there are fewer moving parts. Most people, however, will start with request-based authorization, wherein you secure the application as a whole, at the network protocol level, before control flow ever enters one of your methods. A good many of the things you want to secure are HTTP-based, and Spring Security has excellent authorization support for path-based matching. But HTTP is often just the beginning: gRPC is built on HTTP, and yet it makes no sense to use HTTP path matchers for gRPC. Ditto the Model Context Protocol (MCP).

So, keeping that in mind, let's look at a trivial HTTP example and see how we can secure access to that application - enforcing both authentication and authorization.

Here's our controller, with two endpoints whose access we intend to limit with authorization rules.

```
package com.example.security.authorization.requests;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class ProfileController {

    static final String USER_PATH = "/profiles/user";

    static final String ADMIN_PATH = "/profiles/admin";

    @GetMapping(USER_PATH)
    Map<String, String> user(Principal principal) {
        return principal("user", principal.getName());
    }

    @GetMapping(ADMIN_PATH)
    Map<String, String> admin(Principal principal) {
        return principal("admin", principal.getName());
    }

    private Map<String, String> principal(String attribute, String principal) {
        return Map.of(attribute, principal);
    }

}
```

There are two controller handler methods: one (`/profiles/admin`), which we intend only for users with the `ADMIN` role, and the other (`/profiles/user`), which anybody who has successfully authenticated - that is, anybody we know to be who they say they are - can access. Note that we're using static final constants to describe the route paths so that we can share them in the forthcoming authorization configuration.

Speaking of which: here's a first cut at the authorization rules to enforce these access conditions.

```
package com.example.security.authorization.requests;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Profile("simpleauthorization")
@Configuration
class ProfileControllerConfiguration {

    @Bean
    Customizer<HttpSecurity> httpSecurityCustomizer() {
        return http -> http.authorizeHttpRequests(a -> a
①
            .requestMatchers(ProfileController.ADMIN_PATH)
            .hasRole("ADMIN")
②
            .requestMatchers(ProfileController.USER_PATH)
            .authenticated());
    }

}
```

1. the first rule requires that all access to the `/profiles/admin` path be by users who have `ROLE_ADMIN`.
2. the second requires that all access to the `/profiles/user` path be by users known to the system.

Two things about that first rule are worth pausing on, because both trip people up.

The first is the `ROLE_` prefix. We wrote `hasRole("ADMIN")`, but the authority the rule actually looks for is `ROLE_ADMIN`. Spring Security has a long-standing convention that *roles* are simply *authorities* that begin with `ROLE_`, and `hasRole(String)` prepends that prefix for you. Its sibling, `hasAuthority(String)`, does not: `hasAuthority("ADMIN")` and `hasRole("ADMIN")` look for two different strings, and only one of them will match a user granted `ROLE_ADMIN`. If a rule that looks obviously correct is rejecting everybody, this is the first place to look.

The second is what a rejection looks like from the outside. Unlike the method-level checks we just saw, these rules are evaluated in the filter chain, long before your controller is called, so there's no exception for your code to catch. Spring Security answers on your behalf, and it distinguishes two cases: if the request arrived with no established identity at all, you get a `401 Unauthorized` (or a redirect to the login page, for a browser session), because the fix is to log in. If the request arrived from a perfectly well-authenticated user who simply lacks the required authority, you get a `403 Forbidden`, because logging in again won't help.

Easy! The simple stuff is simple, as it should be. But Spring Security supports a good deal more than this. Let's turn to a slightly more elaborate example involving a content-management system. Or, at least, an API that one could imagine might one day be used for a content-management system.

Here's the `CmsController`.

```
package com.example.security.authorization.requests;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;

①
@Controller
@ResponseBody
@RequestMapping("/api")
class CmsController {

②
    @GetMapping("/articles/{articleId}")
    Article article(@PathVariable int articleId) {
        IO.println("looking up article #" + articleId);
        return new Article(articleId);
    }

③
    @DeleteMapping("/admin/users/{userId}")
    void delete(@PathVariable long userId) {
        IO.println("going to delete " + userId);
    }

④
    @GetMapping("/files/file.{fileId}")
    ArticleFile file(@PathVariable int fileId) {
        IO.println("looking up file #" + fileId);
        return new ArticleFile(fileId);
    }

    record Article(int id) {
    }

    record ArticleFile(int id) {
    }

}
```

1. all the endpoints in this controller live under `/api`.
2. this is a simple endpoint that requires nothing more than the garden-variety `USER` role.
3. this is a supersensitive endpoint that only admins can access.
4. this endpoint is even more privileged: only administrators should be allowed to access it, and even then we need to be sure they have rights to this particular file.

This is a toy example, meant only to demonstrate some concepts. Let's see how we wire up the configuration.

```
package com.example.security.authorization.requests;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authorization.AuthorizationDecision;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import java.util.List;
import java.util.Objects;

@Configuration
class CmsControllerConfiguration {

    @Bean
    Customizer<HttpSecurity> cmsHttpSecurityCustomizer() {
        return http -> http.authorizeHttpRequests(auth -> auth
①
            .requestMatchers(HttpMethod.POST, "/api/**")
            .hasRole("EDITOR")
②
            .requestMatchers("/api/admin/**")
            .hasRole("ADMIN")
③
            .requestMatchers("/api/files/file.{fileId}")
            .access(//
                    (supplier, authorizationContext) -> {
                        var authentication = supplier.get();
                        var isAdmin = Objects.requireNonNull(authentication)
                            .getAuthorities()//
                            .stream() //
                            .anyMatch(ga -> Objects.requireNonNull(ga.getAuthority())
.equals("ROLE_ADMIN"));
                        var variables = Objects.requireNonNull(authorizationContext)
.getVariables();
                        if (!variables.isEmpty()) {
                            var fileId = parseFileId(variables.getOrDefault("fileId",
"0"));
                            IO.println("the file id is " + fileId);
                            return new AuthorizationDecision(isAdmin && canAccess
(fileId));
                        }
                        return new AuthorizationDecision(false);
                    })
④
            .requestMatchers("/api/**")
            .hasRole("USER"));
    }
```

```
⑤
    private long parseFileId(String value) {
        try {
            return Long.parseLong(value);
        } //
        catch (NumberFormatException e) {
            return 0;
        }
    }

    private boolean canAccess(long fileId) {
        var goodFileIds = List.of(1L, 2L, 3L);
        return fileId > 0 && goodFileIds.contains(fileId);
    }

}
```

1. first, we'll ensure that any `POST` request to any endpoint under `/api/**` requires the `EDITOR` role.
2. next, we'll require the `ADMIN` role for any request under `/api/admin/**`.
3. then we'll guard the file endpoint with a rule of our own: the user must have the `ADMIN` role *and* the file being requested must meet some (in this case, admittedly arbitrary) criteria.
4. finally, `/api/**` catches everything else, requiring the `USER` role.
5. the path variable arrives as a `String`, and nothing guarantees it's a number: `/api/files/file.` matches this pattern perfectly well and binds `fileId` to an empty string. Parsing that would throw, and an exception raised inside an `AuthorizationManager` becomes a `500` rather than a clean denial - an unhelpful answer to give an attacker who is probing you. We swallow the parse failure and return `0`, which `canAccess` rejects. This example is meant to demonstrate that you can do almost anything in these authorization rules.

Those patterns are not interchangeable, and the difference between one asterisk and two is the difference between a rule that fires and a rule that doesn't. A single `*` matches within one path segment. A double `**` matches any number of segments, including none at all. It's worth committing the following to memory, because every one of these rows has bitten somebody:

Request path	`/api`	`/api/*`	`/api/**`
`/api`	yes	no	yes
`/api/`	no	yes	yes
`/api/foo`	no	yes	yes
`/api/articles/1`	no	no	yes
`/api/foo/bar/baz`	no	no	yes

Two rows deserve a second look. An exact pattern like `/api` really is exact: it does not match `/api/`, so a stray trailing slash walks straight past a rule you thought you'd written. And `/api/*` matches `/api/foo` but not `/api/articles/1`, because the latter is two segments deep - which is precisely why the second rule in our listing is `/api/*` and not `/api/*`. Patterns with variables in them, like

`/api/files/file.{fileId}`, follow the single-asterisk rule: the variable is confined to one segment, so it matches `/api/files/file.1` and not `/api/files/file.1/thumbnail`.

Order matters here, and it matters more than almost anything else in this chapter. `authorizeHttpRequests` evaluates these rules from top to bottom and stops at the first one whose matcher accepts the request. That's why the broad `/api/**` rule has to come last. Had we declared it first, it would shadow the three more specific rules beneath it and they would never run at all: every request under `/api` would sail through on nothing more than the `USER` role, and the `ADMIN` check and the file check would quietly become dead code. Nothing would fail, no exception would be thrown, and the application would start up perfectly happily. It would simply be less secure than it looks. So: declare the most specific matchers first, and let the general ones fall to the bottom.

There's a companion question worth asking of any rule set: what happens to a request that matches *nothing*? Our rules here all begin with `/api`, so a request to `/somewhere/else` runs off the end of the list without ever being decided. Spring Boot's defaults are kind to us and require authentication for whatever we haven't spoken for, but relying on that is a bet on a default you didn't write and may not control forever. The habit worth forming is to terminate the chain explicitly, with `anyRequest().authenticated()` when you want a sane floor, or `anyRequest().denyAll()` when you'd rather a forgotten endpoint fail loudly than quietly let people in. A new controller added six months from now will land in whichever of those you chose.

Granular Multi-Factor Authentication

What's this? Multifactor *authentication*? "I thought we were talking about *authorization*, Josh!" I hear you lament. And you're right. We already talked about how to apply multifactor authentication globally. In this section, we're going to look at how to carve out more granular authorization rules that selectively apply multifactor authentication. We express that granular, selective application of multifactor authentication using the same API we use for authorization. After all, once a user is authenticated, they carry a set of *authorities*, and those authorities are what our rules read. The fact that they authenticated with a given factor - a one-time token, a password, a passkey - is itself stored as an authority.

Imagine: a user hits your website and is able to use most of the site securely after authenticating with a username and password or with a one-time token. Then the user decides to visit *the danger zone* (cue Kenny Loggins's music): they want to delete their account! (*Danger, Will Robinson!*) At this point, we need to be extra sure that the user is who they say they are and is taking an action that they, and they alone, should be allowed to take. The same could be said for tons of other actions:

- changing a password
- changing access keys
- changing the MFA configuration itself
- changing the credit cards on file
- viewing audit logs
- sending high-impact communications, such as changing a verified sender or blasting out a mass message
- granting legal or consequential approvals, such as accepting contracts, signing documents

electronically, submitting regulated information, or approving another user's sensitive request

The list goes on and on. MFA is a way of asking: *are you sure?* Let's return to the `ProfileController` introduced earlier, with its two endpoints: `/profiles/user`, for all users, and `/profiles/admin`, for only those users with the `ADMIN` role. We want to require MFA before anyone can access the `/profiles/admin` endpoint.

```
package com.example.security.authorization.requests;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.authorization.AuthorizationManagerFactories;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authorization.EnableMultiFactorAuthenti
cation;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.authority.FactorGrantedAuthority;

①
@Profile("requestsmfa")
@Configuration
@EnableMultiFactorAuthentication(authorities = {})
class MfaConfiguration {

    @Bean
    Customizer<HttpSecurity> mfaCustomizer() {
②
        var mfaAmf = AuthorizationManagerFactories.multiFactor()
            .requireFactors(FactorGrantedAuthority.PASSWORD_AUTHORITY,
FactorGrantedAuthority.OTT_AUTHORITY)
            .build();
        return security -> security.authorizeHttpRequests(a -> a
③
            .requestMatchers(ProfileController.ADMIN_PATH)
            .access(mfaAmf.hasRole("ADMIN"))
④
            .requestMatchers(ProfileController.USER_PATH)
            .authenticated());
    }

}
```

1. in this example we enable MFA as before, but this time we specify no required `authorities`.
2. for our particular `/profiles/admin` endpoint, we want to require that both the *one-time token* and *password* authorities be present on the authenticated user. These can be added only if the user has successfully completed both types of authentication. We'll create a custom `AuthorizationManager` specifically for requests routed to the `/profiles/admin` endpoint by using

the convenient factory methods on the `AuthorizationManagerFactories` class. We did something similar earlier when we locked down the `CmsController`s `/api/files/file.{fileId}` path. Here, though, we can rely on built-in factory methods.

3. we craft the `requestMatchers` call as before, but now pass in the `AuthorizationManagerFactory` instance and terminate it with a call to one of the final builder methods. In this case, I want to enable MFA *and* require that the user be an `ADMIN`, so I use the `hasRole(String)` method.
4. finally, we declare a rule that allows all requests to the `/profiles/user` endpoint to proceed, assuming the request is authenticated.

In the example above, we created an `AuthorizationManager` using `AuthorizationManagerFactories`. If you define that factory as a bean in the context, it will apply to all requests.

Earlier, we established that if a user can authenticate with passkeys, it's not unreasonable to spare them the second authentication attempt, since passkeys are so much more secure and tamperproof. We can ask Spring Security to authenticate somebody multiple times, to establish multiple factors. And this implies an interesting corollary: not all authentications are created equal, and some are more valuable than others. We want to establish that the user is who they say they are. Our primary factors, and even some supporting ones, should be focused on authenticating our users - on proving that they are who they say they are. Since Spring Security supports arbitrarily many factors, we can afford to spend some of them establishing *other* things - secondary factors - about users, such as how recently they accessed the system. We could also build other factors if we wanted. Spring Security ships with exactly one such factor, and it's the one we'll use here: how recently the user authenticated. It would make a poor primary factor - knowing that somebody logged in eleven seconds ago tells you nothing about *who* they are - but as a secondary factor it's precisely what the danger zone calls for. It lets us say that a password from an hour ago was fine for browsing and is not fine for deleting an account. Let's see how that works, and then return to the factors you might write yourself.

```
package com.example.security.authorization.requests;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authorization.AuthorizationManagerFactories;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authorization.EnableMultiFactorAuthenti
cation;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.authority.FactorGrantedAuthority;

import java.time.Duration;

①
@EnableMultiFactorAuthentication(authorities = {})
@Configuration
class TimeboxedMfaConfiguration {

    @Bean
```

```
    Customizer<HttpSecurity> mfaCustomizer() {
②
        var mfaAmf = AuthorizationManagerFactories.multiFactor().requireFactors
(objectBuilder -> {
            var duration = Duration.ofSeconds(30);
            var factors = new String[] { FactorGrantedAuthority.OTT_AUTHORITY,
                    FactorGrantedAuthority.PASSWORD_AUTHORITY };
            for (var fga : factors) {
③
                objectBuilder.requireFactor(b -> b.authority(fga).validDuration
(duration));
            }
        }).build();
        return security -> security.authorizeHttpRequests(a -> a.requestMatchers
(ProfileController.ADMIN_PATH)
            .access(mfaAmf.hasRole("ADMIN"))
            .requestMatchers(ProfileController.USER_PATH)
            .authenticated());
    }

}
```

1. just as before: enable MFA, but don't specify any global factor requirements.
2. we build a custom `AuthorizationManagerFactory`…
3. …but this time we'll use the slightly more, um, *detailed* DSL to specify both the required factors *and* the duration during which each one is considered valid.

Launch the program and authenticate. Access `/profiles/admin`, and you'll be prompted to authenticate again. Do so, then stay on the page and do nothing for 30 seconds - that number is the `Duration.ofSeconds(30)` in the listing above, and it's the whole of the window. Access `/profiles/admin` again, and you'll be prompted to authenticate, *again*. Thirty seconds is an absurd figure for a real application, chosen here only so you don't have to wait around; in production you'd measure this in minutes.

Laid out against the clock, with both endpoints in view, the behavior looks like this:

Elapsed	**State of the session**	`/profiles/user`	`/profiles/admin`
before sign-in	no authentication at all	sent to sign in	sent to sign in
t = 0s	password and one-time token both just accepted	allowed	allowed
t = 20s	both factors held, still inside the window	allowed	allowed
t = 31s	both factors still held, but both now stale	allowed	challenged for the factors again
after re-authenticating	factors refreshed, the window restarts	allowed	allowed

The fourth row is the one to sit with. At thirty-one seconds the user has not been signed out, and nothing about their session has been discarded - `/profiles/user` never so much as blinks, because all it ever asked for was `authenticated()`. What expired is narrower than that: the *factors* went stale, and only the rule that demanded fresh ones noticed. This is the practical shape of the danger zone we started the section with. The user browses uninterrupted all day, and the one moment they reach for something consequential, they're asked to prove themselves again.

Recency is the factor Spring Security hands you, but it's not the only one worth having. Nothing stops you from writing your own and having the framework enforce it exactly as it enforces the built-in ones. Is this request coming from the same place - or at least the same IP address - as the last one? Can whoever is on the other end pass a CAPTCHA? Are they a human being at all? Each of those is an assertion about a user that a rule can require, and none of them is authentication in the ordinary sense.

Authentication Events in Spring Security

Everything so far has been about *deciding*: this user may pass, that one may not. Decisions are worth recording as well as making, and a rule that quietly denies a thousand requests an hour is telling you something you'd want to know. Spring Security will tell you, if you ask it to.

One of the benefits of authentication is that it helps establish who's using the system. The lifecycle of a given user's session is interesting in and of itself. When did they sign in? Fail to sign in? When did they sign out? Each of those is a published event, and each is worth something different to you: a sign-in tells you a session has begun and any per-user state you cache can be warmed, a failure is the raw material of lockout policies and intrusion alerts, and a sign-out is your cue to release whatever you were holding on that user's behalf. Spring Security publishes a ton of these events, and you would do well to pay attention to them. I'm listening to some of them in a separate class, `SecurityEvents`:

```
package com.example.security.events;

import org.springframework.context.ApplicationEvent;
import org.springframework.context.event.EventListener;
import org.springframework.security.authentication.event.AbstractAuthenticationEvent;
import org.springframework.security.core.context.SecurityContextChangedEvent;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

@Component
class SecurityEvents {

    @EventListener
    void onSecurityEvent(ApplicationEvent securityEvent) {
        var eventMessage = switch (securityEvent) {
            case AbstractAuthenticationEvent abstractAuthenticationEvent -> message
(abstractAuthenticationEvent);
            case SecurityContextChangedEvent securityContextChangedEvent -> message
(securityContextChangedEvent);
            default -> null;
```

```
        };
        IO.println("security result: " + eventMessage);
    }

    private static String message(ApplicationEvent applicationEvent) {
        Assert.notNull(applicationEvent, "the event should not be null");
        return applicationEvent.getClass().getName() + " ::: " + applicationEvent;
    }

}
```

Run the program and then hit `/profiles/user`. You'll see interesting events such as `InteractiveAuthenticationSuccessEvent`, `SessionFixationProtectionEvent`, and `AuthenticationSuccessEvent`. Those three arrive from a single sign-in, which should give you a sense of how much detail is on offer here for the cost of one `@EventListener`.

Long-Term Credentials vs. Short-Term Credentials

For most of this chapter, to keep things simple, we've been using username- and password-based authentication. We're asking the system to take the password sent by the user and then encode it. Most of the good `PasswordEncoder` algorithms, such as BCrypt, intentionally take a long time to compute. The whole point is to stop would-be attackers from brute-forcing their way in by sending every password in the dictionary and asking the system to try each one. This is one of the reasons SHA-256 is effectively deprecated for this purpose: it's still very hard to reverse or *crack* (though SHA-1 *is* easy to crack, so *never* use that one!), but it's also very quick to encode. A hacker could submit millions of guesses, and the algorithm would encode them all as quickly as the hardware allows. So we use slow password encoders, and this works for the primary authentication - the one that happens at the user-facing boundary of the system.

That boundary is the important word, because it's crossed exactly once. Behind it, a single incoming request may fan out into a dozen calls from one service to another, and each of those has the same question to answer: on whose behalf is this happening? Answering it by passing the username and password along won't do. Deliberately slow is a virtue when you're checking a human being's password once at the front door; it's a catastrophe when you're doing it twelve times an internal hop. And every service that handles a password is another service that can leak one, which is a poor trade for credentials that are good until the user thinks to change them. What those internal hops want is a credential that's cheap to verify, narrow in what it permits, and short-lived enough that losing one is an incident rather than a breach.

In the next chapter, which concentrates on OAuth, we'll look at how to exchange long-term credentials, such as usernames and passwords, for exactly that: short-term credentials, such as tokens. = Introducing OAuth

I have been using Yelp [https://www.yelp.com] for a *very* long time.

Since 2007, I'd guess. I was living in Phoenix, Arizona at the time, and I'd started making regular pilgrimages to my hometown of Los Angeles, California. While I was there, I wanted to make the most of the limited time I had in town, so I started looking for tools to help.

Enter: Yelp.

This was the beginning of my journey as a prolific traveler. I love travel. I maintained my Yelp account and even uploaded a profile photo - one that's still there to this day, I think - of me from a trip to Manila, Philippines.

I loved my account. I love Yelp! It connected me with the world. But there's a limit to my love, and a limit to how much I want it connecting with *my* world.

Did you know there was a (mercifully) brief period when Yelp asked users for their Gmail usernames and passwords so it could scan their contacts and help them connect with friends already on Yelp? Yes, that's right. Yelp wanted you to authenticate with your Gmail account on their website so they could log in to your email and *poke around*, basically.

Insanity! Or at least, by today's standards, it is. But back then, there really wasn't an alternative.

The All-or-Nothing Problem

The internet was becoming a more connected place. Companies were beginning to build on the promise of interconnected services, and these kinds of hurdles were common. Yelp wasn't the only site that asked you to enter credentials on its property so it could log in on your behalf somewhere else.

And we all just sort of went with it because, after all, what was the alternative?

This was the all-or-nothing problem. And it led to even more problems.

I trusted Yelp, so I'm sure it would have handled my data with care. But the same couldn't be said of every service that asked. This created a fraught situation where, with every integration and authentication, you were potentially opening yourself up to catastrophic data loss, a loss of privacy, or worse. Yelp would've been fine, but what about some less-than-scrupulous website? What if they went rogue with your credentials? You might not even have had a way to undo the damage if you later changed your mind, because they had your password; they could lock you out of your own account. Also: how would most of these websites have stored your password? Presumably, they'd have to use a two-way hash, encoded using some sort of algorithm and then decoded with the key. But that means that your Google password was one stolen key away from being stored in plaintext! As bad as the AI-driven CVE situation is today, can you imagine if every site you integrated with had access to your *Google password*? You'd have to trust that every one of them doesn't get *pwned*! Sleeping would be *much* more difficult! OK, I'll stop. I know it's all too deeply upsetting to consider. Mercifully, things improved.

Open Authorization for an Open Web

Developers across the SaaS community - particularly at Twitter and Ma.gnolia - realized there had to be a better way for systems and services to federate identity without sharing passwords.

OpenID, the reigning identity protocol of the day, handled *who you are* well enough, but it didn't cleanly cover the problem Twitter and Ma.gnolia kept hitting: third-party apps that wanted to *act on a user's behalf* without ever holding the user's password. By 2007, Chris Messina (yes, the

hashtag guy), David Recordon of Six Apart, Twitter's Blaine Cook, and Ma.gnolia's Larry Halff were sketching out something new. Yahoo!, AOL, Flickr, and Google all pitched in engineering effort.

In December 2007, the final draft of OAuth 1.0 was released to the world.

OAuth

OAuth (Open Authorization) is a valet key for the internet. A valet key starts the car and opens the driver's door, but it won't open the glove box or the trunk. OAuth works the same way: it lets a third-party application act on your behalf with a narrowly scoped key, without ever handing over your password.

You have probably used OAuth before. If you've ever clicked "Log in with Facebook" or "Log in with Google" or "Log in with Apple" on a website, you were using OAuth. Instead of creating a new username and password for that site, you're granting it permission to read a small slice of information from your account at the identity provider - usually just enough for the site to put together a profile and know what to call you.

People incorporate OAuth into their applications in all manner of different ways, but the basics are the same. You're using an *OAuth Client* - the application running in your browser, on your phone, or on your desktop - and it tries to access a resource that requires a token.

The OAuth Client redirects you to an *OAuth Authorization Server* (like Okta, Keycloak, Auth0, Active Directory, Clerk, etc.) where you authenticate. It is this OAuth Authorization Server that knows who you are, or can talk to something that does. Once you've authenticated, the OAuth Authorization Server issues a token and sends it back to the OAuth Client. There are several different flows for that handoff, each with its own trade-offs; we'll meet them later.

The OAuth client can now make requests on behalf of the user to APIs (these are called *Resource Servers*) using that OAuth token.

These *Resource Servers* will receive requests and need to validate them. If they receive a request that doesn't have a token, then the request is rejected. If the request has a token, and the token is a stateless JSON Web Token (JWT), then the token is signed and can be validated cryptographically without talking to the Authorization Server for each request. This works because the resource server has obtained the Authorization Server's public cryptographic keys on startup. The resource server validates the token *in-memory* using these public keys. It verifies the signature, the expiration date, and the issuer completely offline.

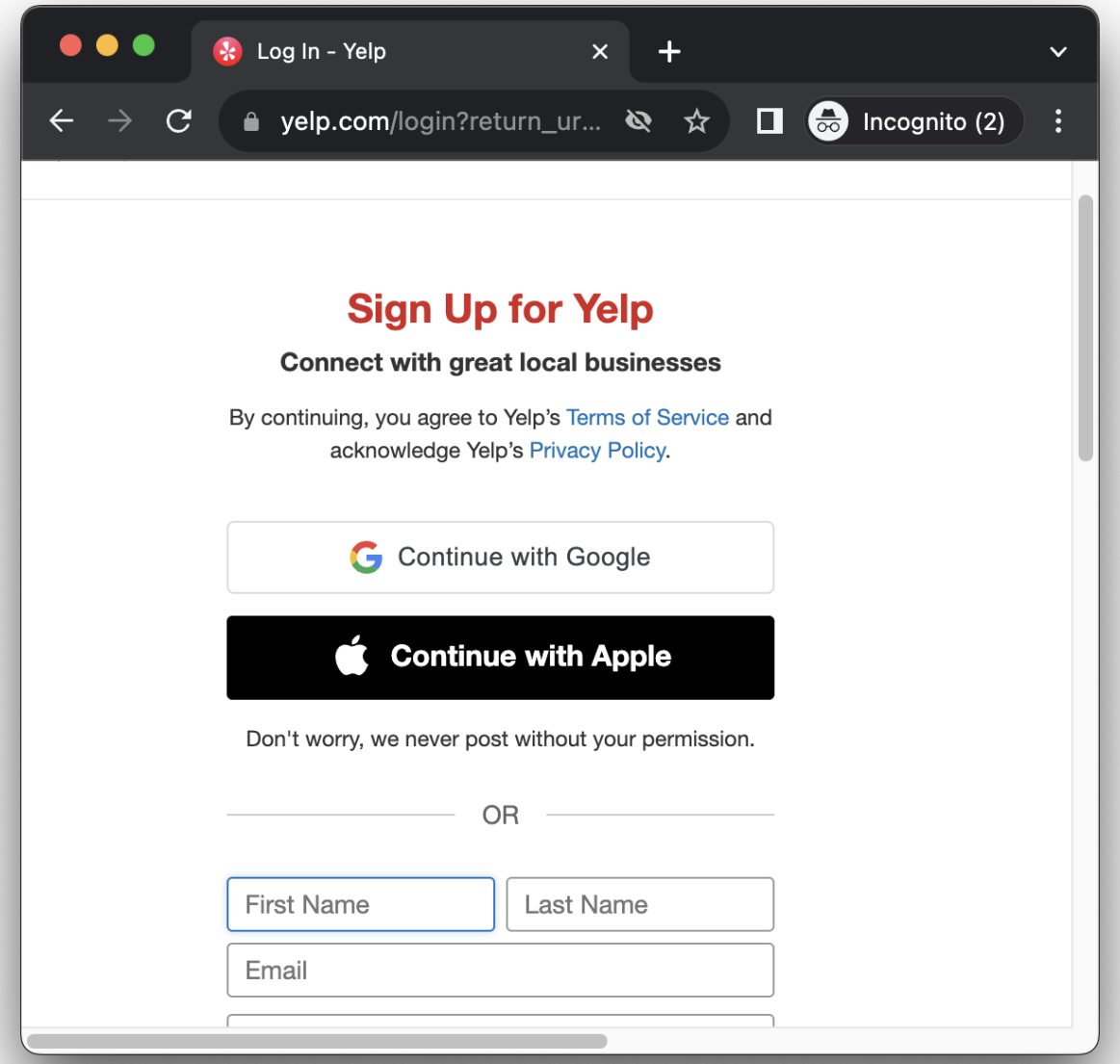
Otherwise, if the token is stateful (an opaque reference token), then the resource server *must* contact the Authorization Server on every single request.

The tokens contain information about what rights (*claims*) the user has. These tokens support *authorization*.

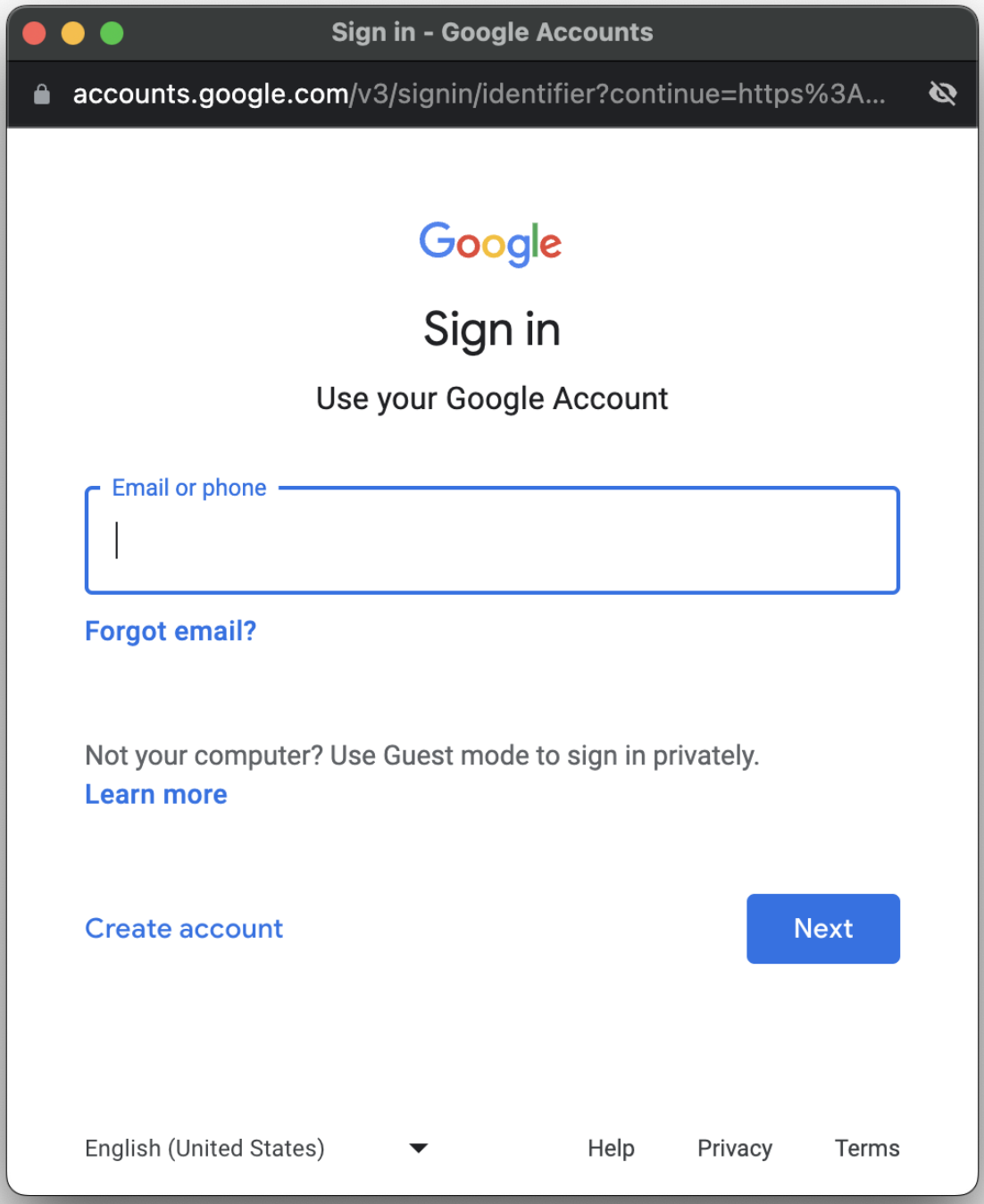
"Continue with Google," Step by Step

That's the abstract version. Let's watch it happen.

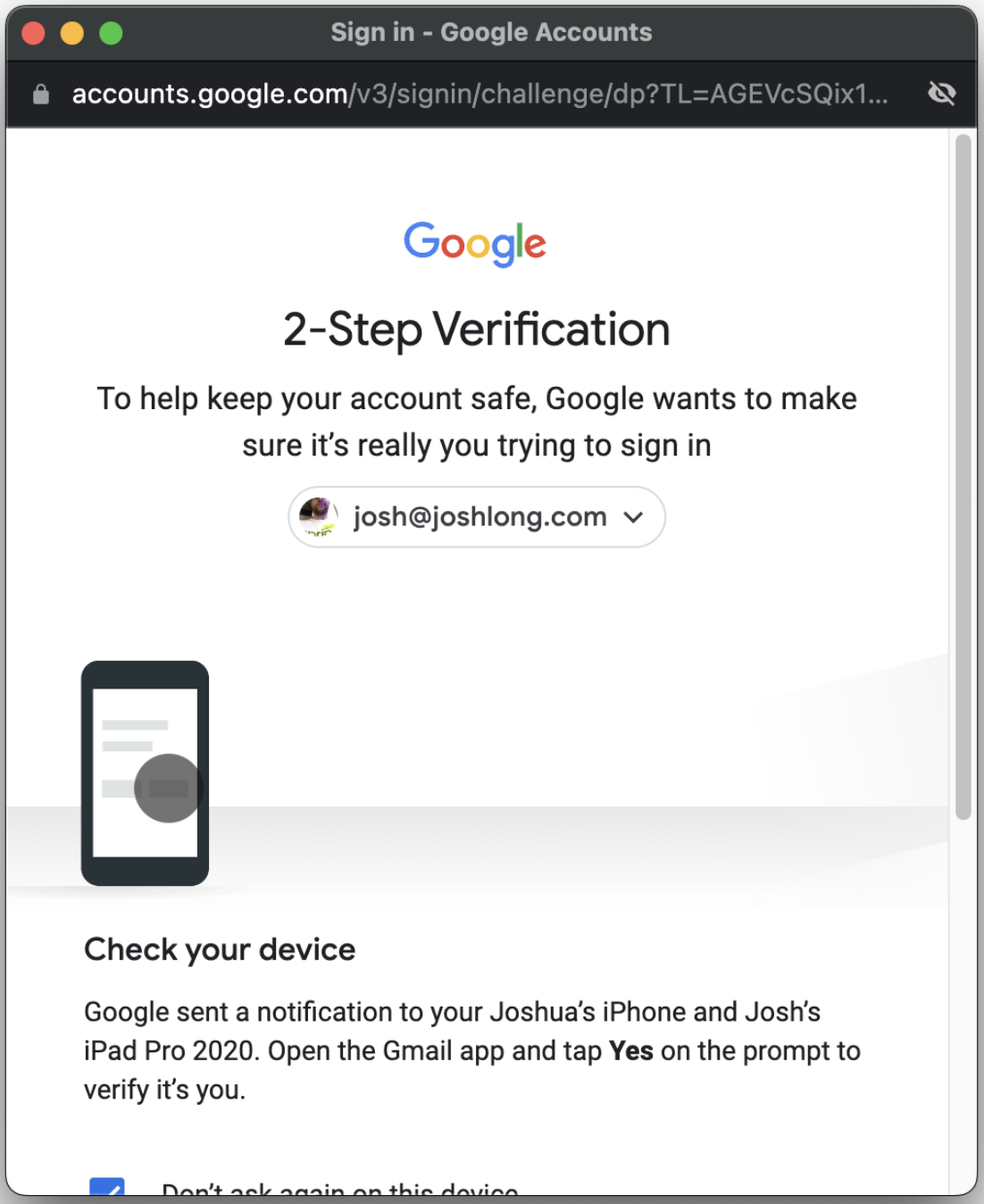
Remember Yelp, from the top of this chapter? Suppose you've stumbled back onto it years later and want to log in to see your history. You *could* create a brand-new account, grinding through yet another signup form and re-entering information you've already typed in a dozen other places - information that starts going stale the moment you forget to come back and update it. Yelp knows this, so it offers another path forward: `Continue with Google` and `Continue with Apple`.

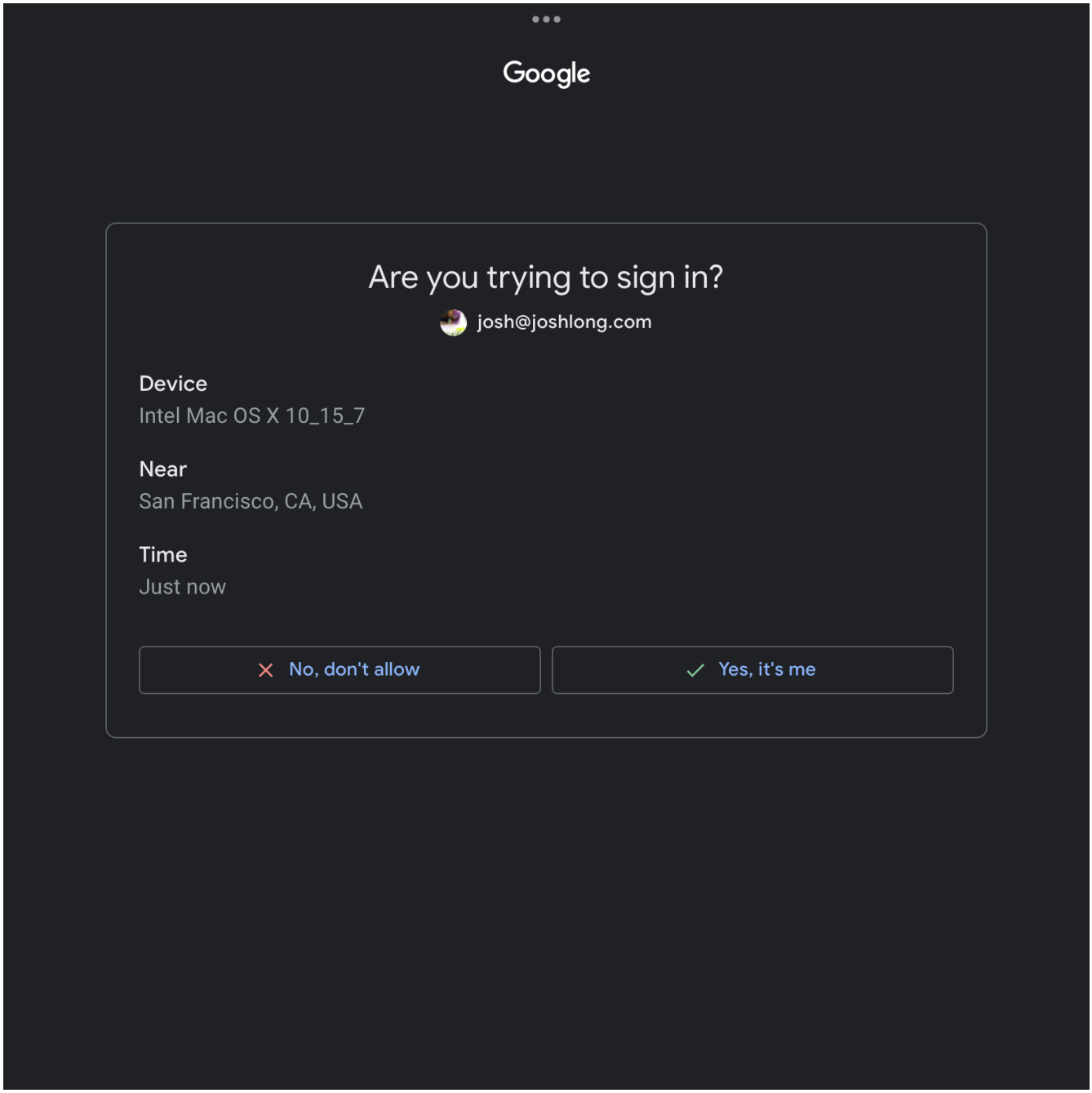


Click the button and another window on Google's or Apple's site pops up.

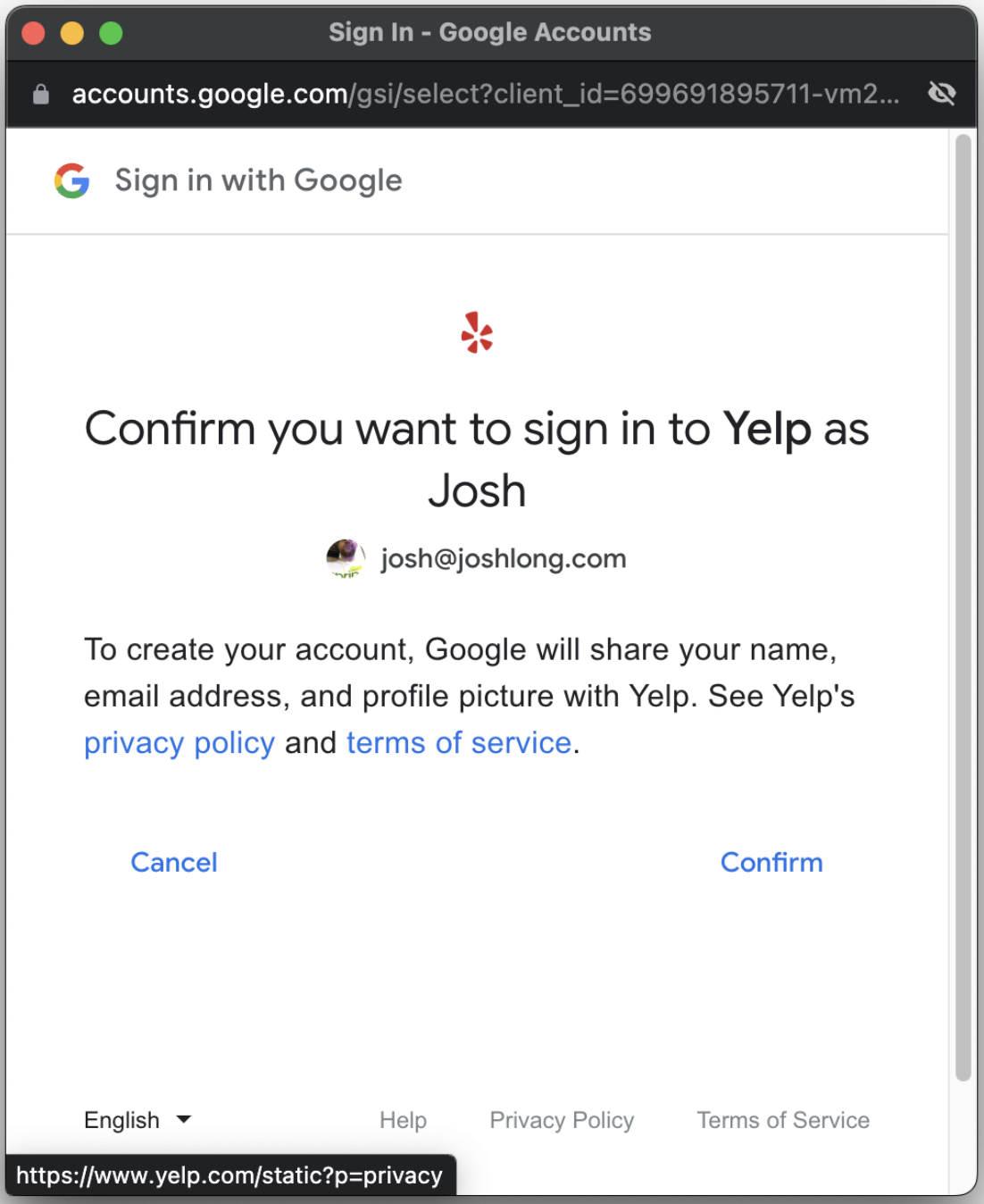


You know what to do here: you're in familiar territory. It's Google.com! You *know* Google. You use it for your daily email, after all. You've even got that reassuring little padlock icon in the browser's location bar giving you the warm-and-fuzzies about the site's authenticity: it is who it claims to be. So you enter your credentials and do whatever multifactor-auth dance Google asks of you. You might get a prompt on another device; you might use a passkey. Let's suppose the prompt lands on a completely different device - an iPad.

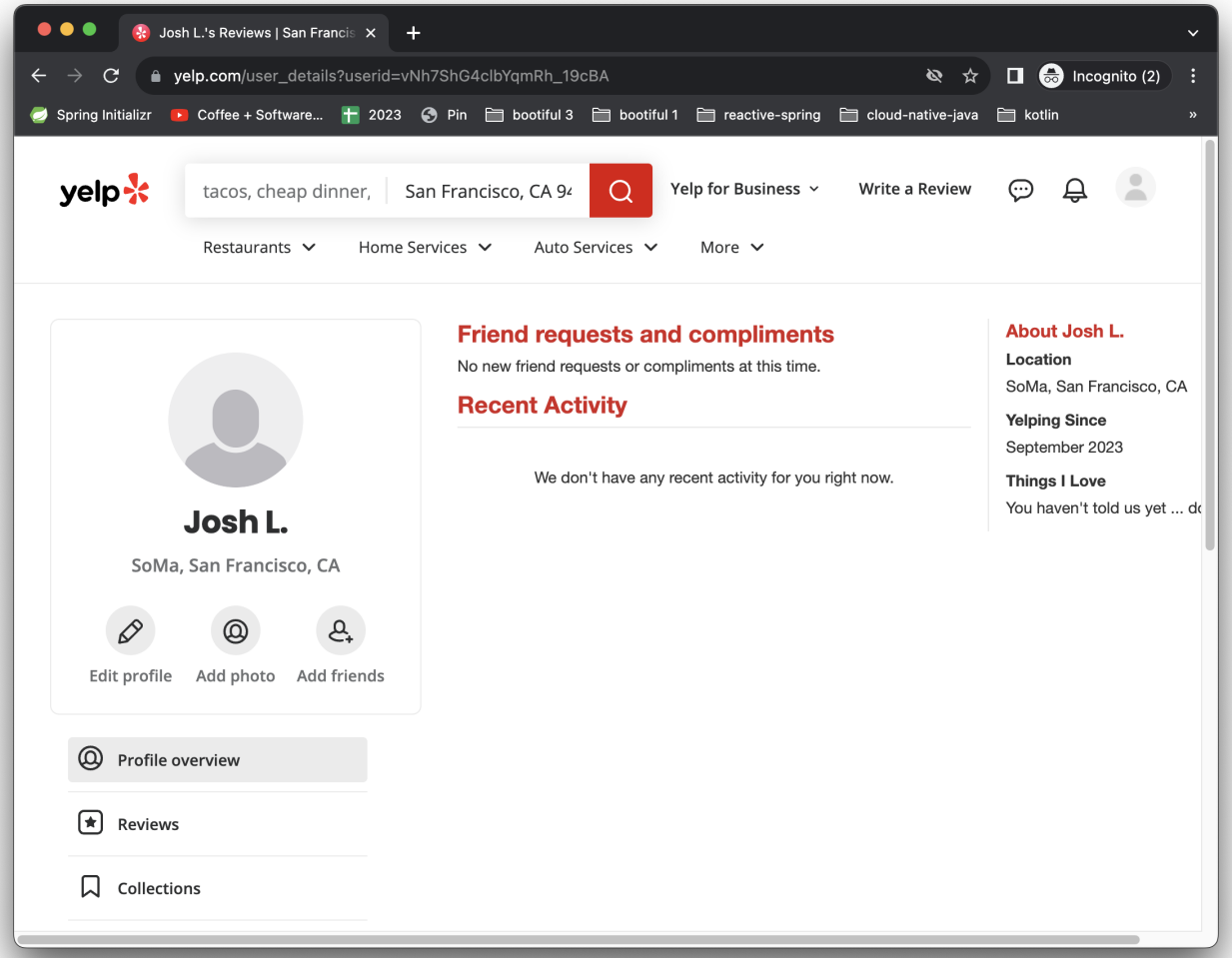

Sign in - Google Accounts
accounts.google.com/v3/signin/challenge/dp?TL=AGEVcSQix1...
Google
2-Step Verification
To help keep your account safe, Google wants to make sure it's really you trying to sign in
josh@joshlong.com
Check your device
Google sent a notification to your Joshua's iPhone and Josh's iPad Pro 2020. Open the Gmail app and tap Yes on the prompt to verify it's you.
Don't ask again on this device



You approve the login, so Google knows it really is you. Now Google has to make sure you realize you're about to hand some of your identity over to this new website, so it presents a consent form.



You click `Confirm` and are finally dropped into Yelp.com, logged in with your Google identity.



In OAuth's vocabulary, Yelp is the *OAuth Client*, Google is the *OAuth Authorization Server*, and the Google APIs that Yelp can now call are *Resource Servers*. At the end of the exchange, Google transmitted a *token* to Yelp. Armed with it, Yelp can call the Google APIs to ask questions about you - your name, your email - and, if you granted them, richer things like your calendar events or location data. What Yelp may and may not touch is governed by the *scopes* it requested: the valet key from a moment ago, opening the driver's door but not the trunk. Occasionally Google will expire the token - tokens, like milk, go stale - but Yelp was also handed a *refresh* token it can quietly exchange for a fresh one.

Now skip ahead a week. You return to Yelp, your session long since expired, and you're logged out. No problem: click `Continue with Google` again, and this time you're dropped straight in, fully authenticated, with no ceremony at all. Both Google and Yelp remember you. Better still, if you changed your home address on Google in the meantime, Yelp sees the new address and can offer you updated recommendations - you kept *one* account current, and the benefit rippled outward. OAuth, then, is invaluable both for establishing a new account and for logging back into an existing one.

OpenID Connect (OIDC)

OAuth was a huge leap forward, but it didn't handle every interesting use case. One of the first things people did once they had a valid access token was use it to ask the provider's API *who is this*

user? - their email, their name, a profile picture - and then sign them into their own application based on the answer. The implication was that providers like Google or Facebook had already done the work of confirming the email belonged to the person (and sometimes even checked government-issued ID along the way), so an application could trust that a returned email really *was* that person's email. If Google and Facebook both returned the same email, it really was the same person across two different ecosystems. This pattern became the basis of the early *Sign In with <X>* capabilities all around the internet, and let services focus on building interesting things instead of solving identity over and over again.

People were using OAuth to do *authentication*, and it caught on fast. The trouble was that every provider exposed those identity details in its own bespoke way, and it became pretty self-evident after OAuth 2.0 that there was a gap.

Enter OpenID Connect (OIDC).

OIDC launched in 2014 as a thin, standardized identity layer on top of OAuth 2.0. Its creation was a massive collaborative effort involving engineers from Google, Microsoft, Salesforce, Ping Identity, and Yahoo, with Nat Sakimura, John Bradley, Michael B. Jones, Chuck Mortimore, and Breno de Medeiros among the key contributors.

What OAuth Gives You

OAuth introduced three things you didn't have when applications were just trading passwords around.

First, **tokens instead of credentials**. Even if an attacker steals a token, they only have access for a limited time and only to the data the token was issued for.

Second, **scopes**. The user (or the developer on their behalf) declares up front what an application is allowed to do - read your calendar, post on your behalf, see your profile - and the token carries that entitlement. The valet key gets the car moving without unlocking the trunk.

Third, **revocation**. The user can cut off an application's access at any time without changing their password and without affecting any of their other connected apps.

There are many scenarios where this matters:

- Third-party applications and services: applications that want to access services like Google Drive, Twitter feeds, or Facebook posts.
- Mobile applications: smartphone applications that access web services but don't want to store passwords.
- Content aggregation: an app that pulls data from multiple sources (e.g., various email accounts).
- Federation: if you need to maintain secure integrations with a number of OAuth platforms, it's possible to use an OAuth IDP like Spring Authorization Server to act as a facade.

Evolution of OAuth

OAuth has evolved over the years to address shortcomings and to better meet the requirements of

developers and organizations. There are two main versions of OAuth:

OAuth 1.0: The original specification, finalized in December 2007 and later refined as RFC 5849 in 2010. It was complex, required clients to sign every request cryptographically, and was painful to implement correctly. OAuth 1.0a tightened a few security gaps, but we'll leave both behind.

OAuth 2.0: Standardized as RFC 6749 in 2012 as a successor, it simplifies the protocol and separates the roles of obtaining credentials from using them. It requires TLS for the authorization and token endpoints - the whole protocol assumes you're operating over HTTPS - which is a large part of what lets it drop OAuth 1.0's request-signing machinery.

OAuth vs. SAML

You'll often see OAuth contrasted with SAML (Security Assertion Markup Language), and you'll often see the contrast drawn as "SAML is for authentication, OAuth is for authorization." That's a useful starting point but it overstates the case - as you just saw, OAuth was being used for authentication well before OIDC formalized it.

The more practical differences:

	OAuth 2.0 / OIDC	**SAML 2.0**
Era and ecosystem	2012 onward; mobile, SPAs, APIs	2005 onward; enterprise single sign-on (SSO)
Wire format	JSON, with tokens that may be opaque or JWTs	XML assertions, signed and sometimes encrypted
Transport	HTTP redirects and JSON over HTTPS	HTTP redirects, form POSTs, and SOAP bindings
Typical use	App-to-API delegation; "Sign in with…"	Enterprise SSO into web apps from a central IdP

Both are still actively deployed. SAML is deeply entrenched in enterprise SSO; OAuth 2.0 and OIDC dominate almost everything newer.

Towards a Passwordless Future

One of OAuth's quiet superpowers is that it centralizes authentication. Authentication is a hard problem, and there are a million things to get right for it to even begin to be considered secure. Okta, Google, Meta, GitHub, and the Spring team have all committed serious engineering effort to getting it right so that you don't have to. Done well, a good OAuth integration can dramatically reduce the number of passwords you need to maintain to live on the internet.

In this book, we're going to focus on the Spring Authorization Server. Once it's at the heart of your SSO solution, identity for your organization collapses down to a single credential.

But what about *no* credentials? Or better, more secure alternative credentials?

That's where things are heading. Large providers like Apple, Google, and Microsoft are pushing

passwordless logins tied to device biometrics, built on WebAuthn - and, increasingly, on passkeys, which are discoverable WebAuthn credentials synced across a vendor's ecosystem so you can sign in from any of your devices.

Spring Authorization Server can be extended to participate in this future too, and by the end of the book you'll do exactly that. Let's look at how to setup the Spring Authorization Server. :code: ..

Introducing the Spring Authorization Server

In the last chapter, we watched OAuth from the outside: Yelp asking Google for permission, Google asking you, and a token coming back to prove that it all happened. Google, in that story, was the *OAuth Authorization Server* - the term comes directly from the OAuth specifications - and it did every interesting thing in the flow. It knew who you were. It asked what you were willing to share. It issued the token that made the whole dance mean something.

This chapter is about becoming that server. Not Google's, of course. Yours.

Concretely, the Spring Authorization Server is a Spring Boot application that implements the specifications a modern OAuth deployment needs. It issues tokens through the authorization code grant (with PKCE), the client credentials grant, and the device authorization grant. It refreshes those tokens, and it revokes them on request. It speaks OpenID Connect, so it publishes a discovery document at `/.well-known/openid-configuration`, a JWKS endpoint carrying the public half of its signing keys, and a UserInfo endpoint. It can mint self-contained JWTs or opaque tokens that resource servers hand back for introspection. That's the surface area. Nearly everything else in this book is a variation on it.

To be fair, you could use *any* valid, modern OAuth authorization server to do most of the things we'll discuss. OAuth is an open protocol, after all. So before we write a line of code, let's talk about why the server we're going to use is the Spring Authorization Server.

Why Spring Authorization Server?

Technically, the Spring Authorization Server is an OAuth- and OIDC-compliant solution just like any other. But as is so often the case, the decision rests on intangibles that go well beyond just the technical merit of the project.

Really, there are *two* questions before us: why should existing applications move to the Spring Authorization Server, and why should new projects start with it?

It's 2026 as I write this, and if you've got software in production, you probably have an OAuth authorization server deployed somewhere. We are spoiled for choice these days!

The space is crowded. On the commercial side, you've got Okta (and the Auth0 product it acquired), Microsoft Entra ID, Amazon Cognito, Google's Identity Platform, and Ping Identity (which now includes ForgeRock), alongside a newer cohort of developer-first SaaS shops like Clerk, WorkOS, Stytch, Frontegg, and Descope. On the self-hosted side, Keycloak is the 800-pound gorilla, with Ory, Authentik, Zitadel, Dex, and Apereo CAS each holding meaningful turf. There are dozens more.

Most of them would work just fine in place of the Spring Authorization Server. I love the Spring Authorization Server precisely because it works just fine in place of most of *them*, and it works in a way that's natural for anyone already living in the Spring ecosystem.

Let's look at the technical and legal facets that drive that choice.

Technical

The pitch is simple: it's a Spring Boot starter.

If you've ever deployed a Spring Boot application, you already know how to deploy the Spring Authorization Server. The same configuration model, the same Actuator endpoints, the same Micrometer metrics, the same container image story, the same CI/CD pipeline. Your team's existing muscle memory transfers over.

That's the familiar half of the pitch. The other half is what changes when the identity provider (IDP) is *yours*. Owning it unlocks a few things that are awkward, expensive, or simply not possible with a SaaS:

- **Your data stays where you put it.** Users are persisted to the same database your application already uses - the same Postgres instance, the same backup story, the same migration tooling, no data leaving your network. That matters in regulated industries, and it matters anywhere a vendor's data-residency story doesn't line up with yours.
- **Custom claims from your business systems.** You can mint tokens that carry the exact claims your services need - tenant ID, role hierarchy, feature flags - read straight from your domain model, without paying a SaaS for a "rules engine" and writing JavaScript inside a vendor's UI.
- **Federation on your terms.** Sign users in against your existing LDAP directory, your Active Directory, a social provider, or a customer's SAML IDP - Spring Security speaks all of them - and then present one consistent OAuth surface to your application code. Identity federates *in* through Spring Security; OAuth and OIDC flow *out* through the authorization server. Your services only ever have to understand the latter.
- **The same scaling story as the rest of your fleet.** Horizontal scaling, blue-green deploys, traffic shaping, observability - all the things you already do with Spring Boot apply to your auth layer.

When something doesn't fit, you reach for the same autoconfiguration override pattern you use everywhere else in Spring Boot - declare a bean of the type you want to replace, and Spring Boot backs off its default. There's no support ticket, no quarterly contract renegotiation, no rate limit on your own data.

None of this is free, though, and I'd be doing you a disservice to pretend otherwise. When you own the authorization server, you own its uptime - and its uptime is the uptime of every login in your organization. You own patching it, monitoring it, backing it up, and rotating its signing keys. You own the pager. A SaaS charges you money to make those someone else's problem, and for plenty of teams that is a perfectly rational trade. What you're really weighing is an invoice against a set of responsibilities, and only you know the exchange rate at your company.

Legal

Quick disclaimer, then the substance: *I am not a lawyer*, I am not your security consultant, and nothing in this book is legal or compliance advice. I work on the Spring team, and I'm here to teach you the technology. I'll point at good practices where I can, but I can't cover every jurisdiction, every industry regulation, and every threat model, so I'm not going to try.

What I will defend is a narrow claim: running your own authorization server does not put you at a

compliance disadvantage. It moves the work around. It does not create work that a vendor would otherwise have done for you.

Here's why that's worth saying out loud. User management is a minefield, and when something goes wrong - and eventually, in some form, it will - regulators will ask whether you did everything reasonable to prevent it. Could the Spring Authorization Server be configured and operated in a way that complies with HIPAA, PCI DSS, GDPR, SOC 2, or whatever else applies to you? Almost certainly yes. Will the out-of-the-box defaults get you there? No. Nobody's defaults get you there - not mine, not anyone else's. That isn't the job of an OAuth server.

To actually pass an audit, you need operational things like:

- backups
- multifactor authentication
- a hardware security module (HSM), or a cloud key management service (KMS), for your signing keys
- comprehensive monitoring and alerting
- a patching cadence and the discipline to keep to it
- a documented incident-response program
- (and a lot more besides)

The Spring Authorization Server can participate in all of that, and I'll point at the hooks where it does. Signing keys in particular get a chapter to themselves later on. The operational discipline around the rest is on you.

And before you assume a managed SaaS makes this go away: it doesn't. Providers like Okta supply genuinely useful compliance artifacts - SOC 2 reports, ISO 27001 certifications, audit logs, and security white papers. But regulators still hold *your* organization accountable for how you configure the service, who has access to it, your business processes, your regulatory obligations, and your overall compliance program. The vendor's compliance posture is a useful input. It is not a substitute for your own.

So: Those Two Questions

For a new project, the answer is easy. It's one dependency on the Spring Initializr and a process your team already knows how to run, monitor, and deploy. Start there, and if you outgrow it, you'll be migrating away from something you understand.

For an existing deployment, it's a real migration, and you should be honest with yourself about what that costs. Clients have to be re-registered. Users have to be moved, or federated to while you move them. Tokens already in the wild have to age out. That's a project, not an afternoon.

The case for doing it anyway is that everything on the far side of the migration is code you own, in a language your team writes, running on infrastructure you already operate. Whether that's worth the trip is a question about your organization, not a question about OAuth.

The Spring Security OAuth Ship of Theseus

The Spring Authorization Server is one of my favorite projects in the whole Spring portfolio. It's a full-blown OAuth identity provider, distributed as Spring Boot autoconfiguration.

But getting to this oh-so-approachable place took more than a decade! It started way back in 2011, when the Spring team volunteered to take on, and evolve, an awesome third-party open-source community project. We wanted to support the project and use it for some of the use cases we had in the popular open-source cloud platform Cloud Foundry, where it eventually became the UAA [https://github.com/cloudfoundry/uaa] component.

This project had three interesting pieces: *OAuth Resource Server* support, *OAuth Client* support, and *OAuth Authorization Server* support. We took it on and even had legends like the great Dr. Dave Syer (cofounder of Spring Boot and Spring Cloud, for a start) work on it. The project worked, but by 2015 it was starting to get long in the tooth. It was built before Spring Boot. It didn't support OIDC and other improvements in the OAuth space. It didn't work in a reactive world. Generally, it needed an overhaul.

So we embarked upon a project to rebuild and reintroduce OAuth support in Spring Security. This time, we said, the OAuth support would live as part of Spring Security itself, not as a separate project. The first piece to debut, in the Spring Security 5.0 and Spring Boot 2.0 generation, was the OAuth Client and OIDC support. That was 2017. Resource server support followed. And there we stopped: we said we didn't feel the need to build an OAuth Authorization Server, as the market had many good choices already.

That position did not survive contact with the community. In the GitHub issue on `spring-security` in which we declined to build it, they told us we were wrong. Loudly [https://github.com/spring-projects/spring-security/issues/6320#issuecomment-614218694]. This issue was one of the most vibrant in the Spring ecosystem's history! Clearly, there was demand. And so in 2020, work began in earnest, eventually landing as a fully supported, easily autoconfigured project in Spring Boot that you can use today.

The Spring Authorization Server originally lived as a separate project, with its own lifecycle, release cadence, and so on, but once it stabilized, it was moved under Spring Security with the Spring Security 7 release in late 2025. If you're already running it, that consolidation is mostly good news: one release train to track instead of two, and one version number to reason about. It also made the configuration friendlier, since there's now an `oauth2AuthorizationServer()` method right on the `HttpSecurity` DSL where older code had to reach for a separate configurer. We'll use it in the next chapter.

Which brings me back to the title of this section. Not one plank of the original project is still in place - not the client support, not the resource server support, not the authorization server, not even the repository it all lived in. Every piece was replaced, one at a time, over the better part of fifteen years. And yet the thing that sailed out the other side does the same job, for the same people, under very nearly the same name.

The Spring Security OAuth project is dead. Long live Spring Security's OAuth support!

One Less `users` Microservice for All

Having a real IDP at the center of your architecture is *liberating*.

The obvious win is consolidation. Every team that would otherwise have built its own `users` table, its own password-reset flow, its own session story, and its own permission system can stop. There's one identity surface for the whole organization: one place to onboard a new employee, one place to revoke their access when they leave, one place to add SSO into a customer's IDP. You'll have one less bespoke token-management system, too.

The less obvious win is isolation. Password hashing is expensive *on purpose* - bcrypt, scrypt, and Argon2 are designed to burn CPU so that attackers can't brute-force their way through stolen hashes. In a typical monolith, that CPU is competing with every business-logic request you're serving. Move authentication into a dedicated authorization server, and the expensive work happens on a node you can scale, monitor, and harden independently of everything else.

That isn't in tension with keeping your users in your own database, by the way. The database can be shared; the process burning CPU on top of it doesn't have to be.

If you knew the agony I've endured learning and wielding OAuth over the years, you'd understand what it means when I say I love this project. It's the first OAuth experience I've had that doesn't feel like work. And, dear reader, I want that for you too: the trick is to *stop having to think about OAuth* - to set it up well once and get back to building the thing you actually meant to build.

So let's set it up. In the next chapter we'll stand up a working Spring Authorization Server and point a real OAuth client at it, keeping everything in memory so that nothing gets in the way of the moving parts. Then we'll give it a PostgreSQL database, so that users, clients, and authorizations survive a restart. Then we'll take control of its signing keys, which is where "it works on my machine" turns into something you can put in front of a customer.

Let's take that journey to production together, with the Spring Authorization Server at our backs.

Implementing the Spring Authorization Server

Now that we understand the grand role a good OAuth authorization server can play in the scheme of your larger system, you might be worried that standing up an OAuth authorization server might be similarly grand in effort. Fear not! Nothing could be further from the truth. In this section, we'll get a trivial (but working) Spring Authorization Server up and running, and then we'll use that Spring Authorization Server instance to vend and validate tokens for an OAuth client.

Go to the Spring Initializr (start.spring.io) [https://start.spring.io], specify a group ID and an artifact ID (`authorization-server` might be a good one), and add `Web` and `OAuth2 Authorization Server` as dependencies. I'd add `GraalVM Native Support` for good measure. Later, we'll revisit these dependencies to add persistence and a few other fortifications to the system. For now, we'll favor simplicity, not security, and keep everything in-memory.

Click `Generate` and then open the resulting `.zip` project in your favorite IDE.

We'll need to enable the Spring Authorization Server itself by customizing the `HttpSecurity` instance.

```
package com.example.authorization_server;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
class SecurityConfiguration {

    @Bean
    Customizer<HttpSecurity> httpSecurityCustomizer() {
        return http -> http
①
            .oauth2AuthorizationServer(a -> a.oidc(Customizer.withDefaults()));
    }

}
```

1. Did you blink and miss it?

Note: In Spring Security 7, Spring Authorization Server became part of Spring Security, enabling the `oauth2AuthorizationServer` addition to the `HttpSecurity` DSL. Older code may use `with(OAuth2AuthorizationServerConfigurer.authorizationServer())`, but this is no longer necessary.

Congratulations! You've got the humble beginnings of a new Spring Boot Authorization Server. But it won't work *yet*. We will need to specify two more things - **users**, and **clients** - to get started.

Users

We've already talked at length about users. We'll use an `InMemoryUserDetailsManager` to create some hardcoded users (for now). This will in turn require a working `PasswordEncoder`.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

import java.util.List;

@Configuration
class UsersConfiguration {

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

    @Bean
    InMemoryUserDetailsManager userDetailsManager(PasswordEncoder passwordEncoder) {
        var josh = User //
            .builder()//
            .username("josh@joshlong.com") //
            .roles("USER")//
            .password(passwordEncoder.encode("pw"))//
            .build();
        var rob = User //
            .builder()//
            .username("rob@spring.security") //
            .roles("USER", "ADMIN")//
            .password(passwordEncoder.encode("pw"))//
            .build();
        var userDetails = List.of(rob, josh);
        return new InMemoryUserDetailsManager(userDetails);
    }

}
```

We've installed two users, `josh@joshlong.com` and `rob@spring.security`. Both have the password `pw`. Moving on...

OAuth Clients

An OAuth client defines how a program or process interacts with an OAuth Authorization Server (like the Spring Authorization Server). Clients correspond, more or less, to the programs that would like to act on behalf of users but need those users' permission - their *access tokens* - to do so.

We met clients in detail back in *Introducing OAuth*, where we walked, screen by screen, through a `Continue with Google` login on Yelp.com. There, Yelp was the client and Google was the authorization server. Here, the roles are reversed: *our* Spring Authorization Server plays Google's part, and we get to register the clients.

From the authorization server's perspective, a client is just a registration - a small bundle of configuration. Every client must stipulate a client ID and a client secret, which are transmitted in the request that initiates an authentication flow, signaling to the authorization server which client is asking. A client also declares the *scopes* it wants - OAuth's version of rights, permissions, or authorities, basically arbitrary strings that mean something to the APIs the token will eventually be presented to - and the *redirect URIs* the authorization server is allowed to send the user back to once they've authenticated.

One scope is special: `openid`, defined by OpenID Connect (OIDC), asks the authorization server to authenticate the user in a standard way. It has two effects: it mints an *ID token* (a JWT that client applications can consume to understand who the user is), and it grants the client access to the `UserInfo` endpoint, which serves up additional information about a given user. Because it's standardized, a client can reuse the same sign-in code across any OIDC-compliant provider, changing only the client ID, the client secret, and the issuer URI (the provider's root URL).

So, if you built a backoffice process, you'd register a client for that backoffice process. If you built a new web application that you intend to support automatic sign-in with OAuth, you'd register a new client for that web application. A new iOS application? Yes, you guessed: a new client for that, too!

The simplest way to register clients in the Spring Authorization Server is to use properties in the `application.properties` (or `application.yaml` file), like in this example:

```
spring.security.oauth2.authorizationserver.client.spring.registration.client-id=spring
①
spring.security.oauth2.authorizationserver.client.spring.registration.client-
secret={bcrypt}$2a$10$oV5p3sSEcmf/PEwv7a0Rq.yqBrDLY2XzidxCitMWDWvhE9KLPhLtG
spring.security.oauth2.authorizationserver.client.spring.registration.client-
authentication-methods[0]=client_secret_basic
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[0]=authorization_code
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[1]=client_credentials
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[2]=refresh_token
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[3]=urn:ietf:params:oauth:grant-type:device_code
spring.security.oauth2.authorizationserver.client.spring.registration.redirect-
uris[0]=http://127.0.0.1:8080/login/oauth2/code/spring
```

```
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[0]=openid
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[1]=profil
e
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[2]=messag
e
```

1. The `client-secret` is `spring`, but it's encoded using the BCrypt `PasswordEncoder`.

I will admit: this *might* be the first (and only) time I've ever wanted to use YAML in my configuration. But that's usually a sign something's very wrong in your life! And indeed, something *is* very wrong! You won't want to store client registrations in inflexible `.properties` or (*gasp*!) `.yaml` files! Later, we'll see how to configure clients using the underlying repository. This will serve to get us off the ground in the meantime.

The client registration registers one client called `spring` with a client secret of `spring`. It supports four authorization grant types. And, assuming a user authenticates with this client successfully, the OAuth authorization server will redirect the client back to one of the URLs specified in the `redirect-uris` array.

But where? You specify the possible redirect URIs in the client definition, as one of the (possibly many) `redirect-uris`. The client, later on, will ask the OAuth authorization server to redirect to precisely one of these URLs. But you could specify multiple such URLs here because you might have different applications and different workflows specifying the same client.

We haven't looked at the application yet, so this is a bit of foreshadowing, but the redirect URI specified here is designed to line up with Spring Security's OAuth client support, which we'll use on the web application.

Run the application in the usual way. Congratulations on your first deployment of the Spring Authorization Server! You know how you know it's a valid OAuth Authorization Server? Visit the localhost:9090/.well-known/openid-configuration [http://localhost:9090/.well-known/openid-configuration] endpoint.



if you want to carry on using property files, then perhaps consider the Spring Cloud Config Server or the Spring integration with Hashicorp Vault, so at least secure values are stored in an external, central, refreshable place. The Spring Cloud Config Server is another piece of Spring Boot-powered middleware that, once stood up, mediates access to configuration files via an HTTP API. The configuration files live in a version control system, like Git, which the Spring Cloud Config Server monitors. When the files change, the Spring Cloud Config Server serves up the new configuration data. Even better, the Spring Cloud Config Server can, via the Spring Cloud Bus abstraction, publish notifications to your microservices (like the Spring Authorization Server) on an event bus like RabbitMQ or Apache Kafka so that you can automatically reload the new configuration. This works particularly well in tandem with the Spring Cloud's `@RefreshScope`. In such a configuration, the configuration for everything still lives in a `.properties` or `.yaml` file, as it does now, but the files are centralized and can be changed without reloading the Spring Authorization Server. Storing files in a version control system gives us niceties like versioning, auditing, rollbacks, etc.,

> for very sensitive configuration data. And, going a step further, you can even use the Spring Cloud Config Server to store data encrypted at rest. For more on these possibilities, check out this video I did some years ago [https://www.youtube.com/watch?v=aC_siBP8rx8&list=PLgGXSWYM2FpPw8rV0tZoMiJYSCiLhPnOc&index=31]. And *all* of these possibilities are enabled entirely because the Spring Authorization Server is delivered as just another Spring Boot autoconfiguration!

The OAuth Client

We've stood up the Spring Authorization Server, added some users, and registered an OAuth client. Let's now build an OAuth client web application that, before we can use it, will require that we authenticate on the Spring Authorization Server. Go to the Spring Initializr [https://start.spring.io], add `Spring Web`, `OAuth2 Client`, choose your build tool, and then click `Generate`. Open the resulting `.zip` file and open the project in your favorite IDE.

Add a single controller.

```
package com.example.oauth_client;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class GreetingsController {

    @GetMapping("/")
    Map<String, String> me(Principal principal) {
        return Map.of("name", principal.getName());
    }

}
```

The only interesting thing in this whole controller is that we're injecting the currently authenticated `java.security.Principal` and using it to reflect back to the user who is authenticated.

We'll need to tell the Spring Security OAuth client autoconfiguration how to behave, so here's the client `.properties`.

```
spring.application.name=oauth-client
#
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:9090
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=spring
```

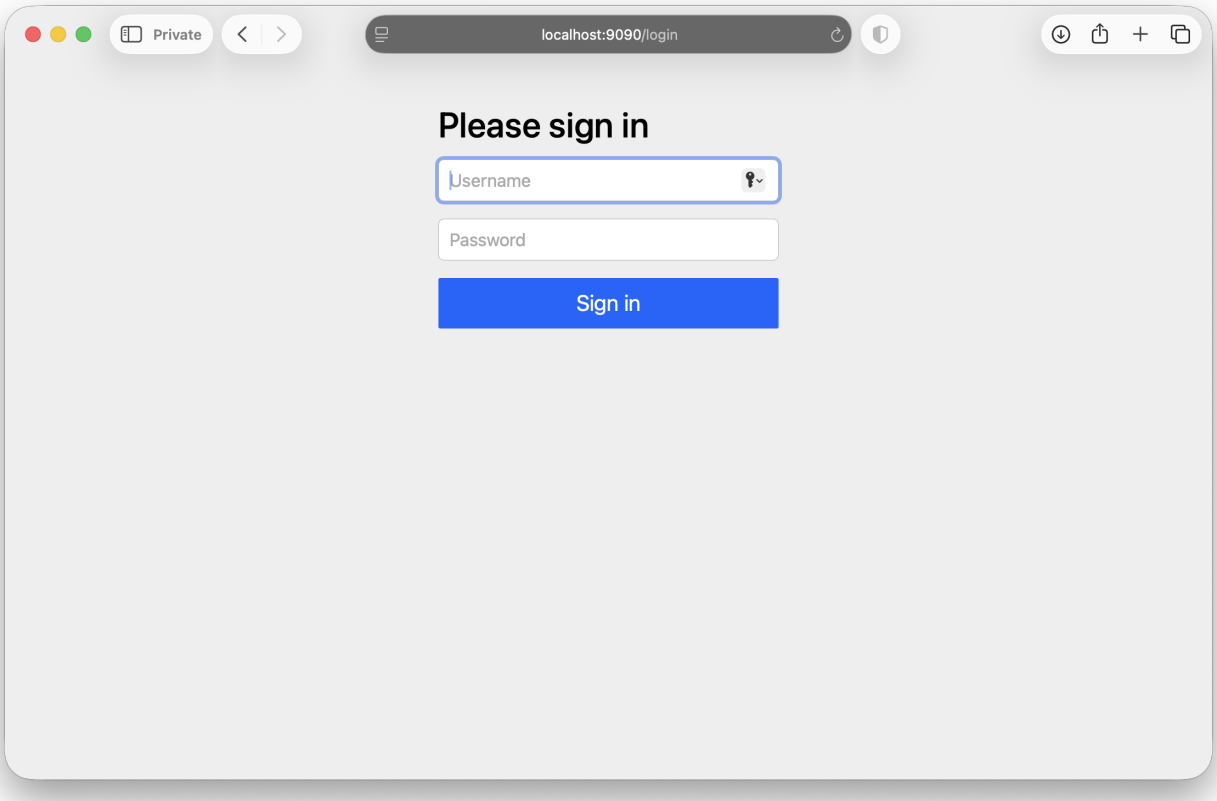
```
spring.security.oauth2.client.registration.spring.client-secret=spring
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=openid,profile
#
logging.level.org.springframework.security=trace
spring.threads.virtual.enabled=true
```

And that should be it! Start the client. The OAuth authorization server should be on port `9090`, and the OAuth client should be on port `8080`.

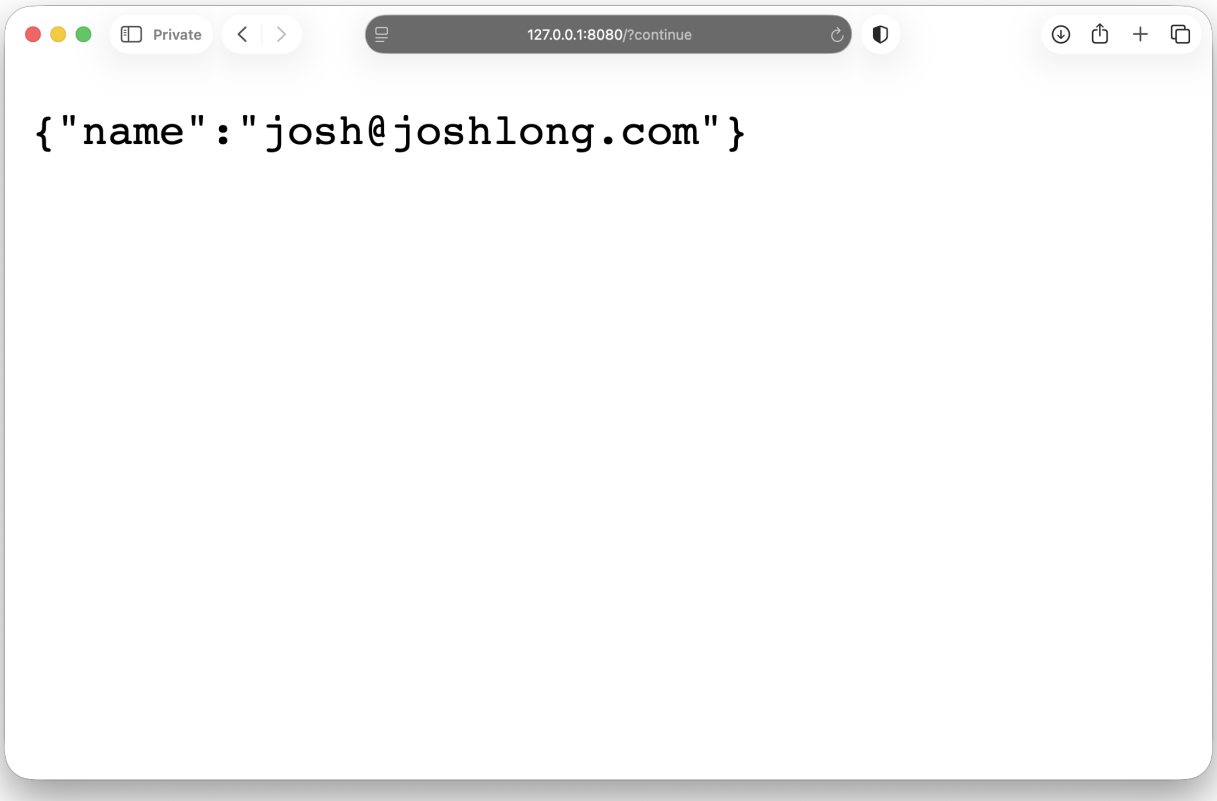


Notice that we reach the client at `127.0.0.1:8080` but the authorization server at `localhost:9090`. That's deliberate. Browser cookies are scoped to a *host*, not a host-and-port, so `localhost:9090` and `localhost:8080` share a single cookie jar; since both apps set a `JSESSIONID` cookie, running them on the same host makes each clobber the other's session, and you'll get redirect loops and `400` errors mid-login. Giving them different hostnames keeps their cookies apart, and `localhost` and `127.0.0.1` conveniently count as different hosts. That trick runs out of road fast, though: two services is fine, but a realistic system has many, and juggling `127.0.0.1`, `localhost`, and hand-maintained `/etc/hosts` aliases (`auth.local`, `app.local`, …) gets old quickly. Two options scale further. A wildcard DNS service like nip.io [https://nip.io] or sslip.io [https://sslip.io] maps any name back to an address for free - `auth.127.0.0.1.nip.io` and `app.127.0.0.1.nip.io` both resolve to `127.0.0.1` with no `/etc/hosts` edits at all. And when you need a real, *public*, HTTPS hostname - for a passkey origin, an external callback, or a webhook - a tunneling tool like ngrok [https://ngrok.com] hands you one in seconds. In production the problem disappears anyway, since every service already has its own DNS name.

Visit `127.0.0.1:8080` in the browser, and you should be redirected to the Spring Authorization Server where you can authenticate with one of the aforementioned users (e.g., `josh@joshlong.com` and `pw`).



You'll then be redirected back to the web application where you'll be greeted with the message from the controller.



The End of the Beginning

We've come a long way! We've learned the basics of Java, Spring, Spring Security, OAuth, and the Spring Authorization Server. We've stood up a working Spring Authorization Server and a working OAuth-protected web application. Not bad! But there's *so much* more to look at!

- where did the Spring Authorization Server store its state? What happens when we restart the Spring Authorization Server?
- can we support programmatically registering OAuth clients?
- what happens if we want to then take that authenticated session, grab the token, and call other backend (protected) applications?

We'll look at all of this, and *so much* more, in the chapters ahead! :code: ..

Persistent State in the Spring Authorization Server with PostgreSQL and JDBC

Spring makes it easy to substitute implementations with *dependency injection*, which only works thanks to the magic of polymorphism. This is one of the key reasons we use Spring. Spring Boot-based configuration makes this doubly powerful. While the Spring Authorization Server does amazing things out of the box, its real power lies in all the knobs and levers available to you for customization. At the end of the day, it's just another Spring Boot autoconfiguration and starter. We have already established that key parts of the Spring Authorization Server work out of the box with in-memory implementations. This is great for a demo but quickly becomes a liability as soon as you need to load-balance to another node, or restart the service. What becomes of all the connected clients? Their keys? Their sessions? Their state? In this section, we'll swap those out for implementations using JDBC. If you don't want to use JDBC, then feel free to implement the interfaces for yourself, deferring to whatever underlying storage mechanism you want. Spring Data is your friend here.

We'll work through a good many SQL schema files along the way, one for nearly every piece of state we persist. Each of them ships on the classpath inside the various Spring Authorization Server `.jar` artifacts, so you could source them from there. I've had to tweak some to sit better with PostgreSQL, though, so I've reprinted them here throughout the chapter, with those slight changes, for posterity.

PostgreSQL as the Database

We've already introduced Flyway for Database Migrations, PostgreSQL, its Docker compose file, and the basics of JDBC. We'll continue with this setup.

I've used PostgreSQL thus far because it's a very common database. In this section, I want to remind you that it's also a very *good* database. It's widely deployed. Its community is larger than that of any other database. Its SQL syntax support is very good, though it does have its own nonstandard and inconsistent quirks, to be sure.

It's also the engine on which a whole wave of so-called NewSQL database engines have been built. So you can start with PostgreSQL, and if you decide you want better availability, scalability, performance, etc., you'll have no trouble finding all of that in various other vendors' implementations.

Most extra capabilities for PostgreSQL are delivered as extensions to classic PostgreSQL, preserving most of if not all standard PostgreSQL features in exchange for some new capability. PostgreSQL's codebase is well known and well understood. PostgreSQL may not tout its customizability, but the community has certainly taken it to horizons unimaginable before. Let's briefly talk about some of the things you might look for in your PostgreSQL database service. Some of these can be found in open source or proprietary distributions. Some are self-hosted, and some are SaaS offerings. Some are a combination thereof.

- **Bottomless PostgreSQL** - some vendors offer "bottomless" storage. You can create tables with as much data as you want, and magically the disk on which your data is stored will just automatically grow.

- **Cloud-native storage** - related, but not necessarily the same: some PostgreSQL services offer storage backed by cloud storage, like Amazon's S3.
- **Horizontal scalability** - some PostgreSQL services, like YugabyteDB, offer horizontal scalability on par with Cassandra.
- **Worldwide availability** - some database services, like CockroachDB, purport to offer the same super-high availability as Google's Spanner database.
- **Custom data types** - some PostgreSQL services (or at least the extensions for those services) support time-series databases, durable execution primitives, geospatial mapping primitives, vector store capabilities, and so much more.

Before we can get started, we'll need a PostgreSQL database — some place in which to store our state. As always, you can use the Spring Initializr to generate a Docker `compose.yml` for PostgreSQL, use the one we provide, or use your own.

Adding Persistence

Now, we're going to need to retool our project to accommodate JDBC, so add the following dependencies to the build:

- `org.springframework.boot` : `spring-boot-starter-flyway`
- `org.springframework.boot` : `spring-boot-starter-restclient`
- `org.springframework.boot` : `spring-boot-starter-jdbc`
- `org.springframework.boot` : `spring-boot-starter-security-oauth2-authorization-server`
- `org.springframework.boot` : `spring-boot-starter-session-jdbc`
- `org.flywaydb` : `flyway-database-postgresql`
- `org.postgresql` : `postgresql` (make it `runtime` scoped)

Modify `application.properties` or `application.yaml` to point to the newly configured PostgreSQL database and enable Flyway database migrations, too.

```
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.username=myuser
spring.datasource.password=secret
```

We're using Flyway for database migrations. Make sure to create the `src/main/resources/db/migration` folder.

Spring Session

We met Spring Session in an earlier chapter, where we used it to lift a web application's `HttpSession` state out of the Servlet container and into a shared, external store, so that sessions survive both restarts and load balancing. The Spring Authorization Server has exactly the same need. It assumes a Servlet container — Apache Tomcat, by default — is present, and it relies on the HTTP session throughout the OAuth 2.0 authorization flow. If we want to run more than one instance behind a

load balancer, or even just survive a restart, that session state can't stay pinned to the memory of a single node.

So, just as we did before, we'll reach for Spring Session JDBC and keep this state in PostgreSQL. All that's left is to install the schema. Just as we did in the Spring Session chapter, put the following in `src/main/resources/db/migration/V1__session.sql`.

```
-- sessions
create table if not exists spring_session
(
    primary_id            character(36) primary key not null,
    session_id            character(36)             not null,
    creation_time         bigint                    not null,
    last_access_time      bigint                    not null,
    max_inactive_interval integer                   not null,
    expiry_time           bigint                    not null,
    principal_name        character varying(100)
);
create unique index if not exists spring_session_ix1 on spring_session using btree
(session_id);
create index if not exists spring_session_ix2 on spring_session using btree
(expiry_time);
create index if not exists spring_session_ix3 on spring_session using btree
(principal_name);

-- session attributes
create table if not exists spring_session_attributes
(
    session_primary_id character(36)          not null,
    attribute_name     character varying(200) not null,
    attribute_bytes    bytea                  not null,
    primary key (session_primary_id, attribute_name),
    foreign key (session_primary_id) references spring_session (primary_id)
        match simple on update no action on delete cascade
);
```

When the application starts up, Flyway runs this migration and Spring Session has everything it needs.



Spring Session can install its own schema for you if you set `spring.session.jdbc.initialize-schema=always`. It works (surprise!), but I'd rather keep every table under version control alongside our other migrations, where I can review and audit it.

Users

We're going to source our users not from the `InMemoryUserDetailsManager` but from users stored in PostgreSQL. Let's install some SQL schema for the users table in

`src/main/resources/db/migration/V2__users-schema.sql`.

```
create table if not exists users
(
    username varchar(200) not null primary key,
    password varchar(500) not null,
    enabled  boolean      not null
);

create table if not exists authorities
(
    username  varchar(200) not null,
    authority varchar(50)  not null,
    constraint fk_authorities_users foreign key (username) references users (username
),
    constraint username_authority UNIQUE (username, authority)
);
```

Then define some user data in `src/main/resources/db/migration/V3__users-data.sql`.

```
delete
from authorities;
delete
from users;

insert into users (username, password, enabled)
values ('josh@joshlong.com',
'{bcrypt}$2a$10$i4a5jw4y9LnKS//8DNDVYe2VfC6Jr2m0Ds1j5un5JNhnkJxg6jAT.', true);

insert into users (username, password, enabled)
values ('user@anotherdomain.site',

'{sha256}0a598ce65440a7c4ad18b48fdc2cf28c4e58aec803780f59548535696a23b16624e47ff4f59e6
f39', true);

--
insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_USER');

insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_ADMIN');

insert into authorities (username, authority)
values ('user@anotherdomain.site', 'ROLE_USER');

insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_EDITOR');
```

We'll need to use the proper `UserDetailsService` implementation, of course.

```
package com.example.authorization_server;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.provisioning.JdbcUserDetailsManager;

import javax.sql.DataSource;

@Configuration
class UsersConfiguration {

    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
        var userDetailsManager = new JdbcUserDetailsManager(dataSource);
        userDetailsManager.setEnableUpdatePassword(true);
        return userDetailsManager;
    }

}
```

This will result in two new users being added to the system: `user@anotherdomain.site` and `josh@joshlong.com`. Both have the password `pw`.

Persisting Clients

When we set up the Spring Authorization Server, we defined our clients in the `.properties` or `.yml` file as a simple demonstration. More typically, you'll want to register and remove clients programmatically so application teams can add or remove them on demand - without restarting anything or editing source code.

To do this, back a `RegisteredClientRepository` with a persistent store. This lets you build a self-service registration form or workflow, just like Google and Apple do, for your organization's developers.

We'll use the JDBC-powered implementation of this interface, `JdbcRegisteredClientRepository`. Implementing alternatives on top of other persistence mechanisms like MongoDB or Hashicorp Vault would be a fairly trivial project.

The `JdbcRegisteredClientRepository` implementation depends on some SQL schema that we've defined in `src/main/resources/db/migration/V4__oauth2-registered-client.sql`.

```
CREATE TABLE if not exists oauth2_registered_client
(
    id                            varchar(100)                            NOT NULL,
    client_id                     varchar(100)                            NOT NULL,
    client_id_issued_at           timestamp     DEFAULT CURRENT_TIMESTAMP NOT NULL,
```

```
    client_secret                varchar(200)  DEFAULT NULL,
    client_secret_expires_at     timestamp     DEFAULT NULL,
    client_name                  varchar(200)                            NOT NULL,
    client_authentication_methods varchar(1000)                          NOT NULL,
    authorization_grant_types    varchar(1000)                           NOT NULL,
    redirect_uris                varchar(1000) DEFAULT NULL,
    post_logout_redirect_uris    varchar(1000) DEFAULT NULL,
    scopes                       varchar(1000)                           NOT NULL,
    client_settings              varchar(2000)                           NOT NULL,
    token_settings               varchar(2000)                           NOT NULL,
    PRIMARY KEY (id)
);
```

And here's the definition of the `RegisteredClientRepository` and a runner that uses it to install a client, more or less identical to the client we registered earlier in `application.properties`.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.core.AuthorizationGrantType;
import org.springframework.security.oauth2.core.ClientAuthenticationMethod;
import org.springframework.security.oauth2.core.oidc.OidcScopes;
import
org.springframework.security.oauth2.server.authorization.client.JdbcRegisteredClientRe
pository;
import
org.springframework.security.oauth2.server.authorization.client.RegisteredClient;
import
org.springframework.security.oauth2.server.authorization.client.RegisteredClientReposi
tory;
import
org.springframework.security.oauth2.server.authorization.settings.ClientSettings;

import java.util.Set;
import java.util.UUID;

@Configuration
class ClientsConfiguration {

①
    @Bean
    JdbcRegisteredClientRepository registeredClientRepository(JdbcTemplate template) {
        return new JdbcRegisteredClientRepository(template);
    }

②
```

```
    @Bean
    ApplicationRunner clientsRunner(PasswordEncoder passwordEncoder,
RegisteredClientRepository repository) {
        return _ -> {
            var clientId = "spring";
            if (repository.findByClientId(clientId) == null) {
                var crmClientSecret = passwordEncoder.encode("spring");
                var authorizationGrantTypes = Set.of( //
                        AuthorizationGrantType.CLIENT_CREDENTIALS, //
                        AuthorizationGrantType.AUTHORIZATION_CODE, //
                        AuthorizationGrantType.REFRESH_TOKEN, //
                        AuthorizationGrantType.DEVICE_CODE //
                ); //
                var scopes = Set.of(OidcScopes.OPENID, OidcScopes.PROFILE, //
                        OidcScopes.EMAIL, "user.write", "user.read");
                var client = RegisteredClient//
                    .withId(UUID.randomUUID().toString())//
                    .clientId(clientId)//
                    .clientSecret(crmClientSecret) ①
                    .clientAuthenticationMethod( //
                            ClientAuthenticationMethod.CLIENT_SECRET_BASIC)//
                    .authorizationGrantTypes(gts -> gts.addAll(
authorizationGrantTypes))//
                    .redirectUri("http://127.0.0.1:8080/login/oauth2/code/spring") //
                    .scopes(s -> s.addAll(scopes))//
                    .clientSettings(ClientSettings.builder()//
                        .requireAuthorizationConsent(true) ②
                        .build() //
                )
                    .build();
                repository.save(client);
            }
        };
    }

}
```

1. define the JDBC-backed `RegisteredClientRepository`.
2. use it to programmatically add some clients to the system.
3. and, of course, I'm hardcoding the `client-secret` to be `spring`. Don't do this! You should source the actual client secret from an external configuration source like an environment variable, Hashicorp Vault, etc.

Persisting Authorizations

With clients now living in the database, the next piece of state to move out of memory is the authorizations themselves. When we used the Spring Authorization Server before, we hit a secured page, were redirected to the Spring Authorization Server, authenticated, and then were redirected

back to the secure page, able to access it. We granted an OAuth 2 ***authorization*** to the OAuth client. By authenticating, we were granting permission to the client to have whatever scopes it asked for. But how do we know what it asked for? And what if we don't want to sign off on all of them? We can stipulate that when we define the client itself. We could've added the following property, and we'd be shown a screen confirming access after authenticating. Granting individual scopes is ***authorization consent***.

Here's how you'd stipulate that you want a consent screen in which the user explicitly grants consent as a property:

```
spring.security.oauth2.authorizationserver.client.spring.require-authorization-
consent=true
```

When we created the `RegisteredClient` programmatically above, we specified the same thing with the `ClientSettings` reference. The Spring Authorization Server keeps track of both authorizations and authorization consents, and - surprise! - persists them.

There are two similar interfaces of note here: `OAuth2AuthorizationService` and `OAuth2AuthorizationConsentService`.

Here are the relevant bean definitions:

```
package com.example.authorization_server;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcOperations;
import
org.springframework.security.oauth2.server.authorization.JdbcOAuth2AuthorizationConsen
tService;
import
org.springframework.security.oauth2.server.authorization.JdbcOAuth2AuthorizationServic
e;
import
org.springframework.security.oauth2.server.authorization.client.RegisteredClientReposi
tory;

@Configuration
class AuthorizationConfiguration {

①
    @Bean
    JdbcOAuth2AuthorizationService jdbcOAuth2AuthorizationService(JdbcOperations
jdbcOperations,
            RegisteredClientRepository rcr) {
        return new JdbcOAuth2AuthorizationService(jdbcOperations, rcr);
    }

②
```

```
    @Bean
    JdbcOAuth2AuthorizationConsentService jdbcOAuth2AuthorizationConsentService
(JdbcOperations jdbcOperations,
            RegisteredClientRepository repository) {
        return new JdbcOAuth2AuthorizationConsentService(jdbcOperations, repository);
    }

}
```

1. the `OAuth2AuthorizationService` backed by JDBC
2. the `OAuth2AuthorizationConsentService` backed by JDBC

`OAuth2AuthorizationService` handles representations of an authorization - the JWT token, the client, etc. We're going to swap out the default autoconfigured implementation for one that uses JDBC for persistence. First, let's install the requisite schema for the `OAuth2AuthorizationService`. This file is `src/main/resources/db/migration/V5__oauth2-authorization-schema.sql`.

```
create table if not exists oauth2_authorization
(
    id                            varchar(100) NOT NULL,
    registered_client_id          varchar(100) NOT NULL,
    principal_name                varchar(200) NOT NULL,
    authorization_grant_type      varchar(100) NOT NULL,
    authorized_scopes             varchar(1000) DEFAULT NULL,
    attributes                    text          DEFAULT NULL,
    state                         varchar(500)  DEFAULT NULL,
    authorization_code_value      text          DEFAULT NULL,
    authorization_code_issued_at  timestamp     DEFAULT NULL,
    authorization_code_expires_at timestamp     DEFAULT NULL,
    authorization_code_metadata   text          DEFAULT NULL,
    access_token_value            text          DEFAULT NULL,
    access_token_issued_at        timestamp     DEFAULT NULL,
    access_token_expires_at       timestamp     DEFAULT NULL,
    access_token_metadata         text          DEFAULT NULL,
    access_token_type             varchar(100)  DEFAULT NULL,
    access_token_scopes           varchar(1000) DEFAULT NULL,
    oidc_id_token_value           text          DEFAULT NULL,
    oidc_id_token_issued_at       timestamp     DEFAULT NULL,
    oidc_id_token_expires_at      timestamp     DEFAULT NULL,
    oidc_id_token_metadata        text          DEFAULT NULL,
    refresh_token_value           text          DEFAULT NULL,
    refresh_token_issued_at       timestamp     DEFAULT NULL,
    refresh_token_expires_at      timestamp     DEFAULT NULL,
    refresh_token_metadata        text          DEFAULT NULL,
    user_code_value               text          DEFAULT NULL,
    user_code_issued_at           timestamp     DEFAULT NULL,
    user_code_expires_at          timestamp     DEFAULT NULL,
    user_code_metadata            text          DEFAULT NULL,
    device_code_value             text          DEFAULT NULL,
```

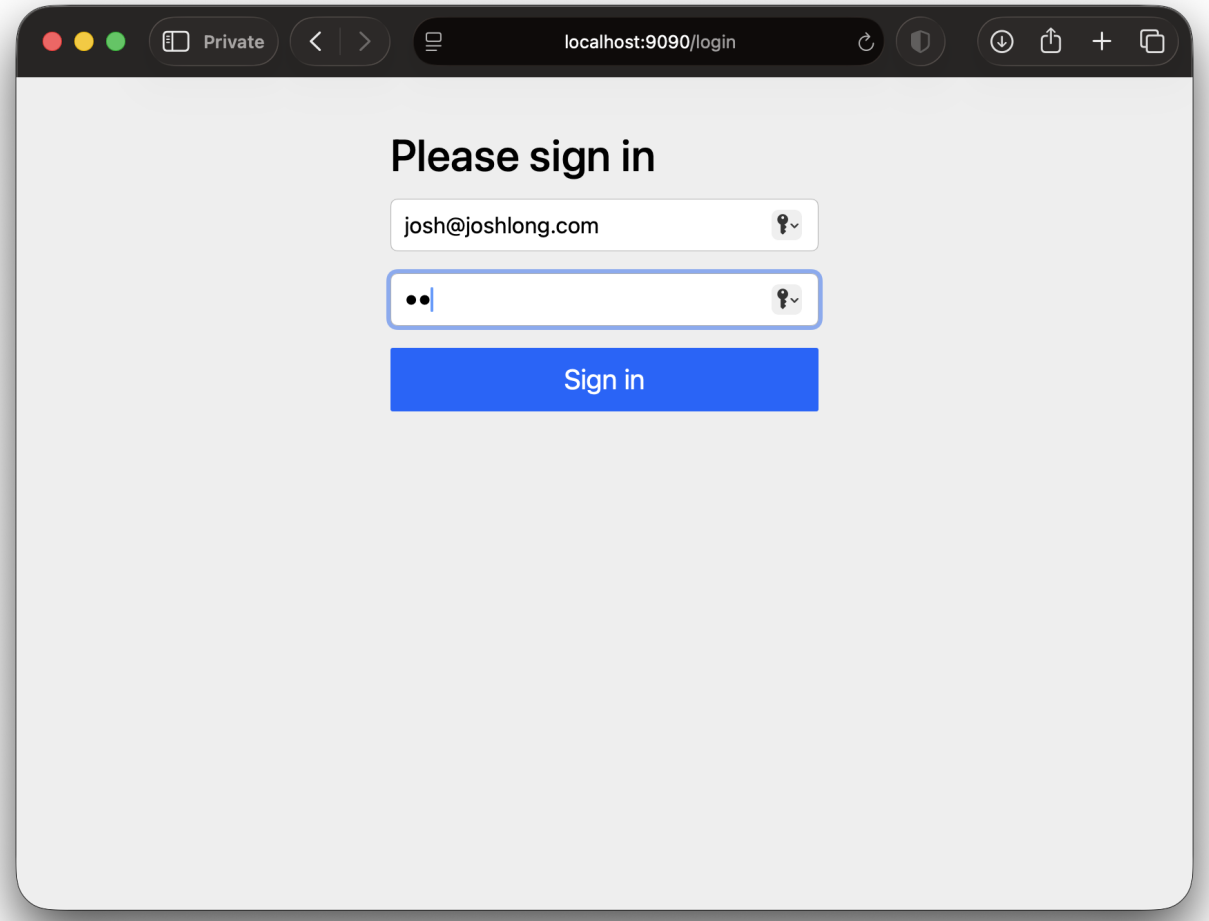
```
    device_code_issued_at         timestamp     DEFAULT NULL,
    device_code_expires_at        timestamp     DEFAULT NULL,
    device_code_metadata          text          DEFAULT NULL,
    PRIMARY KEY (id)
);
```

`OAuth2AuthorizationConsentService` handles representations of an OAuth 2.0 "consent" to an authorization request, which holds state related to the set of authorities granted to a client by the resource owner. Now, let's install the requisite schema for the `OAuth2AuthorizationConsentService`. Put the following in `src/main/resources/db/migration/V6__oauth2-authorization-consent-schema.sql`.

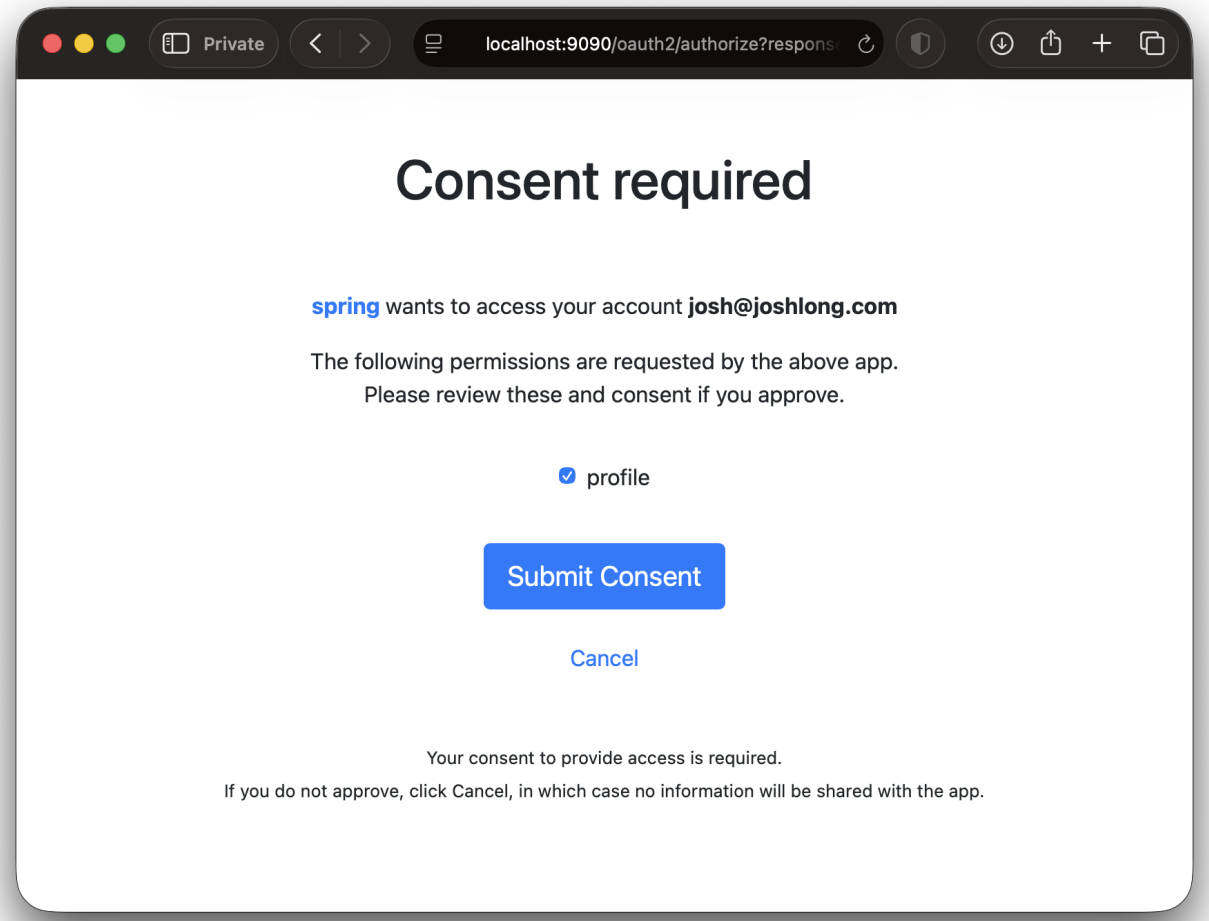
```
create table if not exists oauth2_authorization_consent
(
    registered_client_id varchar(100)  NOT NULL,
    principal_name       varchar(200)  NOT NULL,
    authorities          varchar(1000) NOT NULL,
    PRIMARY KEY (registered_client_id, principal_name)
);
```

Start the application. In an incognito window, attempt to access /, and you'll be redirected as usual to the Spring Authorization Server instance where you'll again authenticate. Then, you'll be prompted to consent to the scopes being granted. Here's what that looks like on my system.

First we sign in.



Then we consent.



Inspect the database and the newly minted tables, and you'll see all of this authorization activity reflected there.

We've now moved sessions, users, clients, and authorizations out of memory and into PostgreSQL. One important piece of state still isn't safe across restarts and load balancing: the cryptographic keys the Spring Authorization Server uses to sign its tokens. That's where we'll turn next. :code: ..

Signing Keys in the Spring Authorization Server

In the last chapter, we moved the Spring Authorization Server's session, user, client, and authorization state out of memory and into PostgreSQL. One crucial piece of state remains: the cryptographic keys the server uses to sign the tokens it issues.

Every time the Spring Authorization Server starts up, the Spring Boot autoconfiguration kicks in generating new keys for our application. It uses the keys to sign the JWT tokens that it vends for our other applications. Other applications - clients, microservices, etc. - retain these tokens, sometimes for hours or days or even weeks! What happens when you restart the Spring Authorization Server - which automatically creates new keys - and then a client with a JWT signed with the old keys tries to connect? It *fails*. Having the Spring Authorization Server generate random keys and store them in-memory on every restart can be quite a boon to getting started, but we should furnish our own stable, central key if we want the JWTs to survive restarts or load balancing. Otherwise, each instance of the JVM would, by default, have different keys. We're going to add two new files, a private and a public key, and use that to create a `JWKSource<SecurityContext>`. It will be this `JWKSource<SecurityContext>` that provides the JWK. JWK stands for *JSON Web Key* - it's a standardized way (RFC 7517) to represent a cryptographic key as a JSON object.

Generating a Key Pair

To get started, we'll need to generate a key pair.

A key pair is two mathematically linked numbers - a private key and a public key - generated together by tools like `ssh-keygen` or `openssl`. The whole point is an asymmetry: the two keys are related, but you can't feasibly derive the private one from the public one. That gap is what makes the whole thing useful.

You can achieve the result using either `ssh-keygen` or `openssl`. We'll use `openssl`.

```
①
openssl genpkey -algorithm RSA -out private_key.pem
openssl pkcs8 -topk8 -inform PEM -outform PEM -in private_key.pem -out
private_key_pkcs8.pem -nocrypt
openssl rsa -pubout -in private_key_pkcs8.pem -out public_key.pem
mv private_key.pem app.key
mv public_key.pem app.pub
②
openssl req -new -x509 -key private_key_pkcs8.pem -out cert.pem -days 365
```

1. Make sure you execute all the following commands in a new, empty directory. It's important that the generated files aren't lost in the wilderness of whatever jam-packed folder you happen to have created them in.
2. This command exports a self-signed certificate (valid for 365 days) that you could (optionally) use if you wanted to store the file in the Java `KeyStore` as a PKCS12 file.

You'll get two artifacts, `app.pub` and `app.key`. Copy them both, the private (`app.key`) and public key (`app.pub`), to the `src/main/resources` folder of your Spring Authorization Server instance.

I created three (arbitrary) properties in `application.properties` to describe these keys and assign them an ID.

```
①
jwk.key.id=bootiful-key
jwk.key.private=classpath:app.key
jwk.key.public=classpath:app.pub
```

1. these are *our* properties. they're arbitrary. we'll refer to them at various points. We could've used any property name scheme.

We'll use these new properties when we define the `JWKSource<SecurityContext>` bean.

```
package bootiful.authorizationserver.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;

import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;

@Configuration
class SimpleKeyConfiguration {

    @Bean
    JWKSource<SecurityContext> jwkSource(
            @Value("${jwt.key.id}") String id,
            @Value("${jwt.key.private}") RSAPrivateKey privateKey,
            @Value("${jwt.key.public}") RSAPublicKey publicKey) {
        var rsa = new RSAKey.Builder(publicKey)
                .privateKey(privateKey)
                .keyID(id)
                .build();
        var jwk = new JWKSet(rsa);
        return new ImmutableJWKSet<>(jwk);
    }

}
```

In this example, I've kept the private key (`app.key`) in the source code of this repository, but as you can imagine, this is a **very bad** idea in production. Don't forget my admonition earlier: take care to handle your credentials securely. If you need it available for this program, it should be passed into the application in such a way that it's very difficult for others to lay hands on it. One such way to approach this is to encrypt the file in such a way that it can only be decrypted with, yes, you guessed it, *yet another key*.

Rotating Keys

In the last example we plugged in a fixed key we generated by hand using `openssl`. It works, for now, but what happens when we need to rotate the keys? Rotating cryptographic keys is a well-established security practice, and it's often recommended to enhance the security posture of systems that use encryption.

Here are a few reasons:

Regularly rotating keys limits exposure and ensures that whatever key an attacker possesses becomes obsolete after a certain time.

Key rotation also mitigates the risk of weak keys or, as sometimes happens, cryptographic erosion. Cryptographic erosion is when you have keys that are susceptible to gains in computing power that make older keys easier to compromise.

Some industry standards and regulations require periodic key rotation. For instance, the Payment Card Industry Data Security Standard (PCI DSS) has requirements around key management practices, which includes periodic key rotation.

So, we agree it's important, but how do we do it? Conceptually, it's easy: we're going to swap out the implementation of the `JWKSource` for an implementation that defers to a repository. In practice, there are a lot more moving parts involved because we have to actually persist the keys somewhere! And the caution from the last section still holds: we don't want those keys lying around on disk in their plain and natural state, so whatever we persist, we'll want to encrypt.

Also, if we're going to rotate keys, we need a way to generate them in code. After all, using `openssl` was easy enough for a one-time thing, but we wouldn't want to shell out for each new key generated. And, finally, we'll need to store these keys somewhere. We're using PostgreSQL, so we'll use that.

A Brief Look at the State of Cryptography with PostgreSQL

Since we're using PostgreSQL to store the keys, we might consider using some of its cryptography support. It's a good idea! We're going to go a different direction, *but* let's take a quick beat to explore what using PostgreSQL for cryptography might look like.

Most databases support column-level encryption of data at rest. That is, nothing is ever written to disk in an unencrypted form. PostgreSQL does not, at least not out of the box. There are plugins, like PGCrypto [https://www.postgresql.org/docs/current/pgcrypto.html], that provide functions that you can

use to encrypt text.

You can use PGCrypto pretty easily. It's a trusted module that can be loaded into the database even if you're not a superuser. Log in to your PostgreSQL instance. If you're using the `compose.yml` file, then you can run:

```
PGPASSWORD=secret psql -U myuser -h localhost mydatabase
```

Once in, you'll need to load the plugin. It's usually bundled with your PostgreSQL distribution.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

You can then confirm that it's worked by using one of the functions provided, like this:

```
SELECT crypt('hello, world', gen_salt('bf'));
```

This example uses the Blowfish algorithm to encrypt some text. This is an awesome option, but it's unique to PostgreSQL.

Some cloud providers have PostgreSQL offerings that support transparent encryption at rest. For instance, AWS RDS for PostgreSQL supports encryption at rest using AWS Key Management Service.

I love PostgreSQL and, if I were building something for production, I'd have no trouble trusting PostgreSQL. But, in the interest of further exploring Spring Security, and keeping our code as generic as possible, we'll use Spring Security's equally rich encryption support to encrypt the private key (and the public one, though we don't really need to) and store it in the database. We'll need a password and a salt for the encryption, however, and those values should ultimately not be stored at rest unencrypted. Again: you'll want to store *those* values somewhere like the Spring Cloud Config Server, Hashicorp Vault, or your hyperscaler's key management solution.

Generating New Keys Programmatically

First thing's first: we'll need a way to generate new keys programmatically. We can't rotate 'em if we can't generate 'em! Mercifully, the Java Security APIs are pretty remarkable here. Here's a class that'll do this for us.

```
package com.example.authorization_server.keys;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

class Keys {
```

```
    RsaKeyPair generateKeyPair(String keyId, Instant created) {
        var keyPair = generateRsaKey();
        var publicKey = (RSAPublicKey) keyPair.getPublic();
        var privateKey = (RSAPrivateKey) keyPair.getPrivate();
        return new RsaKeyPair(keyId, created, publicKey, privateKey);
    }

    private KeyPair generateRsaKey() {
        try {
            var keyPairGenerator = KeyPairGenerator.getInstance("RSA");
            keyPairGenerator.initialize(2048);
            return keyPairGenerator.generateKeyPair();
        } //
        catch (Exception ex) {
            throw new IllegalStateException(ex);
        }
    }

}
```

Easy! The only externally visible method is `generateKeyPair`, which takes a key ID and a timestamp and returns an instance of `RsaKeyPair`, which is our repository's domain type. It's a record to hold both the generated public and private keys.

```
package com.example.authorization_server.keys;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

①
record RsaKeyPair(String id, Instant created, RSAPublicKey publicKey, RSAPrivateKey
privateKey) {
}
```

1. no notes. I just love Java records and want us to take a moment to appreciate them.

Key Pair Persistence

So now we have the ability to generate keys entirely in Java, with no need for `openssl` or `ssh-keygen`, but how (and *where*) do we store them? That's where the rubber meets the secured road!

We're going to implement a repository to make working with, and persisting, `RsaKeyPair` instances. We'll need some schema in which to store everything. Let's add a new Flyway migration, `src/main/resources/db/migration/V7__rsa-key-pairs.sql`.

```
create table if not exists rsa_key_pairs
(
```

```
    id          varchar(1000) not null primary key,
    private_key text          not null,
    public_key  text          not null,
    created     date          not null,
    unique (id, created)
);
```

With the table in place, here's the repository interface we'll implement against it.

```
package com.example.authorization_server.keys;

import java.util.List;

interface RsaKeyPairRepository {

①
    List<RsaKeyPair> findKeyPairs();

②
    void save(RsaKeyPair rsaKeyPair);

}
```

1. Find the key pairs and preserve ordering. We want the latest key to be first.
2. This method looks so simple. Don't trust it. It's a lie!

Saving the `RsaKeyPair`, in this example, means writing it to our database, which means we'll need a way to serialize the `java.security.*` key objects. It's not as straightforward as you'd think! But I got it working and put that logic in two implementations of Spring Framework's handy `Converter<T>` interface: `RsaPrivateKeyConverter` for private keys, and `RsaPublicKeyConverter` for public keys.

Here's the `RsaPrivateKeyConverter`:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.util.Base64;
```

```
class RsaPrivateKeyConverter implements Serializer<RSAPrivateKey>, Deserializer
<RSAPrivateKey> {

    private final TextEncryptor textEncryptor;

    RsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }

①
    @Override
    public void serialize(RSAPrivateKey object, OutputStream outputStream) throws
IOException {
        var pkcs8EncodedKeySpec = new PKCS8EncodedKeySpec(object.getEncoded());
        var string = "-----BEGIN PRIVATE KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(pkcs8EncodedKeySpec
.getEncoded())
                + "\n-----END PRIVATE KEY-----";
        outputStream.write(this.textEncryptor.encrypt(string).getBytes());
    }

②
    @Override
    public RSAPrivateKey deserialize(InputStream inputStream) {
        try {
            var pem = this.textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
            var privateKeyPEM = pem.replace("-----BEGIN PRIVATE KEY-----", "").
replace("-----END PRIVATE KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(privateKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new PKCS8EncodedKeySpec(encoded);
            return (RSAPrivateKey) keyFactory.generatePrivate(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }
    }

}
```

1. serialization involves adding a header and footer to the content of the key and then serializing, but not before Base64 encoding the content of the key, and then passing it through a Spring Security `TextEncryptor`
2. deserialization is basically the same, in reverse. Decrypt the text by passing it through a Spring Security `TextEncryptor`, then strip out the header and footer, and then use the Base64 decoder to decode the content, and then finally create a key using the Java Security `KeyFactory`.

The code for `RsaPublicKeyConverter`. We won't rehash it since its structure is basically identical to the `RsaPrivateKeyConverter`.

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPublicKey;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;

class RsaPublicKeyConverter implements Serializer<RSAPublicKey>, Deserializer
<RSAPublicKey> {

    private final TextEncryptor textEncryptor;

    RsaPublicKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }

    @Override
    public RSAPublicKey deserialize(InputStream inputStream) throws IOException {
        try {
            var pem = textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
            var publicKeyPEM = pem.replace("-----BEGIN PUBLIC KEY-----", "").replace("
-----END PUBLIC KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(publicKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new X509EncodedKeySpec(encoded);
            return (RSAPublicKey) keyFactory.generatePublic(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }

    }

    @Override
    public void serialize(RSAPublicKey object, OutputStream outputStream) throws
IOException {
```

```
        var x509EncodedKeySpec = new X509EncodedKeySpec(object.getEncoded());
        var pem = "-----BEGIN PUBLIC KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(x509EncodedKeySpec.
getEncoded())
                + "\n-----END PUBLIC KEY-----";
        outputStream.write(this.textEncryptor.encrypt(pem).getBytes());
    }

}
```

We'll look at the configuration of the `TextEncryptor` used in both implementations in a bit.

We'll need to register these converters. I suppose I could've added a `@Component` annotation to each of them, but instead I chose to register them with Java configuration, like this:

```
package com.example.authorization_server.keys;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.security.crypto.encrypt.TextEncryptor;

@Configuration
class ConvertersConfiguration {

    @Bean
    JdbcRsaKeyPairRepository jdbcRsaKeyPairRepository(RsaKeyPairRowMapper
rsaKeyPairRowMapper,
            RsaPublicKeyConverter rsaPublicKeyConverter, RsaPrivateKeyConverter
rsaPrivateKeyConverter,
            JdbcClient template) {
        return new JdbcRsaKeyPairRepository(rsaKeyPairRowMapper,
rsaPublicKeyConverter, rsaPrivateKeyConverter,
                template);
    }

    @Bean
    RsaKeyPairRepositoryJWKSource rsaKeyPairRepositoryJWKSource(RsaKeyPairRepository
repository) {
        return new RsaKeyPairRepositoryJWKSource(repository);
    }

    @Bean
    RsaKeyPairRowMapper rsaKeyPairRowMapper(RsaPrivateKeyConverter
rsaPrivateKeyConverter,
            RsaPublicKeyConverter rsaPublicKeyConverter) {
        return new RsaKeyPairRowMapper(rsaPrivateKeyConverter, rsaPublicKeyConverter);
    }
```

```
    @Bean
    RsaPublicKeyConverter rsaPublicKeyConverter(TextEncryptor textEncryptor) {
        return new RsaPublicKeyConverter(textEncryptor);
    }

    @Bean
    RsaPrivateKeyConverter rsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        return new RsaPrivateKeyConverter(textEncryptor);
    }

}
```

Let's look at the JDBC-powered repository implementation.

```
package com.example.authorization_server.keys;

import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.util.Date;
import java.util.List;

class JdbcRsaKeyPairRepository implements RsaKeyPairRepository {

    private final JdbcClient jdbcClient;

    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;

    private final RowMapper<RsaKeyPair> keyPairRowMapper;

    JdbcRsaKeyPairRepository(RowMapper<RsaKeyPair> keyPairRowMapper,
RsaPublicKeyConverter publicConverter,
            RsaPrivateKeyConverter privateConverter, JdbcClient jdbcClient) {
        this.jdbcClient = jdbcClient;
        this.keyPairRowMapper = keyPairRowMapper;
        this.rsaPublicKeyConverter = publicConverter;
        this.rsaPrivateKeyConverter = privateConverter;
    }

①
    @Override
    public List<RsaKeyPair> findKeyPairs() {
        return this.jdbcClient.sql("select * from rsa_key_pairs order by created desc
```

```
")
            .query(this.keyPairRowMapper)
            .list();
    }

②
    @Override
    public void save(RsaKeyPair keyPair) {
        var sql = """
                insert into rsa_key_pairs (id, private_key, public_key, created)
                values (?, ?, ?, ?)
                on conflict on constraint rsa_key_pairs_id_created_key
                do nothing
                """;
        try (var privateBaos = new ByteArrayOutputStream(); var publicBaos = new
ByteArrayOutputStream()) {
            this.rsaPrivateKeyConverter.serialize(keyPair.privateKey(), privateBaos);
            this.rsaPublicKeyConverter.serialize(keyPair.publicKey(), publicBaos);
            var updated = this.jdbcClient.sql(sql)
                .param(keyPair.id())
                .param(privateBaos.toString())
                .param(publicBaos.toString())
                .param(new Date(keyPair.created().toEpochMilli()))
                .update();
            Assert.state(updated == 0 || updated == 1, "no more than one record should
have been updated");
        } //
        catch (IOException e) {
            throw new IllegalArgumentException("there's been an exception", e);
        }
    }

}
```

1. in order to find the records, we'll issue a query and pass in the `RowMapper<RsaKeyPair>`, which is an instance of `RsaKeyPairRowMapper`, which we'll explore shortly.
2. writing the record is pretty easy, too. The converters do most of the work. Then it's a simple matter of lining up the converted values as arguments for the SQL update.

We saw that the repository implementation uses a Spring JDBC `RowMapper<T>` instance, which looks like this:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Component;

import java.io.IOException;
```

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Date;

class RsaKeyPairRowMapper implements RowMapper<RsaKeyPair> {

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;

    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    RsaKeyPairRowMapper(RsaPrivateKeyConverter rsaPrivateKeyConverter,
RsaPublicKeyConverter rsaPublicKeyConverter) {
        this.rsaPrivateKeyConverter = rsaPrivateKeyConverter;
        this.rsaPublicKeyConverter = rsaPublicKeyConverter;
    }

    private static <T> T loadKey(ResultSet rs, String fn, Deserializer<T> f) throws
SQLException, IOException {
        var privateKeyBytes = rs.getString(fn).getBytes();
        return f.deserializeFromByteArray(privateKeyBytes);
    }

    @Override
    public RsaKeyPair mapRow(ResultSet rs, int rowNum) throws SQLException {
        try {

①
            var privateKey = loadKey(rs, "private_key", this.rsaPrivateKeyConverter);
            var publicKey = loadKey(rs, "public_key", this.rsaPublicKeyConverter);

②
            var created = new Date(rs.getDate("created").getTime()).toInstant();
            var id = rs.getString("id");

③
            return new RsaKeyPair(id, created, publicKey, privateKey);
        }
        catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

}
```

1. again, one of the nice things about this implementation is that most of the heavy lifting is in the converters.
2. this stuff is thankfully much easier to deal with
3. I've extracted the logic of extracting a field from the `ResultSet`, passing it to a converter, and then returning it, into a separate method.

Now we've done everything we need to do to support reading and writing key pairs. Let's put it all to good use in an implementation of `JWKSource`, which - when you look at it - is almost anticlimactic. Two concerns are being addressed in this implementation. The first is telling Nimbus — the library whose JWT support underpins the Spring Authorization Server — how to load a new key given a query (called a `JWKSelector`). The second is telling the Spring Authorization Server that we want a chance to post-process, to *customize*, the OAuth 2.0 token attributes. We'll plug that `OAuth2TokenCustomizer` functionality in later.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.KeySourceException;
import com.nimbusds.jose.jwk.JWK;
import com.nimbusds.jose.jwk.JWKSelector;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import
org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
import
org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

import java.util.ArrayList;
import java.util.List;

class RsaKeyPairRepositoryJWKSource implements JWKSource<SecurityContext>,
OAuth2TokenCustomizer<JwtEncodingContext> {

    private final RsaKeyPairRepository keyPairRepository;

    RsaKeyPairRepositoryJWKSource(RsaKeyPairRepository keyPairRepository) {
        this.keyPairRepository = keyPairRepository;
    }

①
    @Override
    public List<JWK> get(JWKSelector jwkSelector, SecurityContext context) throws
KeySourceException {
        var keyPairs = this.keyPairRepository.findKeyPairs();
        var result = new ArrayList<JWK>(keyPairs.size());
        for (var keyPair : keyPairs) {
            var rsaKey = new RSAKey.Builder(keyPair.publicKey())//
                .privateKey(keyPair.privateKey())//
                .keyID(keyPair.id())//
                .build();
            if (jwkSelector.getMatcher().matches(rsaKey)) {
                result.add(rsaKey);
            }
        }
```

```
        return result;
    }

②
    @Override
    public void customize(JwtEncodingContext context) {
        var keyPairs = this.keyPairRepository.findKeyPairs();
        Assert.state(!keyPairs.isEmpty(),
                () -> "there should be at least one element in the " + RsaKeyPair
.class.getName() + " collection");
        var kid = keyPairs.get(0).id();
        context.getJwsHeader().keyId(kid);
    }

}
```

1. when asked, we pass through and return the list of `RsaKeyPair` instances from the database, turning them into `RSAKey.Builder` instances.
2. when asked, we customize the JWTs that are generated by specifying the key ID, so that it lines up with the keys in the repository.

Let's see how all of this gets plugged into Spring Authorization Server through configuration.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.security.oauth2.core.OAuth2Token;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.authorization.token.*;

@Configuration
class KeyConfiguration {

    @Bean
    Keys keys() {
        return new Keys();
    }

①
    @Bean
    TextEncryptor textEncryptor(@Value("${jwk.persistence.password}") String pw,
```

```
            @Value("${jwk.persistence.salt}") String salt) {
        return Encryptors.text(pw, salt);
    }

②
    @Bean
    OAuth2TokenGenerator<OAuth2Token> delegatingOAuth2TokenGenerator(JwtEncoder
encoder,
            OAuth2TokenCustomizer<JwtEncodingContext> customizer) {
        var generator = new JwtGenerator(encoder);
        generator.setJwtCustomizer(customizer);
        return new DelegatingOAuth2TokenGenerator(generator, new
OAuth2AccessTokenGenerator(),
                new OAuth2RefreshTokenGenerator());
    }

③
    @Bean
    NimbusJwtEncoder jwtEncoder(JWKSource<SecurityContext> jwkSource) {
        return new NimbusJwtEncoder(jwkSource);
    }

}
```

1. this is the `TextEncryptor` that's doing so much of the work for us in the converters we looked at earlier. It's a `TextEncryptor`, as opposed to a `BytesEncryptor`. Ultimately, it's using an algorithm called AES, which is the gold standard in ciphers today, succeeding a long line of other algorithms - like DES and 3DES - that go all the way back to the 1970s. It's both space efficient and cryptographically secure.
2. We only want to plug in the `OAuth2TokenCustomizer` for the `JwtGenerator`, but we need to redeclare the bean that uses it, the `OAuth2TokenGenerator`. Most of this is boilerplate that we're copying from the defaults.
3. and finally we want to plug in our new `JWKSource` implementation



In this last example we injected some properties, `jwk.persistence.password` and `jwk.persistence.salt`, which are credentials that need to be stored in a secure fashion!

The machinery is in place, but something needs to start it! We want the application to start up and automatically register a new `RsaKeyPair` if none exists in the database (otherwise the Spring Authorization Server wouldn't work!), and we want to be able to rotate the keys automatically. So, I've created a new Spring `ApplicationEvent`, called `RsaKeyPairGenerationRequestEvent`. We'll rig up a listener to rotate the keys whenever an instance of that event is published.

Any code anywhere in the Spring Authorization Server could publish that event and trigger a key rotation. You could have a `@Scheduled` method that runs every 24 hours and rotates the keys. You could create a (secured) HTTP endpoint that publishes the event. You could create a (secured) Actuator endpoint that publishes the event. You could write some code to listen to new messages

coming in from Kafka and then publish the event in response. The sky is the limit! So when will we first publish the event, though? On startup, but only if we discover there are no keys in the repository. Let's see that event wiring.

```
package com.example.authorization_server.keys;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.ApplicationListener;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

import java.time.Instant;

@Configuration
class LifecycleConfiguration {

①
    @Bean
    ApplicationListener<RsaKeyPairGenerationRequestEvent>
keyPairGenerationRequestListener(Keys keys,
            RsaKeyPairRepository repository, @Value("${jwk.key.id}") String keyId) {
        return event -> repository.save(keys.generateKeyPair(keyId, event.getSource()
));
    }

②
    @Bean
    ApplicationListener<ApplicationReadyEvent> applicationReadyListener
(ApplicationEventPublisher publisher,
            RsaKeyPairRepository repository) {
        return event -> {
            if (repository.findKeyPairs().isEmpty())
                publisher.publishEvent(new RsaKeyPairGenerationRequestEvent(Instant
.now()));
        };
    }

}
```

1. this `ApplicationListener` listens for the aforementioned event and writes a new `RsaKeyPair` to the repository, using the injected `jwk.key.id`, which should be specified externally, and should remain constant. That is, after all, the key ID.
2. this `ApplicationListener` runs when the service starts and publishes a `RsaKeyPairGenerationRequestEvent`, but only if there are no `RsaKeyPairs` in the repository already.

At this point you can delete the key files we generated with `openssl` earlier. You can also delete the

relevant configuration in your `application.properties` or `application.yml`; the application can now create and rotate its own keys.

To Production… and Beyond!

And that's it! Restart the Spring Authorization Server. Now you can run more than one instance behind a load balancer. It won't matter which instance a client (and its cookies) lands on: each one resolves the session, the authorizations, the information about clients, and more from the PostgreSQL database. It'll also sign keys in a consistent fashion. Is now a good time to remind you not to stash sensitive stuff in the HTTP session? You never know when it's going to be serialized to some datastore… :code: ..

Protecting a Simple HTTP API

We ended the last few chapters with a known end state. We stood up a Spring Authorization Server, gave it users and a client registration, taught it to persist its state, and let it manage its own signing keys. We also built a small web application that redirects to that authorization server, authenticates, and comes back holding an access token. Then we asked the obvious next question: what do we actually *do* with that token?

An access token is only interesting if something on the other end insists on seeing one. So in this chapter we'll build that something - a backend HTTP API that refuses to answer unless the caller presents a valid token - and we'll teach our web application to attach the token it's already holding to every call it makes.

A typical OAuth system has three moving parts, and it's worth naming them precisely before we write any code, because the rest of this book leans hard on these distinctions.

The *authorization server* authenticates users and issues tokens. That's the Spring Authorization Server we've spent the last several chapters building, running on port `9090`.

The *client* is the application that acts on the user's behalf. It's the piece that faces the user, initiates the OAuth dance, and ends up holding an access token. Ours will be a Spring Boot application on port `8080`.

The *resource server* is the backend API the client calls once it has that token. It doesn't authenticate anybody; it just inspects the token it's handed and decides whether to answer. Ours will run on port `8081`.

One caveat before we start. Our "client" is going to expose a REST endpoint, and in the real world that's not what a client looks like. Your users won't `curl` your API. They'll load a React or Vue application in a browser, open an iPhone or Android app, launch a desktop application, or run a CLI. A plain REST endpoint stands in for all of those here because it's the shortest path to watching a token move from one process to another, and watching that happen is the whole point of the exercise. We'll graduate to a more realistic arrangement in the second half of this chapter, when we put a gateway in front of everything.

The Flow

Here's what we're building toward.

You visit the client at `127.0.0.1:8080/headers`. It sees you're not authenticated and redirects you to the authorization server on `localhost:9090`, where you sign in and consent. The authorization server redirects you back to the client, which exchanges the authorization code for an access token and stores it against your session. From that point on the client is holding a credential it can use to talk to anything protected by this authorization server. When you hit its endpoint, it pulls the access token out of your session, puts it in an `Authorization` header, and calls the resource server on `localhost:8081`. The resource server validates the token and answers.



Note that we reach the client at `127.0.0.1:8080/headers` and the authorization

> server at `localhost:9090` - the same cookie-scoping trick we set up in *Implementing the Spring Authorization Server*. The rule is simple: anything that holds a session gets its own hostname. That applies to the gateway later in this chapter, too, since it holds a session of its own. The resource server needs no such treatment, because it's stateless and never sets a session cookie.

Let's build the client first, then give it something to call.

The OAuth Client

Go to the Spring Initializr [https://start.spring.io] and add `OAuth2 Client`, `Spring Web`, and `HTTP Client`. Throw in `GraalVM Native Support` while you're there. Click `Generate` and open the resulting project in your favorite IDE.

The API we're calling returns a single JSON object with a single field, so a record will do.

```
package com.example.oauth_client.headers;

record Message(String message) {
}
```

Now the interesting part: the component that actually makes the downstream call.

```
package com.example.oauth_client.headers;

import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component("headersMessageClient")
class MessageClient {

    private final RestClient http;

    MessageClient(RestClient.Builder http) {
        this.http = http.build();
    }

①
    Message getMessage(String token) {
        return this.http //
            .get()//
            .uri("http://localhost:8081/message") //
            .headers(h -> h.setBearerAuth(token)) ②
            .retrieve() //
            .body(Message.class);
    }
```

```
}
```

1. Where does this token come from?
2. We need to be sure to relay the token in the outgoing request's `Authorization` header, as a bearer token.

There's almost nothing here, which is the point. It's an ordinary `RestClient` making an ordinary `GET` request, with exactly one line - `.headers(h → h.setBearerAuth(token))` - that turns it into an authenticated one. That line writes the `Authorization: Bearer …` header the resource server is going to expect. The token arrives as a parameter, because `MessageClient` has no idea where tokens come from (and shouldn't).

Getting the token is the controller's job. Spring Security supports parameter injection of certain relevant data into Spring MVC controller handler methods.

```
package com.example.oauth_client.headers;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.security.oauth2.client.OAuth2AuthorizedClient;
import
org.springframework.security.oauth2.client.annotation.RegisteredOAuth2AuthorizedClient
;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller("headersMessageController")
@ResponseBody
class MessageController {

    private final MessageClient client;

    MessageController(@Qualifier("headersMessageClient") MessageClient client) {
        this.client = client;
    }

    @GetMapping("/headers")
    Message getMessage(@RegisteredOAuth2AuthorizedClient OAuth2AuthorizedClient
client) {
        var token = client.getAccessToken().getTokenValue();
        return this.client.getMessage(token);
    }

}
```

The `@RegisteredOAuth2AuthorizedClient` annotation is doing the heavy lifting. Spring Security keeps the tokens it obtained during the OAuth dance in an `OAuth2AuthorizedClientRepository`, keyed by the current user and the client registration they authenticated with. That annotation resolves the right

entry and hands you an `OAuth2AuthorizedClient`, from which `getAccessToken().getTokenValue()` yields the raw token string. No token juggling, no session lookups, no plumbing.

Finally, the client needs to know which authorization server it's talking to and who it claims to be when it gets there.

```
spring.application.name=oauth-client
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:9090
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=spring
spring.security.oauth2.client.registration.spring.client-secret=spring
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=openid,profile
spring.threads.virtual.enabled=true
```

This should look familiar - it's the same client registration we configured when we first stood this application up. The `issuer-uri` points at the authorization server, and the client ID and secret match the registration we created on it.

The Resource Server

Now for the API on the other end of that call.

Back to the Spring Initializr [https://start.spring.io], this time for `OAuth2 Resource Server`, `Spring Web`, and again `GraalVM Native Support`. Note that this project needs no HTTP client of its own - it's the end of the line, not a link in the middle of it. Generate the project and open it in your IDE.

The endpoint itself is unremarkable.

```
package com.example.resource_server;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;

@Controller
@ResponseBody
class MessageController {

    @GetMapping("/message")
```

```
    Map<String, String> message(Principal principal) {
        return Map.of("message", "Hello " + principal.getName());
    }

}
```

The one thing worth dwelling on is that `Principal` parameter. This service has no user database, no password encoder, and no login form. It has never met the person it's greeting by name. It knows who they are because the token it was handed says so - Spring Security validated that token, unpacked its `sub` claim, and built a `Principal` out of it. That's the trade at the heart of OAuth: the resource server outsources the hard question of *who is this* and gets to spend its energies on the business at hand.

And here's the entire security configuration.

```
spring.application.name=resource-server
server.port=8081
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:9090
spring.threads.virtual.enabled=true
```

That really is all of it. There's no `SecurityConfiguration` class in this project, no `SecurityFilterChain` bean, not a line of Java devoted to security. Adding the `OAuth2 Resource Server` dependency and setting `spring.security.oauth2.resourceserver.jwt.issuer-uri` is enough for Spring Boot to autoconfigure a filter chain that requires a valid JWT on every request and rejects everything else with a `401`.

How does it decide whether a token is valid? The tokens the Spring Authorization Server issues are JWTs, which are self-contained and signed. On startup, the resource server takes that `issuer-uri`, fetches the authorization server's `/.well-known/openid-configuration` document, and reads the JWK Set URI out of it - the endpoint that publishes the authorization server's public signing keys. From then on it validates signatures locally, in-process, with no network call per request. It also checks the usual claims along the way: that the token hasn't expired and that its `iss` claim matches the issuer we configured.



Because the resource server discovers the authorization server's configuration at startup, the authorization server needs to be running *first*. Start it before the resource server, or the resource server will fail to come up.

Two chapters ago we went to some trouble to give the authorization server durable, rotatable signing keys. This is where that pays off. The resource server never sees a shared secret and never phones home to ask permission; it just needs the public half of a key pair it can fetch once and cache.

Running the Whole Thing

Start the three applications in order: the authorization server on `9090`, the resource server on `8081`, and the OAuth client on `8080`.

Visit `127.0.0.1:8080/headers` in a browser and let the flow we sketched out earlier play through. Sign in as one of the users we created earlier - `josh@joshlong.com`, with the password `pw` - and consent. You'll land back on the client, and it'll hand you something like this:

```
{"message":"Hello josh@joshlong.com"}
```

That string traveled farther than it looks. Your identity was established on one service, minted into a token by that service, carried across a session on a second service, and finally verified by a third service that has never heard of you and never will. Nothing in the middle had to be told your password.

To convince yourself the protection is real, call the resource server directly with `curl http://localhost:8081/message`. You'll get a `401` and a `WWW-Authenticate` header for your trouble.

Centralizing the `RestClient` Authentication Logic

The `MessageClient` works, but it's a little cumbersome having to propagate the token from the Spring MVC controller layer to the client. In this pattern, we'll have to source the token for every single request and pass that token down into the client code. Instead, let's modify the `RestClient` and tell it to source the token for us. After all, if Spring MVC can read the information in the `SecurityContextHolder` and eventually resolve the OAuth access token, then why can't the `RestClient`? The `RestClient` supports interceptors that can pre- and post-process outbound HTTP requests. Spring Security has just such an interceptor for us.

Let's see how things come together. The controller is basically unaffected, so we'll ignore it. Here's the updated `MessageClient`.

```
package com.example.oauth_client.restclient;

import org.springframework.security.oauth2.client.OAuth2AuthorizedClientManager;
import
org.springframework.security.oauth2.client.web.client.OAuth2ClientHttpRequestIntercept
or;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

import static
org.springframework.security.oauth2.client.web.ClientAttributes.clientRegistrationId;

@Component("restclientMessageClient")
class MessageClient {

    private final RestClient http;

    MessageClient(OAuth2AuthorizedClientManager am, ①
            RestClient.Builder http) {
        this.http = http.requestInterceptor(new OAuth2ClientHttpRequestInterceptor(am
)) ②
```

```
            .build();
    }

    Message getMessage() {
        return this.http //
            .get()//
            .uri("http://localhost:8081/message") //
            .attributes(clientRegistrationId("spring"))③
            .retrieve() //
            .body(Message.class);
    }

}
```

1. The `OAuth2AuthorizedClientManager` is a key object that we'll see many, many more times in this book.
2. With it, the `OAuth2ClientHttpRequestInterceptor` will obtain the token and pre-process the outbound HTTP request, inserting an `Authorization` header.
3. We don't need to pass the token around anymore, but we *do* need to tell the `RestClient` which OAuth client to use to make the request. We *could*, in theory, have more than one defined in, say, our `application.properties`. In practice, we've just got the one.

Secure Declarative HTTP Service Clients

Much better. But can we clean things up further still? One of my favorite new features in Spring Boot 3.x - and even more so in Spring Boot 4.x - is the declarative HTTP service client. If you've ever used Spring Cloud's OpenFeign support, then you'll recognize the basic pattern: you define an interface, annotate it, and Spring will implement it and handle outbound HTTP requests for you.

Here's our revised `MessageClient`.

```
package com.example.oauth_client.interfaces;

import org.springframework.security.oauth2.client.annotation.ClientRegistrationId;
import org.springframework.web.service.annotation.GetExchange;

@ClientRegistrationId("spring") ①
interface MessageClient {

②
    @GetExchange("http://localhost:8081/message")
    Message getMessage();

}
```

1. The `@ClientRegistrationId` annotation allows us to associate an OAuth client with all HTTP invocations from this HTTP service client.

2. The annotations on the client are similar to those used on Spring MVC controller handler methods. They're sort of the mirror image.

It's no more than an interface! Lovely. The `@ClientRegistrationId` annotation requires that we define a single bean in the Spring context - just once, regardless of how many service interfaces we define.

```
package com.example.oauth_client.interfaces;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client.OAuth2AuthorizedClientManager;
import
org.springframework.security.oauth2.client.web.client.support.OAuth2RestClientHttpServ
iceGroupConfigurer;
import org.springframework.web.service.registry.ImportHttpServices;

@Configuration
@ImportHttpServices(MessageClient.class) ①
class MessageClientConfiguration {

    @Bean
    OAuth2RestClientHttpServiceGroupConfigurer
auth2RestClientHttpServiceGroupConfigurer(
            OAuth2AuthorizedClientManager aacm) { ②
        return OAuth2RestClientHttpServiceGroupConfigurer.from(aacm);
    }

}
```

1. Tell Spring to turn our interface into an implementation…
2. …and here's our friend `OAuth2AuthorizedClientManager` again.

I love this feature! Go back up a few pages and compare our original `MessageClient` to this finished product. The essence of what we were trying to do was buried under all the boilerplate. It's so much better to see all that stuff stripped away.

What Survives, and What Doesn't

We now have a working end-to-end OAuth system, and it's worth being clear-eyed about which parts of it will survive contact with production.

The security model will. The client obtains a token, the resource server validates it, and neither one is doing anything unusual. Add a fourth service tomorrow and the pattern doesn't change. The declarative client survives, too: any service that needs to call another service with a token can be written exactly the way we just wrote `MessageClient`, and we'll write it that way many more times before this book is done.

The wiring won't. `MessageClient` has the resource server's address compiled into it, which is fine for two processes on a laptop and untenable for twenty across a fleet. In production you'd use property placeholder resolution or DNS to substitute real values.

And the front door won't. We said at the outset that a REST endpoint is a stand-in for a real user interface, and we've now gotten everything out of that stand-in that it had to give. Time to replace it.

A Real OAuth Client with Spring Cloud Gateway

Shut down the existing `oauth-client`. What we're about to build takes over its job - originating the OAuth dance and attaching the token to outbound calls - but it does that job for a browser instead of for a `curl` invocation, and it does it without a controller of any kind.

In my mind, a user should be able to point their browser at our domain, get an HTML page with JavaScript and CSS, and have that page present information sourced from calls to backend APIs. I don't want that JavaScript code burdened with obtaining OAuth tokens or performing cross-origin resource sharing gymnastics.

To keep things nice and tidy, we'll front both our backend API (the resource server) *and* the HTML content with a Spring Cloud Gateway-powered `gateway` service.

In our super secure OAuth onion, the `gateway` is the outermost layer. It is the first port of call for all requests destined for the microservices in our system. When visitors to our site punch our domain name into a browser, it'll resolve to a load balancer serving this gateway service, and it is here that we'll originate an OAuth token. The gateway acts as a proxy, allowing us to forward requests to different hosts and ports, concealing from the user that the responses they're seeing are from different services.

The `gateway` service's job is to handle the OAuth dance - if it detects that the request is not authenticated - and to otherwise proxy requests to two downstream HTTP endpoints: the `resource-server` we just built, which is running on port `8081`, and a folder of static HTML we'll call `ui`, which we'll write in a moment and serve on port `8020`. Keep that second one in mind as we read the routing table; the gateway needs to know where it lives before it exists.

Building a Working Spring Cloud Gateway Instance

Here's the routing table. I've used two logical names for the hosts, just to be clear. But for our demo everything will run locally, as before.

HTTP origin	HTTP destination
http://gateway:8080/api/message	http://api:8081/message
http://gateway:8080/index.html	http://static:8020/index.html

Here's the Spring Cloud Gateway configuration. This configuration uses the functional Spring MVC style. This style is an alternative to the Spring MVC controller handler methods we've seen thus far. Each `RouterFunction<ServerResponse>` bean defines predicates that match a given request, filters that act on the request, and a handler. In this case the handler is provided by Spring Cloud Gateway

and comes from the `http()` static method.

```
package com.example.gateway;

import org.springframework.cloud.gateway.server.mvc.filter.BeforeFilterFunctions;
import org.springframework.cloud.gateway.server.mvc.filter.TokenRelayFilterFunctions;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.web.servlet.function.RouterFunction;
import org.springframework.web.servlet.function.ServerResponse;

import static
org.springframework.cloud.gateway.server.mvc.handler.HandlerFunctions.http;
import static org.springframework.web.servlet.function.RouterFunctions.route;

@Configuration
class RouteConfiguration {

①
    @Bean
    @Order(Ordered.LOWEST_PRECEDENCE)
    RouterFunction<ServerResponse> ui() {
        return route().GET("/**", http()).before(BeforeFilterFunctions.uri(
"http://localhost:8020")).build();
    }

②
    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    RouterFunction<ServerResponse> api() {
        return route().GET("/api/**", http())
            .before(BeforeFilterFunctions.uri("http://localhost:8081"))
            .before(BeforeFilterFunctions.rewritePath("/api", "/")) ③
            .filter(TokenRelayFilterFunctions.tokenRelay()) ④
            .build();
    }

}
```

1. This route has the lowest *ordering*. It'll be evaluated last, *after* all the other routes have been evaluated. This is important because it's the most greedy, and it would potentially match *all* requests, short-circuiting the evaluation of the other routes. The effect is that, unless a more specific route answers, this route will attempt to proxy the incoming request (and its path) to the server running on `8020`, which will serve our static `.html` page.
2. This route handles requests that have a prefix of `/api`.
3. This filter strips off the prefix…

4. And this filter is what triggers the OAuth client behavior we saw earlier. If there's no authenticated user, this'll force the client to redirect to the Spring Authorization Server. If there is one, this will encode an `Authorization: Bearer …` header in the request before it's proxied.

And here's the `application.properties` file. The client registration is character-for-character what the old `oauth-client` used, which is the point worth noticing: as far as the authorization server is concerned, nothing has changed. We swapped out the entire front end of the system and the OAuth configuration didn't move.

```
spring.application.name=gateway
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:9090
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=spring
spring.security.oauth2.client.registration.spring.client-secret=spring
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=openid,profile
spring.threads.virtual.enabled=true
```

The Good, the Bad, and the Ugly: JavaScript

First, we've got to serve up the static HTML assets. In this case it's just a single little `.html` page, but you could easily imagine it being a proper single-page application. I've put it in a folder called `ui`. Here's the `.html`.

```
<html>
<title>
    The Message Delivery Room
</title>
<body>
①
<div id="message"></div>
<script>
    addEventListener('load', async () => {
②
        const json = await fetch('/api/message')
        const data = await json.json()
        document.getElementById('message').innerHTML = data['message']
    })
</script>
</body>
</html>
```

1. This `<div>` element will contain the message retrieved from the server.

2. Plain `window.fetch`, and a relative path.

See how clean that is? No absolute hosts and ports, no CORS preflight, no tokens anywhere in the JavaScript. That's the promise from a few pages ago being kept: because the gateway serves both the page and the API from one origin, the browser considers this a same-origin request and asks no further questions. By the time the user sees this page in the browser, we'll have a cookie that corresponds to our HTTP session on the server side (the gateway). When you use the browser's `window.fetch` function, it sends the browser's cookies to the server. So when we make this request, the server will see the cookie, connect it to the HTTP session, realize the user's authenticated, and then allow the request to go through.

You'll want to serve that static `.html` page on port `8020`. In production you'd probably stash it on a CDN, or put it behind NGINX or something. But for our demo, a very convenient and easy way to serve it is to use Java's friendly little HTTP web server, `jwebserver`. I've written a little wrapper script that takes two optional parameters: the port to serve on and the directory to serve. If you don't specify them, then the defaults are `8020` and the current directory.

```
#!/bin/bash
p=${1:-8020}
d=${2:-$PWD}
cd $d
jwebserver -d $d -p $p
```

Don't forget to make the script executable: `chmod a+x serve.sh`.

In your shell, enter the directory where `index.html` is stored and then run `./serve.sh`.

All Together Now

You should have the backend resource server on port `8081`, the Spring Authorization Server on `9090`, the `index.html` page on `8020`, and the Spring Cloud Gateway on `8080`. If you visit `127.0.0.1:8080`, you should be redirected to the Spring Authorization Server to authenticate and consent, then redirected back to the Spring Cloud Gateway instance, where you'll see the result of the successfully proxied gateway routes.

Take a moment to appreciate what is *not* in that browser tab. There's no token in the JavaScript, no CORS configuration, no login form of our own, and not one line in the resource server that knows the first thing about who you are. What's actually there is a cookie between the browser and the gateway, a bearer token between the gateway and the API, and a public signing key the API fetched once and cached. Three services, three ports, and exactly one place where a password was ever typed.

That arrangement is the one we'll keep. What changes from here is the protocol on the wire, not the pattern underneath it: the chapters ahead take this same authorization server and this same token and put them to work over other transports entirely. The dance you just watched is the dance.

PKCE with JavaFX Desktop-based Applications

GraphQL

In this section, we'll look at how to secure Spring GraphQL services with OAuth and method security, alike. :code: ..

gRPC

In this section, we'll look at how to secure Spring gRPC applications with Spring Security, OAuth, and more. :code: ..

Secure SOAP-based Web Services

Protecting Spring AI-based Model Context Protocol (MCP) Services

Messaging

This chapter is my favorite one. Not because it's going to be the most useful, but because it's the one that took the longest for me to figure out. You see, I love backoffice code. I love messaging, and integration. I love analytics. I love workflow. I love stream and batch processing. I love workflow engines and business process management. I love grid computing. I love short-lived tasks like CRON jobs. I love all the sorts of stuff that has no business being anywhere near an HTTP request or taking up space on an HTTP webservice. We used to call these sorts of things "backoffice" jobs. And you've probably built some of these in your career, too.

And I didn't know how OAuth fit in this world of backoffice code. I mean, just look at the Spring portfolio! We've got Spring Integration, Spring AMQP, Spring for Apache Kafka, Spring Cloud Data Flow, Spring Cloud Task, Spring Batch, and even Spring Shell. You can build all sorts of cool stuff with those libraries and never see an HTTP header! So how does OAuth fit here? And, more precisely, how does Spring Security's OAuth support plugin? We've already met the usual suspects: `spring-boot-starter-oauth-client`, `spring-boot-starter-resource-server`, etc. But those all assume the presence of an HTTP server and the Spring Security web filter chain.

In this chapter we're going to expand our sample application a bit with some Spring Integration code that will receive a message that the API (the resource server) will send. Each message will contain the JWT token associated with the authenticated user and it'll contain a payload tat the processor will.., you know, process. In this example, we'll imagine that this processor is busy doing the work of sending emails. We're not going to actually write that bit of the code. But it is a good example. After all, sending email can sometimes take a long time and we don't want to keep the API busy doing this when it should be fielding HTTP requests.

When the message arrives at this `processor` module, we'll validate the JWT token by talking to the Spring Authorization Server (our OAuth IDP) through its issuer URI. Does this sort of sound familiar? It should! We're going to basically do the same trick as the resource server did, albeit a bit more granuarly. Well get to see how the resource server support does some of its work. Its good to know this because, outside of an HTTP environment, there's no one-sized fits all approach. It's convenient then that we can easily plug this stuff in ourselves.

Spring Integration

Let's talk again about Spring Integration. we looked at it ever so briefly when we introduced the API, but let's review some basics.

Remember this code from the API we built earlier?

It's a bean of type `IntegrationFlow`, from the Spring Integration project. The bean describes how we handle messages intended for a RabbitMQ broker: messages come, then we turn it into JSON, then we send it over AMQP. Simple.

Spring Integration is an old project, from 2007. The core conceit fo Spring Integration is that as we move forward in time the body of systems and services with which we need to integrate - to maximally retain value - grows, and the protocols and paradigms required for integration also grow. Spring Integration is an enterprise application technnology. It's designed to help glue systems

together, particularly those things that wouldn't otherwise know about and work with each other.

In 2004, Gregor Hohpe and Bobby Woolf wrote the book *Enterprise Integration Patterns* [https://enterpriseintegrationpatterns.com], which gave us the names for the patterns typical of integration solutions. Broadly, the book said, there are four different kinds of integration styles.

- **RPC**: in this style, network services are made to work like a local object. In such a style, a client could invoke a method on a local Java object and have that translated into remote procedure calls on another object on another object, presumably running on another host. This style feels simple but it hides the reality of the network - that it will fail. It makes assumptions that it shouldn't, namely that the service will always be available. If the service - the consumer - is not available then has to retry or abandon the integration. Because of these design tradeoffs, RPC is a poor choice for service integration.
- **Shared Database**: in this style, a client connects the same database (e.g.: Oracle, PostgreSQL, MongoDB, etc), writes data there that another client then reads. This is fragile because a schema change by one client might break another client. This approach completely violates the principles of encapsulation, exposing the peculiarities data storage to consumers, and is therefore a poor choice for service integration. There's a reason nobody ever says "put your best liver forward!" It's nobody's business what your liver looks like! They should be shaking your hand, instead. The same is true in integration: don't share too much.
- **File synchronization**: in this style, a client deposits a file on a filesystem (NFS, FTP, FTPS, SFTP, SMB, etc.) that a client then consumes. This approach works alright but care must be taken so that the consumer of a file doesn't start processing it before the producer has finished writing it. Additionally, this approach lacks any sort of sophistication around message delivery (once and only once, transactions, message rollback) or routing.
- **Messaging**: in this style, integration is done in terms of a messaging system like Apache Kafka or RabbitMQ. This is usually the best approach if you can get access to it. Messaging systems support transactions, they can be made to ensure a message is delivered, that it is delivered at most once, or at least once, etc. It can hanndle routing, allowing you to send the message to different consumers as required. And of course it does not couple producer or consumer; a consumer can send a message to the messaging system, and it'll be recorded there. When the consumer is available and able, it can read and process the message. We can say that this style of integration is the most decoupled: producers and consumers don't need to both be available at the same time; producers and consumers only see and agree upon message payloads, and not the internals of their state management schemes.

Generally, speaking, messaging is the most flexible approach to building and integrating systems. So it is that Spring Integration models everything as `Message` objects that pass through `MessageChannel` objects. A `MessageChannel` is the connective tissue between components that act on `Message` objects, in a sort of pipeline. These components are written in terms of the `Message`s they accept and the `Message`s they produce. They're otherwise usually stateless. In a way, Spring Integration encourages a lot of the same discipline and conventions as any functional programming language might. You're encouraged to write your business logic in terms of granular, composable, and reusable functions.

Where do these messages come from, and where do they go? Well, the real world of course! They have to come from somewhere. It is Spring Integration's job to connect our code to events in the

real world and translate them into `Message` objects: a microwave turned on (MQTT), a new file appeared in an FTP service, a new row appeared in a SQL database, a new email was sent to an inbox, a message arrived on a JMS destination, etc. It does this work with adapters which *adapt* events into `Message` objects. Each `Message` typically contains a payload (the inbound file adapter might contain a `java.io.File` payload, for example) and headers telling us about the payload (the folder in which the file was found, or the timestamp of when the message was produced).

Once we have a `Message` ,we can do all sorts of things to it. We could split it into smaller messages, route it to other handlers, add information to it, filter it, etc.

And then when we're finally done with it, we use an outbound adapter - which does the reverse of an inbound adapter -to send the message onward to some place in the real world.

So, to review the review: events come via inbound adapters, they're turned into `Message` objects with headers and payloads. In this form they're processed, and ultimately sent out via outbound adapters.

Now back to our regularly scheduled programming.

Defining RabbitMQ Infrastructure

First thing's first: we're using RabbitMQ. In RabbitMQ, you send messages to an exchange which then can route it anyway it wants. In our case, it's going to be sent to a single queue, which is where consumers will know to look for it. So, we have an exchange and a queue, and they're bound together through something called a binding. We'll use the Spring AMQP project to make short work of defining these things. If we define them as beans, Spring AMQP will automaticall create the real structures on the RabbitMQ broker.

```
package com.example.messaging;

import org.springframework.amqp.core.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import static com.example.messaging.Constants.RABBITMQ_DESTINATION_NAME;

@Configuration
class AmqpConfiguration {

    @Bean
    Queue queue() {
        return QueueBuilder.durable(RABBITMQ_DESTINATION_NAME).build();
    }

    @Bean
    Exchange exchange() {
        return ExchangeBuilder.directExchange(RABBITMQ_DESTINATION_NAME).build();
    }
```

```
    @Bean
    Binding binding() {
        return BindingBuilder.bind(queue()).to(exchange()).with
(RABBITMQ_DESTINATION_NAME).noargs();
    }

}
```

You'll notice that this code uses a constant variable I've defined. There are a few others…

A Few Well Known Constants

There are a few things that I've defined as constants: a header name, a `MessageChannel` bean ID, and the RabbitMQ destination.

```
package com.example.messaging;

public class Constants {

    public final static String REQUESTS_MESSAGE_CHANNEL = "requests";

    public static final String RABBITMQ_DESTINATION_NAME = "emails";

    public static final String AUTHORIZATION_HEADER_NAME = "jwt";

}
```

The Integration

Now we're into the meat of the example, the actual Spring Integration code.

In Spring Integration, an `IntegrationFlow` is the definition of a processing pipeline. You may have more than one `IntegrationFlow` in your application. `IntegrationFlow` objects chain together different components that act on the `Message` objects within the flow. You can route messages from one `IntegrationFlow` to another using `MessageChannel` instances.

Conceptually, all we want to do is take a message from the inbound AMQP (that's the protocol that RabbitMQ speaks) in and then print out the message. One might send an email or do something useful here, but we'll leave that as an exercise for another day… And yet, I've written it a little more obtusely: we have one `IntegrationFlow` that takes the message rom AMQP and then stuffs the message into a `MessageChannel`. It pops out the other side and *then* we print out the message's payload and headers. Why the indirection? Why add the `MessageChannel` into the mix, I hear you query. `MessageChannel` objects can have interceptors that can, in effect, *veto* messages. So we'll configure some interceptors to act on the message, validate the attached JWT token, and if it doesn't match, to reject the message *before* it gets to whatever important business logic we've got downstream of thee `MessageChannel`.

Let's see it all in action.

```
package com.example.messaging;

import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.amqp.dsl.Amqp;
import org.springframework.integration.dsl.DirectChannelSpec;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.MessageChannels;
import org.springframework.messaging.MessageChannel;
import org.springframework.security.authorization.AuthenticatedAuthorizationManager;
import
org.springframework.security.messaging.access.intercept.AuthorizationChannelIntercepto
r;
import
org.springframework.security.messaging.context.SecurityContextChannelInterceptor;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationPr
ovider;

import static com.example.messaging.Constants.AUTHORIZATION_HEADER_NAME;
import static com.example.messaging.Constants.RABBITMQ_DESTINATION_NAME;

@Configuration
class IntegrationConfiguration {

①
    @Bean
    IntegrationFlow inboundAmqpRequestsIntegrationFlow(
            @Qualifier(Constants.REQUESTS_MESSAGE_CHANNEL) MessageChannel requests,
            ConnectionFactory connectionFactory) {
        var inboundAmqpAdapter = Amqp.inboundAdapter(connectionFactory,
RABBITMQ_DESTINATION_NAME);
        return IntegrationFlow.from(inboundAmqpAdapter).channel(requests).get();
    }

②
    @Bean
    IntegrationFlow requestsIntegrationFlow(@Qualifier(Constants
.REQUESTS_MESSAGE_CHANNEL) MessageChannel requests) {

        return IntegrationFlow.from(requests)//
            .handle((payload, headers) -> {
                IO.println("----");
                headers.forEach((key, value) -> IO.println("%s=%s".formatted(key,
value)));
                return null;
            })//
            .get();
```

```
    }

③
    @Bean(Constants.REQUESTS_MESSAGE_CHANNEL)
    DirectChannelSpec requests(JwtAuthenticationProvider jwtAuthenticationProvider) {
④
        var jwtAuthInterceptor = new JwtAuthenticationInterceptor
(AUTHORIZATION_HEADER_NAME, jwtAuthenticationProvider);
⑤
        var securityContextChannelInterceptor = new SecurityContextChannelInterceptor
(AUTHORIZATION_HEADER_NAME);
⑥
        var authorizationChannelInterceptor = new AuthorizationChannelInterceptor(
                AuthenticatedAuthorizationManager.authenticated());
        return MessageChannels.direct()
            .interceptor(jwtAuthInterceptor, securityContextChannelInterceptor,
authorizationChannelInterceptor);
    }

}
```

① the first `IntegrationFlow` defines an AMQP Inbound adapter which listens for new `Messages` on our auto configured RabbitMQ connection. As soon as a message comes in, it gets sent to the next step in the `INtegrationFlow`. IN this case, that next step is to travel through an injected `MessageChannel` to another `IntegrationFlow`.

② the second `IntegrationFlow` takes whatevers been stuffed into the `MessageChannel` and passews it to the next step, which is a simple handler that inspects the message payload and headers.

③ both `IntegrationFlow` objects are connected by the `MessageChannel` whoe bean ID is `Constants.REQUESTS_MESSAGE_CHANNEL`, and whose definition we see here. It's a bit complicated but this is where we do the work of validating the JWT token.

④ most of the important woerk is done in these interceptors. Each interceptr can inspect, transform, or reject the `Message` objects flowign through the `MessageChannel` on which they're configured. I wrote this first interceptor. We'll look at it shortyl, but suffice it to say that it in turn is using the injected `JwtAuthenticationProvider` that is provided by Spring Security and whose configuration we'll examine momentarily to do the work of validating the JWT token against the Spring Authorization Server. If the token is valid, the interceptor creates a new message with a valid Spring Security `Authentication` in the header. If the token is not valid, then the interceptor will create a new message with a null value for the header.

⑤ this next interceptor hoists the `Authentication` from the message header and puts it into the Spring Security `SecurityContextHolder`, which is a well-known `ThreadLocal`-like holder of the current thread's authenticated user. Now, all the other machinery in Spring Security that consults this thread local will work correctly.

⑥ the final interceptor does the actual check, rejecting the message if the current thread doesn't have a valid, authenticated `Authentication`.

The `JwtAuthenticationInterceptor` leaves nothing to the imagination:

```
package com.example.messaging;

import org.springframework.messaging.Message;
import org.springframework.messaging.MessageChannel;
import org.springframework.messaging.support.ChannelInterceptor;
import org.springframework.messaging.support.MessageBuilder;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.AuthorityUtils;
import
org.springframework.security.oauth2.server.resource.authentication.BearerTokenAuthenti
cationToken;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationPr
ovider;
import org.springframework.util.Assert;

class JwtAuthenticationInterceptor implements ChannelInterceptor {

①
    private final JwtAuthenticationProvider authenticationProvider;

②
    private final String headerName;

    JwtAuthenticationInterceptor(String headerName, JwtAuthenticationProvider ap) {
        this.headerName = headerName;
        this.authenticationProvider = ap;
    }

    @Override
    public Message<?> preSend(Message<?> message, MessageChannel channel) {
③
        var token = (String) message.getHeaders().get(headerName);
        Assert.hasText(token, "the token must be non-empty!");

④
        var authentication = this.authenticationProvider.authenticate(new
BearerTokenAuthenticationToken(token));

⑤
        if (authentication.isAuthenticated()) {
            var upt = UsernamePasswordAuthenticationToken.authenticated(
authentication.getName(), null,
                    AuthorityUtils.NO_AUTHORITIES);
            return MessageBuilder.fromMessage(message).setHeader(headerName, upt)
.build();
        }

⑥
```

```
        return MessageBuilder.fromMessage(message).setHeader(headerName, null).build(
);
    }

}
```

① We've defined this bean elsewhere and injected it here

② in which header shall we place the authentication once we've validated the token?

③ extract the token from the header

④ use the `AuthenticationProvider` to talk to the Spring Authorization Server.

⑤ if the JWT is in fact valid then we'll create a new `Message`, cloning the headers and payload from the incoming message, but specufying one new header, whose value is an object of type `Authentication`.

⑥ if the JWT is not valid, then create a new `Message` with a null header value. :code: ..

Command Line Interfaces

In this section, we're going to look at how to secure command line interfaces (CLIs) with OAuth and Spring Security. We'll use Spring Shell to build up the CLI and Spring Security's OAuth support, as usual, to do the rest. :code: ..

Federated OAuth with the Spring Authorization Server

Multitenancy

Multitenancy is a hard topic to write about because it intersects with so many other important concepts. Where to begin? Broadly, multitenancy refers to a grouping, or a cohort, of users within a larger, shared system. Think of a workspace in Slack, or a server in Discord. You can use only one at a time, and each one is a place where data is shared with almost everyone inside the tenant, but with nobody beyond it. Logically, this is a perfectly reasonable way to think about a system's architecture, and it overlaps somewhat with the notions of identity and security. But multitenancy has repercussions in plenty of other areas, too. You can use multitenancy to describe deployment regions, availability zones, billing units, data isolation policies, backup policies, and so much more.

From One to Many Users

Spring Security makes it easy to authenticate and authorize a single user, but does that user act alone? Multitenancy gives us a framing with which to understand how that user relates to the other users in the system. If the user works for a company in Singapore, then perhaps it makes more sense to home that user's data in data centers in APAC. If the user works for a company in France, then it makes more sense to home that user's data in data centers in EMEA. You can use tenants to understand where the data for a given user lives and to handle the routing that follows from that.

And once you've established where the data lives, you'll need to sort out who has access to it. If you're building software as a service, then it makes a lot of sense - and it's quite useful - to provision resources and isolate data based on customers and the seats they've purchased as part of their license.

Earlier Is Easier

In order to integrate multitenancy into your system, you'll need to carve up the data in that system based on the tenant of a given user. The earlier in the request flow that you establish the tenant, the earlier you can route the request to potentially disparate systems and services, which act - from their perspective - as though there's only one tenant.

Imagine an HTTP request from a browser - or from an iPhone, desktop, or Android client - headed to your *service*. This service lives behind a gateway, which lives behind a load balancer. Access to that system requires authentication and authorization.

Where does a request flowing through the system go?

First stop: the gateway. The gateway is a natural place to route the request to the appropriate availability zone. Here, you can make decisions about which data center should be tapped to provide the response in the first place.

Once that's decided, you'll (probably) need to authenticate. The identity provider itself can be a seam along which to define multitenancy. Plenty of really "enterprise-y" SaaS products do exactly this to deepen their integration with their customers. Why add your company's users to the SaaS's user database - not to mention all their data - when you can just have the SaaS trust the users in your identity provider? Imagine: you go to `https://mytenant.thesaas.com`, and it redirects you to the identity provider used by *your* organization. From there, the request eventually makes its way to a

service, and it is this service that must ultimately hand you back the right data. But from where? Even here, the data might be doled out along multitenant lines.

Everything from the first request on down could be made multitenant-aware.

Arconia's Multitenancy Module

If we agree that designing for multitenancy would be a good thing, then how do we get started? I thought about this a fair bit. There's not a lot of explicit support in Spring itself, which is odd, because Spring provides exactly the level of control required to enable a good multitenancy story. It surfaces the moving parts we'd need to rework in order to support multitenancy. It wouldn't be possible to build multitenancy effectively without a flexible component model like Spring's.

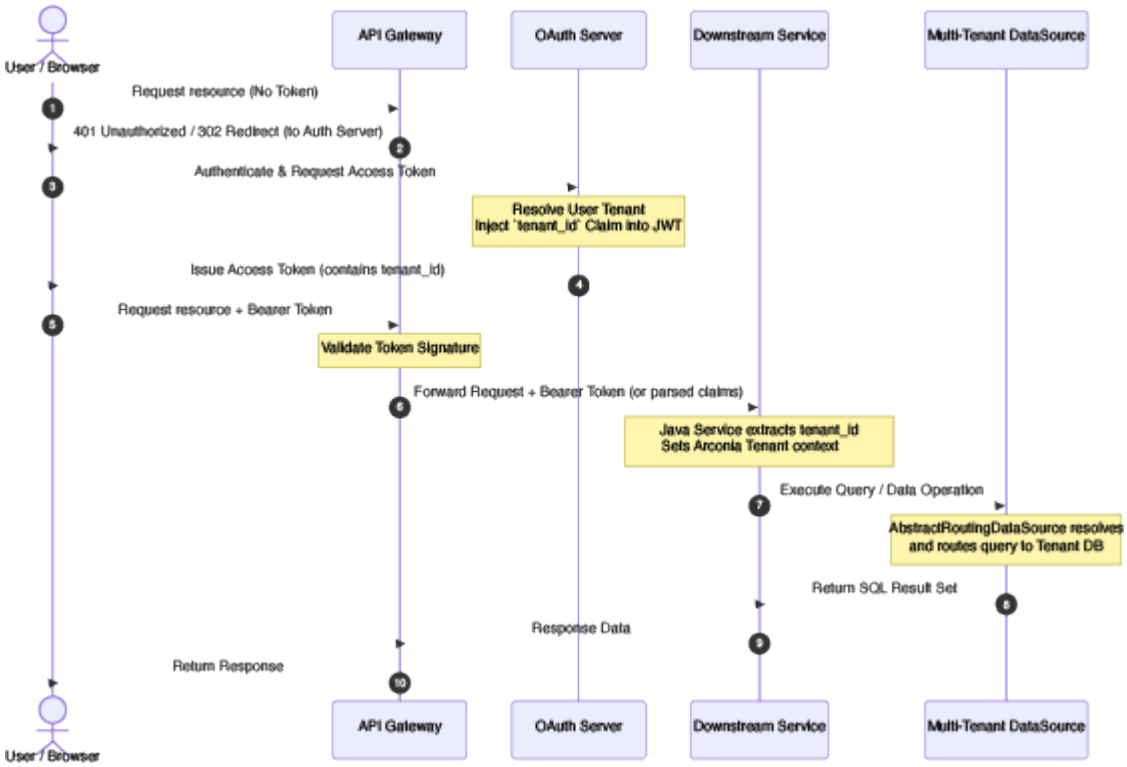
But there are always ways to improve, and Arconia - an open-source project created by my friend and fellow Java Champion Thomas Vitale - provides just such an improvement. Arconia [https://arconia.dev] is a suite of modules delivered in the typical Spring Boot style - as starters, autoconfiguration, properties, etc. It provides a lot of functionality beyond multitenancy, but that's the only part we'll focus on here.

The core abstraction in Arconia is the `TenantContext`, which contains a static reference to a Java 25 `ScopedValue<T>`. It acts like a super-powered thread local, containing the current resolved tenant. Arconia's web filter installs it when the request arrives. We'll use `TenantContext.getTenantIdentifier()` *everywhere*.

What We're Building

To understand everything, we're going to build the usual trinity:

- a Spring Authorization Server - standard stuff we've seen before, with one important proviso: we're going to customize the OAuth tokens that it issues to include information about tenants.
- a Spring Cloud Gateway instance that acts as an OAuth client, routing requests from the client to the downstream services and relaying the OAuth token as it does.
- a Spring MVC-powered HTTP API, acting as an OAuth resource server, that'll serve up information about the `customer` records in our `customer` table. The data will be sourced according to the tenant of the currently authenticated user.



Through it all, we'll keep track of the tenant to which a given request should be assigned.

We're going to need a PostgreSQL server. Actually, several of them. So I've defined a `compose.yml` that spins up four different PostgreSQL instances. We'll refer to those instances by their Docker Compose service names, which - and this is the important part - correspond to the names of the tenants in our system.

```
services:
①
  main:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'

②
  petclinic:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=petclinic'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '4002:5432'

  coffeeshop:
    image: 'postgres:latest'
```

```
    environment:
      - 'POSTGRES_DB=coffeeshop'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '4001:5432'

  acmewidgets:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=acmewidgets'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '4003:5432'
```

① the first and main database (on port `5432`), which serves requests that have no tenant assigned

② the other databases (on ports `4001` through `4003`), each specific to a particular tenant in the system

The Spring Authorization Server

As you've seen many times before, we'll need a centralized OAuth IDP to vend and validate tokens for us. We'll use the Spring Authorization Server, as usual.

The Setup

Hit the Spring Initializr and add `JDBC`, `Flyway`, `PostgreSQL`, `Web`, and `Authorization Server`. And `DevTools`, too.

Give the new project an artifact of `auth`.

Open the `pom.xml` and let's add the Arconia dependencies. First, the bill of materials (BOM):

- `io.arconia` : `arconia-bom` : `0.30.0`

Make sure to specify that the `type` is `pom` and the `scope` is `import` for the BOM dependency.

Then add the individual, version-managed dependencies:

- `io.arconia` : `arconia-multitenancy-web` - this brings in the filter that intercepts inbound requests and attempts to establish a tenant before requests are routed downstream.
- `io.arconia` : `arconia-multitenancy-details-jdbc` - this lets Arconia resolve the tenants in the system by reading from a SQL database. Otherwise, and by default, tenants are configured in memory or with properties.

The Database Schema

We're going to need to create some SQL tables. Here's the Flyway database migration to do that, in

`src/main/resources/db/migration/V0__user_tenants.sql`. This `.sql` file installs the standard tables we need for Spring Security's `JdbcUserDetailsManager`. It also creates the standard tables required for Arconia's multitenancy module. And, finally, it adds a custom table of our own, `users_tenant_details`, that maps users to tenants.

```
create table if not exists users
(
    username text    not null primary key,
    password text    not null,
    enabled  boolean not null
);

create table if not exists authorities
(
    username  text not null references users (username),
    authority text not null
    );

create unique index if not exists index_auth_username ON authorities USING btree
(username, authority);

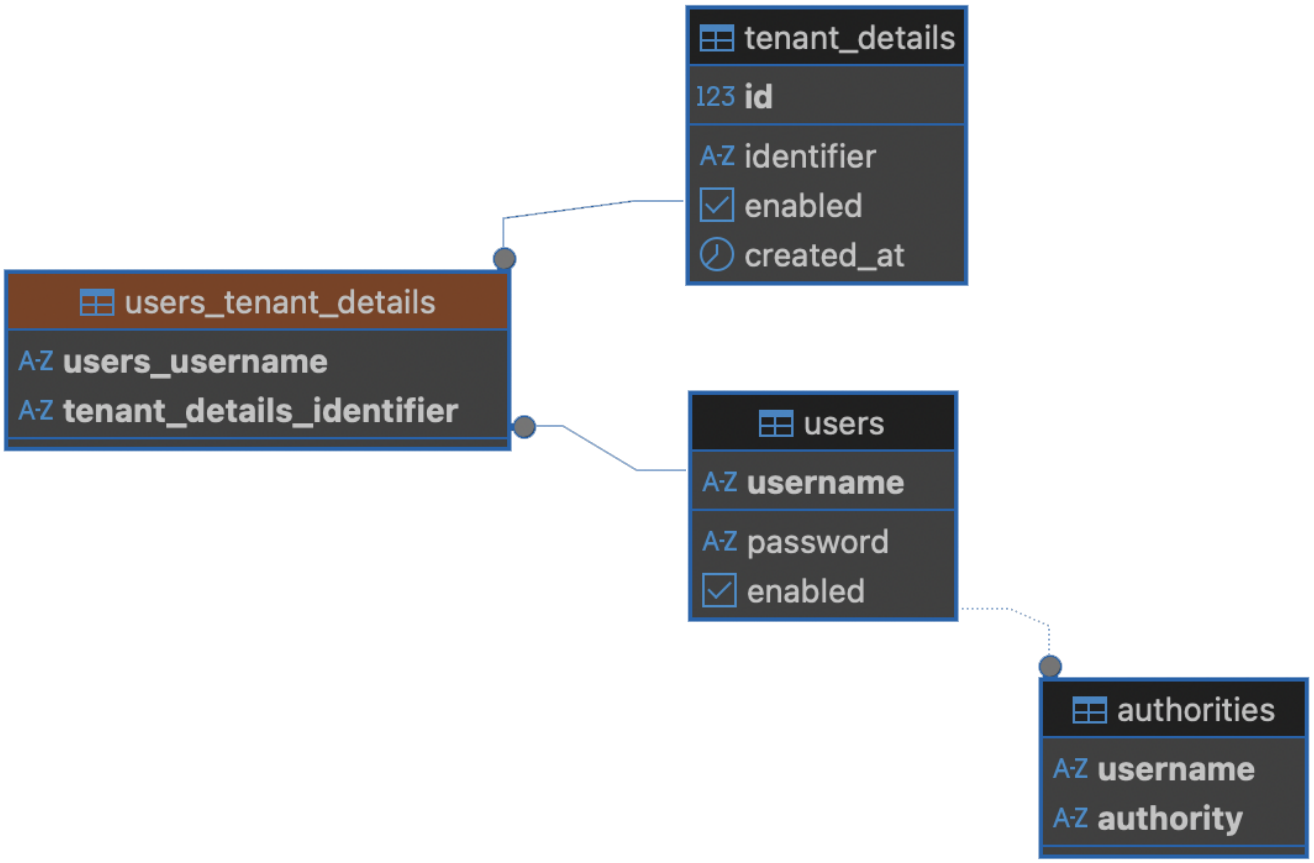
create table if not exists tenant_details
(
    id         serial primary key,
    identifier text not null unique default pg_catalog.gen_random_uuid(),
    enabled    boolean              default true,
    created_at timestamp            default now()
    );

create table if not exists tenant_details_attributes
(
    tenant_id       integer not null references tenant_details (id),
    attribute_name  text    not null,
    attribute_value text    not null,
    primary key (tenant_id, attribute_name)
    );

create table if not exists users_tenant_details
(
    users_username            text not null references users (username),
    tenant_details_identifier text not null references tenant_details (identifier),
    primary key (users_username, tenant_details_identifier)
);
```

In the source code for this book, I've also included `V1__user_tenants_data.sql`, which contains some rather lengthy DML to insert sample data. I won't reprint it here, but you should apply that as well by sourcing it from the GitHub repository for this book [https://github.com/secure-all-the-things-book] and for this chapter, in particular [https://github.com/secure-all-the-things-book/multitenancy].

Here's what the moving parts look like in the ERD diagram:



1. all of these tables *except* `users_tenant_details` come from either Spring Security or Arconia. I created `users_tenant_details` to link the two schemas.

The Implementation

Let's look at the Java code. I'm not going to reprint the `main` method. It's exactly as it was handed to me by the Spring Initializr.

We'll need to define an OAuth client. This time around, I've chosen to use the `.properties`-based mechanism for doing so.

```
spring.application.name=auth
#
spring.datasource.url=jdbc:postgresql://localhost:5432/mydatabase
spring.datasource.username=myuser
spring.datasource.password=secret
#
spring.threads.virtual.enabled=true
#
```

```
spring.security.oauth2.authorizationserver.client.spring.registration.client-id=spring
spring.security.oauth2.authorizationserver.client.spring.registration.client-
secret={noop}spring
spring.security.oauth2.authorizationserver.client.spring.registration.client-
authentication-methods[0]=client_secret_basic
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[0]=authorization_code
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[1]=client_credentials
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[2]=refresh_token
spring.security.oauth2.authorizationserver.client.spring.registration.authorization-
grant-types[3]=urn:ietf:params:oauth:grant-type:device_code
spring.security.oauth2.authorizationserver.client.spring.registration.redirect-
uris[0]=http://127.0.0.1:8080/login/oauth2/code/spring
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[0]=openid
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[1]=profil
e
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[2]=email
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[3]=user.w
rite
spring.security.oauth2.authorizationserver.client.spring.registration.scopes[4]=user.r
ead
#
server.port=9090
#
spring.flyway.table=auth_flyway_schema_history
```

While we're at it, I've:

- set the port to `9090`
- defined the credentials for connectivity to the database
- enabled virtual threads
- named the administrative table that Flyway uses to track which migrations have run

Here are the relevant beans.

```
package com.example.auth;

import io.arconia.multitenancy.details.jdbc.JdbcTenantDetailsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.provisioning.JdbcUserDetailsManager;

import javax.sql.DataSource;

@Configuration
```

```
class AuthServerConfiguration {

①
    @Bean
    Customizer<HttpSecurity> customizer() {
        return http -> http.oauth2AuthorizationServer(a -> a.oidc(Customizer
.withDefaults()));
    }

②
    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
        var u = new JdbcUserDetailsManager(dataSource);
        u.setEnableUpdatePassword(true);
        return u;
    }

③
    @Bean
    JdbcTenantDetailsService jdbcTenantDetails(DataSource dataSource) {
        return JdbcTenantDetailsService.builder().dataSource(dataSource).build();
    }

}
```

① you've seen this before (many times); it just establishes that we want this to be a Spring Authorization Server, a fully compliant OAuth authorization server

② and this too should be familiar: we're sourcing the users from a SQL database (in this case, PostgreSQL)

③ here's our first glimpse of Arconia: we're also sourcing the tenants themselves from PostgreSQL

All of this wires up Arconia, and it wires up the Spring Authorization Server. But where do the two meet? It's in the OAuth tokens that the Spring Authorization Server issues. Let's look at the oh-so-handy `OAuth2TokenCustomizer` implementation.

```
package com.example.auth;

import org.springframework.jdbc.core.simple.JdbcClient;
import
org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
import
org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
import org.springframework.stereotype.Component;

@Component
class TenantOAuth2TokenCustomizer implements OAuth2TokenCustomizer<JwtEncodingContext>
{

    private final JdbcClient db;
```

```
    TenantOAuth2TokenCustomizer(JdbcClient db) {
        this.db = db;
    }

    @Override
    public void customize(JwtEncodingContext context) {
①
        var tenant = this.db //
            .sql(//
                    " select tenant_details_identifier from users_tenant_details utd
" + //
                            " where users_username = ? "//
            ) //
            .params(context.getPrincipal().getName())
            .query((rs, _) -> rs.getString("tenant_details_identifier"))
            .single();
②
        context.getClaims().claim("tenant", tenant);
    }

}
```

① given the current username, figure out to which tenant that user belongs

② and then set that tenant as a *claim*, an attribute, on the OAuth token

This claim is what we'll use for all subsequent downstream requests to ascertain the current tenant. No need for another trip to the database!

The OAuth Client

For our OAuth client, we'll use two Spring Cloud Gateway routes, which will proxy requests from the client to two different downstream endpoints:

- the UI (imagine a CDN, or some Nginx service, serving static assets like those from a Vue or React application), running on port `8020`. This UI won't need an OAuth token to do its work, though of course adding it doesn't really hurt anything.
- the OAuth resource server, running on port `8081`. It serves information about all the `customer` records assigned to a particular authenticated user. It does so by routing the request to the right tenant-specific database and schema.

Source URL	Target URL
`:8080/index.html`	`:8020/index.html`
`:8080/api/customers`	`:8081/customers`

The Setup

Visit the Spring Initializr [https://start.spring.io] and add `OAuth2 Client`, `Gateway`, `Spring Web`, `HTTP`

`Client`, and `Spring Boot DevTools`. I named this module `client`. Click `Generate` and open it in your favorite IDE.

You could add Arconia here if you wanted to use its integration with the OAuth token to establish tenancy, as we'll see when we discuss the resource server. We don't need Arconia in this module, though, since we're proxying requests unchanged from the browser client to the downstream service and sending the OAuth token along for the ride.

The Implementation

This Spring Cloud Gateway module acts as a proxy between client requests and the downstream endpoints, intercepting, enhancing, and redirecting requests as necessary. It'll require some configuration, too.

```
spring.application.name=client
①
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=spring
spring.security.oauth2.client.registration.spring.client-secret=spring
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=openid,profile
②
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:9090
#
spring.threads.virtual.enabled=true
```

① we'll need to stipulate which OAuth client we intend to run as, so we register a client - named `spring` - and, since we're using a custom OAuth provider, we point that registration at a provider of the same name

② we'll also need to point the Spring Security OAuth client support to the Spring Authorization Server running on port `9090`

And, finally, we enable virtual threads. Always.

Here's the configuration for the two Spring Cloud Gateway routes.

```
package com.example.client;

import org.springframework.cloud.gateway.server.mvc.filter.BeforeFilterFunctions;
import org.springframework.cloud.gateway.server.mvc.filter.TokenRelayFilterFunctions;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
```

```
import org.springframework.web.servlet.function.RouterFunction;
import org.springframework.web.servlet.function.ServerResponse;

import static
org.springframework.cloud.gateway.server.mvc.handler.HandlerFunctions.http;
import static org.springframework.web.servlet.function.RouterFunctions.route;

@Configuration
class GatewayConfiguration {

①
    @Bean
    @Order(Ordered.LOWEST_PRECEDENCE)
    RouterFunction<ServerResponse> uiRoute() {
        return route()//
            .GET("/**", http())
            .before(BeforeFilterFunctions.uri("http://localhost:8020"))//
            .build();
    }

②
    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    RouterFunction<ServerResponse> apiRoute() {
        return route().GET("/api/**", http())//
            .before(BeforeFilterFunctions.uri("http://localhost:8081"))//
            .before(BeforeFilterFunctions.rewritePath("/api", "/"))//
            .filter(TokenRelayFilterFunctions.tokenRelay())//
            .build();
    }

}
```

① this route forwards *all* HTTP `GET` traffic from `/**` to `http://localhost:8020`. Because it matches *everything* going to the gateway, it must be evaluated *after* all the more specific routes; otherwise, they won't ever be evaluated or match.

② this route forwards all HTTP `GET` requests arriving at the gateway under the path `/api/**` to our resource server on port `8081`. Before requests can be forwarded, we must strip the `/api` prefix and be sure to include the OAuth token for the currently authenticated user as a header on the outgoing request - hence the token relay filter. This route is more specific, so we've made sure that it gets evaluated *before* the catch-all route for `/**`.

The result of these two routes is that both the backend API and the JavaScript that interacts with it are served from the same host and port. There's no need for CORS headers. And if you hit the `/api` endpoint, the OAuth client machinery will kick in, and you'll be redirected to the Spring Authorization Server, if you haven't already authenticated there.

The UI

I hate JavaScript. But I wrote some for you anyway. Create a new folder adjacent to the other Spring Boot projects, called `ui`.

Create the index page, `ui/index.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Customers</title>
    <style>
        #m {
            display: grid;
            grid-template-columns: auto 2fr ;
            grid-column-gap: 1em;
        }

        .customer {
            display: grid;
            grid-column: 1 / -1;
            grid-template-columns: subgrid;
            border-top: 1px solid;
        }
    </style>
</head>
<body>
<template id="row">
    <div class="customer">
        <div class="id"></div>
        <div class="name"></div>
    </div>
</template>

<div id="m"></div>

<script type="module">
①
    const response = await fetch('/api/customers')
    const customers = await response.json()
    m.append(...customers.map(({id, name}) => {
        const customer = row.content.cloneNode(true)
        customer.querySelector('.id').textContent = id
        customer.querySelector('.name').textContent = name
        return customer
    }))
</script>
</body>
```

```
</html>
```

① the only thing of note is that this client makes requests to the gateway endpoint called `/api/customers` and then modifies the local DOM to show the records it gets back

You could host this particular single-page asset nearly anywhere. I'm using it as a stand-in for a CDN or an Nginx server. During development, I like to use the handy `jwebserver` that ships with Java.

Here's my script to do so, `ui/serve.sh`.

```
#!/bin/bash
p=${1:-8020}
d=${2:-$PWD}
cd $d
jwebserver -d $d -p $p
```

Run it from the `ui` directory, and it'll serve `index.html` on port `8020`, which is right where the gateway expects to find it. Pass a different port as the first argument, or a different directory as the second, if you'd like.

The OAuth Resource Server

Here's where things get interesting. The request has come in to the backend API. The user's expecting a response, but whence should the data come? It's specific to the tenant, so we'll need to use Arconia to resolve the current tenant and then look at ways to work that knowledge into the data layer.

We can access the tennant from the OAuth token. The claim rides the token through the gateway; Arconia's filter pulls it off and stashes it in `TenantContext` here in the service.

The Setup

Go to the Spring Initializr and build a new project. Specify the following dependencies: `Spring Boot DevTools`, `OAuth2 Resource Server`, `Spring Web`, `JDBC API`, `PostgreSQL Driver`, and `Flyway Migration`. Name the project `service`.

Click `Generate` and open it in your favorite IDE.

Next, we'll need to add some dependencies the old-fashioned way: by hand.

First off, Arconia, which - once again - we'll need to wire up manually. Start with the bill of materials:

- `io.arconia` : `arconia-bom` : `0.30.0`

Make sure to specify that the `type` is `pom` and the `scope` is `import` for the BOM dependency.

Then add the individual, version-managed dependencies:

- `io.arconia : arconia-multitenancy-web` - the same module as before, bringing in the filter that intercepts inbound requests and establishes the tenant before those requests reach our controllers.

Now, we're going to add the Docker Java client libraries. Add a property to the `properties` element called `docker-java.version`, and set it to `3.3.6`. Then add the following dependencies:

- `com.github.docker-java : docker-java-core : ${docker-java.version}`
- `com.github.docker-java : docker-java-transport-zerodep : ${docker-java.version}`
- `com.github.docker-java : docker-java-transport-httpclient5 : ${docker-java.version}`

We'll look into why we need those in a bit.

First Flight

Let's see an end-to-end Arconia-powered request to prove everything works. We've got some configuration, of course.

```
spring.application.name=service
①
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:9090
②
server.port=8081
③
spring.threads.virtual.enabled=true
④
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.username=myuser
spring.datasource.password=secret
#
⑤
spring.flyway.enabled=false
```

① this is a standard Spring Security OAuth resource server, so we'll point it to the Spring Authorization Server

② this service will run on port `8081`

③ we're going to enable virtual threads, because of course we will

④ we're going to connect to the main database for *some* things, but not all

⑤ we'll keep Flyway on the classpath, but we *do not* want it running on startup. We'll handle initializing our many `DataSource` instances ourselves.

First, we'll need to enable Arconia's mapping of an OAuth token claim to a tenant.

```
package com.example.service;

import io.arconia.multitenancy.web.context.resolvers.OAuth2TenantResolver;
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;

@Configuration
class ArconiaConfiguration {

①
    @Bean
    OAuth2TenantResolver oauth2TenantResolver() {
        return OAuth2TenantResolver.builder().tenantClaimName("tenant").build();
    }

}
```

① this comes from the Arconia module. The default claim name is `tenant_id`, but we've customized it here to show that you can

Here's a simple controller that returns the currently authenticated user *and* their tenant.

```
package com.example.service;

import io.arconia.multitenancy.core.context.TenantContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.context.SecurityContextHolderStrategy;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Map;

@Controller
@ResponseBody
class TenantController {

    @GetMapping("/")
    Map<String, String> me() {
        var contextHolderStrategy = SecurityContextHolder.getContextHolderStrategy();
        return Map.of(//
                "tenant", TenantContext.getTenantIdentifier(), //
                "user", contextHolderStrategy.getContext().getAuthentication().
getName() //
        );
    }

}
```

If everything's been done correctly, it will show you the user as whom you authenticated and the tenant to which that user belongs.

Let's try it out. Start everything up: first the Spring Authorization Server, then everything else. Visit `http://127.0.0.1:8080/api/`, and you should be redirected to the Spring Authorization Server. If

you've used the data I supplied in the GitHub repository for this book [https://github.com/secure-all-the-things-book/multitenancy], then you can log in as `josh` / `pw`.

Do you see your username, and the tenant to which you belong, in the JSON that comes back? Good. You're on your way!

Now, let's get to the meat of the matter: data.

The Customer's Always Right - If You've Picked the Right Tenant

Here's a simple record, `Customer`.

```
package com.example.service;

record Customer(int id, String name) {
}
```

And here's a hand-rolled repository to read those records.

```
package com.example.service;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Repository;

import java.util.Collection;

@Repository
class CustomerRepository {

    private final JdbcClient db;

    CustomerRepository(JdbcClient db) {
        this.db = db;
    }

    Collection<Customer> findAll() {
        return this.db//
            .sql("select * from customer") //
            .query((rs, _) -> new Customer(rs.getInt("id"), //
                    rs.getString("name")) //
            )//
            .list();
    }

}
```

Finally, here's a controller that'll return all the requisite `customer` data to the aforementioned `index.html` client.

```
package com.example.service;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Collection;

@Controller
@ResponseBody
class CustomerController {

    private final CustomerRepository repository;

    CustomerController(CustomerRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/customers")
    Collection<Customer> customers() {
        return repository.findAll();
    }

}
```

Believe it or not, this stuff won't change from here on down. You'll be able to read data from the same `/customers` endpoint and get different data tied entirely to the current tenant.

Multitenancy in the Data Layer

First, let me admit something: this chapter can't possibly hope to teach you everything about integrating multitenancy at the data layer. It happens at so many layers, and in so many functions of a system, that no single chapter could enumerate them all, let alone demonstrate them. Anything your system depends on that produces data specific to a cohort of your users is a good candidate for multitenancy.

In this chapter, we'll focus on just one of those layers: JDBC's `DataSource` type. A `DataSource` represents a source of connections to your SQL database. You know this. If you've been in the Java space long enough, you've probably used dozens, or maybe even hundreds, of different `DataSource` implementations (I'm not exaggerating). A `DataSource` is usually the second thing you create when working with SQL, after the `java.sql.Driver` itself. It's the thing that creates the `java.sql.Connection` instances that point to a SQL database. Sometimes they're authenticated, sometimes not. Sometimes they're over a network socket, sometimes over a UNIX socket. Sometimes they're connection pools, wrapping and adapting an underlying `DataSource`, sometimes not. In our example, we will provide wrapper `DataSource` implementations that give our clients the illusion that they're working with a

single `DataSource`, and with data that belongs to them alone.

Tenant-specific DataSource Initialization

While we wouldn't normally handle the provisioning of databases at this layer of our system, we *can* handle the setup of the database once it's ready. To that end, I've extracted a `DataSourceInitializer` interface. We'll use it in both of the strategies we look at in this chapter.

```
package com.example.service;

import javax.sql.DataSource;

public interface DataSourceInitializer {

①
    DataSource initialize(String tenantId, DataSource dataSource);

}
```

① given a tenant ID and a `DataSource`, initialize it

We'll use Flyway to provision the tables for each tenant-specific `DataSource`.

```
package com.example.service;

import org.flywaydb.core.Flyway;
import org.springframework.stereotype.Component;

import javax.sql.DataSource;

@Component
class FlywayDataSourceInitializer implements DataSourceInitializer {

    @Override
    public DataSource initialize(String tenantId, DataSource dataSource) {

①
        Flyway.configure(getClass().getClassLoader())
            .dataSource(dataSource)
            .locations("classpath:db/tenants/common", "classpath:db/tenants/" +
tenantId + "/")
            .load()
            .migrate();
        return dataSource;
    }

}
```

1. use Flyway to initialize the SQL tables for a particular tenant

This will be our default `DataSourceInitializer`. It takes a tenant ID and a `DataSource` and uses them to drive Flyway, which in turn ensures that all the database machinery has been established. The `FlywayDataSourceInitializer` class uses the tenant to load both a common directory of SQL database migrations that should apply to *all* tenants *and* any tenant-specific SQL that should be applied to only one given tenant. This way, we can insert tenant-specific customer data.

Here's the one and only common migration, which lives - naturally - in the `common` folder.

```
create table customer
(
    id                serial primary key,
    name              text not null,
    tenant_identifier text not null
);
```

Then, under folders named for the tenants, we see migrations like the following. All this one does is insert some tenant-specific data for our demo. (You probably wouldn't need this capability in a production system...)

```
insert into customer (name,tenant_identifier) values ('bob' ,'acmewidgets');
insert into customer (name,tenant_identifier) values ('zhen' , 'acmewidgets');
```

Flyway keeps track of everything and won't run the same migration against the same `DataSource` more than once, but in order to ascertain that, it has to talk to our SQL database and read the administration table it maintains. That lookup alone can be costly in terms of CPU time, bandwidth, and so on - especially if it happens on every single request. So we'll add one more layer of caching around our `DataSourceInitializer`, one that keeps track, in memory, of which `DataSource` instances have already been initialized. Here is `DataSourceInitializers`, a class with a convenient factory method.

```
package com.example.service;

public class DataSourceInitializers {

①
    public static DataSourceInitializer caching(DataSourceInitializer initializer) {
        return new CachingDataSourceInitializer(initializer);
    }

}
```

This will eliminate the vast majority of Flyway interactions, though not all of them. If you deploy this service across multiple JVMs, the Flyway initialization logic will run once per JVM. Even then, Flyway itself will stop it from doing anything if that tenant's `DataSource` is already up to date.

This `CachingDataSourceInitializer` is yet another composite, delegating to the underlying `DataSourceInitializer` only when it determines that the call hasn't already happened since this JVM started.

```
package com.example.service;

import javax.sql.DataSource;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

class CachingDataSourceInitializer implements DataSourceInitializer {

    private final Map<String, Boolean> initialized = //
            new ConcurrentHashMap<>();

    private final DataSourceInitializer dataSourceInitializer;

    CachingDataSourceInitializer(//
            DataSourceInitializer dataSourceInitializer) {
        this.dataSourceInitializer = dataSourceInitializer;
    }

    @Override
    public DataSource initialize(String tenantId, //
            DataSource dataSource //
    ) {
        var initialized = this.initialized //
            .computeIfAbsent(tenantId, _ -> false);
        if (!initialized) {//
            this.dataSourceInitializer //
                .initialize(tenantId, dataSource);
            this.initialized.put(tenantId, true);
        }
        return dataSource;
    }

}
```

Tenant-aware DataSources

A `DataSource` is a very natural place in which to make decisions about multitenancy. You can route to and set up individual `DataSource` instances as they come into use.

Let's look at a few strategies.

Database per Tenant

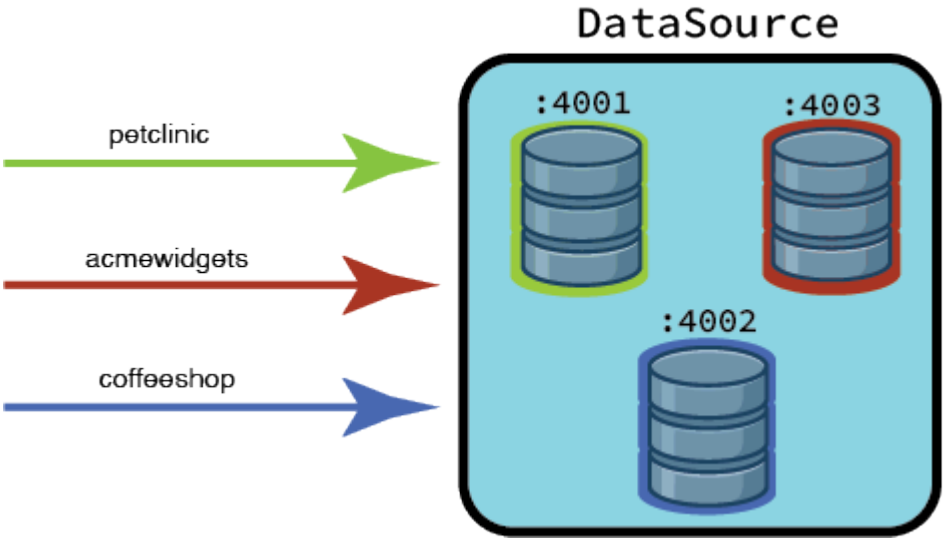
The simplest form is also the most isolated - and the most expensive: give each tenant its own database. Is that a logical database sharing a single, dense node with its neighbors? An isolated

virtual machine on a dense node? Or a standalone server with its own operating system and system resources? It's entirely up to you. There is, of course, operational complexity in provisioning databases dynamically, and that complexity is well beyond the scope of this book. Let's just imagine you had such a capability.



if you are using Cloud Foundry and its marketplace of service brokers, provisioning new databases dynamically is a trivial affair. You can achieve similar results, with far more moving parts and complexity, using Kubernetes, too.

`DataSource` references typically connect to a single host and port. From the perspective of the code, it's a completely different `DataSource` for each tenant. In a database-per-tenant scenario, then, we would have a variable and ever-growing number of hosts and ports. We don't want to expose the rest of our code to that complexity, so we'll encapsulate the routing and maintenance of the individual `DataSource` instances inside a wrapper `DataSource` implementation, as shown in the diagram below.



1. the requests all proceed to the same outer `DataSource`, but they are forwarded to individual `DataSource` instances based on which tenant they're assigned to

```
package com.example.service.db;

import io.arconia.multitenancy.core.context.TenantContext;
import org.aopalliance.intercept.MethodInterceptor;
import org.springframework.aop.framework.ProxyFactoryBean;
import org.springframework.jdbc.datasource.DelegatingDataSource;

import javax.sql.DataSource;
import javax.sql.XADataSource;
import java.lang.reflect.Method;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Function;

①
class DatabasePerTenantDataSource extends DelegatingDataSource {
```

```
②
    DatabasePerTenantDataSource(Function<String, DataSource> supplier) {
③
        var map = new ConcurrentHashMap<String, DataSource>();
        var pfb = new ProxyFactoryBean();
        pfb.addInterface(XADataSource.class);
        pfb.addInterface(DataSource.class);
        pfb.addAdvice((MethodInterceptor) methodInvocation -> {
④
            var tenantIdentifier = TenantContext.getTenantIdentifier();
            var targetDatabase = map.computeIfAbsent( //
                    tenantIdentifier, supplier);
            var method = methodInvocation.getMethod();
            return method.invoke(targetDatabase, //
                    methodInvocation.getArguments());
        });
        var db = (DataSource) pfb.getObject();
        setTargetDataSource(db);
    }

}
```

① the `DelegatingDataSource` is a convenient wrapper type included in Spring Framework that simply delegates all invocations to a contained `DataSource`. This way you can enjoy the benefits of composition without having to implement *every* method in the `DataSource` hierarchy.

② the sole constructor argument is a `java.util.function.Function<String,DataSource>` that takes a tenant ID and returns a `DataSource`. It is with this callback that you'll determine to which `DataSource` a request should be routed.

③ our `DataSource` will simply be a JDK proxy that forwards invocations to the `DataSource` returned from the provided function

④ we'll use Arconia's `TenantContext` to source the current tenant

That's the generic class, and you could use it in any scenario. Register a bean of this type *instead* of the `DataSource` that Spring Boot autoconfigures for you, and every consumer downstream gets a `DataSource` that lazily resolves the right tenant's connections on demand.

But where does the rubber meet the road? How do we arrive at a `DataSource` in the first place? As I mentioned before, there is some implied operational complexity here. As new tenants are added to the system, something needs to provision the PostgreSQL server to which we'll connect when defining our `DataSource` instances. You'd do well to keep track of those instances, perhaps storing the host, port, username, and password in an encrypted form somewhere so that you can hydrate new `DataSource` instances on demand.

In this example, I've extracted an interface called `DatabaseRegistry`, which returns a Spring Boot `JdbcConnectionDetails` given a `String` that we assume will be a tenant ID.

To keep things within the realm of an easy-enough explanation for a printed book, I've chosen to use Docker to back the one and only implementation of this `DatabaseRegistry` type. Most people have access to something that can run Docker containers and Docker Compose. In the `compose.yml`

file for this chapter, we've defined unique PostgreSQL instances and given them `service` names that line up with the tenants in our `tenant_details` table: `coffeeshop`, `acmewidgets`, and `petclinic`. We'll implement the `DatabaseRegistry` interface in such a way that it resolves the credentials for a `DataSource` by talking to the Docker process running on your local machine.



a single, giant Docker Compose file is *not* a viable solution for production. You'll need something that can provision PostgreSQL instances and scale up and out, like Cloud Foundry, Kubernetes, or even an infrastructure-as-a-service cloud provider like Amazon Web Services (AWS).

Here's the implementation.

```
package com.example.service.db;

import com.github.dockerjava.core.DockerClientConfig;
import com.github.dockerjava.core.DockerClientImpl;
import com.github.dockerjava.transport.DockerHttpClient;
import org.jspecify.annotations.Nullable;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.boot.jdbc.autoconfigure.JdbcConnectionDetails;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

class DockerDatabaseRegistry implements DatabaseRegistry, InitializingBean {

    private final DockerClientConfig dockerClientConfig;

    private final Map<String, String> mapping = new ConcurrentHashMap<>();

    private final DockerHttpClient dockerHttpClient;

    DockerDatabaseRegistry(DockerHttpClient dockerHttpClient, DockerClientConfig
dockerClientConfig) {
        this.dockerClientConfig = dockerClientConfig;
        this.dockerHttpClient = dockerHttpClient;
    }

    @Nullable
    @Override
    public JdbcConnectionDetails getConnectionDetails(String tenantId) {
        if (this.mapping.containsKey(tenantId)) {
            return new JdbcConnectionDetails() {
                @Override
                public String getUsername() {
                    return "myuser";
                }

                @Override
```

```
                public String getPassword() {
                    return "secret";
                }

                @Override
                public String getJdbcUrl() {
                    return mapping.get(tenantId);
                }
            };
        }
        return null;
    }

    private void refresh() throws Exception {

①
        try (var dockerClient = DockerClientImpl.getInstance(this.dockerClientConfig,
this.dockerHttpClient)) {
            var containers = dockerClient.listContainersCmd()
                .withShowAll(false)
                .withLabelFilter(List.of("com.docker.compose.service"))
                .exec();
            for (var container : containers) {
                var image = container.getImage();
                if (image == null || !image.contains("postgres")) {
                    continue;
                }
                var labels = container.getLabels();
                var serviceName = labels.get("com.docker.compose.service");
                var ports = container.getPorts();
                for (var port : ports) {
                    this.mapping.put(serviceName,
                            "jdbc:postgresql://localhost:" + port.getPublicPort() + "
/" + serviceName);
                }
            }
        }
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        this.refresh();
    }

}
```

① on startup, this implementation lists all the running services and stores the port on which each one is listening in a map, keyed by the service name. That service name should later line up with the tenant ID.

Given a tenant ID, `getConnectionDetails` looks up the JDBC URL in that internal map and wraps it up

in a `JdbcConnectionDetails`. Note that it hardcodes the same username and password for every tenant. This is *not* a production-worthy solution.

Now that we have the Docker-backed `DatabaseRegistry`, and a `DataSource` that can route requests to a tenant-specific `DataSource`, we just need to tie it all together.

```
package com.example.service.db;

import com.example.service.DataSourceInitializer;
import com.example.service.DataSourceInitializers;
import com.github.dockerjava.core.DefaultDockerClientConfig;
import com.github.dockerjava.core.DockerClientConfig;
import com.github.dockerjava.transport.DockerHttpClient;
import com.github.dockerjava.zerodep.ZerodepDockerHttpClient;
import com.zaxxer.hikari.HikariDataSource;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

import javax.sql.DataSource;
import java.util.function.Function;

①
@Profile("db")
@Configuration
class DatabasePerTenantConfiguration {

    @Bean
    DatabasePerTenantDataSource dataSource(//
            DataSourceInitializer dataSourceInitializer, //
            DatabaseRegistry registry) {//

②
        var initializer = DataSourceInitializers.caching(dataSourceInitializer);

③
        var dataSourceForTenantFunction = (Function<String, DataSource>) tenantId -> {

④
            var connectionDetails = registry //
                .getConnectionDetails(tenantId);
            var db = DataSourceBuilder.create()//
                .url(connectionDetails.getJdbcUrl())//
                .username(connectionDetails.getUsername())//
                .password(connectionDetails.getPassword())//
                .type(HikariDataSource.class)//
                .build();
⑤
            return initializer.initialize(tenantId, db);
```

```
        };
        return new DatabasePerTenantDataSource(dataSourceForTenantFunction);
    }

⑥
    @Bean
    DefaultDockerClientConfig dockerClientConfig() {
        return DefaultDockerClientConfig//
            .createDefaultConfigBuilder()
            .build();
    }

    @Bean
    ZerodepDockerHttpClient zerodepDockerHttpClient(DockerClientConfig config) {
        return new ZerodepDockerHttpClient.Builder() //
            .dockerHost(config.getDockerHost())//
            .sslConfig(config.getSSLConfig())//
            .build();
    }

    @Bean
    DockerDatabaseRegistry dockerDatabaseRegistry(DockerHttpClient httpClient,
DockerClientConfig config) {
        return new DockerDatabaseRegistry(httpClient, config);
    }

}
```

① I've guarded this behind a Spring profile, so remember to activate the `db` profile when you run the code

② we take the default Flyway-powered `DataSourceInitializer` and wrap it in a `CachingDataSourceInitializer` so that it never runs more than once per tenant per JVM

③ N.B.: the `Function<String,DataSource>` is itself called only once per JVM for a given tenant. So we have multiple layers of - admittedly redundant - caching keeping us from creating and initializing a given tenant's `DataSource` more than once in a given JVM.

④ look up the Docker-bound database using the Docker-backed `DatabaseRegistry` implementation, then use the resulting credentials and connection details to build a `DataSource` with this convenient Spring Boot helper class

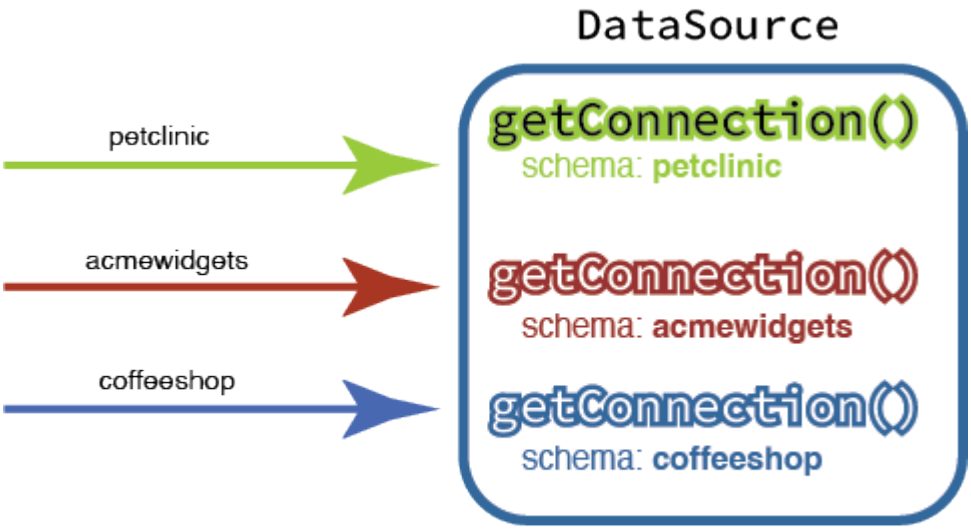
⑤ then, finally, we call the `DataSourceInitializer` on the resulting `DataSource`

⑥ the rest of these beans configure the Docker Java library that we use in the `DockerDatabaseRegistry` implementation we saw earlier

If you restart the service and visit `127.0.0.1:8080`, you'll be prompted to authenticate on the Spring Authorization Server. Enter `josh` / `pw` and you'll see one set of `customer` records. Enter `trisha` / `pw`, and you'll get another. Nothing changed in the SQL query, nor in the `JdbcClient`. It's all hidden behind the `DataSource`, which works in tandem with the Arconia tenant module to route you to the right `DataSource`.

Schema per Tenant

The database-per-tenant strategy is the *purest*. If you truly wanted to guarantee that there would be no noisy-neighbor issues, you could do so by putting each tenant's data in a completely separate, tenant-specific database. But it is also the costliest, in terms of both computation and cloud infrastructure spend. And it isn't always required. Often, partitioning tenants by database schema is *good enough*. A schema is a logical subdivision of a database server process. We're using PostgreSQL, for the most part, in this book, and it just so happens that PostgreSQL has a literal concept called a schema that acts as a sort of namespace. Database objects like views, tables, and functions can all live in individual schemas. It *is* possible for one schema to reach into another, but you can lock that down with the right privileges. In such an arrangement, each tenant gets its own schema, and each tenant controls its own data. A tenant that wanted a backup could ask for just its own records to be dumped, exactly as with the database-per-tenant strategy. Schema per tenant is also easier to pull off from a Spring Boot perspective, because we no longer need to spin up countless `DataSource` instances. Instead, we need only change the schema on the JDBC `Connection` object before handing it back to the client.



We can switch schemas in a `DataSource` wrapper, before ever handing the `Connection` back to the user. We could even go a step further, handing them back a connection whose schema can't subsequently be changed.



astute readers might already have spotted the issue with this arrangement: databases have limits on how many schemas you can have! Sure, some don't specify a particular upper bound, but eventually you're constrained by RAM, disk, and CPU capacity. So sooner or later you'll have to combine schema-per-tenant and database-per-tenant. That's a bin-packing problem for another day. And another book! Moving on…

Let's see what a trivial implementation of this pattern looks like as a `DataSource` wrapper.

```
package com.example.service.schema;
```

```
import com.example.service.DataSourceInitializer;
import com.example.service.DataSourceInitializers;
import io.arconia.multitenancy.core.context.TenantContext;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DelegatingDataSource;
import org.springframework.jdbc.datasource.SingleConnectionDataSource;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.SQLException;

class SchemaPerTenantDataSource extends DelegatingDataSource {

    private final DataSourceInitializer dataSourceInitializer;

①
    SchemaPerTenantDataSource(DataSource dataSource, DataSourceInitializer dbi) {
        this.dataSourceInitializer = DataSourceInitializers.caching((tenantId,
dataSource1) -> {
②
            var jdbc = JdbcClient.create(dataSource1);
            jdbc.sql("create schema if not exists " + schema()).update();
            return dbi.initialize(tenantId, dataSource1);
        });
        this.setTargetDataSource(dataSource);
    }

③
    private String schema() {
        return "schema_" + TenantContext.getTenantIdentifier();
    }

④
    protected Connection initializeUnderlyingConnection(Connection connection) throws
SQLException {
        var tenantIdentifier = TenantContext.getTenantIdentifier();
        connection.setSchema(schema());
        var singleConnectionDataSource = new SingleConnectionDataSource(connection,
true);
        this.dataSourceInitializer.initialize(tenantIdentifier,
singleConnectionDataSource);
        return connection;
    }

    @Override
    public Connection getConnection(String username, String password) throws
SQLException {
        return this.initializeUnderlyingConnection(super.getConnection(username,
password));
    }
```

```
    @Override
    public Connection getConnection() throws SQLException {
        return this.initializeUnderlyingConnection(super.getConnection());
    }

}
```

① this implementation takes a pointer to the main `DataSource` - typically the one autoconfigured by Spring Boot - and the `DataSourceInitializer` we used in the last example

② we need to ensure that there's a schema for this tenant, so we issue a `create schema` DDL statement and *then* call the Flyway-backed `DataSourceInitializer`

③ the `schema()` method gives us a central place to derive the schema name from the tenant

④ both `getConnection` variants eventually call this method. In it, we invoke the `DataSourceInitializer` and change the `schema` on the connection.

That last method - `initializeUnderlyingConnection` - is `protected` for a reason: we *could* further refine the `Connection` that's returned. One such use case might be to adapt that `Connection` so that it does nothing if somebody besides our wrapper tries to call `setSchema`. You could even have it throw an exception. Here are the humble beginnings of something that implements this basic idea; it extends `SchemaPerTenantDataSource`.

```
package com.example.service.schema;

import com.example.service.DataSourceInitializer;
import org.aopalliance.intercept.MethodInterceptor;
import org.springframework.aop.framework.ProxyFactoryBean;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.SQLException;

class ImmutableSchemaPerTenantDataSource extends SchemaPerTenantDataSource {

    ImmutableSchemaPerTenantDataSource(DataSource dataSource, DataSourceInitializer
dbi) {
        super(dataSource, dbi);
    }

    @Override
    protected Connection initializeUnderlyingConnection(Connection connection) throws
SQLException {
        var initialized = super.initializeUnderlyingConnection(connection);
        return this.buildImmutableSchemaConnection(initialized);
    }

①
    private Connection buildImmutableSchemaConnection(Connection connection) {
        var pfb = new ProxyFactoryBean();
```

```
        pfb.addInterface(Connection.class);
        for (var i : connection.getClass().getInterfaces())
            pfb.addInterface(i);
        pfb.addAdvice((MethodInterceptor) invocation -> {
            var method = invocation.getMethod();
            if (method.getName().equals("setSchema"))
                return null;
            return invocation.getMethod().invoke(connection, invocation.getArguments(
));
        });
        pfb.setTarget(connection);
        return (Connection) pfb.getObject();
    }

}
```

① this does everything as before, but before handing the `Connection` back to the client, it wraps it in a proxy that quietly ignores calls to `setSchema`

For our purposes, the first implementation - `SchemaPerTenantDataSource` - is suitable enough, so we'll stick with that.

Remember, in this case the goal is *not* to replace the default `DataSource` that Spring Boot provides, since there's really only one database we need to talk to. Instead, it's to *wrap* that `DataSource` and change its behavior when it returns `Connection` objects. A `BeanPostProcessor` is the ideal tool for the job: it can spot Spring's autoconfigured `DataSource` on its way into the application context and swap in our wrapper.

```
package com.example.service.schema;

import com.example.service.DataSourceInitializer;
import org.jspecify.annotations.Nullable;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.BeanFactoryAware;
import org.springframework.beans.factory.ObjectProvider;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.sql.DataSource;
import java.util.concurrent.atomic.AtomicReference;

@Configuration
class SchemaPerTenantDataSourceConfiguration {

①
    @Bean
    static SchemaPerTenantDataSourceBeanPostProcessor sptBPP() {
        return new SchemaPerTenantDataSourceBeanPostProcessor();
```

```
    }

    static class SchemaPerTenantDataSourceBeanPostProcessor implements
BeanPostProcessor, BeanFactoryAware {

        private final AtomicReference<ObjectProvider<DataSourceInitializer>>
dataSourceInitializer //
                = new AtomicReference<>();

        @Override
        public @Nullable Object postProcessAfterInitialization(Object bean, String
beanName) throws BeansException {
            if (bean instanceof DataSource dataSource) {
②
                var dsi = this.dataSourceInitializer //
                    .get()
                    .getIfAvailable();
                return new SchemaPerTenantDataSource(//
                        dataSource, dsi);
            }
            return bean;
        }

        @Override
        public void setBeanFactory(BeanFactory beanFactory) throws BeansException {
            var beanProvider = beanFactory //
                .getBeanProvider(DataSourceInitializer.class);
            this.dataSourceInitializer.set(beanProvider);
        }

    }

}
```

① this configuration class registers a `BeanPostProcessor`, which must run much earlier than everything else. We declare the `@Bean` method `static` so that Spring can create it early in the lifecycle without inadvertently triggering the creation of other beans.

② here's where we make the switch: as soon as we've stumbled upon the default `DataSource`, we wrap it

The `BeanPostProcessor` also takes a reference to Spring's underlying `BeanFactory` and uses it to acquire an `ObjectProvider<DataSourceInitializer>`. That provider isn't the `DataSourceInitializer` itself, but it can be used to obtain one on demand. We resolve the reference only when we know we need it, and no sooner, because we don't want to force the initializer's creation too early in the lifecycle.

Now we've got a smart, schema-centric, multitenant `DataSource`. If you've kept everything in the same codebase, make sure you've deactivated the `db` profile so that the `DatabasePerTenantDataSource` and its related configuration stay out of the way. Then restart the service. Visit `127.0.0.1:8080` and you should again see different `customer` records for different logins - but this time you'll also see

new schemas, each with its own SQL tables, being created inside the one main PostgreSQL database.

Next Steps

We've looked at how to use Spring Security and Arconia's handy multitenancy module to design a system that is aware of a given authenticated user's tenant, and we used that to route the requests - in a way that was minimally invasive - to the appropriate database or schema.

There's more we could do.

Another idea I toyed with showing, but didn't, is row-level tenancy. All the data sits in one global, shared table, but clients see only the records that belong to their tenant, as determined by, say, a `tenant_identifier` column. You *could* build this by forcing `tenant_identifier` into your data model and modifying every query in the system, but it's precisely this sort of invasiveness that we want to avoid. Instead, we could use row-level security.

Row-level security is a mechanism specific to a handful of databases, PostgreSQL among them, and the implementations differ enough that you couldn't move from one to another without porting your work. The basic gist, in PostgreSQL anyway, is that the database can automatically qualify the queries an application issues against a given table with an extra `where` clause. The application sends `select id, name from customer`, and the database, seeing a rule attached to that table, quietly appends `where tenant_identifier = ⋯`. The ⋯ is a parameter, but the client never sends it. The client doesn't even know the rewriting is happening.

So who does send it, and how? PostgreSQL and other databases support session-specific variables. You could imagine building a `DataSource` that, each time it issues a new `Connection`, sets the session variable that the row-level security policy looks for.

You would issue SQL like this (in PostgreSQL):

```
ALTER TABLE customer ENABLE ROW LEVEL SECURITY;

ALTER TABLE customer FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON customer
   FOR ALL USING (tenant_identifier = current_setting('app.current_tenant', true));
```

Then, before handing out the connection, the `DataSource` would run:

```
SET LOCAL app.current_tenant = 'petclinic';
```

And the client code would issue nothing more exotic than:

```
select id, name from customer
```

And get back results already filtered by tenant. Easy! But PostgreSQL-specific, which is why I didn't spell it out in code. You could, though, and there are ways to get the same effect in SQL Server, Oracle, and others.

Of course, multitenancy affects not just the SQL database but plenty of other places as well. Got a NoSQL database? You could apply the patterns discussed in this chapter there, too. But it all starts with a simple OAuth token.